BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

LUẬN VĂN TỐT NGHIỆP

NGÀNH AN TOÀN THÔNG TIN

Đề tài

NGHIÊN CỨU MÔ HÌNH HỌC SÂU GỌN NHẸ CHO HỆ THỐNG PHÁT HIỆN XÂM NHẬP TẠI BIÊN TRONG MẠNG IOT – PHÂN HỆ 1

A STUDY OF LIGHTWEIGHT DEEP LEARNING MODELS FOR EDGE-BASED INTRUSION DETECTION SYSTEMS IN IOT NETWORKS – MODULE 1

Giảng viên hướng dẫn:

TS. TRẦN THỊ TỐ QUYÊN

Sinh viên thực hiện:

NGÔ ĐỨC THẮNG

B2203737

KHÓA 48

Cần Thơ, 08/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

LUẬN VĂN TỐT NGHIỆP

NGÀNH AN TOÀN THÔNG TIN

Đề tài

NGHIÊN CỨU MÔ HÌNH HỌC SÂU GỌN NHẸ CHO HỆ THỐNG PHÁT HIỆN XÂM NHẬP TẠI BIÊN TRONG MẠNG IOT – PHÂN HỆ 1

A STUDY OF LIGHTWEIGHT DEEP LEARNING MODELS FOR EDGE-BASED INTRUSION DETECTION SYSTEMS IN IOT NETWORKS – MODULE 1

Giảng viên hướng dẫn:

TS. TRẦN THỊ TỐ QUYÊN

Sinh viên thực hiện:

NGÔ ĐỨC THẮNG

B2203737

KHÓA 48

Cần Thơ, 08/2026

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

XÁC NHẬN CHỈNH SỬA LUẬN VĂN THEO YÊU CẦU CỦA HỘI ĐỒNG

Tên luận văn (tiếng Việt và tiếng Anh):

Tiếng Việt: Nghiên cứu mô hình học sâu gọn nhẹ cho hệ thống phát hiện xâm nhập tại biên trong mạng IoT - Phân hệ 1

Tiếng Anh: A study of lightweight deep learning models for edge-based intrusion detection systems in IoT networks – Module 1

Họ tên sinh viên: Ngô Đức Thắng MASV: B2203737

Mã lớp: DI22D2A1

Đã báo cáo tại hội đồng ngành: An toàn thông tin

Ngày báo cáo: 25/08/2026

Luận văn đã được chỉnh sửa theo góp ý của Hội đồng.

Cần Thơ, ngày ..... tháng …… năm 20…
Giáo viên hướng dẫn
(Ký và ghi họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày….tháng 08 năm 2026

Sinh viên thực hiện

Ngô Đức Thắng

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN ..... **i**

MỤC LỤC ..... **ii**

LỜI CẢM ƠN ..... **vi**

TÓM TẮT ..... **vii**

ABSTRACT ..... **ix**

DANH MỤC BẢNG ..... **xi**

DANH MỤC HÌNH ..... **xiv**

DANH MỤC TỪ CHUYÊN NGÀNH ..... **xviii**

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI **1**

1.1. Đặt vấn đề 1

1.2. Lịch sử giải quyết vấn đề 1

1.3. Mục tiêu đề tài 3

1.4. Phạm vi đề tài 3

1.5. Bảng phân chia công việc 4

1.6. Bố cục luận văn 7

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **8**

2.1. Internet of Things (IoT) 8

2.1.1. Khái niệm 8

2.1.2. Đặc điểm của IoT 9

2.1.3. Thách thức của IoT (trong bối cảnh phân tích dữ liệu) 9

2.2. Hệ thống phát hiện xâm nhập và nhu cầu trong IoT 10

2.2.1. Hệ thống phát hiện xâm nhập (IDS) 10

2.2.2. Nhu cầu về hệ thống phát hiện xâm nhập mạng tại biên (edge computing) trong IoT 11

2.3. Học máy và vai trò trong hệ thống phát hiện xâm nhập mạng 11

2.3.1. Học máy (Machine Learning) và học sâu (Deep Learning) 11

2.3.2. Vai trò của học máy và học sâu trong hệ thống phát hiện xâm nhập mạng 12

2.4. Tập dữ liệu CICIoT2023 [14] 12

2.5. Các mô hình huấn luyện được sử dụng 17
2.5.1. XGBoost 17
2.5.2. DNN (Deep Neural Network) 19
2.5.3. CNN (Convolutional Neural network) 21
2.5.4. LSTM 24
2.5.5. MLP 25
2.5.6. AutoEncoder 26
2.5.7. Mô hình học Tương phản tự giám sát có tinh chỉnh (Self-supervised Contrastive Learning DNN with Fine-tuning hay SSL-CL DNN) 27
2.6. Kỹ thuật tiền xử lý dữ liệu 29
2.6.1. Giảm mẫu 29
2.6.2. Mã hóa nhãn 29
2.6.3. Phân chia tập dữ liệu 29
2.6.4. Chuẩn hóa Standard Scaler 29
2.7. Kỹ thuật tối ưu siêu tham số với OPTUNA 30
2.8. Kỹ thuật làm nhẹ (lightweight) cho mô hình 31
2.8.1. Kỹ thuật Chưng cất tri thức (Knowledge Distillation) 31
2.8.2. Teacher đồng huấn luyện – MTKD (Multi-teacher Knowledge Distillation) [8] 34
2.8.3. Phòng thủ IoT nhận thức tài nguyên (Resource-Aware IoT Defense using KL-JS loss or RAID-KL) - Chưng cất tri thức bằng phân kỳ KL-JS (Raid-KL) [5] 37
2.8.4. Chưng cất tri thức qua gợi ý tầng ẩn (FitNets) 42
2.9. Thông số đánh giá hiệu quả các mô hình 45
2.9.1. Độ chính xác toàn cục (Accuracy) 45
2.9.2. Độ chính xác (Precision) 45
2.9.3. Độ nhạy (Recall) 45
2.9.4. F1-Score 46
2.9.5. Diện tích dưới đường cong ROC (AUC – Area Under the ROC Curve) 46
2.9.6. Chỉ số đánh giá về hiệu suất 47
CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN 48
3.1. Tổng quan bài toán 48

3.2. Xây dựng và huấn luyện các mô hình học sâu học sinh theo các phương pháp chưng cất tri thức truyền thống, Raid-KL và FitNets.....49

3.2.1. Xây dựng các mô hình học sinh từ các mô hình giáo viên.....49

3.2.2. Áp dụng kỹ thuật chưng cất tri thức truyền thống trong huấn luyện các mô hình học sinh CNN1D, DNN, SSL-CL-DNN.....53

3.2.3. Áp dụng kỹ thuật chưng cất tri thức sử dụng phân kỳ KL-JS (Raid-KL fixed temperature & adaptive temperature) trong huấn luyện các mô hình học sinh CNN1D, DNN, SSL-CL-DNN.....56

3.2.4. Áp dụng phương pháp chưng cất tri thức sử dụng gợi ý tầng ẩn (FitNets) cho các mô hình học sinh CNN1D, DNN, SSL-CL-DNN.....59

3.3. Quy trình triển khai suy luận bằng dữ liệu mạng PCAP sử dụng các mô hình học sinh giáo viên và học sinh để đánh giá kết quả suy luận, hiệu suất tại thiết bị IoT biên.....62

3.3.1. Mô tả dữ liệu mạng được sử dụng trong quy trình đánh giá, so sánh.....62

3.3.2. Mô tả môi trường triển khai quy trình đánh giá, so sánh.....66

3.3.3. Các thành phần trong quy trình trích xuất đặc trưng, tiền xử lý và suy luận bằng các mô hình học sâu giáo viên và học sinh.....66

3.3.4. Quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh sau chưng cất tri thức trên PCAP có nhãn trên thiết bị.....70

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 75

4.1. Cấu hình của môi trường huấn luyện và thiết bị IoT biên Rasberry Pi 4 B ...75

4.2. Các kết quả huấn luyện các mô hình học sâu giáo viên và học sinh sau khi thực hiện các kỹ thuật chưng cất tri thức 76

4.2.1. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức truyền thống.....76

4.2.2. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức bằng phân kì JS-KL (Raid-KL) 85

4.2.3. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức bằng gợi ý tầng ẩn (FitNets).....96

4.2.4. Đánh giá, so sánh kết quả tổng kết sau quá trình huấn luyện các mô hình 103

4.3. Các kết quả triển khai suy luận bằng các mô hình trên dữ liệu mạng PCAP tại thiết bị biên.....110

4.3.1. Kết quả đánh giá của các mô hình học sinh và giáo viên trên dữ liệu mạng PCAP có nhãn tại thiết bị biên.....110

4.3.2. Kết quả hiệu suất của các mô hình học sinh và mô hình giáo viên trên dữ liệu mạng PCAP tại thiết bị biên. 119

4.3.3. Kết quả thực nghiệm khi suy luận trực tuyến trên dữ liệu mạng tự thu thập trên thiết bị biên.....125

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....137

5.1. Kết luận.....137

5.2. Hạn chế.....138

5.3. Hướng phát triển.....138

TÀI LIỆU THAM KHẢO 140

PHỤ LỤC.....143

Đối chiếu đặc trưng CICIoT2023 giữa mô tả gốc và mã nguồn trích xuất.....143

a) Mục đích 143

b) Kết quả huấn luyện và suy luận trên tập dữ liệu CICIoT2023 46 đặc trưng 143

c) Kết quả huấn luyện và suy luận của các mô hình trên tập dữ liệu CICIoT2023 có 39 đặc trưng.....147

d) Kết quả huấn luyện và suy luận của các mô hình trên tập dữ liệu CICIoT2023 có 37 đặc trưng.....150

LỜI CẢM ƠN

Lời đầu tiên, em muốn nói rằng việc hoàn thành đề tài luận văn tốt nghiệp "Nghiên cứu mô hình học sâu gọn nhẹ cho hệ thống phát hiện xâm nhập tại biên trong mạng IoT – Phân hệ 1" là một thách thức đầy ý nghĩa đối với em. Trong suốt quá trình thực hiện đề tài này, em đã gặp không ít khó khăn, xem như là cơ hội để em được học hỏi và rèn luyện bản thân. Đồng thời em đã được nhận được rất nhiều sự quan tâm và hỗ trợ đến từ rất nhiều cá nhân cũng như tập thể. Vậy nên cho em xin được gửi lời tri ân sâu cắc đến những người và tập thể mà em mang ơn sâu sắc.

Trước tiên hết, em xin gửi lời cảm ơn sâu sắc đến Tiến sĩ Trần Thị Tố Quyên, cô đã tận tình dành rất nhiều thời gian quý báu để hướng dẫn, góp ý rất nhiều và định hướng trong suốt quá trình em thực hiện đề tài. Nhờ sự giúp đỡ và chỉ dẫn của cô, em đã hiểu rõ hơn về các kiến thức chuyên ngành và cách tiếp cận vấn đề một cách khoa học hơn, cũng như cách triển khai, ứng dụng các kiến thức chuyên ngành để hoàn thành đề tài luận văn.

Em cũng xin chân thành cảm ơn Trường Đại học Cần Thơ, đặc biệt là Khoa Mạng máy tính và Truyền thông dữ liệu của Trường Công nghệ thông tin và Truyền thông đã tạo điều kiện học tập và môi trường tốt để em có thể thực hiện và hoàn thành đề tài này.

Bên cạnh đó, em xin cảm ơn gia đình đã luôn là chỗ dựa tinh thần vững chắc, và cảm ơn các bạn bè đã hỗ trợ, chia sẻ tài liệu, góp ý cũng như động viên em trong suốt thời gian thực hiện đề tài này một cách trọn vẹn.

Mặc dù còn nhiều thiếu sót nhưng em thật sự đã cố gắng hết sức mình để hoàn thành đề tài luận văn. Em hy vọng sẽ nhận được những ý kiến đóng góp quý báo từ thầy cô để đề tài này cũng như bản thân em được hoàn thiện hơn trong tương lai ạ.

Cuối lời em xin chân thành chúc thầy cô sức khỏe, thành công và hạnh phúc!

Cần Thơ, ngày….tháng 08 năm 2026

Sinh viên thực hiện

Ngô Đức Thắng

TÓM TẮT

Trong bối cảnh số lượng thiết bị IoT tăng trưởng không ngừng, bề mặt tấn công của các hệ thống mạng ngày càng mở rộng, trong khi phần lớn thiết bị ở biên lại rất hạn chế về tài nguyên tính toán, bộ nhớ và năng lượng. Các mô hình học sâu phát hiện xâm nhập có độ chính xác cao thường có dung lượng lớn và tốn nhiều tài nguyên, nên khó triển khai trực tiếp lên các thiết bị IoT biên. Chưng cất tri thức đang nổi lên như một hướng làm nhẹ mô hình đầy tiềm năng, nhưng vẫn còn thiếu các đánh giá thực nghiệm đa chiều trên dữ liệu dạng bảng và trên thiết bị vật lý thực. Xuất phát từ thực trạng đó, luận văn đặt mục tiêu khảo sát toàn diện khả năng của các kỹ thuật chưng cất tri thức trong việc tạo ra mô hình phát hiện xâm nhập nhẹ, có thể triển khai hiệu quả trên thiết bị IoT biên.

Luận văn sử dụng tập dữ liệu CICIoT2023, tổ chức lại thành bài toán phân loại đa lớp với 8 nhãn nhóm. Nghiên cứu xây dựng và huấn luyện các mô hình giáo viên đa dạng về kiến trúc gồm CNN-1D, DNN, SSL-CL-DNN, XGBoost và LSTM, đều có tối ưu siêu tham số bằng Optuna để làm cơ sở tri thức cho quá trình chưng cất. Trên cơ sở đó, luận văn triển khai ba kỹ thuật chưng cất là KD truyền thống, RAID-KL sử dụng phân kỳ KL-JS (khảo sát cả nhiệt độ cố định và nhiệt độ thích ứng) và FitNets lên các mô hình học sinh có kiến trúc thu gọn tương ứng, …đồng thời khảo sát chưng cất đa giáo viên theo hai cơ chế AW-MTKD và CW-MTKD, trong đó cơ chế CW-MTKD được chọn để thực nghiệm với học sinh là mạng MLP nhỏ. Hiệu quả phân loại được đánh giá qua accuracy, macro-F1, macro-AUC, precision và recall, còn hiệu suất triển khai được đánh giá qua thông lượng, độ trễ, mức chiếm dụng CPU và RAM. Quá trình đánh giá thực hiện trên cả dữ liệu PCAP có nhãn và lưu lượng mạng tự thu thập, trong đó hiệu suất được đo trực tiếp trên thiết bị Raspberry Pi 4 B.

Kết quả cho thấy chưng cất tri thức nén được các mô hình từ 92.0% đến 99.65% về số tham số mà chỉ mất từ 1.91 đến 5.58 điểm phần trăm macro-F1. Trong nhóm học sinh, kiến trúc SSL-CL-DNN chịu nén tốt nhất và đạt độ chính xác cao nhất, khoảng 85%, trong khi học sinh MLP chưng cất đa giáo viên CW-MTKD lại đạt macro-F1 cao nhất, khoảng 70%, đổi lại có độ chính xác tổng thể thấp nhất và kích thước nhỏ nhất. Đáng chú ý là lựa chọn kiến trúc và cặp giáo viên tác động đến hiệu quả phân loại mạnh hơn lựa chọn kỹ thuật chưng cất, và cơ chế nhiệt độ thích ứng của RAID-KL vượt nhiệt độ cố định một cách nhất quán trên cả ba kiến trúc. Khi suy luận trên lưu lượng mạng thực, độ chính xác giảm còn khoảng 68% đến 73% do khác biệt của bộ trích xuất đặc trưng và hiện tượng dịch chuyển phân phối, nhưng macro-AUC vẫn giữ ở mức cao khoảng 94% đến 95%, cho thấy năng lực xếp hạng xác suất của mô hình gần như được bảo toàn. Về hiệu suất triển khai trên Raspberry Pi 4 B, các mô hình học sinh đạt thông lượng khoảng 4.100 đến 4.170 gói tin mỗi giây với độ trễ khoảng 2,4 ms cho mỗi cửa sổ mười gói tin, chiếm khoảng 25% CPU và 105 MB RAM, trong khi các mô hình giáo viên chỉ đạt 1.500 đến 3.900 gói tin mỗi giây. Ở chế độ suy luận trực tuyến trên lưu lượng tự thu thập, quy trình vận hành liên

tục và ổn định qua toàn bộ các kịch bản tấn công thử nghiệm với mức chiếm dụng khoảng 26% CPU và dưới 50 MB RAM, chứng minh tính khả thi của việc vận hành mô hình phát hiện xâm nhập ngay tại biên.

Từ các kết quả trên, luận văn kết luận rằng chưng cất tri thức là một hướng tiếp cận khả thi để đưa mô hình phát hiện xâm nhập về kích thước phù hợp với thiết bị IoT biên. Không có mô hình nào tối ưu trên mọi tiêu chí, nên việc lựa chọn mô hình triển khai cần dựa vào mục tiêu ưu tiên cụ thể, chẳng hạn độ trễ thấp, độ chính xác tổng thể, hay khả năng phát hiện các lớp tấn công hiếm.

Từ khóa: phát hiện xâm nhập, IoT, điện toán biên, chưng cất tri thức, mô hình học sâu gọn nhẹ, CICIoT2023, Raspberry Pi 4.

ABSTRACT

As the number of IoT devices keeps growing, the attack surface of network systems keeps expanding, while most edge devices remain very limited in computation, memory and energy. Deep learning intrusion detection models with high accuracy are usually large and resource-heavy, so they are hard to deploy directly on edge IoT devices. Knowledge distillation is emerging as a promising direction for making models lighter, but there is still a lack of multi-faceted empirical evaluation on tabular data and on real physical devices. Motivated by this situation, the thesis aims to comprehensively investigate how well knowledge distillation techniques can produce lightweight intrusion detection models that can be effectively deployed on edge IoT devices.

The thesis uses the CICIoT2023 dataset, reorganized into a multi-class problem with 8 grouped labels. Several teacher models with diverse architectures are built and trained, namely CNN-1D, DNN, SSL-CL-DNN, XGBoost and LSTM, all tuned with Optuna to serve as the knowledge base for distillation. On this basis, three distillation techniques are applied to the correspondingly compact students, namely vanilla KD, RAID-KL with the KL-JS divergence (studied under both fixed and adaptive temperature) and FitNets, together with multi-teacher distillation, in which the AW-MTKD and CW-MTKD schemes are surveyed and CW-MTKD is selected for experimentation with a small MLP student. Classification quality is evaluated by accuracy, macro-F1, macro-AUC, precision, and recall, while deployment efficiency is evaluated by throughput, latency, and CPU and RAM usage. The evaluation is carried out on both labeled PCAP data and self-collected traffic, with the efficiency measured directly on a Raspberry Pi 4 B device.

The results show that knowledge distillation compresses the models by 92.0% to 99.65% in the number of parameters while losing only 1.91 to 5.58 percentage points of macro-F1. Among the students, the SSL-CL-DNN architecture withstands compression best and reaches the highest accuracy, about 85%, whereas the CW-MTKD multi-teacher MLP student reaches the highest macro-F1, about 70%, in exchange for the lowest overall accuracy and the smallest size. Notably, the choice of architecture and teacher pair affects classification quality more strongly than the choice of distillation technique, and RAID-KL with adaptive temperature consistently outperforms the fixed temperature across all three architectures. On real network traffic, accuracy drops to about 68% to 73% due to the feature-extractor discrepancy and distribution shift, yet macro-AUC stays high at about 94% to 95%, showing that the ranking capability of the models is largely preserved. On the Raspberry Pi 4 B, the student models reach a throughput of about 4,100 to 4,170 packets per second with a latency of about 2.4 ms per ten-packet window, while using about 25% CPU and 105 MB of RAM; the teacher models reach only 1,500 to 3,900 packets per second. In the online inference mode on self-collected traffic, the pipeline runs continuously and stably through all tested attack scenarios while using about 26% CPU

and under 50 MB of RAM, demonstrating the feasibility of running intrusion detection right at the edge.

From these results, the thesis concludes that knowledge distillation is a feasible approach for bringing intrusion detection models down to a size suitable for edge IoT devices. No single model is optimal on every criterion, so the deployed model should be chosen according to the specific priority, such as low latency, overall accuracy, or the ability to detect rare attack classes.

Keywords: intrusion detection, IoT, edge computing, knowledge distillation, lightweight deep learning, CICIoT2023, Raspberry Pi 4.

DANH MỤC BẢNG

Bảng 1.1. Phân công công việc của đề tài....7

Bảng 2.1. Danh sách đặc trưng của tập dữ liệu CICIoT2023 tải từ trang chính của UNB CIC....13

Bảng 2.2. Số lượng mẫu mỗi nhãn tấn công (8 nhãn được nhóm) trong tập dữ liệu....14

Bảng 3.1. Bảng kiến trúc mô hình học sinh CNN1D so với giáo viên CNN1D....50

Bảng 3.2. Bảng kiến trúc mô hình học sinh DNN so với giáo viên DNN....51

Bảng 3.3. Bảng kiến trúc mô hình học sinh SSL-CL-DNN so với mô hình giáo viên SSL-CL-DNN....52

Bảng 3.4. Bảng tổng hợp số lượng tham số của kiến trúc mô hình giáo viên so với học sinh....53

Bảng 3.5. Bảng siêu tham số chưng cất tri thức truyền thống....53

Bảng 3.6. Bảng siêu tham số huấn luyện mô hình học sinh CNN....55

Bảng 3.7. Bảng siêu tham số huấn luyện mô hình học sinh DNN....55

Bảng 3.8. Bảng siêu tham số huấn luyện của mô hình học sinh SSL-CL-DNN....55

Bảng 3.9. Bảng siêu tham số huấn luyện chưng cất tri thức Raid-KL nhiệt độ cố định....56

Bảng 3.10. Bảng siêu tham số huấn luyện chưng cất tri thức Raid-KL nhiệt độ thích ứng....56

Bảng 3.11. Bảng siêu tham số huấn luyện chưng cất tri thức bằng gợi ý tầng ẩn FitNets 59

Bảng 3.12. Bảng thông tin tập tin dữ liệu mạng PCAP đầu sử dụng để trích xuất và suy luận (tập tin thứ 1 đến 17)....63

Bảng 3.13. Bảng thông tin tập tin dữ liệu mạng PCAP đầu sử dụng để trích xuất và suy luận (tập tin thứ 18 đến 34)....64

Bảng 3.14. Bảng thông tin các tập tin dữ liệu mạng PCAP cho trích xuất và suy luận theo nhãn nhóm (từ tập tin thứ 1 đến 17)....64

Bảng 3.15. Bảng thông tin các tập tin dữ liệu mạng PCAP cho trích xuất và suy luận theo nhãn nhóm (từ tập tin thứ 18 đến 34)....64

Bảng 3.16. Công cụ của Kali Linux được sử dụng với lưu lượng tạo ra tương ứng....65

Bảng 3.17. Danh sách lệnh cơ bản của live_infer.py....73

Bảng 3.18. Danh sách tham số truyền cho lệnh ‘run’ khi bắt đầu suy luận....74

Bảng 4.1. Cấu hình chi tiết môi trường huấn luyện Kaggle và thiết bị Raspberry Pi 4 B.75

Bảng 4.2. Kết quả huấn luyện mô hình học sinh CNN KD so với giáo viên....77

Bảng 4.3. Kết quả huấn luyện mô hình học sinh DNN KD so với giáo viên 79
Bảng 4.4. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN KD so với giáo viên 81
Bảng 4.5. Bảng kết quả huấn luyện các mô hình học sinh bằng chưng cất tri thức truyền thống.....83
Bảng 4.6. Kết quả huấn luyện mô hình học sinh CNN Raid-KL so với giáo viên.....86
Bảng 4.7. Kết quả huấn luyện mô hình học sinh DNN Raid-KL so với giáo viên.....88
Bảng 4.8. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN Raid-KL so với giáo viên 91
Bảng 4.9. Kết quả huấn luyện các mô hình học sinh chưng cất tri thức Raid-KL 93
Bảng 4.10. Kết quả mô hình học sinh CNN FitNets so với giáo viên.....97
Bảng 4.11. Kết quả huấn luyện mô hình học sinh DNN FitNets so với giáo viên 99
Bảng 4.12. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN FitNets so với giáo viên 100
Bảng 4.13. Kết quả huấn luyện các mô hình học sinh chưng cất tri thức FitNets.....101
Bảng 4.14. Bảng tổng hợp kết quả huấn luyện tất cả các mô hình.....108
Bảng 4.15. Kết quả đánh giá các mô hình học sinh CNN trên dữ liệu mạng PCAP.....110
Bảng 4.16. Kết quả đánh giá của các mô hình học sinh DNN trên dữ liệu mạng PCAP 112
Bảng 4.17. Kết quả đánh giá của các mô hình học sinh SSL-CL-DNN trên dữ liệu mạng PCAP.....113
Bảng 4.18. Kết quả đánh giá của các mô hình học sinh LSTM trên dữ liệu mạng PCAP 114
Bảng 4.19. Kết quả chỉ số số gói tin trên giây của các mô hình trên thiết bị so với trên môi trường huấn luyện.....120
Bảng 4.20. Kết quả chỉ số tốc độ thực thi khi suy luận bằng các mô hình trên thiết bị so với trên môi trường huấn luyện.....121
Bảng 4.21. Kết quả chỉ số độ trễ quy trình khi suy luận bằng các mô hình trên thiết bị so với trên môi trường huấn luyện.....122
Bảng 4.22. Kết quả chỉ số chiếm dụng CPU và RAM của quy trình suy luận bằng mô hình trên thiết bị biên 123
Bảng 4.23. Kết quả suy luận và hiệu suất của các hệ thống IDS liên quan đến đề tài 124
Bảng phụ lục 1. Danh sách đặc trưng của tập dữ liệu CICIoT2023 có 46 đặc trưng 145

Bảng phụ lục 2. Kết quả huấn luyện các mô hình giáo viên và học sinh trên tập dữ liệu CICIoT2023 có 46 đặc trưng 146

Bảng phụ lục 3. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 46 đặc trưng 147

Bảng phụ lục 4. Khác biệt trong định nghĩa đặc trưng giữa tập dữ liệu 39 đặc trưng so với tập dữ liệu 46 đặc trưng 148

Bảng phụ lục 5. Kết quả huấn luyện mô hình SSL-CL-DNN trên tập dữ liệu CICIoT2023 39 đặc trưng.....149

Bảng phụ lục 6. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng 149

Bảng phụ lục 7. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng 150

Bảng phụ lục 8. Kết quả suy luận của mô hình SSL-CL-DNN trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng.....151

Bảng phụ lục 9. Kết quả suy luận của mô hình SSL-CL-DNN trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng.....152

Bảng phụ lục 10. Kết quả suy luận của mô hình LSTM trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng 153

Bảng phụ lục 11. Kết quả suy luận của mô hình LSTM trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng 154

DANH MỤC HÌNH

Hình 2.1. Số lượng mẫu mỗi nhãn trên 34 loại lưu lượng trong tập dữ liệu 14
Hình 2.2. Minh họa quy trình giải thuật XGBoost 19
Hình 2.3. Kiến trúc cơ bản của mô hình DNN 20
Hình 2.4. Kiến trúc cơ bản của mô hình CNN.....22
Hình 2.5. Cách dữ liệu đầu vào CNN được biểu diễn qua lớp tích chập.....23
Hình 2.6. Kiến trúc cơ bản mô hình LSTM 24
Hình 2.7. Kiến trúc hoạt động của mô hình MLP.....26
Hình 2.8. Cấu trúc cơ bản của mô hình AutoEncoder 27
Hình 2.9. Minh họa kiến trúc của mô hình SSL-CL DNN 28
Hình 2.10. Nguyên lý hoạt động cơ bản của quá trình Chưng cất tri thức 34
Hình 2.11. Minh họa 3 loại đường ROC.....47
Hình 3.1. Hình minh họa bài toán tổng quan.....48
Hình 3.2. Luồng xử lý của CNN1D.....49
Hình 3.3. Luồng xử lý của DNN.....50
Hình 3.4. Biểu đồ cột thể hiện số gói tin mỗi loại nhãn theo nhóm (8 nhóm) 65
Hình 3.5. Minh họa sơ đồ mạng của môi trường triển khai quy trình 66
Hình 3.6. Ảnh minh họa quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh trên dữ liệu PCAP có nhãn trên thiết bị.....70
Hình 3.7. Cấu trúc cây thư mục của quy trình suy luận trên dữ liệu mạng tự thu thập.....71
Hình 3.8. Ảnh minh họa quy trình thử nghiệm suy luận mô hình trên dữ liệu mạng PCAP tự thu thập tại thiết bị 72
Hình 3.9. Giao diện chương trình live_infer.py khi khởi động (phần đầu) 73
Hình 3.10. Giao diện chương trình live_infer.py khi khởi động (phần cuối) 73
Hình 4.1. Ảnh thiết bị Raspberry Pi 4 B 75
Hình 4.2. Biểu đồ đường mất mát trong quá trình huấn luyện CNN KD.....76
Hình 4.3. Ma trận nhầm lẫn của CNN KD 78
Hình 4.4. Biểu đồ đường mất mát quá trình huấn luyện DNN KD 79
Hình 4.5. Ma trận nhầm lẫn DNN KD.....80
Hình 4.6. Biểu đồ đường mất mát quá trình huấn luyện SSL-CL-DNN KD 81

Hình 4.7. Ma trận nhầm lẫn của SSL-CL-DNN KD 82
Hình 4.8. Biểu đồ cột Accuracy, Macro-F1 và inference time của các mô hình học sinh KD 83
Hình 4.9. Ma trận nhầm lẫn của các mô hình học sinh sau chưng cất tri thức truyền thống 84
Hình 4.10. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức truyền thống 84
Hình 4.11. Hai biểu đồ đường mất mát huấn luyện của mô hình CNN Raid-KL 85
Hình 4.12. Ma trận nhầm lẫn CNN Raid-KL (fix t) 87
Hình 4.13. Ma trận nhầm lẫn CNN Raid-KL (adapt t) 87
Hình 4.14. Hai biểu đồ đường mất mát huấn luyện mô hình DNN Raid-KL.....88
Hình 4.15. Ma trận nhầm lẫn DNN Raid-KL (fix t) 89
Hình 4.16. Ma trận nhầm lẫn DNN Raid-KL (adapt t) 90
Hình 4.17. Hai biểu đồ đường mất mát huấn luyện mô hình SSL-CL-DNN Raid-KL.....90
Hình 4.18. Ma trận nhầm lẫn SSL-CL-DNN Raid-KL (fix t) 92
Hình 4.19. Ma trận nhầm lẫn SSL-CL-DNN Raid-KL (adapt t) 92
Hình 4.20. Biểu đồ cột Accuracy, Macro-F1 và Inference time các mô hình học sinh chưng cất tri thức Raid-KL 93
Hình 4.21. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức Raid-KL (fix t) ...94
Hình 4.22. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức Raid-KL (adapt t) 94
Hình 4.23. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức Raid-KL (fix t) 95
Hình 4.24. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức Raid-KL (adapt t) 95
Hình 4.25. Biểu đồ đường mất mát ở 2 giai đoạn huấn luyện CNN FitNets.....96
Hình 4.26. Ma trận nhầm lẫn CNN FitNets 97
Hình 4.27. Biểu đồ đường mất mát 2 giai đoạn huấn luyện DNN FitNets.....98
Hình 4.28. Ma trận nhầm lẫn DNN FitNets.....99
Hình 4.29. Biểu đồ đường mất mát 2 giai đoạn huấn luyện SSL-CL-DNN FitNets 100
Hình 4.30. Ma trận nhầm lẫn SSL-CL-DNN FitNets 101

Hình 4.31. Biểu đồ cột Accuracy, Macro-F1 và Inference time các mô hình học sinh chưng cất tri thức FitNets.....102
Hình 4.32. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức bằng FitNets 103
Hình 4.33. Các biểu đồ đường ROC các mô hình học sinh chưng cất tri thức bằng FitNets 103
Hình 4.34. Biểu đồ cột thể hiện Accuracy giảm dần của tất cả các mô hình học sinh và giáo viên.....104
Hình 4.35. Biểu đồ cột thể hiện Macro-F1 giảm dần của tất cả các mô hình học sinh và giáo viên.....105
Hình 4.36. Biểu đồ cột thể hiện Inference time (µs) giảm dần của các mô hình học sinh 106
Hình 4.37. Kết quả đánh giá các mô hình học sinh CNN trên dữ liệu mạng PCAP 111
Hình 4.38. Kết quả đánh giá các mô hình học sinh DNN trên dữ liệu mạng PCAP 112
Hình 4.39. Kết quả đánh giá các mô hình học sinh SSL-CL-DNN trên dữ liệu mạng PCAP.....113
Hình 4.40. Kết quả đánh giá mô hình LSTM học sinh so với giáo viên trên dữ liệu mạng PCAP.....114
Hình 4.41. Kết quả accuracy của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn 115
Hình 4.42. Kết quả macro-F1 của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn 116
Hình 4.43. Kết quả macro-AUC của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn 117
Hình 4.44. Dự đoán của mô hình học sinh cho các gói tin warm-up ban đầu.....125
Hình 4.45. Dự đoán của mô hình giáo viên sau bước warm-up 126
Hình 4.46. Dự đoán của mô hình học sinh sau khi ping.....127
Hình 4.47. Dự đoán của mô hình giáo viên sau khi ping.....127
Hình 4.48. Dự đoán của mô hình học sinh khi bị quét host trên nhánh mạng bằng nmap 128
Hình 4.49. Dự đoán của mô hình giáo viên khi bị quét host trên nhánh mạng bằng nmap 128
Hình 4.50.Dự đoán của mô hình khi bị quét Stealth scan bằng nmap.....129
Hình 4.51. Dự đoán của mô hình giáo viên khi bị quét Stealth Scan bằng nmap 129

Hình 4.52. Dự đoán của mô hình học sinh khi bị quét hệ điều hành bằng nmap 130
Hình 4.53. Dự đoán của mô hình giáo viên khi bị quét hệ điều hành bằng nmap.....130
Hình 4.54. Dự đoán của mô hình học sinh khi bị tấn công BruteForce trên dịch vụ SSH bằng hydra.....131
Hình 4.55. Dự đoán của mô hình giáo viên khi bị tấn công BruteForce trên dịch vụ SSH bằng hydra.....132
Hình 4.56. Dự đoán của mô hình học sinh khi bị tấn công làm ngập bằng hping3.....132
Hình 4.57. Dự đoán của mô hình giáo viên khi bị tấn công làm ngập bằng hping3 133
Hình 4.58. Dự đoán của mô hình khi bị tấn công làm ngập với IP ngẫu nhiên bằng hping3 134
Hình 4.59. Dự đoán của mô hình giáo viên khi bị tấn công làm ngập bằng IP ngẫu nhiên với công cụ hping3.....134
Hình 4.60. Kết quả đo hiệu suất sau khi chạy chương trình suy luận trực tuyến với mô hình học sinh DNN_XGB.....135
Hình 4.61. Kết quả đo hiệu suất sau khi chạy chương trình suy luận trực tuyến với mô hình giáo viên DNN 135

DANH MỤC TỪ CHUYÊN NGÀNH

Từ viết tắt	Dạng đầy đủ	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
ML	Machine Learning	Học máy
DL	Deep Learning	Học sâu
Edge AI	Edge Artificial Intelligence	Trí tuệ nhân tạo triển khai tại biên mạng
IoT	Internet of Things	Internet vạn vật
UAV	Unmanned Aerial Vehicle	Thiết bị bay không người lái
LLMs	Large Language Models	Các mô hình ngôn ngữ lớn
IDS	Intrusion Detection System	Hệ thống phát hiện xâm nhập
NIDS	Network-based IDS	IDS dựa trên mạng
HIDS	Host-based IDS	IDS dựa trên máy chủ
DoS	Denial of Service	Tấn công từ chối dịch vụ
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
MITM	Man-in-the-Middle	Tấn công người đứng giữa
XSS	Cross-Site Scripting	Tấn công chèn mã kịch bản chéo trang
UNB	University of Brunswick	Đại học Brunswick
CIC	Canadian Institute for Cybersecurity	Viện An ninh mạng Canada (đơn vị công bố các bộ dữ liệu CICIoT2023, CIC-IDS2017...)
CICIoT2023	CIC IoT dataset 2023	Bộ dữ liệu tấn công mạng IoT do CIC công bố năm 2023
CICIoMT2024	CIC IoMT dataset 2024	Bộ dữ liệu tấn công mạng IoT y tế do CIC công bố năm 2024
CIC-IDS2017	CIC Intrusion Detection dataset 2017	Bộ dữ liệu phát hiện xâm nhập do CIC công bố năm 2017
NSL-KDD	NSL-KDD dataset	Bộ dữ liệu chuẩn cho bài toán phát hiện xâm nhập
UNSW-NB15	UNSW-NB15 dataset	Bộ dữ liệu phát hiện xâm nhập của Đại học New South Wales
IoTID20	IoT Intrusion Dataset 2020	Bộ dữ liệu xâm nhập IoT năm 2020
PCAP	Packet Capture	Tệp ghi lại (bắt) gói tin mạng
CSV	Comma-Separated Values	Định dạng tệp dữ liệu phân tách bằng dấu phẩy

DNN	Deep Neural Network	Mạng nơ-ron sâu (mạng truyền thẳng đa lớp)
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CNN-1D / CNN1D	1-Dimensional CNN	Mạng CNN một chiều
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài–ngắn hạn
BiLSTM	Bidirectional LSTM	Mạng LSTM hai chiều
MLP	Multilayer Perceptron	Mạng perceptron đa lớp
SSL-CL-DNN / SSL_CL_DNN	Self-supervised Contrastive Learning DNN (with Fine-tuning)	Mô hình DNN học tương phản tự giám sát có tinh chỉnh (mô hình giáo viên/baseline)
CW-DNN	Confidence-Weighted DNN (student)	Mô hình học sinh DNN chưng cất theo cơ chế trọng số tin cậy
XGBoost / XGB	eXtreme Gradient Boosting	Thuật toán/thư viện tăng cường gradient cực trị
RF	Random Forest	Rừng ngẫu nhiên
AdaBoost	Adaptive Boosting	Thuật toán tăng cường thích ứng
CART	Classification and Regression Tree	Cây phân loại và hồi quy
AutoEncoder	Autoencoder	Bộ tự mã hóa (mạng nơ-ron học biểu diễn không giám sát)
ReLU	Rectified Linear Unit	Hàm kích hoạt tuyến tính chỉnh lưu
Softmax	Softmax function	Hàm kích hoạt chuẩn hóa xác suất đầu ra ở lớp phân loại
Adam	Adaptive Moment Estimation	Thuật toán tối ưu Adam
SGD	Stochastic Gradient Descent	Hạ gradient ngẫu nhiên
BatchNorm / BatchNormalization	Batch Normalization	Chuẩn hóa theo lô
MSE	Mean Squared Error	Sai số bình phương trung bình
CE	Cross-Entropy	Hàm mất mát entropy chéo
NT-Xent (Loss)	Normalized Temperature-scaled Cross-Entropy Loss	Hàm mất mát entropy chéo chuẩn hóa theo nhiệt độ (dùng trong học tương phản)
PEFT	Parameter-Efficient Fine-Tuning	Tinh chỉnh hiệu quả tham số
KD	Knowledge Distillation	Chưng cất tri thức

RAID-KL	RAID-KL (technique name)	Kỹ thuật chưng cất tri thức dùng phân kỳ KL-JS
FitNets	FitNets	Kỹ thuật/phương pháp chưng cất tri thức bằng gợi ý tầng ẩn FitNets
MTKD	Multi-teacher Knowledge Distillation	Chưng cất tri thức đa giáo viên
AW-MTKD	Average-Weighted MTKD	MTKD trọng số trung bình (giữa các giáo viên)
CW-MTKD	Confidence-Weighted MTKD	MTKD trọng số theo độ tin cậy của từng giáo viên
KL	Kullback-Leibler (divergence)	Phân kỳ Kullback-Leibler
JS	Jensen-Shannon (divergence)	Phân kỳ Jensen-Shannon
KL-JS / JS-KL	KL-JS divergence	Phân kỳ lai kết hợp Kullback-Leibler và Jensen-Shannon
Optuna	Optuna	Thư viện tối ưu siêu tham số mã nguồn mở (giấy phép MIT)
TPE	Tree-structured Parzen Estimator	Thuật toán ước lượng Parzen dạng cây
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
UDP	User Datagram Protocol	Giao thức dữ liệu người dùng
IP	Internet Protocol	Giao thức Internet
ICMP	Internet Control Message Protocol	Giao thức thông điệp điều khiển Internet
IGMP	Internet Group Management Protocol	Giao thức quản lý nhóm Internet
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HTTP Secure	Giao thức HTTP có bảo mật (mã hóa)
DNS	Domain Name System	Hệ thống phân giải tên miền
DHCP	Dynamic Host Configuration Protocol	Giao thức cấu hình máy chủ động
SMTP	Simple Mail Transfer Protocol	Giao thức truyền tải thư điện tử đơn giản
IRC	Internet Relay Chat	Giao thức trò chuyện trực tuyến qua Internet

SSH	Secure Shell	Giao thức truy cập từ xa bảo mật (cổng dịch vụ 22)
ARP	Address Resolution Protocol	Giao thức phân giải địa chỉ
LLC	Logical Link Control	Giao thức tầng liên kết dữ liệu
GRE	Generic Routing Encapsulation	Giao thức đóng gói định tuyến chung
VNC	Virtual Network Computing	Giao thức điều khiển máy tính từ xa
MAC	Media Access Control	Địa chỉ điều khiển truy cập môi trường truyền
GSM	Global System for Mobile Communications	Chuẩn thông tin di động toàn cầu
RFID	Radio-Frequency Identification	Công nghệ nhận dạng qua tần số vô tuyến
SYN	Synchronize (TCP flag)	Cờ đồng bộ trong bắt tay TCP
ACK	Acknowledgement (TCP flag)	Cờ xác nhận trong TCP
FIN	Finish (TCP flag)	Cờ kết thúc kết nối TCP
RST	Reset (TCP flag)	Cờ đặt lại kết nối TCP
PSH	Push (TCP flag)	Cờ đẩy dữ liệu ngay trong TCP
URG	Urgent (TCP flag)	Cờ dữ liệu khẩn trong TCP
ECE	ECN-Echo (TCP flag)	Cờ báo hiệu tắc nghẽn mạng (Explicit Congestion Notification Echo)
CWR	Congestion Window Reduced (TCP flag)	Cờ báo đã giảm cửa sổ tắc nghẽn
TTL	Time To Live	Thời gian sống của gói tin
IAT	Inter-Arrival Time	Khoảng cách thời gian giữa các gói tin liên tiếp
AVG	Average	Trung bình
ACC	Accuracy	Độ chính xác
AUC	Area Under the Curve	Diện tích dưới đường cong
ROC	Receiver Operating Characteristic	Đường cong đặc trưng hoạt động thu nhận
CPU	Central Processing Unit	Bộ xử lý trung tâm
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên

VRAM	Video RAM	Bộ nhớ đồ họa (của GPU)
OS	Operating System	Hệ điều hành
MicroSSD	Micro Solid State Drive	Thẻ nhớ lưu trữ dùng trên Raspberry Pi
TFLite	TensorFlow Lite	Phiên bản TensorFlow tối ưu cho thiết bị biên/di động
API	Application Programming Interface	Giao diện lập trình ứng dụng

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong bối cảnh bùng nổ công nghệ điện tử hiện nay, số lượng các thiết bị IoT góp mặt trong mọi lĩnh vực đời sống đang tăng trưởng không ngừng, cho đến năm 2025 đạt tới hơn 20 tỷ thiết bị toàn cầu [1]. Bên cạnh sự hữu ích, tiện lợi mang lại, những thiết bị thông minh này luôn tiềm ẩn rủi ro bị tấn công và lấy cắp thông tin bởi việc chỉ tập trung vào hiệu suất nó mang lại mà chưa chú trọng đến các biện pháp bảo mật hiệu quả. Theo SonicWall, số vụ tấn công mã độc IoT toàn cầu năm 2022 đã tăng 87% [2], trong khi Việt Nam liên tục nằm trong nhóm quốc gia có tỉ lệ thiết bị IoT bị nhiễm mã độc cao nhất khu vực Đông Nam Á [3]. Các hệ thống sử dụng IoT thường sử dụng thiết lập mặc định trên các thiết bị IoT và dồn tất cả lượng thông tin tình trạng thiết bị về máy chủ giám sát sẽ dễ bị quá tải, lượng băng thông lớn khó xử lý kịp thời biến chúng trở thành con mồi ngon cho các cuộc tấn công. Vì thế, việc gia tăng bảo mật và thiết lập hệ thống phân tích cảnh báo thời gian thực trên các thiết bị biên là vấn đề cấp thiết hiện nay.

Trước tình hình đó, máy học không phải là hướng đi mới, nhưng hiệu quả trong lĩnh vực an toàn thông tin mạng, bởi khả năng học tập và phân tích, cảnh báo sớm các dấu hiệu tấn công từ các trích xuất đặc trưng mạng, log hệ thống. Tuy nhiên, các mô hình nhận diện, phân loại và cảnh báo nguy cơ bất thường trên các thiết bị điện tử có cấu hình mạnh (laptop, PC, server,…) sẽ có điểm chung là dung lượng lớn và tốn nhiều tài nguyên khi triển khai, sẽ là điểm khuyết lớn khi triển khai chúng trên các thiết bị IoT biên vốn ít tài nguyên.

Xuất phát từ lý do trên, đề tài tập trung vào nghiên cứu để đề xuất các mô hình máy học nhẹ có thể đảm bảo năng suất trên các thiết bị biên, chủ yếu dựa vào logic chưng cất tri thức để tạo các mô hình học sinh gọn nhẹ có thể kế thừa hoàn hảo tri thức từ các mô hình giáo viên. Đồng thời, nghiên cứu sẽ thiết kế thực nghiệm trên thiết bị thật Raspberry Pi 4 để cung cấp cái nhìn thực tế về hiệu suất mô hình trên thực tế.

1.2. Lịch sử giải quyết vấn đề

Như đã đề cập trước đó rằng chủ đề dùng máy học trong giám sát và đảm bảo an ninh mạng là không mới, cũng đã có không ít rất nhiều nghiên cứu đóng góp vào các khảo sát mô hình máy học nhẹ trên các thiết bị biên. Nhưng bên cạnh đó vẫn còn một số khoảng trống để tiếp tục nghiên cứu. Một số nghiên cứu lâu đời là cơ sở để phát triển kỹ thuật làm nhẹ nhưng chỉ có thể tham khảo vì sự phát triển tinh vi các kỹ

thuật tấn công mới qua hàng năm khiến dataset và mô hình cũ không còn giữ được sự bao quát tốt ở hiện tại.

Nghiên cứu của Alzahrani cùng cộng sự [4] đã đề xuất mô hình lai CNN-LSTM đã tối ưu tham số tối đa để phù hợp trên thiết bị nhỏ, đạt độ chính xác trên 92% và triển khai thành công trên Raspberry Pi3 làm nút fog, khảo sát mức độ tiêu tốn năng lượng rành mạch. Nhưng điểm yếu là mô hình vẫn còn khá lớn do không có các kỹ thuật làm nhẹ nâng cao như Knowledge Distillation, Pruning hay Quantization.

Okey [5] đã có một nghiên cứu khác sử dụng kỹ thuật Chưng cất tri thức (Knowledge Distillation) với hàm mất mát lai KL-JS divergence để cải thiện sự hội tụ và tổng quát hóa nhưng cuối cùng dừng lại ở đánh giá trên máy chủ chứ chưa triển khai trên thiết bị biên vật lý để đánh giá số liệu thực tế.

Nghiên cứu của Kuo & Wu, CMC 2026 [6] đã đề xuất khung hai giai đoạn: Stage 1 (Decision Tree) lọc DDoS, Stage 2 (soft-voting ensemble RF+XGBoost+AdaBoost) phân loại phần còn lại, có sử dụng Mutual Information để giảm chiều dữ liệu đã đạt kết quả rất tốt: độ chính xác 99.67% trên CICIoT2023, hơn 94.7% trên CICIoMT2024 và Edge-IioTse. Tuy nhiên nghiên cứu chỉ khảo sát trên các mô hình máy học (machine learning) truyền thống mà chưa thử nghiệm độ hiệu quả khi sử dụng deep learning.

Hai nghiên cứu khác áp dụng kỹ thuật làm nhẹ được tham khảo là Misrak & Melaku, Discover Internet of Things 2025 [7] và Ogab cùng cộng sự [8]. Nghiên cứu của Misrak & Melaku đề xuất mô hình DNN-BiLSTMQ với kỹ thuật lượng tử hóa (quantizied) giảm được kích thước student 250 lần và số tham số 20 lần so với teacher. Bên cạnh là nghiên cứu của Ogab et al. (2025) triển khai multi-teacher với dataset có tiền xử lý giảm mẫu với 2 kỹ thuật khác nhau và so sánh. Nhìn chung, cả 2 nghiên cứu đều cho ra mô hình nhẹ với chất lượng tốt nhưng chưa có số liệu kiểm thử trên thiết bị thật, và kết quả có nhiều chỗ mơ hồ cần kiểm nghiệm lại.

Wisanwanichthan & Thammawichai, Computers 2025 [9] tập trung khảo sát hiệu quả của chưng cất tri thức (KD) trong việc nén các mô hình DNN nhiều lớp (teacher) thành mạng nông hai lớp (student) áp dụng nhất quán trên năm bộ dữ liệu khác nhau, bao gồm NSL-KDD, UNSW-NB15, CIC-IDS2017, IoTID20 và đặc biệt là tập dữ liệu chuyên biệt cho UAV (UAV IDS). Kết quả cho thấy KD giúp giảm số lượng tham số từ 92-95% và tăng tốc độ suy luận 7-11%, đồng thời cải thiện đáng kể hiệu năng phân loại so với mô hình student không qua chưng cất - ví dụ trên IoTID20, F1-score tăng từ 86,20% lên 93,13%; trên UAV IDS đạt F1-score 98,59%, chỉ thấp hơn teacher 1,34 điểm phần trăm. Nghiên cứu cũng có đóng góp đáng chú ý khi là

một trong số ít công trình áp dụng KD đồng nhất trên cả miền IoT truyền thống lẫn miền UAV để chứng minh tính tổng quát liên miền (cross-domain). Tuy nhiên, hạn chế lớn nhất của nghiên cứu là toàn bộ quá trình huấn luyện và đánh giá chỉ được thực hiện trên máy trạm (workstation) có cấu hình mạnh (Intel Core i9, GPU RTX 2070 Super) mà chưa triển khai mô hình student thực tế lên thiết bị biên vật lý nào, nên các số liệu về độ trễ, mức tiêu thụ năng lượng thực tế trên phần cứng giới hạn tài nguyên vẫn còn bỏ ngỏ.

1.3. Mục tiêu đề tài

Trước thực trạng các mô hình học sâu hiệu năng cao khó triển khai trực tiếp lên thiết bị IoT biên do ràng buộc tài nguyên, và trong bối cảnh chưng cất tri thức đang nổi lên như một hướng giải quyết đầy tiềm năng nhưng còn thiếu đánh giá thực nghiệm đa chiều đặc biệt trên dữ liệu tabular và thiết bị vật lý thực. Luận văn đặt mục tiêu khảo sát toàn diện khả năng của các kỹ thuật chưng cất tri thức trong việc tạo ra mô hình phát hiện xâm nhập nhẹ, có thể triển khai hiệu quả trên thiết bị IoT biên. Để đạt được mục tiêu đó, luận văn thực hiện các nội dung cụ thể sau:

Xây dựng và huấn luyện các mô hình giáo viên đa dạng về kiến trúc bao gồm CNN, DNN, SSL-CL-DNN, XGBoost và LSTM trên tập dữ liệu CICIoT2023 với bài toán phân loại đa lớp 8 nhãn nhóm, có tối ưu hóa siêu tham số bằng Optuna, làm cơ sở tri thức để chưng cất.

Triển khai ba kỹ thuật chưng cất tri thức: KD [10], RAID-KL [5] và FitNets [11] lên các mô hình học sinh có kiến trúc thu gọn tương ứng với từng giáo viên, đồng thời thực hiện kỹ thuật chưng cất đa giáo viên trên bốn cặp giáo viên với học sinh là mạng MLP nhỏ.

Đánh giá và so sánh hiệu quả phân loại (accuracy, macro-F1, macro-AUC) cùng hiệu suất suy luận (throughput, latency, mức chiếm dụng CPU và RAM) của các mô hình học sinh trên luồng lưu lượng mạng thực dạng PCAP từ CICIoT2023, trong đó hiệu suất được đo trực tiếp trên thiết bị IoT biên Raspberry Pi 4 B. Từ kết quả thực nghiệm, luận văn rút ra nhận xét và kết luận về mức độ phù hợp của từng kỹ thuật chưng cất tri thức đối với bài toán phát hiện xâm nhập IoT trên dữ liệu dạng bảng đặc trưng, cũng như tiềm năng triển khai thực tế của các mô hình học sinh trên thiết bị biên tài nguyên hạn chế.

1.4. Phạm vi đề tài

Luận văn sử dụng tập dữ liệu CICIoT2023 do Đại học New Brunswick (University of New Brunswick - UNB) công bố, được cắt giảm mẫu và tổ chức lại

thành bài toán phân loại đa lớp với 8 nhãn nhóm gộp từ 34 nhãn tấn công gốc. Dữ liệu đầu vào cho quá trình huấn luyện và đánh giá hiệu quả là dữ liệu bảng đặc trưng trích xuất từ luồng PCAP có nhãn tải từ trang chính thức của UNB.

Luận văn triển khai và khảo sát các phương pháp chưng cất tri thức: KD [10], RAID-KL [5] và FitNets [11], AW-MTKD và CW-MTKD [8]. Thực hiện đánh giá nội bộ giữa các mô hình trong phương pháp và đánh giá hiệu quả giữa các phương pháp với nhau. Qua đó củng cố thêm nhận định về độ hiệu quả của Chưng cất tri thức trong các bài toán mô hình nhẹ với tập dữ liệu cụ thể.

Mở rộng xây dựng benchmark đánh giá độ hiệu quả các phương pháp Chưng cất tri thức khi triển khai mô hình trên thiết bị thật là Raspberry Pi 4 B+. Hiệu quả phân loại được đánh giá trên cả dữ liệu PCAP có nhãn và dữ liệu mạng tự thu thập. Luận văn không xây dựng hệ thống IDS hoàn chỉnh mà giới hạn ở việc triển khai và đo benchmark module phân loại trên thiết bị thực.

1.5. Bảng phân chia công việc

STT	Tên công việc	Thành viên phụ trách
Giai đoạn 1: Nghiên cứu cơ sở lý thuyết và phương pháp		
1.1	Nghiên cứu bối cảnh vấn đề	Cả nhóm
1.2	Khảo sát công trình liên quan đến IDS nhẹ cho an ninh mạng IoT	Cả nhóm
1.3	Xác định khoảng trống nghiên cứu và mục tiêu cụ thể cho đề tài	Cả nhóm
1.4	Tìm hiểu tập dữ liệu CICIoT2023	Cả nhóm
1.5	Tìm hiểu các phương pháp làm nhẹ, so sánh ưu nhược điểm và chốt hướng phù hợp đề tài	Cả nhóm
1.6	Nghiên cứu thiết bị Raspberry Pi 4B, phương pháp kiểm thử	Cả nhóm
Giai đoạn 2: Xử lý dữ liệu		
2.1	Phân tích cấu trúc, nhãn, đặc trưng và đặc tính mất cân bằng của tập dữ liệu	Cả nhóm
2.2	Tiền xử lý tập dữ liệu CICIoT2023	Cả nhóm
Giai đoạn 3: Xây dựng và huấn luyện các mô hình		

3.1	Xây dựng các kiến trúc mô hình giáo viên CNN, DNN, LSTM, SSL-CL-DNN, XGBoost	Trương Đăng Huy
3.2	Tối ưu siêu tham số mô hình với OPTUNA (xác định không gian tìm kiếm, thiết lập hàm mục tiêu, chiến lược tìm kiếm, chạy tối ưu cho từng mô hình)	Trương Đăng Huy
3.3	Xây dựng kiến trúc các mô hình học sinh: MLP2x64, CNN, DNN, SSL-CL-DNN	Đăng Huy (MLP2x64) Đức Thắng (CNN, DNN, SSL-CL-DNN)
3.4	Huấn luyện mô hình học sinh với phương pháp Chưng cất đa giáo viên (AW-MTKD và CW-MTKD)	Đăng Huy (MLP2x64)
3.5	Đánh giá nội bộ giữa các biến thể, bộ tham số trong phương pháp Chưng cất đa giáo viên	Trương Đăng huy
3.6	Huấn luyện các mô hình học sinh với phương pháp: Chưng cất KD truyền thống của Hinton, RAID-KL và Fitness	Đức Thắng (CNN, DNN, SSL-CL-DNN)
3.7	Đánh giá nội bộ giữa các biến thể, bộ tham số trong phương pháp Chưng cất đơn giáo viên	Ngô Đức Thắng
3.8	Nhận xét các kết quả huấn luyện của tất cả các mô hình	Từng thành viên
3.9	So sánh kết quả huấn luyện giữa mô hình học sinh với mô hình giáo viên để đưa ra đánh giá ban đầu về hiệu quả của các phương pháp chưng cất tri thức	Từng thành viên
3.10	So sánh kết quả huấn luyện giữa các mô hình học sinh được huấn luyện bằng các phương pháp chưng cất tri thức khác nhau với nhau	Cả nhóm
3.11	Đánh giá tổng kết quá trình xây dựng các mô hình, kết quả huấn luyện các mô hình giáo viên và học sinh bằng các phương pháp chưng cất tri thức	Cả nhóm
Giai đoạn 4: Thực hiện suy luận bằng các mô hình giáo viên và học sinh		

4.1	Tìm hiểu và xây dựng quy trình đánh giá các mô hình giáo viên và học sinh (độ trễ, thông lượng, CPU/RAM) trên Raspberry Pi 4B để so sánh hiệu năng giữa mô hình giáo viên và học sinh trên thiết bị IoT biên.	Ngô Đức Thắng
4.2	Chuyển đổi mô hình sang định dạng phù hợp triển khai trên thiết bị biên (TFLite)	Từng thành viên
4.3	Tiến hành suy luận và thu thập kết quả suy luận kèm hiệu suất của tất cả các mô hình giáo viên và học sinh trên Raspberry Pi 4 B	Cả nhóm
4.4	Nhận xét các kết quả suy luận và hiệu năng của tất cả các mô hình trên dữ liệu PCAP có nhãn và tự thu thập tại thiết bị IoT biên	Cả nhóm
4.5	So sánh kết quả suy luận và hiệu năng của tất cả các mô hình trên thiết bị Raspberry Pi 4 B với kết quả huấn luyện bằng tập dữ liệu CSV CICIoT2023 trên Kaggle	Cả nhóm
4.6	Đánh giá tổng kết mức độ hiệu quả và hiệu suất của các mô hình học sinh chưng cất tri thức trên thiết bị IoT biên Raspberry Pi 4 B, đồng thời đưa ra kết luận về độ hiệu quả và tiềm năng của các phương pháp chưng cất tri thức trong việc nén mô hình để xây dựng hệ thống phát hiện xâm nhập gọn nhẹ cho mạng IoT biên	Cả nhóm
Giai đoạn 5: Tổng hợp các kết quả và hoàn thiện luận văn		
5.1	Tổng hợp các kết quả huấn luyện, xây dựng các bảng và biểu đồ so sánh	Cả nhóm
5.2	Phân tích, thảo luận về kết quả đạt được, xác định các hạn chết của đề tài, trình bày phương hướng phát triển tương lai của đề tài	Cả nhóm
5.3	Hoàn thiện nội dung luận văn theo phần công việc đã trình bày và theo các nhóm mô hình cũng như phương pháp chưng cất tri thức đã phân công	Từng thành viên
5.4	Kiểm tra, thống nhất cách trình bày và hoàn thiện báo cáo	Cả nhóm

Bảng 1.1. Phân công công việc của đề tài

1.6. Bố cục luận văn

Luận văn được bố cục thành 5 chương chính, kèm theo phần tài liệu tham khảo, với nội dung tóm tắt như sau:

Chương 1 – Tổng quan đề tài: trình bày lý do đặt vấn đề, lịch sử giải quyết vấn đề, mục tiêu, phạm vi đề tài, bảng phân chia công việc, cùng với bố cục tổng thể của luận văn.

Chương 2 – Cơ sở lý thuyết: hệ thống hoá lý thuyết nền tảng về IoT, hệ thống phát hiện xâm nhập (IDS) và nhu cầu triển khai tại thiết bị biên (edge computing); trình bày vai trò của học máy, học sâu trong IDS, tập dữ liệu CICIoT2023, các kiến trúc mô hình sử dụng (XGBoost, DNN, CNN, LSTM, MLP, AutoEncoder, SSL-CL-DNN); các kỹ thuật tiền xử lý dữ liệu, tối ưu siêu tham số với Optuna, và đặc biệt là các kỹ thuật làm nhẹ mô hình gồm chưng cất tri thức truyền thống, Raid-KL (phân kỳ KL-JS) và FitNets (gợi ý tầng ẩn), cùng các thông số đánh giá hiệu quả mô hình.

Chương 3 – Phương pháp thực hiện: trình bày chi tiết quy trình xây dựng và huấn luyện các mô hình học sinh (CNN1D, DNN, SSL-CL-DNN) từ các mô hình giáo viên tương ứng, theo ba phương pháp chưng cất tri thức: truyền thống, Raid-KL (nhiệt độ cố định và thích ứng) và FitNets; đồng thời mô tả quy trình triển khai suy luận trên dữ liệu mạng PCAP để đánh giá kết quả và hiệu suất của các mô hình tại thiết bị IoT biên.

Chương 4 – Kết quả thực nghiệm: trình bày cấu hình môi trường huấn luyện và thiết bị biên Raspberry Pi 4B; kết quả huấn luyện các mô hình học sinh theo từng kỹ thuật chưng cất tri thức, kèm đánh giá, so sánh tổng kết; và kết quả triển khai suy luận trên dữ liệu mạng PCAP tại thiết bị biên, bao gồm cả thực nghiệm suy luận trực tuyến trên dữ liệu mạng tự thu thập.

Chương 5 – Kết luận và hướng phát triển: tóm tắt kết quả đạt được, đối chiếu với các mục tiêu ban đầu, đánh giá các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho đề tài.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Internet of Things (IoT)

2.1.1. Khái niệm

Internet of Things (IoT) là một nền tảng cho phép các cảm biến và thiết bị giao tiếp liền mạch trong một môi trường thông minh, tạo điều kiện thuận lợi cho việc chia sẻ thông tin trên nhiều nền tảng (Marjani cùng cộng sự [12]).

Sự phổ biến gần đây của các công nghệ không dây đã đưa IoT trở thành công nghệ cách mạng tiếp theo bằng cách tận dụng tối đa các cơ hội của môi trường internet mang lại. Các thiết bị IoT hiện hữu ngày càng nhiều xung quanh chúng ta, từ đồng hồ đeo tay thông minh, máy bán hàng tự động, báo động cảm biến, cửa tự động, đến thiết bị gia dụng như tủ lạnh, lò vi sóng, điều hòa, máy giặt thông minh, góp phần giúp công việc và cuộc sống con người trở nên tự động hóa mang lại sự tiện lợi, tiết kiệm thời gian và công sức.

Công nghệ IoT hướng tới 3 quy tắc cốt lõi:

- Định hướng Cảm biến (Sensor): Xác định nguồn gốc của dữ liệu, bao gồm các thiết bị cảm biến và đối tượng được nhúng để thu thập các loại dữ liệu thực tế (dữ liệu địa lý, môi trường, v.v…)
- Định hướng Internet (Internet-oriented): Nhấn mạnh vai trò của kết nối và giao tiếp, xác định khả năng tận dụng tối đa cơ hội do công nghệ Internet mang lại để đảm bảo thiết bị trong môi trường thông minh được giao tiếp liền mạch và được điều khiển từ xa.
- Định hướng Kiến thức (Knowledge): Nhấn mạnh vai trò của việc phân tích và trích xuất các giá trị từ dữ liệu. Dữ liệu được thu thập phải được phân tích và tận dụng để trích xuất được các thông tin có kiến thức giúp đưa ra các dự đoán, quyết định, …

Kiến trúc cơ bản của IoT có 3 lớp chức năng chính (Perception – Network – Application).

- Lớp Cảm biến/Thiết bị (Perception layer): Lớp này chứa tất cả các thiết bị cảm biến và đối tượng (sensor devices and objects) được nhúng vào thế giới thực. Vai trò của chúng là thu thập các loại dữ liệu khác nhau, như dữ liệu địa lý, thiên văn, môi trường và hậu cần, sau đó truyền chúng đi bằng cách sử dụng các thiết bị truyền thông nhúng.

- Lớp Mạng (Network layer): Lớp mạng chịu trách nhiệm kết nối các đối tượng và thiết bị cảm biến thông qua nhiều giải pháp truyền thông khác nhau, bao gồm Bluetooth, Wifi, ZigBee, GSM, và RFID. Cổng IoT (IoT gateway) đóng vai trò quan trọng trong lớp này, cho phép giao tiếp giữa mạng nội bộ của thiết bị và Internet.
- Lớp Ứng dụng/Phân tích Big Data (Application layer): Đây là lớp trên cùng, nơi luượng lớn dữ liệu nhận được từ cảm biến được lưu trữ trên đám mây (cloud). Các ứng dụng phân tích dữ liệu lớn (big data analytics applications) sau đó truy cập và xử lý lượng dữ liệu này. Lớp này thường bao gồm các thành phần như Quản lý API (API Management) và Bảng điều khiển (Dashboard) để hỗ trợ tương tác với công cụ xử lý dữ liệu.

2.1.2. Đặc điểm của IoT

Dữ liệu IoT đóng vai trò quan trọng trong bối cảnh dữ liệu lớn và có các đặc điểm sau:

Có dạng mô hình 3V (Volumn, Variety, Velocity) nghĩa là dữ liệu có dung lượng lớn, đa dạng và tốc độ thu thập dữ liệu rất cao. Ngoài ra số lượng thiết bị và cảm biến dự kiến tăng lên đáng kể.

Không đồng nhất: Do có sự tham gia của nhiều cảm biến và đối tượng khác nhau trong quá trình thu thập, dữ liệu IoT có tính không đồng nhất, nhiễu (noise) và đa dạng.

2.1.3. Thách thức của IoT (trong bối cảnh phân tích dữ liệu)

Quyền riêng tư (Privacy): Các vấn đề riêng tư là thách thức cốt lõi. Các vấn đề này nảy sinh khi hệ thống bị xâm phạm để suy luận hoặc khôi phục thông tin cá nhân bằng các công cụ phân tích dữ liệu lớn. Dữ liệu IoT thường là thông tin nhạy cảm cần được bảo vệ khỏi sự can thiệp bên ngoài.

Khai thác Big Data (Big Data Mining): Thách thức nằm ở việc khám phá dữ liệu và trích xuất thông tin do dung lượng lớn, tốc độ, và sự đa dạng của big IoT data.

Trực quan hóa (Visualization): Khó khăn trong việc thực hiện trực quan hóa do kích thước lớn và chiều cao của dữ liệu lớn.

Tích hợp (Integration): Khó khăn trong việc có được một cái nhìn thống nhất về dữ liệu đến từ nhiều nguồn và định dạng khác nhau (cấu trúc, bán cấu trúc, và phi cấu trúc).

Rủi ro bảo mật: Tính không đồng nhất của IoT đã tạo nên các lỗ hổng mật và số lượng thì ngày càng tăng lên. Hơn nữa, quá trình hoạt động của hệ thống IoT và lưu trữ dữ liệu đôi khi đòi hỏi phải dựa dẫm rất nhiều vào bên thứ ba. Vậy nên, trong bối cảnh dữ liệu lớn, bảo mật và riêng tư là thách thức quan trọng trong việc lưu trữ và xử lý dữ liệu.

2.2. Hệ thống phát hiện xâm nhập và nhu cầu trong IoT

2.2.1. Hệ thống phát hiện xâm nhập (IDS)

Hệ thống phát hiện xâm nhập (Intrusion Detection System -IDS) là một hệ thống bảo mật được thiết kế để theo dõi và phân tích hành vi của hệ thống nhằm phát hiện các hành vi xâm nhập trái phép (Gyamfi cùng cộng sự [13]). Nếu có phát hiện tấn công hệ thống phản ứng và tạo các cảnh báo cho người quản trị hoặc hệ thống quản trị. Thường có hai loại IDS dựa vào chế độ hoạt động:

IDS dựa trên Máy chủ (Host -based IDS – HIDS): HIDS xác định các mối đe dọa và tấn công trên cùng hệ thống hoặc thiết bị mà chúng được cài đặt.

- Nguồn dữ liệu: HIDS sử dụng Nhật ký lệnh gọi hệ thống (System call logs).
- Vị trí: HIDS được đặt cục bộ trên thiết bị/máy chủ được bảo vệ.
- Phạm vi: Chỉ xác định các mối đe dọa và tấn công trên cùng hệ thống đã cài đặt.

IDS dựa trên Mạng (Network-based IDS – NIDS): NIDS là công cụ chính được sử dụng để chống lại xâm nhập mạng và các cuộc tấn công khác.

- Nguồn dữ liệu: NIDS sử dụng Lưu lượng mạng được thu thập (Captured network traffic).
- Vị trí: NIDS được đặt trên các thiết bị IoT cụ thể trên cùng một mạng con.
- Phạm vi: Giám sát lưu luượng Internet qua các thiết bị trong mạng.

Trong hệ thống mạng máy tính hiện nay, NIDS là công cụ chính để bảo vệ hệ thống mạng khỏi xâm nhập mạng và các cuộc tấn công trái phép, cung cấp hệ thống bảo mật với nhiệm vụ giám sát lưu lượng mạng nhằm xác định các rủi ro, phân tích các mối đe dọa và phản ứng, gửi cảnh báo cho bộ phận quản trị mạng khi phát hiện có tấn công mạng.

2.2.2. Nhu cầu về hệ thống phát hiện xâm nhập mạng tại biên (edge computing) trong IoT

Trong kiến trúc IoT truyền thống, toàn bộ dữ liệu lưu lượng mạng được gửi về máy chủ trung tâm hoặc đám mây để phân tích. Cách làm này bộc lộ hai hạn chế lớn: (1) lưu lượng khổng lồ từ hàng nghìn thiết bị dễ gây quá tải máy chủ, làm chậm quá trình phát hiện, khiến hệ thống không phản ứng kịp trước các dấu hiệu tấn công; (2) phụ thuộc hoàn toàn vào kết nối Internet và truyền dữ liệu ra ngoài, làm tăng rủi ro mất an toàn thông tin và gián đoạn dịch vụ.

Điện toán biên (edge computing) khắc phục những điểm yếu trên bằng cách đưa năng lực xử lý xuống sát nguồn dữ liệu. Triển khai hệ thống phát hiện xâm nhập ngay trên thiết bị biên cho phép giám sát theo thời gian thực, phát hiện tức thời hành vi bất thường và đưa ra phản ứng kịp thời ngay tại chỗ, không cần gửi dữ liệu đi xa. Tuy nhiên, chính các thiết bị biên – điển hình như Raspberry Pi 4 – lại có tài nguyên tính toán, bộ nhớ và năng lượng rất hạn chế. Những mô hình IDS học sâu cồng kềnh không thể triển khai trực tiếp trên nền tảng này.

Vì vậy, cần phát triển các mô hình phát hiện xâm nhập nhẹ (NIDS lightweight), được thiết kế để đảm bảo hiệu suất phát hiện cao, tiêu thụ ít tài nguyên, phản hồi nhanh và cho phép ngăn chặn kịp thời các cuộc tấn công ngay tại biên. Đây chính là động lực để thực hiện nghiên cứu huấn luyện mô hình nhẹ trên Raspberry Pi 4 với bộ dữ liệu IoT thực tế CICIoT2023.

2.3. Học máy và vai trò trong hệ thống phát hiện xâm nhập mạng

2.3.1. Học máy (Machine Learning) và học sâu (Deep Learning)

Học máy (Machine Learning - ML) là một lĩnh vực của trí tuệ nhân tạo (Artificial Intelligence - AI) [14], bản chất là một tập hợp các kỹ thuật được sử dụng để phát triển các giải pháp phân tích bảo mật (security analytics solutions). ML cho phép máy tính học hỏi từ dữ liệu và tự động đưa ra các dự đoán hoặc quyết định.

Học sâu (Deep Learning - DL) là một tập con của Học máy, sử dụng các mô hình mạng thần kinh nhân tạo (neural network models) với nhiều lớp để xử lý dữ liệu phức tạp hơn [15].

Học máy và học sâu nhìn chung đã đạt được những tiến bộ lớn và hiện đang được ứng dụng rộng rãi trong nhiều lĩnh vực, từ truyền thông không dây đến thị giác máy tính và hệ thống chăm sóc sức khỏe, đặc biệt quan trọng trong an ninh mạng.

2.3.2. Vai trò của học máy và học sâu trong hệ thống phát hiện xâm nhập mạng

Trong môi trường IoT đầy thách thức, nơi các thiết bị thường bị hạn chế tài nguyên và mạng lưới ngày càng phức tạp, các Hệ thống Phát hiện Xâm nhập (IDS) dựa trên Học máy và Học sâu đóng vai trò cốt lõi có nhiệm vụ phân tích bảo mật cốt lõi, cho phép NIDS học hỏi từ dữ liệu để tự động xác định sự khác biệt quan trọng trong các mẫu bất thường. Nhờ tiềm năng tổng quát hóa mạnh mẽ, một số mô hình ML/DL tiên tiến giúp NIDS phát hiện các cuộc tấn công chưa biết, qua đó cải thiện khả năng phân loại lưu lượng mạng là độc hại hay lành tính.

2.4. Tập dữ liệu CICIoT2023 [14]

Bộ dữ liệu CICIoT2023 được tạo ra bởi Viện an ninh mạng Canada (CIC) bằng cách xây dựng một cấu trúc mạng mở rộng với 105 thiết bị IoT thật, trong đó các thiết bị này có thể đóng vai trò là kẻ tấn công hoặc nạn nhân, mô phỏng một cách chân thực các kịch bản trong thế giới thực.

STT	Tên đặc trưng	Ý nghĩa
1	Header_Length	Độ dài header
2	Protocol Type	Loại giao thức
3	Time_To_Live	Thời gian tồn tại
4	Rate	Tỷ lệ truyền gói tin trong một cửa sổ gói
5	Fin_flag_number	Giá trị cờ FIN
6	Syn_flag_number	Giá trị cờ SYN
7	Rst_flag_number	Giá trị cờ RST
8	Psh_flag_number	Giá trị cờ PSH
9	Ack_flag_number	Giá trị cờ ACK
10	Ece_flag_number	Giá trị cờ ECE
11	Cwr_flag_number	Giá trị cờ CWR
12	Ack_count	Số lượng gói tin có cờ ACK trong cửa sổ gói
13	Syn_count	Số lượng gói tin có cờ SYN trong cửa sổ gói
14	Fin_count	Số lượng gói tin có cờ FIN trong cửa sổ gói
15	Rst_count	Số lượng gói tin có cờ RST trong cửa sổ gói
16	HTTP	Giao thức được sử dụng
17	HTTPS	Giao thức được sử dụng
18	DNS	Giao thức được sử dụng
19	Telnet	Giao thức được sử dụng
20	SMTP	Giao thức được sử dụng

21	SSH	Giao thức được sử dụng
22	IRC	Giao thức được sử dụng
23	TCP	Giao thức được sử dụng
24	UDP	Giao thức được sử dụng
25	DHCP	Giao thức được sử dụng
26	ARP	Giao thức được sử dụng
27	ICMP	Giao thức được sử dụng
28	IGMP	Giao thức được sử dụng
29	IPv	Giao thức IP
30	LLC	Giao thức tầng liên kết dữ liệu
31	Tot sum	Tổng chiều dài gói tin trong cửa sổ gói
32	Min	Chiều dài gói tin nhỏ nhất trong cửa sổ gói
33	Max	Chiều dài gói tin lớn nhất trong cửa sổ gói
34	AVG	Trung bình chiều dài gói tin trong cửa sổ gói
35	Std	Độ lệch chuẩn chiều dài gói tin
36	Tot size	Tổng kích thước gói tin
37	IAT	Khoảng thời gian giữa 2 gói tin liên tiếp
38	Number	Số lượng gói tin trong một luồng
39	Variance	Độ biến thiên giữa độ dài incoming và outgoing

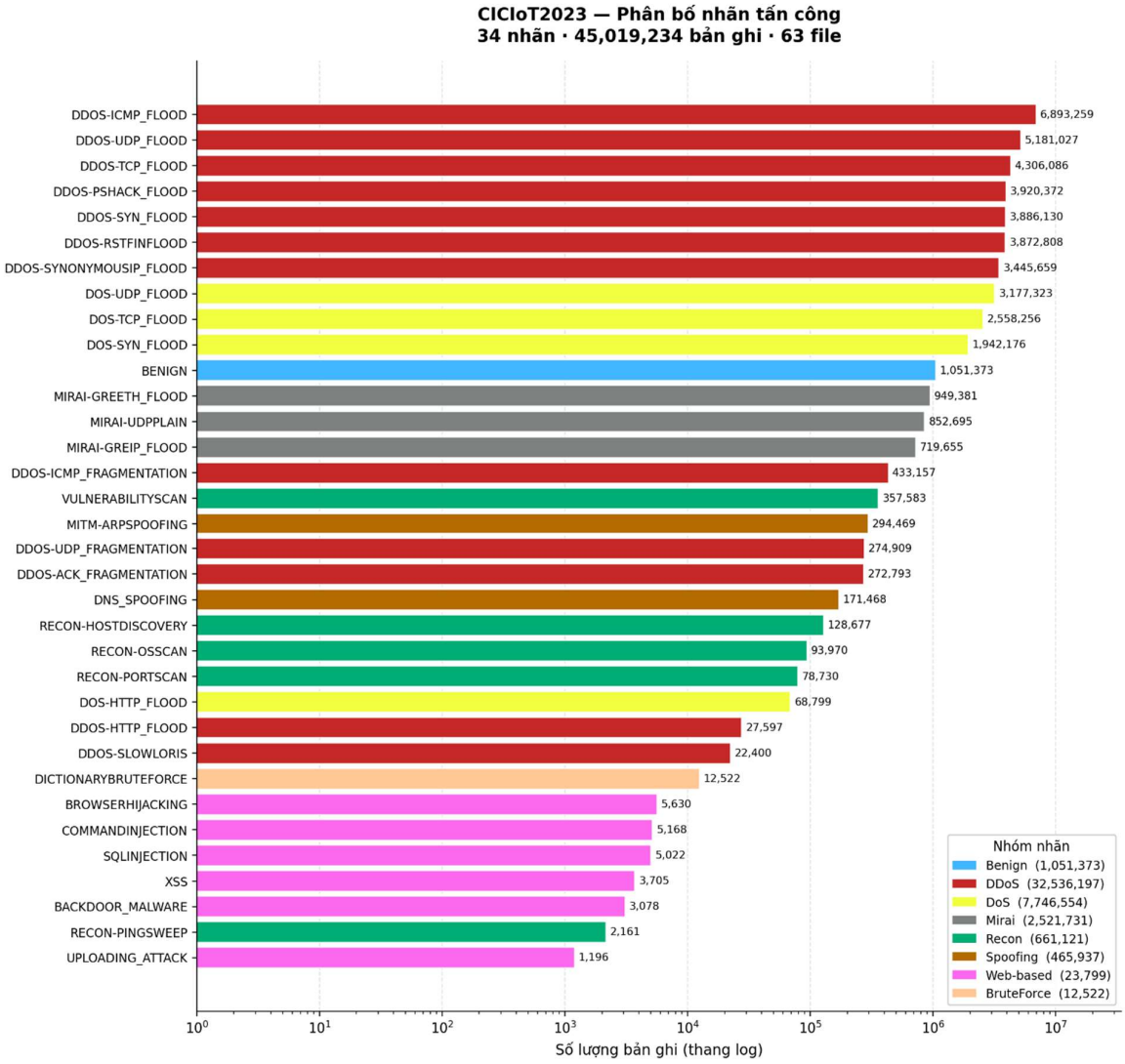
Bảng 2.1. Danh sách đặc trưng của tập dữ liệu CICIoT2023 tải từ trang chính của UNB CIC

Tập dữ liệu gốc với 39 đặc trưng được lấy trực tiếp từ website của UNB sẽ là phiên bản được sử dụng trong nghiên cứu này. Thực nghiệm và ghi nhận 33 loại tấn công: Ghi lại dữ liệu từ những cuộc tấn công khác nhau, được phân loại thành 7 nhóm chính (DDoS, DoS, Recon, Web-based, Brute-force, Spoofing, và Mirai).

Loại	**Số dòng mỗi loại**
DDoS	32,536,197
DoS	7,736,554
Mirai	2,521,731
Benign	1,051,373
Recon	661,121
Spoofing	465,937

Web	23,799
Brute-force	12,522
Tổng cộng	**45,019,234**

Bảng 2.2. Số lượng mẫu mỗi nhãn tấn công (8 nhãn được nhóm) trong tập dữ liệu



Hình 2.1. Số lượng mẫu mỗi nhãn trên 34 loại lưu lượng trong tập dữ liệu

Tấn công DDoS (Distributed Denial-of-Service): Đây là các cuộc tấn công từ chối dịch vụ phân tán, sử dụng nhiều nguồn tấn công đồng thời để làm cạn kiệt tài nguyên của thiết bị hoặc mạng mục tiêu, khiến chúng không thể phục vụ các yêu cầu hợp pháp.

Tấn công DoS (Denial-of-Service): Tương tự như DDoS, tấn công DoS nhằm mục đích làm gián đoạn dịch vụ, nhưng thường được thực hiện từ một nguồn duy nhất.

- ACK/UDP/ICMP Fragmentation: Gửi một lượng lớn các gói tin bị phân mảnh để làm quá tải tài nguyên của mục tiêu khi chúng cố gắng tái hợp các gói tin này.
- SlowLoris: Duy trì nhiều kết nối HTTP mở với máy chủ web mục tiêu bằng cách gửi các yêu cầu không hoàn chỉnh, làm cạn kiệt nhóm kết nối của máy chủ.
- ICMP/HTTP/UDP/TCP/SYN Flood: Gửi một lượng lớn các gói tin (ICMP, HTTP, UDP, TCP, hoặc SYN) từ nhiều nguồn đến mục tiêu để làm bão hòa băng thông hoặc tài nguyên xử lý của nó.
- RST-FIN Flood: Gửi liên tục các gói tin RST-FIN để làm suy giảm khả năng kết nối mạng của mục tiêu.
- PSH-ACK Flood: Làm ngập lụt máy chủ bằng các yêu cầu PUSH và ACK để làm suy giảm hoạt động của nó.
- Synonymous IP Flood: Gửi một số lượng lớn các gói tin TCP-SYN giả mạo với địa chỉ IP nguồn và đích đều là của mục tiêu, khiến máy chủ tự tấn công chính mình.
- TCP/HTTP/SYN/UDP Flood: Gửi một lượng lớn các gói tin từ một máy tấn công duy nhất để làm quá tải tài nguyên của máy chủ mục tiêu.

Tấn công Trinh sát (Reconnaissance): Mục tiêu của các cuộc tấn công trinh sát là thu thập thông tin về mạng và các thiết bị mục tiêu để chuẩn bị cho các cuộc tấn công trong tương lai.

- Ping Sweep: Gửi các yêu cầu ICMP Echo đến một dải địa chỉ IP để xác định các máy chủ đang hoạt động trên mạng.
- OS Scan: Phân tích các phản hồi từ gói tin mạng để xác định loại và phiên bản hệ điều hành đang chạy trên máy chủ mục tiêu.
- Vulnerability Scan: Sử dụng các công cụ tự động để quét và xác định các lỗ hổng bảo mật tiềm ẩn trên một hệ thống hoặc mạng.
- Port Scan: Gửi các gói tin đến nhiều cổng khác nhau trên một máy chủ mục tiêu để xác định các cổng nào đang mở và các dịch vụ nào đang chạy.

- Host Discovery: Sử dụng nhiều kỹ thuật khác nhau để xác định địa chỉ IP của các thiết bị đang hoạt động và kết nối vào mạng.

Tấn công Dựa trên Web (Web-Based): Các cuộc tấn công này nhắm vào các dịch vụ web chạy trên các thiết bị IoT, khai thác các lỗ hổng trong ứng dụng web.

- Sql Injection: Chèn mã SQL độc hại vào các trường nhập liệu của ứng dụng web để truy cập trái phép vào cơ sở dữ liệu.
- Command Injection: Chèn các lệnh hệ thống độc hại vào trường nhập liệu để thực thi mã tùy ý trên máy chủ mục tiêu.
- Backdoor Malware: Cài đặt phần mềm độc hại tạo ra một "cửa hậu" để kẻ tấn công có thể truy cập trái phép vào hệ thống sau này.
- Uploading Attack: Khai thác các lỗ hổng trong chức năng tải tệp lên của ứng dụng web để tải lên các tệp độc hại.
- XSS (Cross-Site Scripting): Chèn mã kịch bản độc hại vào một trang web, mã này sẽ được thực thi bởi trình duyệt của người dùng khác khi họ truy cập trang.
- Browser Hijacking: Sửa đổi cài đặt của trình duyệt web (ví dụ: trang chủ, công cụ tìm kiếm mặc định) để chuyển hướng người dùng đến các trang web độc hại hoặc hiển thị quảng cáo không mong muốn.

Tấn công Giả mạo (Spoofing): Các cuộc tấn công này cho phép kẻ tấn công mạo danh một hệ thống hợp pháp để chặn hoặc thao túng lưu lượng mạng.

- Arp Spoofing: Gửi các thông điệp ARP giả mạo để liên kết địa chỉ MAC của kẻ tấn công với địa chỉ IP của một thiết bị hợp pháp, cho phép chặn hoặc sửa đổi lưu lượng.
- DNS Spoofing: Thay đổi các mục DNS trong bộ đệm của máy chủ DNS để chuyển hướng người dùng đến các trang web độc hại.

Tấn công Brute-Force: Tấn công này bao gồm việc thử lặp đi lặp lại nhiều mật khẩu hoặc cụm mật khẩu để giành quyền truy cập vào một hệ thống.

- Dictionary Brute Force: Thử các từ trong một danh sách được xác định trước (từ điển) để đoán mật khẩu chính xác.

Tấn công Mirai: Mirai là một loại phần mềm độc hại nổi tiếng nhắm vào các thiết bị IoT, biến chúng thành một mạng botnet để thực hiện các cuộc tấn công DDoS quy mô lớn.

- GREIP Flood: Làm ngập lụt hệ thống mục tiêu bằng các gói tin được đóng gói trong Giao thức Đóng gói Định tuyến Chung (GRE), với IP và cổng ngẫu nhiên.
- Greeth Flood: Cuộc tấn công này có quy trình tương tự GREIP. Tuy nhiên, điểm khác biệt chính nằm ở phương pháp đóng gói gói tin, dựa trên tiêu đề ethernet.
- UDPPlain: Làm ngập lụt mục tiêu bằng các gói tin UDP, trong đó phần tải trọng (payload) được thay đổi cho mỗi gói tin.

Ngoài thu thập các loại tấn công thì không thể thiếu dữ liệu lành tính (Benign) khi thiết bị hoạt động ở trạng thái bình thường và có tương tác với con người (ví dụ như dữ liệu cảm biến, yêu cầu echo dot, yêu cầu truy xuất video từ camera).

2.5. Các mô hình huấn luyện được sử dụng

2.5.1. XGBoost

Là mô hình máy học dạng cây duy nhất được đề xuất trong phương pháp chưng cất Đa giáo viên (multi-teacher)[8], XGBoost đã chứng minh độ hiệu quả và thời gian nhanh chóng trên tập dữ liệu dạng bảng (tabular) này.

XGBoost (viết tắt của eXtreme Gradient Boosting) là một thư viện phần mềm mã nguồn mở, được tối ưu hóa cao, cung cấp một framework cho phương pháp Gradient Boosting. Đây là một thuật toán học máy thuộc họ ensemble learning (học tập hợp), dùng để giải quyết các bài toán phân loại (classification), hồi quy (regression) và xếp hạng (ranking).

XGBoost có một hệ thống tham số phong phú, được chia thành 3 loại chính:

- **Tham số tổng quát** để quyết định loại booster sử dụng: booster, nthread, silent.
- **Tham số booster** là nhóm tham số quan trong nhất, ảnh hưởng trực tiếp đến hiệu suất của mô hình: eta, max_depth, gamma, min_child_weight, subsample, colsample_bytree.
- **Tham số tác vụ** quyết định kịch bản đọc: objective, eval_metric.

XGBoost hoạt động dựa trên nguyên lý boosting, xây dựng các mô hình học máy theo dạng chuỗi liên tiếp, trong đó mỗi mô hình mới sẽ cố gắng sửa sai cho mô hình trước đó. Cụ thể, XGBoost sử dụng các cây quyết định (decision trees) làm mô hình cơ sở và xây dựng chúng theo kiểu Gradient Boosting.

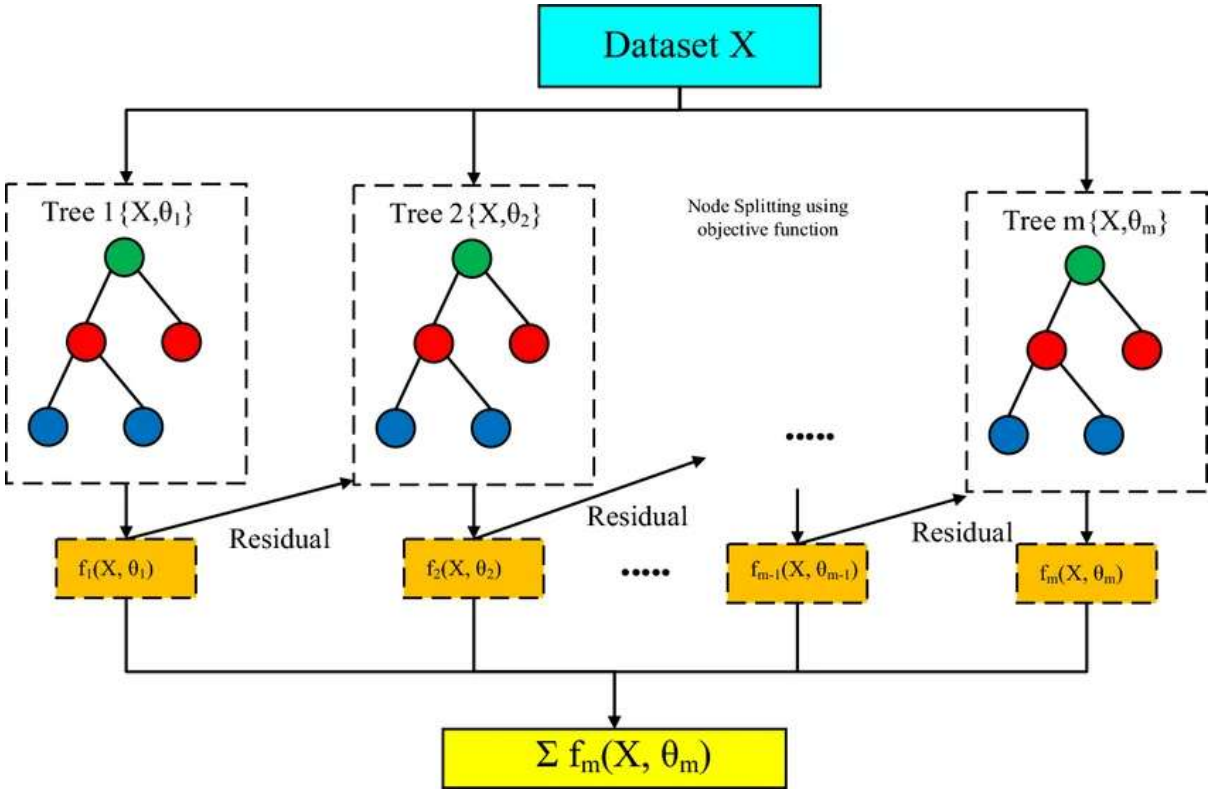
Gradient Boosting là phương pháp xây dựng các mô hình máy học máy bằng cách giảm thiểu hàm mất mát một cách tuần tự: Thuật toán xây dựng cây quyết định mới để dự đoán phần dư (residual) – sai số - từ mô hình trước đó, tiếp theo cây mới sẽ được huấn luyện để giảm thiểu lỗi của mô hình hiện tại, kết quả dự đoán cuối cùng là tổng hợp của tất cả cây quyết định.

Mỗi cây trong XGBoost là một **CART** (Classification and Regression Tree), trong đó mỗi lá có một giá trị điểm số thực thay vì chỉ là quyết định nhị phân như cây quyết định truyền thống. Mô hình tổng quát của XGBoost có dạng:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F}$$

Trong đó:

- K là số lượng cây.
- f_k là hàm dự đoán của cây thứ K.
- x_i là vector đặc trưng của mẫu dữ liệu thứ i.
- $\mathcal{F}$ là không gian các cây quyết định (CART).



Hình 2.2. Minh họa quy trình giải thuật XGBoost

Hàm mục tiêu (Objective Function):

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i\right) + \sum_{k=1}^{K} \Omega\left(f_k\right)$$

Trong đó:

- $l(y_i, \hat{y}_i)$ là hàm tổn thất đo sai số giữa giá trị thực tế và dự đoán.
- $\Omega(f_k)$ là hàm điều chuẩn kiểm soát độ phức tạp của cây, giúp tránh overfitting.

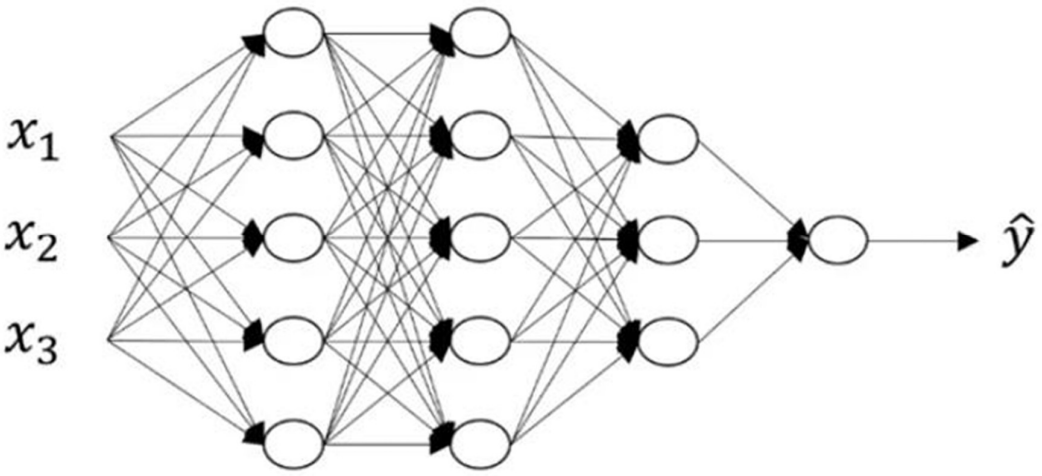
2.5.2. DNN (Deep Neural Network)

Mạng nơ-ron sâu (Deep Neural Network) được lấy cảm hứng từ cấu trúc và chức năng của bộ não con người. DNN là một loại mạng nơ-ron nhân tạo được mô hình hóa như một mạng nơ-ron truyền thẳng đa lớp (multilayer perception – MLP) với nhiều tầng ẩn. Khái niệm sâu "deep" ám chỉ việc sử dụng nhiều tầng ẩn hơn so với các mạng nơ-ron truyền thống, cho phép mô hình học các biểu diễn dữ liệu ở nhiều mức độ trừu tượng khác nhau.

DNN có khả năng tự động học các đặc trưng từ dữ liệu thô mà không cần phải thiết kế thủ công các bộ trích xuất đặc trưng. Các tầng đầu của mạng học các đặc trưng đơn giản, trong khi các tầng sâu hơn kết hợp chúng để nhận diện các cấu trúc phức tạp và từu tượng hơn.

Một mô hình DNN điển hình có cấu trúc phân tầng gồm ba loại tầng chính:

- **Tầng Đầu vào (Input Layer)**: Tầng này nhận dữ liệu thô (ví dụ: vector đặc trưng của một flow mạng) và chuyển tiếp đến tầng ẩn đầu tiên. Số lượng nơ-ron ở tầng này tương ứng với số lượng đặc trưng đầu vào.
- **Tầng Ẩn (Hidden Layers)**: Đây là các tầng ở giữa, nơi diễn ra phần lớn quá trình tính toán và học tập. Một DNN có thể có từ một đến nhiều tầng ẩn. Mỗi tầng ẩn bao gồm một tập hợp các nơ-ron (Neurals) hay còn gọi là các đơn vị (units). Số lượng tầng ẩn và số lượng nơ-ron trong mỗi tầng là các siêu tham số (hyperparameters) quan trọng ảnh hưởng đến khả năng học của mô hình.
- **Tầng Đầu ra (Output Layer)**: Tầng cuối cùng của mạng, đưa ra kết quả dự đoán cuối cùng. Đối với bài toán phân loại, tầng này thường sử dụng hàm kích hoạt Softmax để chuyển đổi đầu ra thành xác suất cho từng lớp.



Hình 2.3. Kiến trúc cơ bản của mô hình DNN

Nguyên lý hoạt động của DNN dựa trên cơ chế truyền thẳng (feedforward) và lan truyền ngược (backpropagation).

Trong quá trình lan truyền thẳng, dữ liệu đầu vào được đưa qua từng tầng của mạng theo thứ tự từ tầng đầu vào đến tầng đầu ra. Tại mỗi nơ-ron, dữ liệu được xử lý qua hai bước:

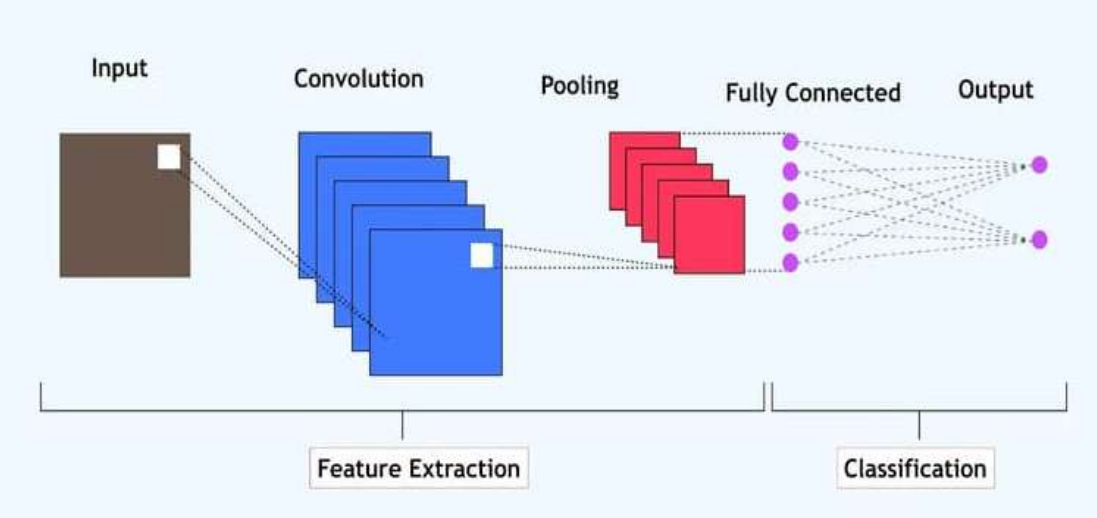
- **Tổ hợp tuyến tính (Linear Combination)**: Mỗi nơ-ron nhận các giá trị đầu vào từ các nơ-ron của tầng trước, nhân chúng với các trọng số (weights) tương ứng, cộng thêm một hệ số điều chỉnh (bias), và tính tổng.
- **Kích hoạt phi tuyến (Non-linear Activation)**: Kết quả của phép tổ hợp tuyến tính được đưa qua một hàm kích hoạt phi tuyến (activation function) để tạo ra đầu ra của nơ-ron. Hàm kích hoạt phổ biến bao gồm ReLU (Rectified Linear Unit), Sigmoid, và Tanh. Hàm này giúp mạng học được các mối quan hệ phi tuyến phức tạp trong dữ liệu.

Mục tiêu của quá trình huấn luyện là tìm ra bộ trọng số và bias tối ưu để mô hình có thể đưa ra dự đoán chính xác. Quá trình này được thực hiện thông qua thuật toán **lan truyền ngược (backpropagation)** kết hợp với các kỹ thuật tối ưu hóa:

- **Tính toán sai số (Loss Calculation):** Sau khi lan truyền thẳng, đầu ra dự đoán của mô hình được so sánh với nhãn thực tế để tính toán sai số (loss) bằng một hàm mất mát (loss function), ví dụ: Cross-Entropy cho bài toán phân loại.
- **Truyền ngược sai số (Backward Pass):** Sai số được lan truyền ngược từ tầng đầu ra về các tầng trước đó. Thuật toán lan truyền ngược sử dụng quy tắc chuỗi (chain rule) trong giải tích để tính toán đạo hàm (gradient) của hàm mất mát đối với từng trọng số và bias trong mạng.
- **Cập nhật trọng số (Weight Update):** Các trọng số và bias được cập nhật theo hướng ngược với gradient để giảm thiểu sai số. Quá trình này được thực hiện bởi một thuật toán tối ưu hóa, phổ biến nhất là Stochastic Gradient Descent (SGD) hoặc các biến thể của nó như Adam.

2.5.3. CNN (Convolutional Neural network)

CNN là mạng nơ-ron tích chập (Convolutional Neural network) một trong những mô hình cơ sở của Deep Learning trích xuất hiệu quả các đặc trưng phân cấp từ dữ liệu cấu trúc lưới, đặc biệt là hình ảnh.



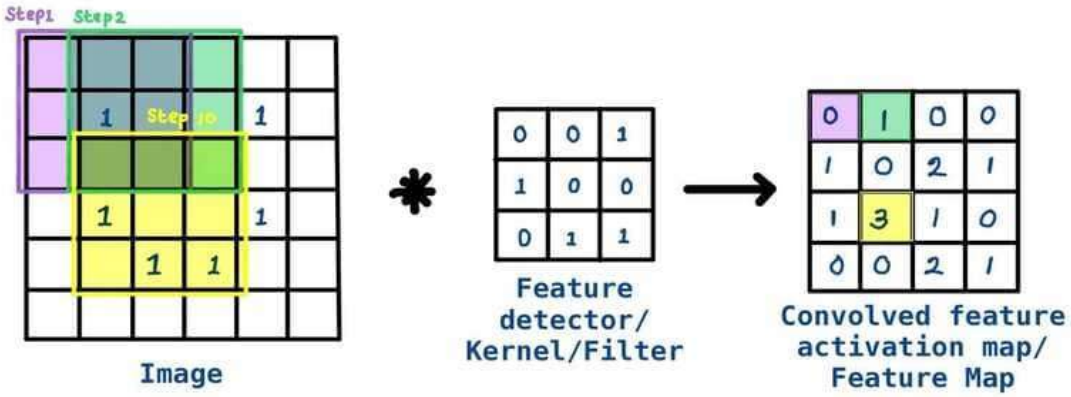
Hình 2.4. Kiến trúc cơ bản của mô hình CNN

Một kiến trúc CNN cơ bản có 3 lớp, được thiết kế đặc trưng cho các tác vụ phân loại hình ảnh và nhận dạng đối tượng trong không gian 3 chiều.

Lớp tích chập (Convolutional layer): là lớp đóng vai trò mấu chốt của CNN, thực hiện phép tích chập giữa ma trận đầu vào với một bộ lọc (filter/kernel) để tạo ra các đặc trưng (feature map). Giả sử dữ liệu đầu vào là một ảnh màu, nó sẽ được biểu diễn dưới dạng môt ma trận trong không gian 3 chiều (3D), với ba chiều tương ứng là chiều cao, chiều rộng và độ sâu (tương ứng với RGB trong ảnh). Các filter/kernel là ma trận trọng số 2 chiều (2D) sẽ di chuyển qua các reception fields (vùng cảm nhận) và tạo các tích vô hướng với giá trị pixel vùng quét để đưa vào mảng đầu ra. Sau đó filter dịch chuyển "trượt" theo một khoảng cách xác định (stride), quá trình được lặp lại cho đến khi filter đã quét qua toàn bộ ảnh.

Feature map (hay activation map hoặc convolved feature) chính là kết quả của chuỗi các phép tính tích vô hướng giữa pixel đầu vào và filter. Có ba siêu tham số quan trọng ảnh hưởng đến kích thước đầu ra và cần được thiết lập trước khi huấn luyện:

- Số lượng filter: quyết định độ sâu đầu ra.
- Stride: bước di chuyển của filter trên ma trận đầu vào, xác định khoảng cách giữa các lần quét. Stride lớn sẽ tạo đầu ra nhỏ hơn.
- Zero-padding: được sử dụng khi kích thước của filter không khớp với kích thước của hình ảnh đầu vào.



Hình 2.5. Cách dữ liệu đầu vào CNN được biểu diễn qua lớp tích chập

Pooling layer (hay downsampling): có nhiệm vụ giảm chiều dữ liệu và giảm số lượng tham số đầu vào. Tương tự như Lớp tích chập, lớp Pooling cũng sử dụng filter để quét qua toàn bộ đầu vào, nhưng filter lớp này không có trọng số, thay vào đó nó áp dụng một phép toán tổng hợp lên các giá trị trong vùng cảm nhận để ra giá trị cho mảng đầu ra. Lớp này làm giảm thông tin dữ liệu nhưng cũng giảm độ phức tạp tính toán, cải thiện hiệu suất và hạn chế hiện tượng overfitting.

- Max Pooling: Khi filter di chuyển qua đầu vào, nó chọn pixel có giá trị lớn nhất trong vùng quét và đưa giá trị đó vào mảng đầu ra. Đây là phương pháp phổ biến nhất trong các mạng nơ-ron.
- Average Pooling: Khi filter di chuyển qua đầu vào, nó tính giá trị trung bình của các pixel trong vùng quét và đưa giá trị trung bình đó vào mảng đầu ra.

Lớp Kết nối đầy đủ (fully connected): đóng vai trò là "bộ phân loại" (classifier). Các đặc trưng trừu tượng từ các lớp trước được "làm phẳng" flattten và đưa qua một hoặc nhiều lớp kết nối đầy đủ để đưa ra dự đoán cuối cùng. Trong khi Convolutional Layers và Pooling Layers thường sử dụng hàm kích hoạt ReLU, thì Fully connected Layers chủ yếu sử dụng hàm kích hoạt Softmax để phân loại.

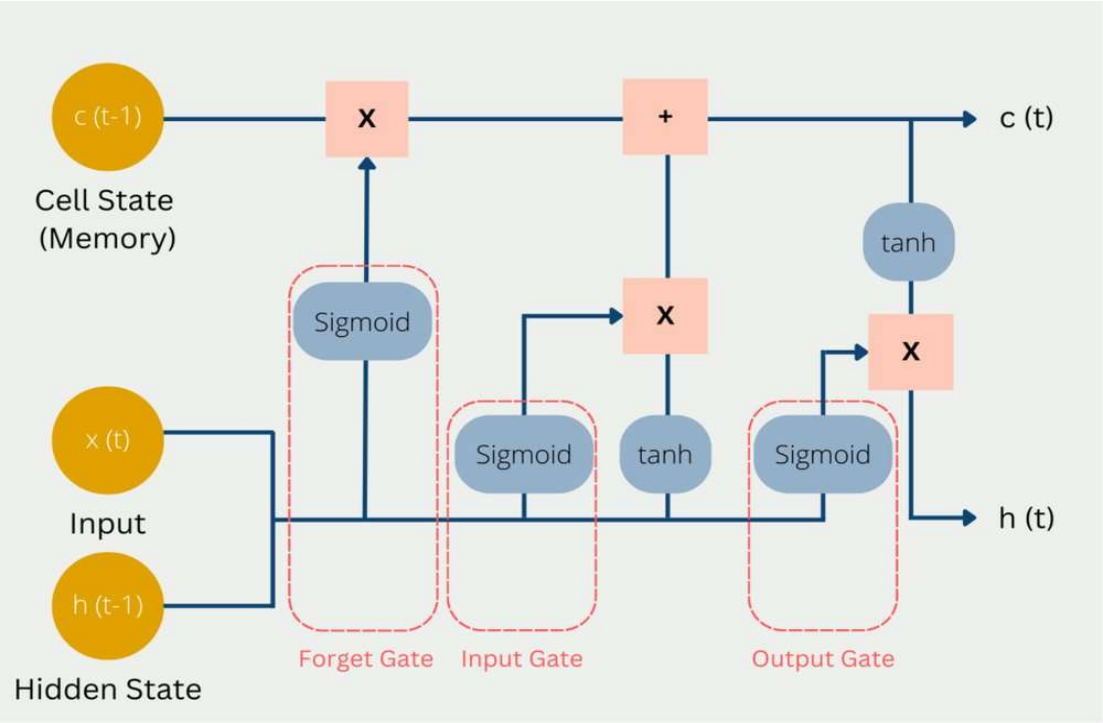
Trong nghiên cứu này chúng tôi sử dụng CNN-1D, là một biến thể của CNN với lớp tích chập 1 chiều (1D) để phù hợp với dữ liệu tabular dạng chuỗi (sequence data) – nơi trật tự của các điểm dữ liệu có ý nghĩa. Về cơ bản, kiến trúc của CNN-1D tương t như CNN-2D nhưng các phép toán được thực hiện trên một chiều không gian.

- **Lớp tích chập 1D (Conv1D)**: Sử dụng các bộ lọc một chiều trượt dọc theo chiều dài của chuỗi dữ liệu để trích xuất các đặc trưng cục bộ.

- **Lớp gộp 1D (MaxPooling1D)**: Tương tự, thực hiện giảm kích thước trên chiều không gian của chuỗi.

2.5.4. LSTM

Mô hình LSTM (Long Short-Term Memory) là biến thể đặc biệt của mạng nơ-ron hồi quy (RNN), được thiết kế để xử lý dữ liệu chuỗi (văn bản, chứng khoáng, âm thanh) và học các phụ thuộc xa của thời gian. Trong khi RNN chỉ sử dụng một trạng thái ẩn đơn giản để truyền thông tin qua các bước thời gian, thì LSTM bổ sung một bộ nhớ tế bào (memory cell) cùng với ba cơ chế cổng kiểm soát: *cổng quên, cổng đầu vào và cổng đầu ra*.



Hình 2.6. Kiến trúc cơ bản mô hình LSTM

Cổng quên quyết định thông tin nào từ trạng thái trước đó cần được giữ lại hoặc loại bỏ. Nó sử dụng một hàm kích hoạt sigmoid để tạo ra giá trị trong khoảng từ 0 đến 1, trong đó: 0 (loại bỏ hoàn toàn thông tin cũ) và 1 (giữ nguyên thông tin đó). Nhờ cơ chế này, LSTM có thể lọc bỏ những thông tin không cần thiết, giúp tối ưu hóa khả năng ghi nhớ dữ liệu quan trọng.

Cổng đầu vào xác định thông tin nào từ đầu vào mới cần thêm vào bộ nhớ. Cơ chế này gồm hai bước chính: Hàm sigmoid (quyết định những phần nào của thông tin sẽ được cập nhật) và hàm tanh (tạo một vector ứng viên chứa dữ liệu mới), sau đó kết

hợp với đầu ra của hàm sigmoid để điều chỉnh bộ nhớ. Nhờ đó, LSTM có thể tích hợp thông tin mới một cách có chọn lọc, giúp mô hình thích ứng với dữ liệu theo thời gian.

Cổng đầu ra kiểm soát thông tin nào sẽ được truyền sang bước tiếp theo. Cơ chế hoạt động gồm: Hàm sigmoid (lọc những thông tin quan trọng từ bộ nhớ) và hàm tanh (điều chỉnh trạng thái bộ nhớ trước khi đưa ra kết quả). Cổng đầu ra giúp LSTM tập trung vào các đặc trưng quan trọng và hạn chế nhiễu, làm tăng độ chính xác của mô hình trong các bài toán xử lý chuỗi.

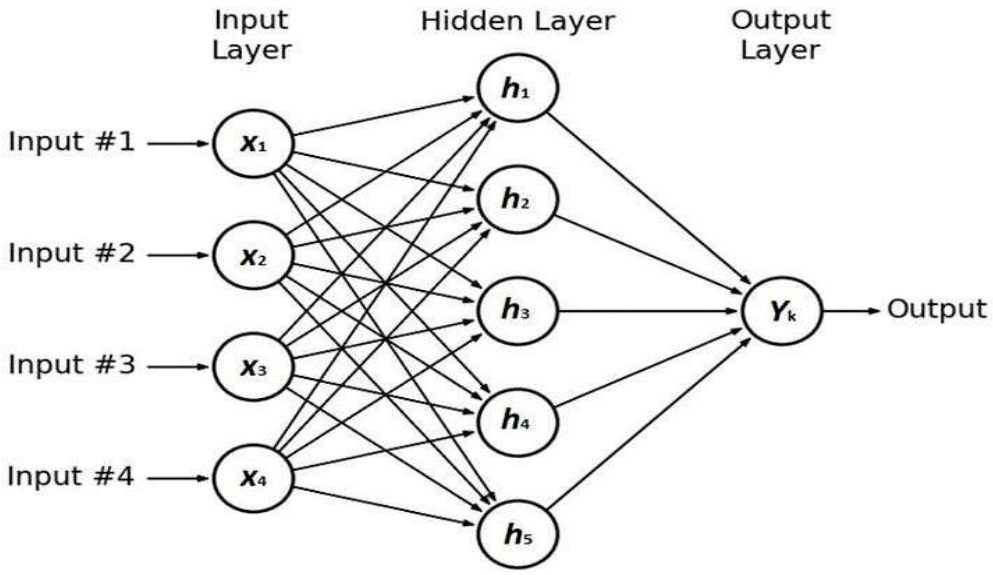
Kiến trúc được tham khảo từ nghiên cứu [11] với kiến trúc 2 lớp, ở mỗi lớp sẽ có BatchNorm để ổn định phân phối dữ liệu và Dropout để tránh quá khớp.

2.5.5. MLP

MLP (Multilayer Perceptron) là một loại mạng nơ-ron nhân tạo (Artificial Neural Network) gồm nhiều lớp Neural (hoặc "node") được kết nối với nhau theo cấu trúc tầng. Mỗi Neural trong mạng có nhiệm vụ nhận đầu vào, xử lý thông qua hàm kích hoạt (activation function) rồi truyền kết quả sang các Neural ở lớp tiếp theo.

Điểm đặc trưng của MLP là có ít nhất một lớp ẩn (hidden layer) nằm giữa lớp đầu vào và lớp đầu ra giúp mô hình có khả năng học được các mối quan hệ phi tuyến giữa dữ liệu. Nhờ đó, MLP có thể giải quyết các bài toán phức tạp hơn nhiều so với mô hình Perceptron đơn tầng (Single-Layer Perceptron).

Một mô hình MLP (Multilayer Perceptron) điển hình bao gồm ba loại lớp chính: lớp đầu vào, lớp ẩn, và lớp đầu ra. Mỗi lớp đảm nhận một vai trò quan trọng trong việc tiếp nhận, xử lý và xuất kết quả của dữ liệu. Các lớp này được kết nối theo kiểu fully-connected (liên kết đầy đủ), nghĩa là mỗi Neural ở một lớp đều có kết nối với tất cả Neural ở lớp kế tiếp.

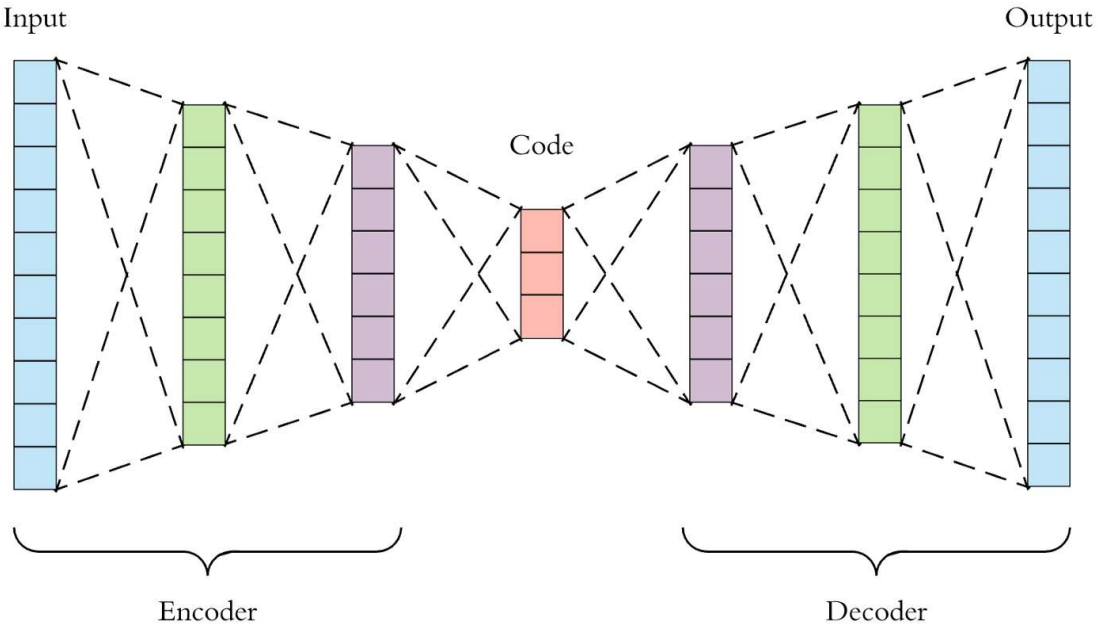


Hình 2.7. Kiến trúc hoạt động của mô hình MLP

- **Lớp đầu vào:** mỗi Neural đại diện cho một đặc trưng (feature) của mẫu dữ liệu, ví dụ như trong bài toán nhận dạng chữ số viết tay MNIST, hình ảnh 28×28 pixels sẽ được làm phẳng thành vec-tơ 784 phần tử, tương ứng với 784 nơ-ron đầu vào.
- **Lớp ẩn (Hidden layer):** đây là trái tim của MLP, diễn ra quá trình học và trích xuất đặc trưng. Mỗi Neural thực hiện phép tính cơ bản $z = Wx + b$, sau đó áp dụng hàm kích hoạt (activation function) để tạo đầu ra $a = f(z)$. Hàm kích hoạt giúp mạng học được mối quan hệ phi tuyến, cho phép MLP mô hình hóa các quy luật phức tạp mà mô hình tuyến tính không thể nắm bắt.
- **Lớp đầu ra:** thực hiện phân loại dự đoán cuối cùng.

2.5.6. AutoEncoder

AutoEncoder là mạng nơ-ron học không giám sát (unsupervised learning) với mục tiêu học một biểu diễn nén, có ý nghĩa của dữ liệu đầu vào bằng cách tái tạo đầu vào đó ở đầu ra. Cấu trúc mô hình đối xứng gồm 3 phần chính:



Hình 2.8. Cấu trúc cơ bản của mô hình AutoEncoder

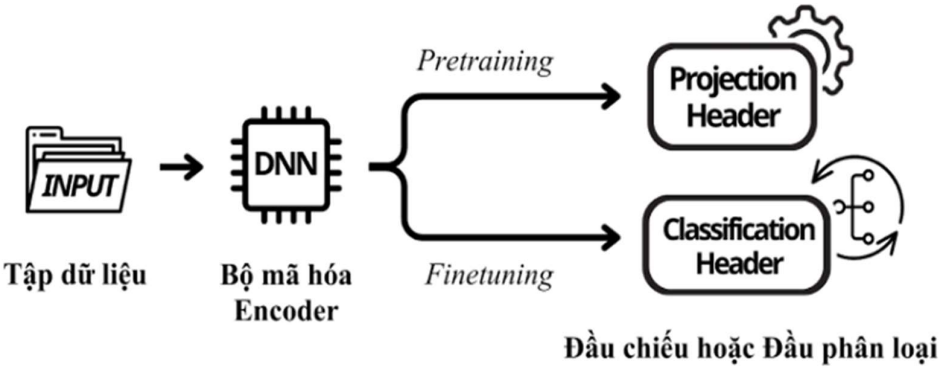
- **Bộ mã hóa (Encoder):** Nén dữ liệu đầu vào x thành một biểu diễn tiềm ẩn (latent representation) h với số chiều thấp hơn.
- **Biểu diễn tiềm ẩn (Latent space / Code):** Là không gian đặc trưng được học, nơi chứa đựng những thông tin quan trọng và cô đọng nhất của dữ liệu.
- **Bộ giải mã (Decoder):** Tái tạo dữ liệu đầu ra r từ biểu diễn tiềm ẩn h, sao cho r càng giống với đầu vào x càng tốt.

Quá trình huấn luyện nhằm tối thiểu hóa sai số tái tạo (reconstruction error) - khoảng cách giữa đầu vào x và đầu ra r. Việc buộc phải nén dữ liệu qua một không gian tiềm ẩn có kích thước nhỏ hơn sẽ khiến mô hình học được các đặc trưng quan trọng và cốt lõi nhất của dữ liệu. Autoencoder được sử dụng như một mô hình Baseline trong thí nghiệm để đánh giá hiệu quả của các kỹ thuật.

2.5.7. Mô hình học Tương phản tự giám sát có tinh chỉnh (Self-supervised Contrastive Learning DNN with Fine-tuning hay SSL-CL DNN)

Mô hình SSL-CL DNN được sử dụng làm baseline trong nghiên cứu gốc của Koukoulis cùng cộng sự [16], cho các lược đồ tự giám sát trên NetFlows. Về cơ bản, kiến trúc này là một Mạng Nơ-ron Sâu (DNN), còn được gọi là Mạng truyền thẳng đa lớp (Multi Layer Perceptron - MLP) khi có nhiều lớp ẩn. Mô hình gồm có 3 thành phần:

- DNN Encoder (Bộ Mã hóa) là một cấu trúc DNN 4 lớp chịu trách nhiệm học và trích xuất biểu diễn dữ liệu.
- Projection Head (Đầu Chiếu) là một MLP 2 lớp sử dụng trong Giai đoạn Tiền huấn luyện. Đầu chiếu ánh xạ biểu diễn đầu ra của Encoder vào không gian nhúng nơi hàm mất mát tương phản được áp dụng.
- Classification Head (Đầu phân loại) là một MLP 2 lớp có nhiệm vụ đưa ra dự đoán.



Hình 2.9. Minh họa kiến trúc của mô hình SSL-CL DNN

Ưu điểm SSL-CL DNN:

- Tổng quát hóa Tốt hơn: Do học được biểu diễn mạnh mẽ từ dữ liệu không nhãn, mô hình SSL-CL DNN giúp tăng khả năng nhận diện các tấn công mới chưa từng thấy (unseen attacks).
- Điểm Khởi đầu Vượt trội: Quy trình Pretraining cung cấp một mô hình vượt trội làm điểm khởi đầu khi dữ liệu được dán nhãn, đặc biệt là các mẫu luồng độc hại, bị khan hiếm.
- Cải thiện Hiệu suất trên Dữ liệu Hạn chế: Pretraining làm tăng điểm AUC (Area Under the Curve).

Nhược điểm:

- Phụ thuộc vào Đặc trưng Thủ công: Mặc dù sử dụng SSL-CL, mô hình DNN vẫn dựa vào đặc trưng thống kê mức luồng (NetFlows), vốn là các đặc trưng thủ công (handcrafted features) có thể hạn chế khả năng thích ứng với các môi trường mạng đa dạng.

2.6. Kỹ thuật tiền xử lý dữ liệu

2.6.1. Giảm mẫu

Undersampling (hay còn gọi là giảm mẫu) là một kỹ thuật xử lý dữ liệu mất cân bằng (imbalanced data) trong học máy, bằng cách giảm số lượng mẫu của lớp đa số để đạt được tỷ lệ cân bằng giữa các lớp. Cụ thể, kỹ thuật này giữ lại toàn bộ dữ liệu của lớp thiểu số (minority class) và loại bỏ bớt các quan sát từ lớp đa số (majority class) cho đến khi số lượng giữa hai lớp tương đương nhau.

2.6.2. Mã hóa nhãn

Là một kỹ thuật chuyển các nhãn dạng chữ/chuỗi (categorical labels) thành các số nguyên để mô hình Machine Learning hay Deep Learning có thể xử lý được. Đối với phân loại đa lớp, các mẫu sau khi được gom thành 8 nhóm nhãn chính theo thống kê dataset trước đó sẽ thực hiện mã hóa về số, bao gồm Benign, DDoS, DoS, BruteForce, Spoofing, Recon, Web-based, Mirai. Sau quá trình tiền xử lý thu được tập dữ liệu đã mã hóa đa nhãn (8 lớp).

2.6.3. Phân chia tập dữ liệu

Quá trình này được thực hiện trực tiếp trên tập dữ liệu còn 34 nhãn trước khi gom nhóm và thực hiện stratified phân chia đồng đều các nhãn, mục đích để đảm bảo tính đa dạng và công bằng cho các tập dữ liệu sau khi chia. Nghiên cứu sẽ thực hiện phân chia tập train và test với tỉ lệ 80-20, sau đó trong tập train lại tiếp tục lấy 20% để làm tập validation.

2.6.4. Chuẩn hóa Standard Scaler

Đề tài sử dụng kỹ thuật StandardScaler để chuẩn hóa các đặc trưng đầu vào (sau khi split sẽ chuẩn hóa theo train để tránh tình trạng rò rỉ dữ liệu). Phương pháp này đưa dữ liệu về phân phối chuẩn N(0,1), giúp mô hình học tốt hơn, tránh trường hợp các thuộc tính có thang đo lớn lấn át các thuộc tính nhỏ. Đây là bước tiền xử lý cần thiết cho các mô hình dựa trên khoảng cách và Gradient Descent như SVM, KNN, và Deep Learning. Cho một giá trị gốc x, sau khi chuẩn hóa ta được giá trị mới x′:

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó:

- μ: giá trị trung bình (mean) của feature
- σ: độ lệch chuẩn của feature

- x′: giá trị sau chuẩn hóa (đưa về phân phối chuẩn).

2.7. Kỹ thuật tối ưu siêu tham số với OPTUNA

Optuna là một framework tối ưu hóa siêu tham số thế hệ mới - giới thiệu lần đầu bởi T. Akiba cùng cộng sự [17], được thiết kế để tự động hóa quá trình tìm kiếm các thiết lập tốt nhất cho các mô hình học máy. Đây là một phần mềm mã nguồn mở dưới giấy phép MIT, cho phép người dùng xây dựng không gian tìm kiếm tham số một cách linh hoạt và hiệu quả thông qua các cơ chế chính sau:

Giao diện lập trình Define-by-run: Đây là đặc điểm nổi bật nhất của Optuna so với các công cụ truyền thống, cho phép người dùng xây dựng không gian tìm kiếm siêu tham số một cách động (dynamic) ngay trong quá trình thực thi hàm mục tiêu (objective function), thay vì phải khai báo tĩnh từ trước như các framework truyền thống.

Cơ chế Sampling (lấy mẫu): là quá trình Optuna quyết định giá trị nào sẽ được thử nghiệm tiếp theo trong không gian siêu tham số. Thay vì mang tính ngẫu nhiên hay vét cạn (grid search), Optuna áp dụng các thuật toán tiên tiến để thu hẹp dần không gian tìm kiếm dựa trên lịch sử các trial trước đó, hướng tới vùng không gian chứa các giá trị cho kết quả tốt hơn.

TPE (Tree-structured Parzen Estimator) là thuật toán sampling mặc định và được sử dụng phổ biến trong Optuna. Đây là một dạng tối ưu Bayesian - một phương pháp tối ưu hàm số có chi phí đánh giá cao, tận dụng kết quả của các lần thử trước để cân bằng giữa thăm dò (exploration) và khai thác (exploitation).

- Xây dựng một mô hình thay thế (surrogate model) - thường là Gaussian Process - để ước lượng hàm mục tiêu dựa trên các điểm đã đánh giá.
- Sử dụng hàm thu nhận (acquisition function – Expected Improvement (EI) hoặc Upper Confidence Bound (UCB) để xác định điểm tiếp theo cần đánh giá, cân bằng giữa vùng có kỳ vọng cao (khai thác) và vùng có độ bất định cao (thăm dò).
- Lặp lại quá trình cho đến khi đạt số lần thử mong muốn.

Cơ chế Pruning - cắt tỉa sớm các trial kém tiềm năng: là cơ chế cho phép Optuna tự động dừng sớm (early stopping) các trial không có triển vọng ngay trong quá trình huấn luyện, thay vì đợi đến khi kết thúc, giúp tiết kiệm đáng kể thời gian và tài nguyên tính toán.

Để kích hoạt pruning, người dùng cần báo cáo (report) các giá trị trung gian (intermediate results) của mô hình – ví dụ: độ mất mát (loss) hoặc độ chính xác (acc) sau mỗi epoch thông qua phương thức trial.report(). Dựa vào đó, Optuna sẽ sử dụng các bộ cắt tỉa (pruner) như MedianPruner, PercentilePruner, SuccessiveHalvingPruner, PatientPruner để phân tích các giá trị trung gian này để quyết định có dừng trial hay không.

2.8. Kỹ thuật làm nhẹ (lightweight) cho mô hình

Kỹ thuật làm nhẹ (Lightweight) được định nghĩa trong máy học là các phương pháp tối ưu hóa giúp mô hình AI nhỏ gọn, nhanh chóng và ít tiêu tốn tài nguyên hơn. Điều này cho phép triển khai các mô hình phức tạp trực tiếp lên các thiết bị hạn chế phần cứng như các thiết bị IoT biên.

Các kỹ thuật làm nhẹ cốt lõi thường được áp dụng hiệu quả hiện nay có thể kể đến:

- Chưng cất tri thức (Knowledge Distillation): Huấn luyện một mô hình học sinh nhỏ gọn (student) để bắt chước và đạt hiệu suất gần bằng một mô hình phức tạp, khổng lồ là giáo viên (teacher) [9].
- Lượng tử hóa (quantization): Giảm độ chính xác của các trọng số mô hình (ví dụ từ số thực 32-bit xuống số nguyên 8-bit), giúp giảm đáng kể dung lượng lưu trữ và tăng tốc độ tính toán mà không giảm nhiều độ chính xác [18].
- Cắt tỉa mạng (Pruning): Loại bỏ các nơ-ron hoặc trọng số ít quan trọng trong mạng nơ-ron, từ đó tinh gọn cấu trúc mô hình [18].
- Các kiến trúc tối ưu sẵn (Efficient Architectures): Sử dụng các kiến trúc được thiết kế chuyên biệt cho thiết bị biên (Edge AI) như MobileNet hay SqueezeNet.
- Phân mảnh tham số (PEFT - Parameter-efficient Fine-tuning): Đối với các mô hình ngôn ngữ lớn (LLMs), kỹ thuật này chỉ tinh chỉnh một số lượng nhỏ các tham số thay vì toàn bộ mô hình, giúp tiết kiệm bộ nhớ.

2.8.1. Kỹ thuật Chưng cất tri thức (Knowledge Distillation)

Nghiên cứu này tập trung vào triển khai làm nhẹ các model với kỹ thuật Chưng cất tri thức (Knowledge Distillation). Trong học máy, giai đoạn huấn luyện cần trích xuất lượng thông tin rất lớn từ dữ liệu học đa dạng và đôi khi là dư thừa. Bản chất của quá trình này không cần chạy thời gian thực và có thể sử dụng các mô hình với

kiến trúc lớn phức tạp để đem lại kết quả tối ưu. Thế nhưng có nghịch lý là ở giai đoạn triển khai thì các mô hình này lại bộc lộ nhược điểm là thời gian xử lý lâu và tiêu thụ nhiều tài nguyên tính toán, khiến mô hình không còn hiệu quả trên các thiết bị có bộ nhớ thấp. Do đó, chưng cất là cần thiết để chuyển đổi "tri thức" từ những mô hình cồng kềnh, chính xác nhưng chậm chạp sang các mô hình nhỏ gọn, hiệu quả để sử dụng thực tế.

Một rào cản khái niệm khiến phương pháp này ít được nghiên cứu trước đây là quan niệm cho rằng tri thức nằm ở các giá trị tham số của mô hình. Tuy nhiên, Hinton đã đưa ra một cái nhìn trừu tượng hơn: tri thức là một ánh xạ (mapping), không phục thuộc vào cấu trúc của mô hình mà là hàm ánh xạ từu vector đầu vào sang vector đầu ra. Ngay cả khi mô hình dự đoán sai, các xác suất tương đối của các lớp sai vẫn chứa đựng thông tin quý giá về cách mô hình tổng quát hóa (generalize). Ví dụ: Một hình ảnh xe BMW có thể có xác suất bị nhầm thành xe rác cao hơn nhiều so với việc nhầm thành củ cà rốt, điều này cho thấy mô hình đã học được cấu trúc tương đồng giữa các phương tiện giao thông. [10]

Các phương pháp Chưng cất hiện đại được phân loại theo nhiều các tiếp cận: Theo nguồn tri thức (Response-based, Feature-based) hay theo kiến trúc huấn luyện ta có Teacher cố định (Offline distillation), Teacher đồng huấn luyện (Online Distillation) và Tự học chính mình (Self-distillation). Nhưng tổng kết lại Chưng cất sẽ có các tham số quan trọng đặc trưng sau:

Tham số Nhiệt độ (Temperature – T) là tham số quan trọng nhất trong Chưng cất tri thức, giúp kiểm soát độ "mềm" (softness) của phân phối xác suất đầu ra từ hàm Softmax.

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Chú thích:

- z_i là logit (đầu ra trước softmax) của lớp i.
- T là tham số nhiệt độ.

Tác động của Nhiệt độ như sau:

- Nếu T = 1: đây là hàm Softmax chuẩn, phân phối xác suất gốc.
- Nếu T → 0: Phân phối tiến dần đến one-hot, tập trung vào lớp có xác suất cao nhất.

- Nếu T > 1: Phân phối trở nên mềm hơn (smooth), các lớp không phải dự đoán chính có xác suất cao hơn.
- Nếu T → ∞: Dần tiến đến phân phối đều (uniform).

Khi nhiệt độ càng cao, phân phối xác suất mềm hơn sẽ tiết lộ các "tri thức ẩn" (dark-knowledge) - thông tin về mức độ tương tự giữa các lớp mà mô hình giáo viên đã học được. Ví dụ, với bài toán phân loại chữ số MNIST, thay vì chỉ biết "đây là số 7", soft target còn cho biết "số 7 có nét tương đồng với số 1 ở mức độ nào".

Hàm Mất mát (Loss Function): Chưng cất tri thức sử dụng một hàm mất mát kết hợp từ hai thành phần.

Soft Target (Distillation Loss) so sánh phân phối xác suất đầu ra của student và teacher sau khi đã áp dụng nhiệt độ T, thành phần này giúp student "bắt chước" hành vi của teacher. Khi T lớn, student sẽ chú ý nhiều hơn đến cấu trúc tương quan giữa các lớp (dark knowledge) thay vì chỉ tập trung vào lớp dự đoán chính.

$$\mathcal{L}_{soft} = T^2 \cdot KL\big(\sigma(z_s/T) \parallel \sigma(z_t/T)\big)$$

Chú thích:

- KL là Kullback-Leibler Divergence.
- σ là hàm softmax.
- z_s, z_t lần lượt là logits của student và teacher.
- Hệ số T^2 giúp cân bằng độ lớn của gradient khi thay đổi nhiệt độ.

Hard Target (Student Loss) so sánh đầu ra của student với nhãn thật (ground truth), sử dụng CE (Cross-Entropy) với T=1, thành phần này đảm bảo student vẫn học được nhãn chính xác từ dữ liệu gốc, không chỉ phụ thuộc vào teacher. Điều này quan trọng vì Teacher cũng có thể mắc sai lầm, không thể chỉ dựa nào đó.

$$\mathcal{L}_{hard} = CE\big(y, \sigma(z_s)\big)$$

Chú thích:

- y là nhãn thật.
- z_s là logit của student.

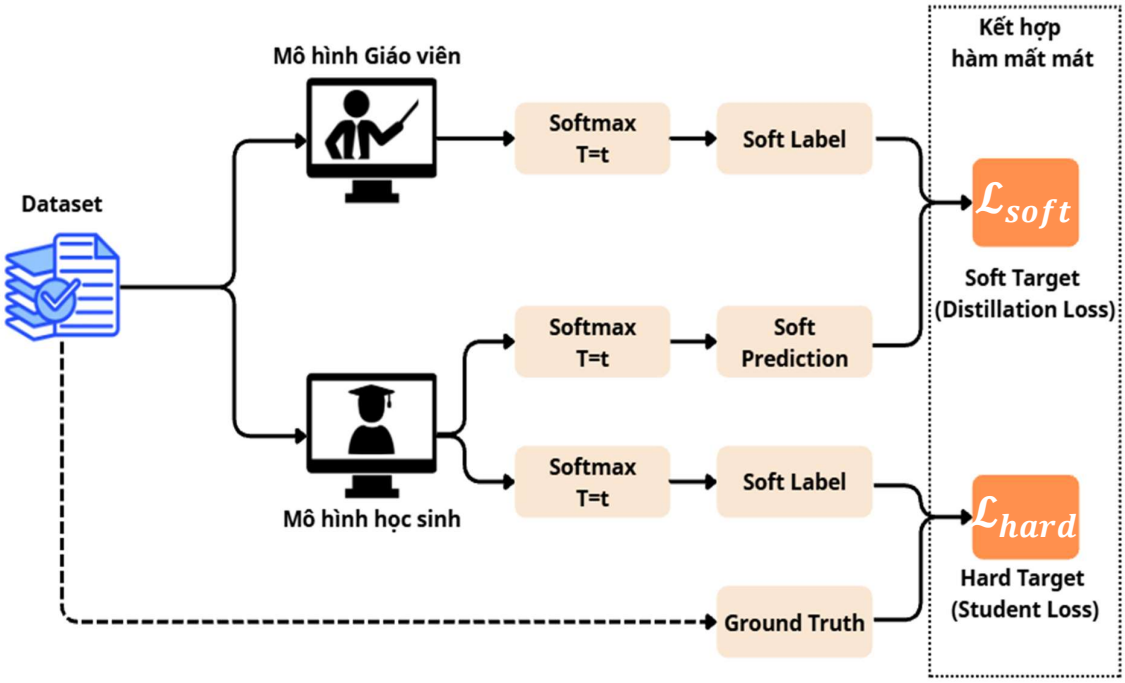
Kết hợp cả hai ta có được hàm mất mát tổng hợp (Loss Function đề xuất bởi Hinton et al. (2015)):

$$\mathcal{L}_{KD} = \alpha \cdot \mathcal{L}_{soft} + (1 - \alpha) \cdot \mathcal{L}_{hard}$$

Ở đây xuất hiện thêm một hệ số là α đóng vai trò kiểm soát tỷ lệ đóng góp giữa hai thành phần loss, tùy vào α càng tiến gần về 1 hay 0 mà việc học của student sẽ phụ thuộc vào teacher nhiều hay ít.

- α = 1.0: student chỉ học từ teacher (pure soft).
- α = 0.0: student chỉ học từ nhãn thật (pure hard).
- α = 0.5: cân bằng giữa 2 nguồn.

$$\mathcal{L}_{KD} = \alpha \cdot \mathcal{L}_{soft} + (1 - \alpha) \cdot \mathcal{L}_{hard}$$



Hình 2.10. Nguyên lý hoạt động cơ bản của quá trình Chưng cất tri thức

2.8.2. Teacher đồng huấn luyện – MTKD (Multi-teacher Knowledge Distillation) [8]

Phương pháp teacher đồng huấn luyện này được đề xuất để student học kiến thức đa dạng từ nhiều góc nhìn teacher khác nhau thay vì một. Lấy ví dụ với bài nghiên cứu của Ogab cùng cộng sự [8] đề xuất 2 teacher là LSTM giỏi phát hiện các phụ thuộc thời gian của luồng dữ liệu, trong khi cây quyết định XGBoost lại mạnh trong việc bắt các tương tác đặc trưng phân cấp và xử lý dữ liệu dạng bảng. Mục tiêu là giúp mô hình học sinh kế thừa được kiến thức bổ sung từ nhiều góc nhìn khác nhau, từ đó đưa ra quyết định tổng hợp phong phú và chính xác hơn, giảm thiểu rủi ro overfitting so với phụ thuộc vào một mô hình giáo viên duy nhất.

Ở trên đã cho thấy cái nhìn tổng quan về hàm phân phối xác suất dựa vào nhiệt độ, vậy khi tăng M số lượng teacher lên thì mỗi teacher k cho ra một phân phối xác suất riêng:

$$p_{i,j}^{k}(T) = \frac{\exp\left(z_{i,j}^{k}/T\right)}{\sum_{c=1}^{C}\exp\left(z_{i,c}^{k}/T\right)}$$

Vậy làm thế nào biến M phân phối xác suất này thành một p_i^{fused} duy nhất để chạy student. Đây chính là nút thắt kỹ thuật, và hai biến thể AW-MTKD và CW-MTKD khác nhau đúng ở cách gán trọng số w_k cho từng teacher. [2]

AW – MTKD (Average Weighted) dựa vào cơ chế trung bình như tên gọi, không có teacher nào được "ưu ái" hơn, thuật toán sẽ lấy trung bình cộng ý kiến mà không xét ai giỏi hơn trong tình huống cụ thể:

$$w_k^{avg} = \frac{1}{M}, \quad \forall k \in \{1, \dots, M\}$$

Suy ra: $$p_i^{AW} = \frac{1}{M}\sum_{k=1}^{M} p_i^k\,(T)$$

CW – MTKD (Confidence Weighted) sẽ có thêm cơ chế tính độ tự tin (confidence score) của mô hình trong một lô (mini-batch) cụ thể được xét để quyết định xem mô hình nào có sức ảnh hưởng với student nhiều hơn trong tình huống đó (c_k càng cao nghĩa là teacher k nhìn chung càng quyết đoán trong quyết định, ít mơ hồ trong các dự đoán của bản thân):

$$c_k = \frac{1}{N}\sum_{i=1}^{N}\max_{j}\ p_{i,j}^{k}(T)$$

Sau đó chuẩn hóa thành trọng số:

$$w_k = \frac{c_k}{\sum_{m=1}^{M} c_m}$$

Hợp nhất soft target theo trọng số:

$$p_i^{CW} = \sum_{k=1}^{M} w_k\ p_i^k(T)$$

Dù là AW hay CW, công thức huấn luyện là như nhau, chỉ khác ở cách xử lý đầu vào soft target.

$$\mathcal{L} = (1-\alpha)\,CE\left(\sigma\left(\frac{S_i}{T}\right), y_i\right) + \alpha\,T^2\,KL\left(\sigma\left(\frac{S_i}{T}\right) \| \, p_i^{fused}\right)$$

Pipeline hai thuật toán được trình bày rõ hơn theo hình sau đây:

Algorithm 1. AW-MTKD

1: Đầu vào: Mô hình học sinh S, các mô hình giáo viên $\mathcal{T} = \{T_1, \dots, T_M\}$, nhiệt độ τ, hệ số trọng số α, tập dữ liệu huấn luyện $\mathcal{D}$

2: Đầu ra: Mô hình học sinh Sđã được tối ưu với tri thức chưng cất

3: với mỗi mini-batch $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$ **thực hiện**

4: **với mỗi** giáo viên $T_k \in \mathcal{T}$ **thực hiện**

5: $z^k \leftarrow T_k(\mathbf{x})$ → *Logits của giáo viên*

6: $p^k \leftarrow \text{softmax}(z^k/\tau)$ → *Dự đoán đã được làm mềm*

7: **kết thúc vòng lặp**

8: $p_{fused} \leftarrow \frac{1}{M}\sum_{k=1}^{M} p^k$ → *Hợp nhất dự đoán các giáo viên*

9: $z^S \leftarrow S(\mathbf{x})$ → *Logits của học sinh*

10: $\mathcal{L}_{CE} \leftarrow CE(z^S, \mathbf{y})$ → *Hàm mất mát nhãn cứng*

11: $\mathcal{L}_{KD} \leftarrow \tau^2 \cdot KL(\sigma(z^S/\tau) \parallel p_{fused})$ → *Hàm mất mát chưng cất*

12: $\mathcal{L} \leftarrow (1-\alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD}$

13: Cập nhật S bằng lan truyền ngược trên $\mathcal{L}$

14: kết thúc vòng lặp

Algorithm 2. CW-MTKD

1: Đầu vào: Mô hình học sinh S, các mô hình giáo viên $\mathcal{T} = \{T_1, \dots, T_M\}$, nhiệt độ τ, hệ số trọng số α, tập dữ liệu huấn luyện $\mathcal{D}$

2: Đầu ra: Mô hình học sinh S đã được tối ưu với tri thức chưng cất

3: với mỗi mini-batch $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$ **thực hiện**

4: Khởi tạo điểm số tự tin của các giáo viên $\mathbf{c} \leftarrow \mathbf{0}$

5: **với mỗi** giáo viên $T_k \in \mathcal{T}$ **thực hiện**

6: $z^k \leftarrow T_k(\mathbf{x})$ → *Logits của giáo viên*

7: $p^k \leftarrow \text{softmax}(z^k/\tau)$ → *Dự đoán đã được làm mềm*

8: $c_k \leftarrow \frac{1}{|\mathbf{x}|}\sum_n \max_j p_{n,j}^k$ → *Điểm số tự tin*

9: **kết thúc vòng lặp**

10: $w_k \leftarrow \frac{c_k}{\sum_{m=1}^{M} c_m}$ → *Trọng số tự tin đã chuẩn hóa*

11: $p_{fused} \leftarrow \sum_{k=1}^{M} w_k \cdot p^k$ → *Hợp nhất có trọng số dự đoán của các giáo viên*

12: $z^S \leftarrow S(\mathbf{x})$ ▷ → *Logits của học sinh*

13: $\mathcal{L}_{CE} \leftarrow CE(z^S, \mathbf{y})$ → *Hàm mất mát nhãn cứng*

14: $\mathcal{L}_{KD} \leftarrow \tau^2 \cdot KL(\sigma(z^S/\tau) \parallel p_{fused})$ → *Hàm mất mát chưng cất*

15: $\mathcal{L} \leftarrow (1-\alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD}$

16: Cập nhật S bằng lan truyền ngược trên $\mathcal{L}$

17: kết thúc vòng lặp

Cách lựa chọn mô hình dựa vào độ tự tin trong một lô cụ thể (mini-batch) bộc lộ được tính thích nghi cục bộ cho student và tối ưu khả năng của việc học multi-teacher nên chúng tôi quyết định bám vào tham khảo phương pháp CW – MTKD để làm thực nghiệm trong nghiên cứu của mình.

2.8.3. Phòng thủ IoT nhận thức tài nguyên (Resource-Aware IoT Defense using KL-JS loss or RAID-KL) - Chưng cất tri thức bằng phân kỳ KL-JS (Raid-KL) [5]

Trong khuôn khổ chưng cất tri thức chuẩn đã trình bày, hàm mất mát chưng cất $\mathcal{L}_{soft}$ sử dụng độ phân kỳ Kullback-Leibler để đo độ lệch giữa phân phối xác suất đã làm mềm đầu ra của giáo viên và học sinh. Đây là phép đo KL nói chung, có dạng tổng quát sau [5]:

$$KL(P_T^\tau(c|x), P_S^\tau(c|x)) = \sum_{c=1}^{C} P_T^\tau(c|x)\, log\, \frac{P_T^\tau(c|x)}{P_S^\tau(c|x)} \quad (1)$$

Trong đó:

$D_{KL}(P \parallel Q)$ là giá trị phân kỳ Kullback-Leibler giữa hai phân phối xác suất $P_T^\tau(c|x)$ (giáo viên) và $P_S^\tau(c|x)$ (học sinh).

C là tổng số lớp, sigma lấy tổng theo chỉ số lớp c từ 1 tới C.

$P_T^\tau(c|x)$ là xác suất (đã làm mềm bởi nhiệt độ τ) của lớp c do mô hình giáo viên dự đoán.

$P_S^\tau(c|x)$ là xác suất (đã làm mềm bởi nhiệt độ τ) của lớp c do mô hình học sinh dự đoán.

Đây là lựa chọn gốc của [10] và vẫn phổ biến nhất trong các nghiên cứu KD hiện nay, nhưng tồn tại hai hạn chế mang tính cấu trúc.

Thứ nhất, KL không đối xứng: $D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$. Trong KD, chỉ chiều $D_{KL}(P_T \parallel P_S)$ được sử dụng, chiều ngược lại bị bỏ qua hoàn toàn, nên sai lệch giữa hai phân phối không được xử phạt cân xứng theo cả hai hướng.

Thứ hai, KL mất ổn định số học. Khi học sinh chưa học được một lớp tấn công hiếm gặp và gán xác suất rất thấp cho lớp đó thì $P_S(c) \to 0$. Số hạng $\log[P_T(c)/P_S(c)]$ sẽ bị phân kỳ về vô cực. Trong dữ liệu lưu lượng IoT vốn mất cân bằng nghiêm trọng giữa lớp bình thường và các lớp tấn công hiếm, học sinh dễ gán xác suất gần 0 cho lớp thiểu số ở giai đoạn đầu huấn luyện, khiến tình huống này xảy ra thường xuyên hơn so với bài toán cân bằng, dẫn đến độ dốc (gradient) hàm mất mát bùng nổ và huấn luyện bất ổn.

Sự chuyển dịch sang phân kỳ Jensen - Shannon

Để khắc phục hai hạn chế trên, [5] sử dụng phân kỳ Jensen-Shannon (JS), đây là phiên bản đối xứng và bị chặn của KL được xây dựng qua một phân phối trung bình M giữa giáo viên và học sinh:

$$M(c \mid x) = \frac{1}{2}[P_T^\tau(c \mid x) + P_S^\tau(c \mid x)] \qquad (2)$$

$$JS(P_T^\tau, P_S^\tau) = \frac{1}{2}KL(P_T^\tau \parallel M) + \frac{1}{2}KL(P_S^\tau \parallel M) \qquad (3)$$

Trong đó:

M là trung bình cộng của hai phân phối xác suất $P_T^\tau(c|x)$ và $P_S^\tau(c|x)$.

$JS(P_T^\tau, P_S^\tau)$ là giá trị phân kỳ Jensen - Shannon, có tính đối xứng: $JS(Q \parallel P) = JS(P \parallel Q)$.

$KL(P_T^\tau \parallel M)$ và $KL(P_S^\tau \parallel M)$ là phân kỳ KL của từng mô hình so với phân phối trung bình M.

M là trung của 2 phân phối xác suất nên luôn dương, cả hai số hạng KL trong công thức (3) đều đo khoảng cách giữa hai phân phối dương, do đó logarit không bùng nổ dù P_T^τ và P_S^τ cách xa nhau.

Hệ quả trực tiếp là $JS(P \parallel Q) = JS(Q \parallel P)$ và $JS(P \parallel Q)$ luôn bị chặn trong một khoảng hữu hạn, không phụ thuộc vào mức độ lệch giữa hai phân phối. Đánh đổi là JS hội tụ chậm hơn KL, do bản chất "thận trọng" của việc so sánh gián tiếp qua M thay vì so sánh trực tiếp giữa P_T^τ và P_S^τ [5].

Hàm mất mát lai KL-JS

Vì KL hội tụ nhanh nhưng dễ mất ổn định, còn JS ổn định nhưng hội tụ chậm hơn, [5] đề xuất kết hợp tuyến tính có trọng số giữa hai thước đo:

$$\mathcal{L}_{KL-JS} = \alpha \cdot KL(P_T^\tau, P_S^\tau) + (1-\alpha) \cdot JS(P_T^\tau, P_S^\tau) \qquad (4)$$

Trong đó:

Hệ số $\alpha \in [0,1]$ là siêu tham số cân bằng đóng góp của hai thước đo.

Khi $\alpha = 1$, hàm mất mát thu về KL thuần túy, tương đương công thức gốc của [10]

khi $\alpha = 0$, hàm mất mát thu về JS thuần túy.

Với $\alpha \in (0,1)$, thành phần KL giữ vai trò dẫn hướng hội tụ nhanh, còn thành phần JS giữ vai trò ổn định hóa khi hai phân phối lệch xa nhau, đặc biệt hữu ích ở các lớp tấn công thiểu số trong giai đoạn đầu huấn luyện.

Cơ chế nhiệt độ thích ứng

Bên cạnh hàm mất mát lai, RAID-KL thay nhiệt độ τ cố định bằng một nhiệt độ thích ứng $\tau_a(x)$, được tính riêng cho từng mẫu dựa trên mức độ tương đồng giữa biểu diễn đặc trưng của giáo viên và học sinh, thay vì cố định cho toàn bộ tập dữ liệu.

Theo [5], gọi $\phi_T(x)$, $\phi_S(x)$ là các vector đặc trưng đã chuẩn hóa L2 tại tầng áp chót (penultimate layer) của giáo viên và học sinh. Độ tương đồng cosine giữa hai đại lượng này là:

$$\cos\big(\phi_T(x), \phi_S(x)\big) = \frac{\phi_T(x) \cdot \phi_S(x)}{\parallel \phi_T(x) \parallel \parallel \phi_S(x) \parallel} \qquad (5)$$

Với hai cận cho trước τ_{min} và τ_{max}, nhiệt độ thích ứng được tính:

$$\tau_a(x) = \tau_{min} + (\tau_{max} - \tau_{min}) \cdot \Big(1 - \cos\big(\phi_T(x), \phi_S(x)\big)\Big) \qquad (6)$$

Công thức (6) cho thấy khi biểu diễn của học sinh và giáo viên đã tương đồng cao ($\cos \to 1$), $\tau_a(x)$ tiến về τ_{min}: hai mô hình đã tương đối "đồng thuận" trên mẫu đó nên không cần làm mềm phân phối nhiều. Ngược lại, khi biểu diễn còn lệch xa nhau ($\cos$ thấp), $\tau_a(x)$ tiến về τ_{max}, làm phân phối mềm hơn để bộc lộ rõ tri thức ẩn về quan hệ giữa các lớp, cung cấp tín hiệu học phong phú hơn cho đúng những mẫu mà học sinh còn hiểu sai cấu trúc dữ liệu. Nói cách khác, cơ chế này thay một siêu tham số nhiệt độ tĩnh, áp dụng đồng loạt cho mọi mẫu, bằng một đại lượng phụ thuộc mẫu và phản ánh trạng thái hội tụ cục bộ tại thời điểm huấn luyện.

Mục tiêu huấn luyện tổng quát

Kết hợp hàm mất mát lai KL-JS với nhiệt độ (cố định hoặc thích ứng), mục tiêu huấn luyện đầy đủ của mô hình học sinh là:

$$\mathcal{L}_{total} = (1 - \lambda) \cdot \mathcal{L}_{CE} + \lambda \cdot \tau^2 \cdot \mathcal{L}_{KL-JS} \qquad (7)$$

Trong đó $\lambda \in [0,1]$ điều phối tỉ trọng giữa học từ nhãn cứng ($\mathcal{L}_{CE}$) và học từ tri thức chưng cất. Hệ số τ^2 bù trừ hiện tượng gradient của $\mathcal{L}_{KL-JS}$ bị thu nhỏ theo tỉ lệ $1/\tau^2$ khi nhiệt độ tăng, cùng lý do đã trình bày ở công thức (5) của KD chuẩn. Khi sử dụng cơ chế nhiệt độ thích ứng, τ trong (7) được thay bằng $\tau_a(x)$ tính riêng cho từng mẫu theo (6).

Quy trình thuật toán

Algorithm 2. Huấn luyện mô hình học sinh bằng chưng cất KL-JS

1: Đầu vào: Tập huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, Mô hình giáo viên đã huấn luyện T_{ϕ^*}, Mô hình học sinh S_θ

2: Đầu vào: Siêu tham số α, λ, τ (hoặc τ_{min}, τ_{max} nếu dùng nhiệt độ thích ứng)

3: Đầu vào: Tốc độ học η_S, số epoch E_S, kích thước lô B

4: Đầu ra: mô hình học sinh tối ưu S_{θ^*}

5: Khởi tạo ngẫu nhiên tham số θ của học sinh

6: Nạp mô hình giáo viên đã huấn luyện trước T_{ϕ^*}

7: Đặt T_{ϕ^*} ở chế độ đánh giá (đóng băng tham số)

8: Tạo bộ nạp dữ liệu với kích thước lô B từ tập $\mathcal{D}$

9: với mỗi epoch $= 1$ đến E_S **thực hiện**

10: $\mathcal{L}_{epoc} \leftarrow 0$ *(khởi tạo mất mát của epoch)*

11: **với mỗi** lô (X_{batch}, Y_{batch}) trong $\mathcal{D}$ **thực hiện**

12: ***//lượt truyền xuôi qua cả hai mô hình***

13: $logits_T = T_{\phi^*}(X_{batch})$ ***//suy luận giáo viên***

14: $logits_S = S_\theta(X_{batch})$ ***//suy luận học sinh***

15: $P_S = \text{Softmax}(logits_S)$ ***//dùng cho*** $\mathcal{L}_{CE}$

16: ***//điều chỉnh nhiệt độ*** τ ***và cơ chế thích ứng***

17: **nếu** dùng nhiệt độ thích ứng τ **thì**

18: $\phi_T = \text{GetFeatures}(T_{\phi^*}, X_{batc}\ )$

19: $\phi_S = \text{GetFeatures}(S_\theta, X_{batch})$

20: $\tau = \text{ComputeAdaptiveTemp}(\phi_T, \phi_S, \tau_{min}, \tau_{max})$ ***//(6)***

21: **kết thúc rẽ nhánh**

22: $P_T^\tau = \text{TemperatureSoftmax}(logits_T, \tau)$ ***//(2)***

23: $P_S^\tau = \text{TemperatureSoftmax}(logits_S, \tau)$ ***//(2)***

24: ***//tính các thành phần hàm mất mát***

25: $\mathcal{L}_{CE} = \text{CrossEntropy}(P_S, Y_{batch})$

26: $\mathcal{L}_{KL} = KLDivergence(P_T^\tau, P_S^\tau)$ ***(1)***

27: $\mathcal{L}_{JS} = JSDivergence(P_T^\tau, P_S^\tau)$ ***(3)***

28: $\mathcal{L}_{KL-JS} \leftarrow$ ***(4)***

29: ***//mục tiêu tổng hợp***

30: $\mathcal{L}_{total} \leftarrow$ ***(7)***

31: $\mathcal{L}_{epoch} \leftarrow \mathcal{L}_{epoch} + \mathcal{L}_{total}$

32: ***//lan truyền ngược và tối ưu hóa***

33: Lan truyền ngược: tính $\nabla_\theta \mathcal{L}_{total}$

34: Cập nhật tham số: $\theta \leftarrow \theta - \eta_S \cdot \nabla_\theta \mathcal{L}_{total}$

35: **kết thúc vòng lặp**

36: ***//đánh giá và ghi log***

37: Đánh giá mô hình học sinh trên tập kiểm định

38: Ghi log mất mát huấn luyện: $\mathcal{L}_{epoch}/|batches|$, $\mathcal{L}_{CE}$, $\mathcal{L}_{KL-JS}$

39: Ghi log các độ đo kiểm định và mức sử dụng tài nguyên

40: **nếu** điều kiện dừng sớm thỏa mãn **thì**

41: dừng vòng lặp

42: **kết thúc rẽ nhánh**

43: kết thúc vòng lặp

44: Lưu tham số tối ưu của mô hình học sinh θ^*

45: trả về S_{θ^*}

2.8.4. Chưng cất tri thức qua gợi ý tầng ẩn (FitNets)

Chưng cất tri thức chuẩn của Hinton chỉ giám sát học sinh tại đầu ra cuối cùng thông qua hàm mất mát $\mathcal{L}_{KD}$ [10]. Với mục tiêu nén giáo viên thành học sinh sâu hơn nhưng mỏng hơn, Romero cùng đồng nghiệp [11] chỉ ra rằng cách giám sát này là không đủ: mạng càng sâu, không gian tối ưu hóa càng phi tuyến, khiến sự giảm độ dốc xuất phát từ điểm khởi tạo ngẫu nhiên dễ mắc kẹt ở vùng nghiệm xấu. Qua nhiều tầng liên tiếp trong lan truyền ngược, độ dốc suy yếu dần và về đến các tầng đầu thường đã quá nhỏ để tạo ra cập nhật có nghĩa. FitNets giải quyết bài toán này bằng cách bổ sung một tín hiệu giám sát tại tầng ẩn trung gian của học sinh, nhằm đưa phần đầu mạng học sinh về điểm khởi tạo tốt trước khi bước vào tối ưu toàn mạng bằng KD chuẩn.

Gợi ý (hint) và tầng được dẫn dắt (guided layer)

Để giải quyết vấn đề trên, FitNets bổ sung một điểm giám sát tại **tầng ẩn trung gian** của học sinh, thay vì chỉ ở đầu ra. Để làm được điều đó, cần một tín hiệu tham chiếu tại chính tầng đó tín hiệu này được lấy từ giáo viên và gọi là **hint**. Hint được định nghĩa là giá trị kích hoạt (activation) tại một tầng ẩn cụ thể h của giáo viên khi đưa mẫu x vào. Đối xứng với hint, một tầng ẩn g của học sinh được chọn làm **guided layer** tầng này sẽ được huấn luyện để tái tạo lại đúng hint đó, tức là làm cho activation của học sinh gần với của giáo viên trên toàn bộ tập dữ liệu.

Vị trí chọn hint layer h và guided layer g không tùy tiện. Vì hint đóng vai trò ràng buộc thêm vào học sinh (một dạng chính quy hóa), guided layer đặt càng sâu thì

học sinh càng bị trói buộc chặt, mất tự do điều chỉnh biểu diễn ở các tầng phía sau dẫn đến giảm khả năng tổng quát hóa. Ngược lại, guided layer đặt quá nông thì các tầng sâu của học sinh vẫn không nhận được tín hiệu học đủ mạnh. [11] chọn hint tại tầng giữa của giáo viên và guided layer tại tầng giữa của học sinh, đảm bảo tỉ lệ chiều sâu đã xử lý tính đến hai điểm đó là tương đương nhau điều kiện cần thiết để độ phức tạp của hàm phi tuyến mà học sinh cần xấp xỉ tương xứng với số tầng mà nó được phép dùng.

Bộ hồi quy (regressor) và hàm mất mát hint

Gọi $u_h(x; W_{Hint})$ là hàm tính activation của giáo viên đến tầng h, $v_g(x; W_{Guided})$ là hàm tính activation của học sinh đến tầng g với W_{Hint} là tham số giáo viên từ tầng đầu đến tầng h, đã cố định sau khi giáo viên được huấn luyện xong.

Giáo viên thường rộng hơn học sinh, nên giá trị kích hoạt u_htại lớp hint so với giá trị kích hoạt v_g tại guided layer thường có số chiều khác nhau, không thể so sánh trực tiếp. Để giải quyết, FitNets thêm một bộ hồi quy r đặt ngay sau guided layer, ánh xạ v_g về đúng kích thước của u_h rồi mới so sánh. Bộ hồi quy này không phải một phần của kiến trúc học sinh cuối cùng, nó chỉ là dụng cụ tạm thời phục vụ việc so sánh trong giai đoạn huấn luyện đầu tiên.

Vậy để guided layer của học sinh học được cách biểu diễn dữ liệu tương tự giáo viên tại hint layer, FitNets tối thiểu hóa sai số bình phương giữa hint và đầu ra đã hồi quy của guided layer:

$$\mathcal{L}_{HT}(W_{Guided}, W_r) = \frac{1}{2} \parallel u_h(x; W_{Hint}) - r\big(v_g(x; W_{Guided}); W_r\big) \parallel^2 \qquad (8)$$

Trong đó W_{Hint} là tham số giáo viên từ tầng đầu đến tầng h, đã cố định sau khi giáo viên được huấn luyện xong.

Gradient của $\mathcal{L}_{HT}$ chỉ chảy ngược qua W_{Guided} và W_r, không chảy qua giáo viên (đóng băng) và không chảy qua các tầng học sinh sau guided layer (không nằm trong đồ thị tính toán của $\mathcal{L}_{HT}$).

Bộ hồi quy r cần dùng cùng hàm kích hoạt với tầng hint của giáo viên, vì u_h và $r\big(v_g\big)$ phải cùng loại phi tuyến để MSE đo khoảng cách có nghĩa trong cùng một không gian biểu diễn. [11] dùng hồi quy tích chập cho dữ liệu ảnh để tránh bùng nổ tham số. Số tham số hồi quy kết nối đầy đủ tăng theo tích toàn bộ kích thước không gian và số kênh ở cả hai phía, trong khi hồi quy tích chập chỉ cần số tham số tỉ lệ với

kích thước nhân chập. Với dữ liệu bảng đặc trưng không có chiều không gian, một tầng tuyến tính là đủ.

Huấn luyện theo hai giai đoạn

FitNets tách quá trình huấn luyện thành hai giai đoạn tuần tự. Giai đoạn 1 chỉ tối thiểu hóa $\mathcal{L}_{HT}$ và không dùng nhãn thật, không so khớp đầu ra phân loại để đưa W_{Guided} về một điểm khởi tạo tốt trong không gian tham số trước khi tối ưu toàn mạng. Kết thúc giai đoạn 1, bộ hồi quy bị loại bỏ. Giai đoạn 2 huấn luyện toàn bộ học sinh bằng $\mathcal{L}_{KD}$ chuẩn. Điểm khác biệt duy nhất là W_{Guided} lúc này đã được giai đoạn 1 dẫn dắt thay vì khởi tạo ngẫu nhiên. Đồng thời, sử dụng một lịch giảm λ (thay cho α của công thức 2.6.1), để giai đoạn đầu huấn luyện học được nhiều từ phân phối mềm của giáo viên hơn, sau đó khi λ giảm dần, mô hình học sinh bắt đầu học nhiều từ nhãn cứng hơn và học cân bằng giữa 2 mất mát chưng cất và mất mát huấn luyện. Nhờ điểm khởi tạo tốt hơn, độ giảm độ dốc (gradient descent) ở giai đoạn 2 bắt đầu từ vùng không gian tham số ít cực tiểu cục bộ kém hơn, giúp quá trình tối ưu toàn mạng hội tụ về nghiệm tốt hơn so với chưng cất tri thức khởi tạo hoàn toàn ngẫu nhiên.

Quy trình thuật toán

Algorithm 1. FitNets Stage-Wise Training

1: Đầu vào: tham số giáo viên đã huấn luyện W_T, tham số học sinh khởi tạo ngẫu nhiên W_S, chỉ số tầng hint h và guided layer g

2: Đầu ra: tham số học sinh tối ưu W_S^*

3: $W_{Hint} \leftarrow \{W_T^1, \dots, W_T^h\}$

4: $W_{Guided} \leftarrow \{W_S^1, \dots, W_S^g\}$

5: Khởi tạo W_r bằng các giá trị ngẫu nhiên nhỏ

6: $W_{Guided}^* \leftarrow \underset{W_{Guided}}{\operatorname{argmin}} \mathcal{L}_{HT}(W_{Guided}, W_r)$ ***(8)***

7: $\{W_S^1, \dots, W_S^g\} \leftarrow \{W_{Guide}^{*1}, \dots, W_{Guided}^{*g}\}$

8: $W_S^* \leftarrow \underset{W_S}{\operatorname{argmin}} \mathcal{L}_{KD}(W_S)$ ***(công thức mục 2.6.1)***

9: trả về W_S^*

2.9. Thông số đánh giá hiệu quả các mô hình

Để đánh giá toàn diện hiệu suất của các mô hình phát hiện xâm nhập (IDS), báo cáo sử dụng hai nhóm chỉ số: nhóm chỉ số hiệu suất phân loại bao gồm Accuracy, Precision, Recall, F1-score và AUC (được áp dụng cho cả mô hình giáo viên và học sinh); và nhóm chỉ số hiệu quả triển khai bao gồm Thời gian suy luận (Inference Time), Kích thước mô hình (Model Size), Số lượng tham số (Parameter Count) và FLOPs được sử dụng riêng để đánh giá tính khả thi khi triển khai học sinh trên các thiết bị biên (edge devices) có tài nguyên hạn chế [5].

2.9.1. Độ chính xác toàn cục (Accuracy)

Accuracy đo lường tỷ lệ số lượng mẫu được phân loại chính xác trên tổng sô mẫu. Đây là chỉ số trực quan nhất, cho biết mô hình hoạt động "đúng" bao nhiêu phần trăm, rất hữu ích khi các lớp tương đối cân bằng. Tuy nhiên, trong các tập dữ liệu mất cân bằng (imbalanced) như CICIoT2023, Accuracy có thể gây hiểu nhầm vì mô hình có thể đạt chính xác cao bằng cách chỉ dự đoán đúng các lớp chiếm đa số.

$$\text{Accuracy} = \frac{\sum_{i=1}^{K} T\,P_i}{\sum_{i=1}^{K}(TP_i + FP_i + FN_i)}$$

2.9.2. Độ chính xác (Precision)

Precision đo lường tỷ lệ các mẫu được dự đoán là thuộc một lớp mà thực sự đúng là lớp đó (dương tính thật). Chỉ số này cao thể hiện hệ thống ít đưa ra cảnh báo sai (dương tính giả) [5].

$$\text{Precision} = \frac{1}{K}\sum_{i=1}^{K}\frac{TP_i}{TP_i + FP_i}$$

2.9.3. Độ nhạy (Recall)

Recall đo lường tỷ lệ các mẫu thực sự thuộc một lớp mà mô hình đã phát hiện đúng. Trong số các mẫu là dương tính thật, mô hình bắt được bao nhiêu phần trăm. Recall đặc biệt quan trọng trong các bài toán an ninh mạng vì nó đo lường khả năng phát hiện ra tấn công. Recall thấp đồng nghĩa với việc có quá nhiều cuộc tấn công bị bỏ sót (âm tính giả), gây nguy hiểm cho hệ thống [5].

$$\text{Recall } = \frac{1}{K}\sum_{i=1}^{K}\frac{TP_i}{TP_i + FN_i}$$

2.9.4. F1-Score

Điểm F1 là trung bình điều hòa (harmonic mean) của Precision và Recall. Trong các tập dữ liệu mất cân bằng, điểm F1 thường được ưa chuộng hơn Accuracy vì nó phản ánh chính xác hơn hiệu suất của mô hình đối với các lớp thiểu số [5].

$$F1\text{-}score = \frac{1}{K}\sum_{i=1}^{K}\frac{2 \times TP_i}{2 \times TP_i + FP_i + FN_i}$$

2.9.5. Diện tích dưới đường cong ROC (AUC – Area Under the ROC Curve)

ROC (Receiver Operating Characteristic) là đường cong biểu diễn mối quan hệ giữa Tỷ lệ dương tính thật trên trục tung và Tỷ lệ dương tính giả trên trục hoành. AUC là diện tích nằm dưới đường cong này ROC này. Nó đo lường khả năng phân biệt giữa các lớp của mô hình, chỉ số này càng gần 1.0 thì mô hình càng có khả năng phân tách hoàn hảo giữa các lớp [5].

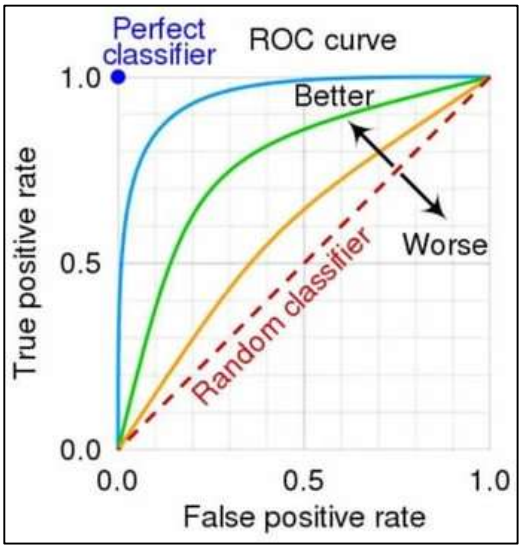
Đồ thị này được xây dựng dựa trên sự đánh đổi giữa hai đại lượng True Positive Rate - TPR và False Positive Rate - FPR.

- TPR hay còn gọi là recall hoặc sensitivity.

$$TPR = \frac{TP}{TP + FN}$$

- FPR là tỷ lệ dự báo sai các trường hợp thực tế là âm tính thành dương tính trên tổng số các trường hợp thực tế là âm tính.

$$FPR = \frac{FP}{FP + TN}$$



Hình 2.11. Minh họa 3 loại đường ROC

- Trục X đại diện cho tỷ lệ dương tính giả FPR.
- Trục Y đại diện cho tỷ lê dương tính thực TPR.
- Phần diện tính phía dưới đường ROC và trên trục hoành là AUC có giá trị nằm trong khoảng [0,1].

2.9.6. Chỉ số đánh giá về hiệu suất

Thời gian suy luận (Inference Time): Là khoảng thời gian mà mô hình cần để đưa ra dự đoán cho một mẫu đầu vào hoặc batch dữ liệu [5].

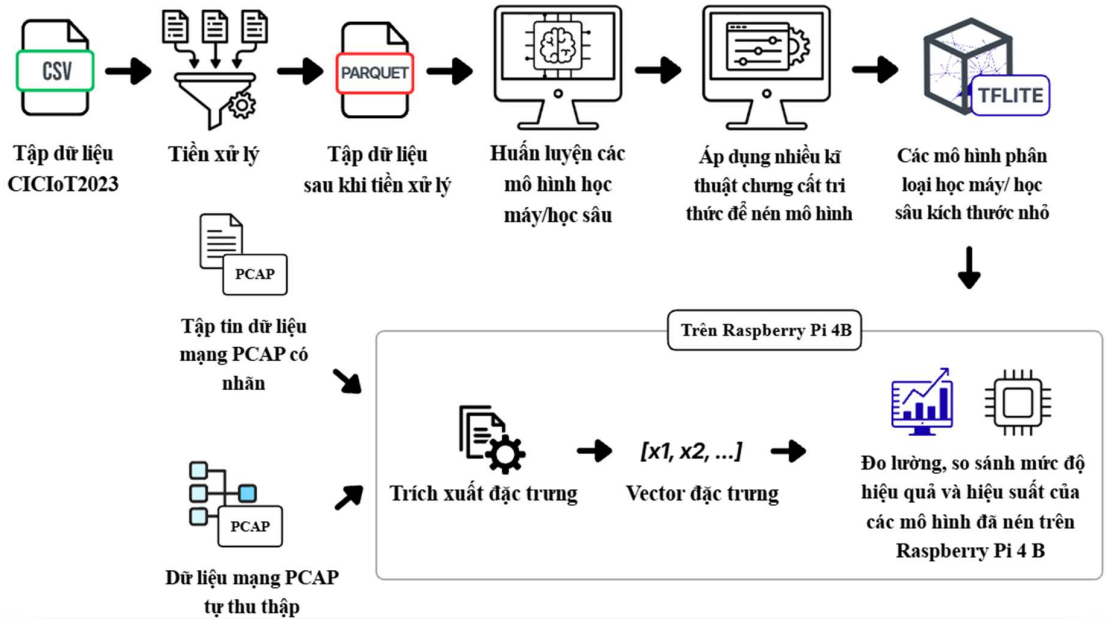
Kích thước mô hình (Model size): Là dung lượng bộ nhớ lưu trữ mà mô hình chiếm dụng trên đĩa hoặc bộ nhớ flash [5].

Số lượng tham số (parameter count): Là tổng số trọng số (weights) và hệ số điều chỉnh (bias) có thể huấn luyện được trong mạng nơ-ron [5].

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

Bài toán tổng quan của luận văn sẽ được trình bày cụ thể ở chương thứ 3 này, bao gồm các công việc như tiền xử lý dữ liệu, huấn luyện các mô hình, áp dụng các kĩ thuật chưng cất tri thức để nén mô hình, so sánh khả năng phân loại, mức độ hiệu quả, và hiệu suất của các mô hình trên dữ liệu mạng tại thiết bị IoT thật CICIoT2023.

3.1. Tổng quan bài toán



Hình 3.1. Hình minh họa bài toán tổng quan

Bước đầu thực hiện sử dụng tập dữ liệu CICIoT2023 có nguồn từ trang chính thức của UNB. Tập dữ liệu sau khi được tiền xử lý sẽ được sử dụng trực tiếp để huấn luyện cho các mô hình học máy/học sâu giáo viên và học sinh.

Luận văn tiếp theo sẽ xây dựng và huấn luyện các mô hình học máy/học sâu giáo viên như CNN, DNN, SSL-CL-DNN, XGBoost và LSTM có tối ưu siêu tham số huấn luyện. Sau khi có các mô hình giáo viên, xây dựng các mô hình học sinh CNN, DNN, SSL-CL-DNN, MLP làm đối tượng để áp dụng các kĩ thuật chưng cất tri thức: KD (truyền thống), RAID-KL, FitNets, AW-MTKD và CW-MTKD.

Các kết quả huấn luyện mô hình giáo viên và học sinh sẽ được thu thập để so sánh, đưa ra nhận xét. Các mô hình phân loại học máy/học sâu giáo viên và học sinh sau khi được huấn luyện hoàn chỉnh sẽ thực hiện phân loại tấn công trên dữ liệu mạng PCAP có nhãn sẵn (nguồn từ trang chính thức của UNB) và trên dữ liệu mạng tự thu

thập. Quy trình thực hiện phân loại tấn công trên được triển khai trực tiếp tại thiết bị IoT biên Rapsberry Pi 4 B có tài nguyên hạn chế với mục tiêu chính là thu thập, so sánh mức độ hiệu quả, hiệu suất phân loại của các mô hình phân loại tấn công trước và sau khi nén bằng chưng cất tri thức.

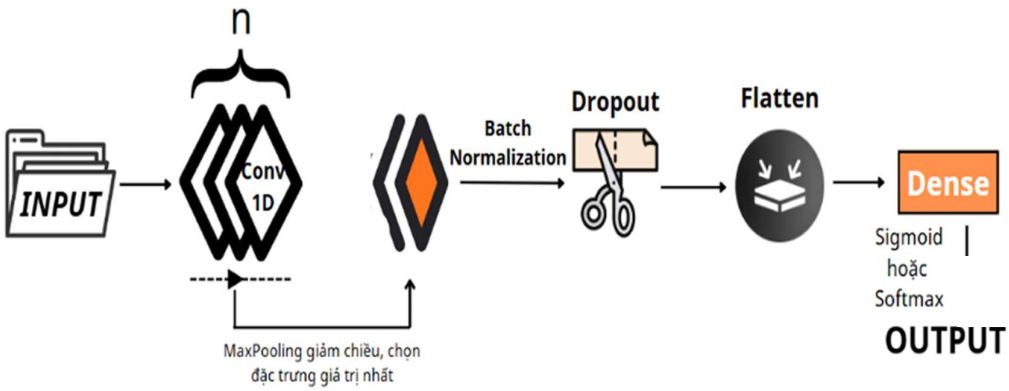
Sau khi thu thập và đã tổng hợp kết quả phân loại, mức độ hiệu quả, hiệu suất phân loại của các mô hình phân loại tấn công trước/sau khi nén bằng các kỹ thuật chưng cất tri thức, luận văn sẽ đưa ra các so sánh, nhận xét và cuối cùng là đưa ra kết luận về mức độ phù hợp của từng kỹ thuật chưng cất tri thức đối với bài toán phát hiện xâm nhập IoT trên dữ liệu dạng bảng đặc trưng, cũng như tiềm năng triển khai thực tế của các mô hình học sinh trên thiết bị biên tài nguyên hạn chế như mục tiêu đề tài đã đề ra.

3.2. Xây dựng và huấn luyện các mô hình học sâu học sinh theo các phương pháp chưng cất tri thức truyền thống, Raid-KL và FitNets

3.2.1. Xây dựng các mô hình học sinh từ các mô hình giáo viên

Luận văn lựa chọn các mô hình học sâu giáo viên CNN, DNN và SSL-CL-DNN tham khảo lựa chọn mô hình của [5], [9] và [10]. Cũng theo [5], phương pháp thiết kế các mô hình học sâu học sinh chính đó là tham khảo kiến trúc mô hình giáo viên tương ứng nhưng cắt giảm theo chiều rộng (giảm số Neural ở các lớp) để có được mô hình học sinh gọn nhẹ hơn, nhằm tối ưu khả năng nén mô hình của các biện pháp chưng cất tri thức. Kiến trúc của các mô hình học sinh sẽ trình bày xoay quanh các thành phần: các tầng/lớp ẩn (hidden layer), số nơ-ron mỗi tầng (Neural per layer), hàm mất mát (loss function), hàm tối ưu (optimizer), chuẩn hóa lô (batch normalization), tỉ lệ cắt bỏ nơ-ron (dropout).

3.2.1.a. Xây dựng mô hình học sinh CNN



Hình 3.2. Luồng xử lý của CNN1D

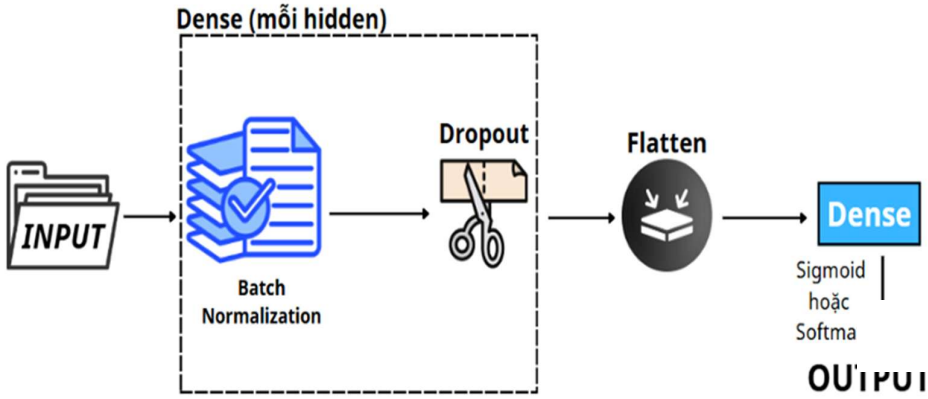
Mô hình học sinh CNN gồm các tầng Conv1D để trích xuất đặc trưng cục bộ theo chiều thời gian hoặc chuỗi, phát hiện các quy luật ngắn hạn (local patterns), tiếp theo là tầng MaxPooling lấy giá trị lớn nhất trên cửa sổ trượt qua các tầng Conv1D nhằm giảm số chiều dữ liệu và loại bỏ nhiễu. Tầng Batch Normalization được áp dụng sau đó để ổn định phân phối dữ liệu, kết hợp với tầng Dropout nhằm hạn chế hiện tượng quá khớp (overfitting). Cuối cùng, dữ liệu được làm phẳng qua tầng Flatten trước khi đưa vào tầng Dense để phân loại đầu ra.

	CNN1D	**CNN1D học sinh**
Optimizer	Adam	Adam
Loss Function	CE	CE
Hidden Layer	3	3
Neurals/ Layer	[128, 128, 256]	[32, 32, 64]
Activation	ReLU	ReLU
BatchNormalization	Yes	Yes
Dropout	0.1	0.1

Bảng 3.1. Bảng kiến trúc mô hình học sinh CNN1D so với giáo viên CNN1D

Mô hình học sinh sử dụng thành phần và kiến trúc giống giáo viên CNN nhưng cắt giảm đi khoảng 4 lần số nơ-ron các lớp tham khảo tương đối sức nén mô hình từ [9] đồng thời tham khảo kích thước của mô hình học sinh MLP khác phân hệ. Kích thước mô hình học sinh CNN là 157,576 tham số, còn với mô hình học sinh là 11,752, vậy mô hình giáo viên đã được nén khoảng 92.5% khi sau đưa về mô hình học sinh. Quá trình huấn luyện học sinh sẽ dùng. Hàm mất mát là cross-entropy có trọng số lớp nhằm giảm thiên lệch về các lớp đa số mà vẫn giữ ổn định huấn luyện.

3.2.1.b. Xây dựng mô hình học sinh DNN



Hình 3.3. Luồng xử lý của DNN

Mô hình học sinh DNN gồm các tầng Dense (nơ-ron dày đặt) nhằm học các mối quan hệ phi tuyến toàn cục giữa các đặc trưng, mỗi tầng sau đó kết hợp với Batch Normalization và để ổn định quá trình huấn luyện và tăng tốc độ hội tụ, cùng với Dropout để hạn chế quá khớp. Dữ liệu sau khi làm phẳng qua tầng Flatten sẽ được đưa vào tầng Dense đầu ra để thực hiện phân loại.

	DNN	**DNN học sinh**
Optimizer	Adam	Adam
Loss Function	CE	CE
Hidden Layer	3	3
Neurals/ Layer	[256, 256, 256]	[64, 64, 64]
Activation	ReLU	ReLU
BatchNormalization	Yes	Yes
Dropout	0.1	0.1

Bảng 3.2. Bảng kiến trúc mô hình học sinh DNN so với giáo viên DNN

Tương tự như CNN, mô hình DNN học sinh cũng được thực hiện giảm đi 4 lần số nơ-ron trên các lớp ẩn với sức nén khoảng 92% (DNN học sinh có 11,656 tham số và DNN giáo viên có 144,904 tham số). DNN vẫn có trọng số lớp cho hàm mất mát như CNN.

3.2.1.c. Xây dựng mô hình học sinh SSL-CL-DNN

Mô hình học sinh SSL-CL-DNN có cấu hình huấn luyện khác nhau tùy vào giai đoạn huấn luyện nhằm phục vụ cho mục tiêu huấn luyện tại giai đoạn đó [16]. Nhưng cơ bản có 3 thành phần chính tạo nên mô hình này:

Encoder (Bộ mã hóa) là một mạng nơ-ron sâu DNN chỉ gồm các tầng Dense (cuối mỗi tầng đầu ra đi qua hàm kích hoạt ReLU).

Projection Head (Đầu Chiếu) là một mạng MLP nhỏ gồm các tầng nơ-ron (cuối mỗi tầng đầu ra đi qua hàm kích hoạt ReLU).

Classification Head (Đầu phân loại): Là một mạng MLP nhỏ gồm các tầng nơ-ron (cuối mỗi tầng đầu ra đi qua Dropout và hàm kích hoạt ReLu), ở tầng cuối đầu ra sẽ là 8 logits (phân loại đa lớp nhãn nhóm 8).

Giai đoạn 1: Huấn luyện học tương phản tự giám sát – Tiền huấn luyện (pretraining)

Kiến trúc mô hình học sinh tại giai đoạn 1 bao gồm 2 phần:

Encoder chịu trách nhiệm trích xuất các biểu diễn véc-tơ (flow representations) từ dữ liệu đầu vào đưa cho đầu chiếu.

Projection Head được gắn vào đầu ra của Encoder nhằm này ánh xạ biểu diễn véc-tơ chiều lớn từ DNN thành một véc-tơ chiều thấp hơn để mô hình học được cách giảm thiểu khoảng cách giữa các mẫu tương tự (cặp dương tính) và tối đa hóa khoảng cách giữa các mẫu không tương tự, điều này hỗ trợ cho việc tính toán hàm mất mát ở bước học tương phản.

Học tương phản tự giám sát:

Nhận vào tập dữ liệu huấn luyện, dùng kỹ thuật Làm hỏng Đặc trưng Ngẫu nhiên (Random Feature Corruption - RFC). Kĩ thuật này tạo ra hai bản (x_1, x_2) từ mỗi véc-tơ đặc trưng ban đầu với x_2 là véc-tơ đặc trưng x_1 nhưng một số các đặc trưng ngẫu nhiên đã bị làm hỏng theo tỉ lệ cố định (gọi là tỉ lệ tăng cường λ) giúp mô hình tăng khả năng tổng quát, cặp (x_1, x_2) này sẽ được gọi là cặp dương tính (positive pair), cặp x_1 với bất kì véc-tơ đặc trưng nào khác sẽ là cặp âm tính (negative pair).

Sử dụng hàm mất mát NT-Xent Loss (Normalized Temperature-scaled Cross-Entropy Loss) để tính toán khoảng cách và tín hiệu điều chỉnh (độ dốc) giữa các cặp mẫu: các cặp mẫu dương tính sẽ được tín hiệu điều chỉnh kéo lại gần nhau hơn, các cặp mẫu sẽ bị tín hiệu điều chỉnh đẩy ra xa hơn.

Giai đoạn 2: Huấn luyện tinh chỉnh có giám sát (Supervised Finetuning)

Kiến trúc khi huấn luyện tinh chỉnh có giám sát:

Classification Head sẽ được kết hợp vào cuối Encoder thay vì Projection Head để chính thức thực hiện phân loại đa lớp, đầu ra trả về logits. Mô hình bắt đầu quá trình tinh chỉnh khi huấn luyện trên tập dữ liệu và giám sát trên tập thẩm định nhiều lần.

	SSL-CL-DNN	SSL-CL-DNN học sinh
Encoder	DNN [512, 512, 256, 256]	DNN [128, 64, 64]
Projaction Head & Classification head	MLP [256, 256]	MLP [64, 32]
Loss Function (Pretraining)	NT-Xent	
Loss Function (Finetuning)	Cross-Entropy	
Optimizer	AdamW	AdamW
Dropout	0.1	0.1

Bảng 3.3. Bảng kiến trúc mô hình học sinh SSL-CL-DNN so với mô hình giáo viên SSL-CL-DNN

Tương tự như CNN, mô hình học sinh SSL-CL-DNN cũng được thực hiện giảm đi 4 lần số nơ-ron trên các lớp ẩn với sức nén khoảng 96.4% (mô hình học sinh SSL-CL-DNN có 19,624 tham số và mô hình giáo viên SSL-CL-DNN có 547,080 tham số). Đặc biệt do kiến trúc giáo viên có kích thước lớn từ khâu tham khảo thiết kế, nên với DNN Encoder của mô hình học sinh thực hiện cắt bỏ một tầng nơ-ron dày trước khi giảm kích thước để tối ưu sức nén. Ngoài ra đầu phân loại và đầu chiếu cũng sẽ được giảm đưa về cùng số lượng nơ-ron so với học sinh MLP khác phân hệ. L_{CE} của mô hình SSL-CL-DNN cũng như 2 mô hình trước đều có trọng số lớp.

Mô hình học sinh	Tham số mô hình giáo viên	Tham số mô hình học sinh	Sức nén
CNN	158,600	11,752	92.50%
DNN	144,904	11,656	92%
SSL-CL-DNN	547,080	19,624	96.42%

Bảng 3.4. Bảng tổng hợp số lượng tham số của kiến trúc mô hình giáo viên so với học sinh

3.2.2. Áp dụng kỹ thuật chưng cất tri thức truyền thống trong huấn luyện các mô hình học sinh CNN1D, DNN, SSL-CL-DNN

Đối với kĩ thuật chưng cất tri thức truyền thống, các mô hình học sinh CNN1D, DNN và SSL-CL-DNN sẽ huấn luyện theo phương pháp chưng cất tri thức offline (offline distillation): mô hình giáo viên tương ứng đã huấn luyện hội tụ và đóng băng sẽ được nạp lại và chạy ở chế độ đánh giá để cung cấp Soft Target hay mất mát chưng cất cho mô hình học sinh, gradient sẽ không chảy ngược qua giáo viên trong suốt quá trình huấn luyện [10]. Về siêu tham số huấn luyện KD, [17] đề xuất một bộ giá trị nhiệt độ và hệ số alpha cho chưng cất tri thức truyền thống của Hinton mà nghiên cứu cho là tối ưu cho phương pháp này, chi tiết siêu tham số KD như bảng.

KD	
Temperature (τ)	3
Alpha α	0.5

Bảng 3.5. Bảng siêu tham số chưng cất tri thức truyền thống

Luồng huấn luyện tổng quát chưng cất tri thức truyền thống:

Quá trình huấn luyện mô hình học sinh diễn ra qua nhiều vòng huấn luyện (epoch), tại mỗi vòng lập huấn luyện thực hiện:

Với mỗi lô dữ liệu (kích thước lô tùy chọn tùy vào mô hình đang huấn luyện) từ đầu vào là tập dữ liệu đã xử lý sẽ thực hiện:

Đi qua 2 khối kiến trúc của mô hình học sinh và giáo viên để thu được 2 giá trị logits đầu ra của cả 2 mô hình phục vụ tính toán mất mát.

Logits của học sinh và giáo viên sẽ đi qua softmax ($\tau = 3$) để làm mềm và kết hợp với nhau để tính Mất mát chưng cất L_{soft}.

Logits của học sinh qua softmax kết hợp với nhãn cứng để tính Mất mát huấn luyện L_{hard}.

Kết hợp 2 giá trị mất mát để tính Mất mát tổng thể theo [5]. Lan truyền ngược gradient để mô hình cập nhật trọng số trên nơ-ron các tầng thông qua Hàm tối ưu được chọn theo tốc độ học đã trình bày.

Sau khi kết thúc duyệt qua toàn bộ tập dữ liệu của lần huấn luyện đó, thực hiện đánh giá và kiểm tra mất mát trên tập kiểm thử (validation). Khi nhật ký (log) huấn luyện lại. Nếu sau vòng huấn luyện hiện tại, giá trị mất mát tốt nhất (thấp nhất) thì lưu lại trạng thái tốt nhất tham số của học sinh.

Sau khi kết thúc số vòng huấn luyện đã cấu hình, in ra biểu đồ Đường biểu diễn mất mát đánh giá trên tập kiểm thử trong quá trình huấn luyện.

Tiếp đến phục hồi lại trạng thái tham số tốt nhất của các mô hình học sinh, kết hợp với kiến trúc của chính học sinh để dựng lại mô hình học sinh. Mô hình học sinh sau huấn luyện hoàn chỉnh sẽ đánh giá, dự đoán tấn công trên tập dữ liệu kiểm tra (test), đánh giá xong lưu ra các kết quả đánh giá, các ma trận nhầm lẫn và biểu đồ cột thể hiện khả năng đánh giá của các mô hình học sinh theo tiêu chí macro-F1 (do tập dữ liệu mất cân bằng). Các kết quả sẽ được thu thập cho công tác thống kê, so sánh và nhận xét ở chương tiếp theo. Đồng thời, lưu ra 3 mô hình học sinh CNN, DNN, SSL-CL-DNN đã chưng cất tri thức truyền thống phục vụ cho việc triển khai trên môi trường Raspberry để đo mức độ hiệu quả, hiệu suất.

Với mô hình học sinh CNN: các lô dữ liệu dạng chuỗi đặc trưng được đưa qua khối Conv1D–MaxPool–BatchNorm–Dropout của cả giáo viên và học sinh (kiến trúc đã trình bày ở phần xây dựng mô hình) đầu ra sau Flatten của mỗi bên đi qua tầng Dense riêng để sinh logits; hai logits này được dùng trực tiếp trong hàm mất mát chưng cất mà không cần thêm bước trung gian nào giữa hai mô hình. Mô hình học sinh CNN được huấn luyện với số vòng huấn luyện, kích thước mỗi lô (batch size)

và tốc độ học tham khảo tham khảo bên dưới. Các giá trị siêu tham số này tham khảo từ mô hình giáo viên để so sánh mức độ hiệu quả của kiến trúc học sinh trực tiếp với giáo viên theo phương pháp lựa chọn siêu tham số của [5].

Batch size	64
Epochs	50
Learning rate	0.00001

Bảng 3.6. Bảng siêu tham số huấn luyện mô hình học sinh CNN

Với mô hình học sinh DNN: các lô dữ liệu được đưa qua các tầng Linear–BatchNorm–Dropout của giáo viên và học sinh (qua kiến trúc đã xây dựng), đầu ra logits ở tầng cuối của hai bên được đối chiếu trực tiếp qua hàm mất mát chưng cất, tương tự CNN1D. Siêu tham số huấn luyện mô hình học sinh DNN tham khảo từ mô hình giáo viên DNN và có cấu hình như sau.

Batch size	512
Epochs	50
Learning rate	0.0017

Bảng 3.7. Bảng siêu tham số huấn luyện mô hình học sinh DNN

Với mô hình học sinh SSL-CL-DNN: Phương pháp KD truyền thống chỉ tác động lên phần tương đương giai đoạn tinh chỉnh có giám sát của học sinh. Ở giai đoạn chưng cất, batch dữ liệu được đưa qua khối Encoder rồi đến Classification Head của cả giáo viên và học sinh (khác nhau về kích thước tầng ẩn, kiến trúc đã trình bày) để sinh logits tính toán, không dùng đến Projection Head nữa. Quá trình huấn luyện chưng cất tri thức trên mô hình học sinh SSL-CL-DNN không đi qua giai đoạn Pretraining một lần nữa do các giá trị huấn luyện của giai đoạn này mô hình giáo viên đã hấp thụ rồi, việc triển khai trực tiếp phương pháp KD truyền thống lên giai đoạn tinh chỉnh có giám sát của học sinh, giúp tách biệt hiệu quả của cơ chế chưng cất khỏi hiệu quả của học biểu diễn tự giám sát khi so sánh kết quả ở chương sau.

Batch size	128
Epochs	50
Learning rate	0.0001487

Bảng 3.8. Bảng siêu tham số huấn luyện của mô hình học sinh SSL-CL-DNN

Siêu tham số huấn luyện của mô hình học sinh SSL-CL-DNN gồm các giá trị kích thước lô, số vòng huấn luyện và tốc độ học tham khảo trực tiếp từ mô hình giáo viên SSL-CL-DNN. Các siêu tham số hỗ trợ cho việc tiền huấn luyện học tương phản

không xuất hiện lại trong quá trình huấn luyện tinh chỉnh và các giá trị siêu tham số này sẽ được tham khảo trực tiếp từ mô hình giáo viên và từ [16].

3.2.3. Áp dụng kỹ thuật chưng cất tri thức sử dụng phân kỳ KL-JS (Raid-KL fixed temperature & adaptive temperature) trong huấn luyện các mô hình học sinh CNN1D, DNN, SSL-CL-DNN

Phương pháp Raid-KL thay hàm chưng cất một chiều sử dụng phân kỳ KL truyền thống bằng kết hợp KL và JS (Jensen-Shannon), đồng thời khảo sát hai trường hợp nhiệt độ: **nhiệt độ cố định (τ fixed)** và **nhiệt độ thích ứng (τ adaptive)**. Tương ứng với mỗi trường hợp nhiệt độ, triển khai huấn luyện chưng cất tri thức offline trên các mô hình giáo viên và học sinh tương ứng. Khi áp dụng kỹ thuật chưng cất tri thức sử dụng phân kỳ KL-JS (Raid-KL fixed temperature & adaptive temperature) trong huấn luyện các mô hình học sinh (CNN1D, DNN, SSL-CL-DNN), số vòng huấn luyện, kích thước lô và tốc độ học của từng mô hình học sinh được giữ nguyên như đã trình bày ở kỹ thuật chưng cất tri thức truyền thống, chỉ bổ sung các siêu tham số chưng cất riêng của Raid-KL dưới đây, tham khảo trực tiếp từ [5].

Raid-KL (fixed temperature)	
Temperature (τ)	7
Alpha (α)	0.5
Lambda (λ)	0.3

Bảng 3.9. Bảng siêu tham số huấn luyện chưng cất tri thức Raid-KL nhiệt độ cố định

Raid-KL (adaptive temperature)	
Min temperature (τ_min)	2
Max temperature (τ_max)	7
Alpha (α)	0.5
Lambda (λ)	0.3

Bảng 3.10. Bảng siêu tham số huấn luyện chưng cất tri thức Raid-KL nhiệt độ thích ứng

Luồng huấn luyện tổng quát chưng cất tri thức Raid-KL:

Quá trình huấn luyện mô hình học sinh diễn ra qua nhiều vòng huấn luyện, áp dụng cho hai trường hợp nhiệt độ như sau:

Trường hợp nhiệt độ cố định: tại mỗi vòng lặp huấn luyện thực hiện:

Với mỗi lô dữ liệu từ đầu vào là tập dữ liệu đã xử lý sẽ thực hiện:

Đi qua 2 khối kiến trúc của mô hình học sinh và giáo viên để thu được logits đầu ra của cả 2 mô hình phục vụ tính toán mất mát.

Logits của học sinh và giáo viên đi qua hàm softmax theo nhiệt độ τ cố định để làm mềm, kết hợp tính đồng thời hai thành phần phân kỳ KL và JS, tổng hợp theo trọng số α để ra Mất mát chưng cất L_{soft}.

Logits của học sinh qua softmax kết hợp với nhãn cứng để tính Mất mát huấn luyện L_{hard}.

Kết hợp 2 giá trị mất mát theo trọng số λ để tính Mất mát tổng thể. Lan truyền ngược gradient để mô hình cập nhật trọng số trên nơ-ron các tầng thông qua Hàm tối ưu được chọn theo tốc độ học đã trình bày.

Sau khi kết thúc duyệt qua toàn bộ tập dữ liệu của lần huấn luyện đó, thực hiện đánh giá và kiểm tra mất mát trên tập kiểm thử (validation). Ghi lại nhật ký huấn luyện. Nếu giá trị mất mát tốt hơn giá trị tốt nhất đã ghi nhận thì lưu lại trạng thái tốt nhất tham số của học sinh.

Trường hợp nhiệt độ thích ứng: tại mỗi vòng lặp huấn luyện thực hiện:

Với mỗi lô dữ liệu từ đầu vào là tập dữ liệu đã xử lý sẽ thực hiện:

Đi qua 2 khối kiến trúc của mô hình học sinh và giáo viên để thu được đồng thời logits đầu ra và đặc trưng trung gian (feature) của cả 2 mô hình.

Tính độ tương đồng cosine trung bình giữa đặc trưng trung gian của học sinh (đã chiếu về cùng số chiều với giáo viên qua một tầng tuyến tính riêng) và đặc trưng trung gian của giáo viên, từ đó suy ra nhiệt độ τ của riêng lô dữ liệu đó, sao cho độ tương đồng càng thấp thì τ càng lớn, độ tương đồng càng cao thì τ càng nhỏ.

Logits của học sinh và giáo viên đi qua softmax theo nhiệt độ τ vừa tính để làm mềm, kết hợp tính đồng thời hai thành phần phân kỳ KL và JS, tổng hợp theo trọng số α để ra Mất mát chưng cất L_{soft}.

Logits của học sinh kết hợp với nhãn cứng để tính Mất mát huấn luyện L_{hard}.

Kết hợp 2 giá trị mất mát theo trọng số λ để tính Mất mát tổng thể. Lan truyền ngược gradient để mô hình cập nhật trọng số trên nơ-ron các tầng thông qua Hàm tối ưu được chọn theo tốc độ học đã trình bày.

Sau khi kết thúc duyệt qua toàn bộ tập dữ liệu của lần huấn luyện đó, thực hiện đánh giá và kiểm tra mất mát trên tập kiểm thử (validation). Ghi lại nhật ký huấn luyện. Nếu giá trị mất mát tốt hơn giá trị tốt nhất đã ghi nhận thì lưu lại trạng thái tốt nhất tham số của học sinh.

Sau khi kết thúc vòng huấn luyện đã cấu hình, in ra biểu đồ Đường biểu diễn mất mát đánh giá trên tập kiểm thử trong quá trình huấn luyện.

Tiếp đến phục hồi lại trạng thái tham số tốt nhất của các mô hình học sinh, kết hợp với kiến trúc của chính học sinh để dựng lại mô hình học sinh. Mô hình học sinh sau huấn luyện hoàn chỉnh sẽ đánh giá, dự đoán tấn công trên tập dữ liệu kiểm tra (test), đánh giá xong lưu ra các kết quả đánh giá, các ma trận nhầm lẫn và biểu đồ cột thể hiện khả năng đánh giá của các mô hình học sinh theo tiêu chí macro-F1 (do tập dữ liệu mất cân bằng). Các kết quả sẽ được thu thập cho công tác thống kê, so sánh và nhận xét ở chương tiếp theo. Đồng thời, lưu ra 6 mô hình học sinh CNN, DNN, SSL-CL-DNN (nhiệt độ cố định và nhiệt độ thích ứng) đã chưng cất tri thức bằng phân kỳ JS-KL (Raid-KL) phục vụ cho việc triển khai trên môi trường thiết bị IoT biên để đo mức độ hiệu quả, hiệu suất.

Với mô hình học sinh CNN1D: đặc trưng trung gian dùng để tính nhiệt độ thích ứng (trường hợp adaptive) lấy tại đầu ra sau Flatten của khối Conv1D-MaxPool-BatchNorm-Dropout cuối cùng, ngay trước tầng Dense phân loại, chiều của mô hình học sinh qua tầng tuyến tính riêng để chiếu lên cùng chiều mô hình giáo viên để nhiệt độ thích ứng để tính độ tương đồng cosine. Ở trường hợp nhiệt độ cố định, bước trích đặc trưng trung gian và tầng chiếu này không được sử dụng, học sinh chỉ cần logits đầu ra như luồng chung đã mô tả.

Với mô hình học sinh DNN: đặc trưng trung gian lấy tại đầu ra của thân mạng, ngay trước tầng Dense phân loại, chiều của mô hình học sinh qua tầng tuyến tính riêng để chiếu lên cùng chiều mô hình giáo viên tính toán nhiệt độ thích ứng. Trường hợp nhiệt độ cố định giữ nguyên luồng chung, không trích đặc trưng trung gian.

Với mô hình học sinh SSL-CL-DNN: đặc trưng trung gian lấy tại đầu ra của khối Encoder (không dùng Projection Head), ngay trước Classification Head, số chiều của học sinh sẽ nhỏ hơn, phải qua tầng tuyến tính riêng để chiếu lên cùng chiều với chiều của mô hình giáo viên nhằm tính nhiệt độ thích ứng. Classification Head vẫn dùng để sinh logits cho cả hai trường hợp như bình thường. Tương tự kỹ thuật KD truyền thống, học sinh SSL-CL-DNN không đi qua giai đoạn Pretraining một lần nữa ở cả hai trường hợp nhiệt độ, chỉ tác động lên phần tương đương giai đoạn tinh chỉnh có giám sát.

3.2.4. Áp dụng phương pháp chưng cất tri thức sử dụng gợi ý tầng ẩn (FitNets) cho các mô hình học sinh CNN1D, DNN, SSL-CL-DNN

Khác với KD truyền thống và RAID-KL chỉ huấn luyện một giai đoạn duy nhất, FitNets huấn luyện học sinh qua hai giai đoạn tuần tự, độc lập: **giai đoạn khớp gợi ý tầng ẩn (hint training)** và **giai đoạn chưng cất tri thức (KD training)**, theo đúng phương pháp gốc của [11]. Kích thước lô, tốc độ học và số vòng huấn luyện của từng mô hình học sinh vẫn giữ nguyên như hai kỹ thuật trên, cả 2 giai đoạn khớp gợi ý tầng ẩn và chưng cất đều sử dụng cùng một tham số vòng huấn luyện cấu hình chung là 50 epochs (lấy bằng số epochs của các mô hình học sinh). Các siêu tham số chưng cất FitNets tham khảo trực tiếp [11].

FitNets	
Temperature (τ)	3
Initial lambda (λ_init)	4
Final lambda (λ_final)	1

Bảng 3.11. Bảng siêu tham số huấn luyện chưng cất tri thức bằng gợi ý tầng ẩn FitNets

Luồng huấn luyện tổng quát chưng cất tri thức gợi ý tầng ẩn:

Quá trình huấn luyện mô hình học sinh diễn ra qua hai giai đoạn tuần tự, mỗi giai đoạn có bộ tối ưu (optimizer) riêng.

Giai đoạn 1: Khớp gợi ý tầng ẩn (Hint training): tại mỗi vòng lặp huấn luyện thực hiện:

Với mỗi lô dữ liệu từ đầu vào là tập dữ liệu đã xử lý sẽ thực hiện:

Đưa lô dữ liệu qua giáo viên để lấy đặc trưng tại tầng ẩn giữa đã chọn làm gợi ý (hint), giáo viên chỉ suy luận thuận, không cập nhật.

Đưa cùng lô dữ liệu qua nhánh thân mạng của học sinh để lấy đặc trưng tại tầng ẩn tương ứng (guided), sau đó đưa qua một tầng hồi quy (regressor) để khớp số chiều/số kênh với đặc trưng gợi ý của giáo viên.

Tính Mất mát khớp gợi ý (hint loss) là sai số bình phương trung bình giữa hai đặc trưng đã cùng kích thước. Lan truyền ngược gradient về thân mạng học sinh để cập nhật trọng số của nhánh thân mạng học sinh và tầng hồi quy thông qua hàm tối ưu riêng của giai đoạn này. (Gradient chỉ chảy ngược về tầng guided sau đó chảy lên phần đầu mô hình, các tầng sau guided không được cập nhật).

Sau khi kết thúc duyệt qua toàn bộ tập dữ liệu, đánh giá mất mát khớp gợi ý trên tập kiểm thử, ghi lại nhật ký huấn luyện của giai đoạn này.

Giai đoạn 2: Chưng cất tri thức (FitNets KD training): tiếp nối ngay sau khi hoàn tất giai đoạn 1, tại mỗi vòng lặp huấn luyện thực hiện:

Xác định hệ số λ của vòng huấn luyện hiện tại theo lịch giảm tuyến tính từ giá trị λ khởi đầu về giá trị λ kết thúc.

Với mỗi lô dữ liệu từ đầu vào là tập dữ liệu đã xử lý sẽ thực hiện:

Đưa lô dữ liệu qua giáo viên và học sinh (đã được khởi tạo lại từ trọng số thân mạng vừa huấn luyện ở giai đoạn 1) để lấy logits đầu ra của cả hai mô hình, tầng hồi quy không còn được sử dụng ở giai đoạn này.

Logits của học sinh và giáo viên đi qua softmax theo nhiệt độ τ cố định để làm mềm, tính Mất mát chưng cất L_{soft} theo phân kỳ KL.

Logits của học sinh kết hợp với nhãn cứng để tính Mất mát huấn luyện L_{hard} (cross-entropy có trọng số lớp).

Kết hợp L_{hard} và L_{soft} theo hệ số λ vừa xác định để tính Mất mát tổng thể theo công thức KD truyền thống [10]. Sau đó, giảm giá trị λ theo lịch tuyến tính.

Lan truyền ngược gradient để cập nhật trọng số học sinh thông qua bộ tối ưu riêng của giai đoạn này theo tốc độ học đã trình bày.

Sau khi kết thúc duyệt qua toàn bộ tập dữ liệu, đánh giá mất mát trên tập kiểm thử, ghi lại nhật ký huấn luyện. Nếu giá trị mất mát tốt hơn giá trị tốt nhất đã ghi nhận thì lưu lại trạng thái tốt nhất tham số của học sinh.

Sau khi kết thúc vòng huấn luyện đã cấu hình, in ra biểu đồ Đường biểu diễn mất mát đánh giá trên tập kiểm thử trong quá trình huấn luyện.

Tiếp đến phục hồi lại trạng thái tham số tốt nhất của các mô hình học sinh, kết hợp với kiến trúc của chính học sinh để dựng lại mô hình học sinh. Mô hình học sinh sau huấn luyện hoàn chỉnh sẽ đánh giá, dự đoán tấn công trên tập dữ liệu kiểm tra (test), đánh giá xong lưu ra các kết quả đánh giá, các ma trận nhầm lẫn và biểu đồ cột thể hiện khả năng đánh giá của các mô hình học sinh theo tiêu chí macro-F1 (do tập dữ liệu mất cân bằng). Các kết quả sẽ được thu thập cho công tác thống kê, so sánh và nhận xét ở chương tiếp theo. Đồng thời, lưu ra 3 mô hình học sinh CNN, DNN, SSL-CL-DNN đã chưng cất tri thức bằng gợi ý tầng ẩn (FitNets) phục vụ cho việc triển khai trên môi trường thiết bị IoT biên để đo mức độ hiệu quả, hiệu suất.

Với mô hình học sinh CNN1D: Ở giai đoạn 1, cặp tầng ẩn gợi ý được chọn tại đầu ra khối Conv1D thứ 2, đây là vị trí ngay ở giữa 3 tầng mạng của kiến trúc mô hình học sinh và giáo viên CNN. Tầng hồi quy dùng 1 khối Conv1D kernel 1×1 để khớp số kênh giữa mô hình học sinh và giáo viên (32 lên 128). Ở giai đoạn 2, quy trình huấn luyện chưng cất chạy qua khối kiến trúc CNN đầy đủ giống như chưng cất tri thức truyền thống đã trình bày.

Với mô hình học sinh DNN: Ở giai đoạn 1, cặp tầng ẩn gợi ý được chọn tại đầu ra tầng ẩn thứ 2 (nằm giữa 3 tầng kiến trúc) của mạng mô hình học sinh và giáo viên. Tầng hồi quy dùng một lớp Linear để khớp số chiều học sinh với giáo viên (64 lên 256). Ở giai đoạn 2, quy trình huấn luyện chưng cất chạy qua khối kiến trúc DNN đầy đủ giống như chưng cất tri thức truyền thống đã trình bày.

Với mô hình học sinh SSL-CL-DNN: Ở giai đoạn 1, cặp tầng ẩn gợi ý được chọn tại đầu ra tầng ẩn giữa (tầng thứ 2 trong 3 tầng kiến trúc DNN) của khối Encoder giáo viên. Tầng hồi quy dùng một lớp Linear để khớp số chiều (64 lên 512). Tương tự hai kỹ thuật trên, học sinh SSL-CL-DNN không đi qua giai đoạn Pretraining tương phản; giai đoạn hint training ở đây đóng vai trò tương đương bước khởi tạo có định hướng cho Encoder, còn giai đoạn KD training tương đương giai đoạn tinh chỉnh có giám sát.

3.3. Quy trình triển khai suy luận bằng dữ liệu mạng PCAP sử dụng các mô hình học sinh giáo viên và học sinh để đánh giá kết quả suy luận, hiệu suất tại thiết bị IoT biên

Để đánh giá khả năng ứng dụng thực tế của các mô hình giáo viên và học sinh sau chưng cất tri thức, luận văn xây dựng một quy trình thống nhất gồm bốn bước: thu thập dữ liệu mạng dạng PCAP, trích xuất đặc trưng theo cửa sổ 10 gói tin, tiền xử lý (chuẩn hóa) véc-tơ đặc trưng, và suy luận bằng mô hình trên thiết bị Raspberry Pi 4. Mục tiêu cuối cùng của quy trình này đó là giải đáp rằng liệu mô hình học sinh sau chưng cất tri thức giữ được bao nhiêu phần trăm độ chính xác so với mô hình giáo viên khi suy luận trên lưu lượng mạng thực và hệ thống có đáp ứng được yêu cầu về tốc độ và tài nguyên khi triển khai trên phần cứng của thiết bị biên hay không.

3.3.1. Mô tả dữ liệu mạng được sử dụng trong quy trình đánh giá, so sánh

Dữ liệu mạng sử dụng cho quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh được chọn là dữ liệu mạng PCAP có sẵn nhãn và PCAP tự thu thập phục vụ 2 nhu cầu khác nhau:

Các tập tin mạng PCAP có sẵn nhãn do Canadian Institute for Cybersecurity (CIC) thu thập trên 105 thiết bị IoT gồm 33 loại tấn công khác nhau, nguồn tải tập dữ liệu PCAP cho tập dữ liệu CICIoT2023 được cấp trực tiếp tại trang chính thức của UNB CIC. Trang chính thức công bố cung cấp rất nhiều thư mục, mỗi thư mục chứa 1 hoặc nhiều tập tin PCAP của cùng một loại lưu lượng (bao gồm 33 lưu lượng tấn công và lưu lượng bình thường).

Các tập tin mạng này phục vụ mục đích đánh giá chất lượng suy luận (ACC, Macro-F1, Macro-AUC) của từng mô hình giáo viên/học sinh, đồng thời đo hiệu năng, tài nguyên tiêu thụ khi chạy trên Raspberry Pi 4 trong điều kiện có kiểm soát (dữ liệu tĩnh, không tranh chấp tiến trình). Ngoài ra, được dùng làm cơ sở để kiểm chứng bộ trích xuất đặc trưng tự triển khai (bám theo mã nguồn của chính tác giả UNB CIC), đánh giá chất lượng suy luận (ACC, Macro-F1, Macro-AUC) của từng mô hình giáo viên/học sinh.

Tương ứng với mỗi loại lưu lượng trên 34 lưu lượng gốc, 1 tập tin dữ liệu mạng PCAP sẽ được chọn cắt mẫu giữ lại khoảng 300,000 gói tin liên tục để phục vụ giai đoạn trích xuất và suy luận trên thiết bị. Tuy nhiên bài toán phân loại chính vẫn là phân loại đa lớp nhãn nhóm (8 nhãn). Việc chỉ chọn một số lượng nhất định gói tin do kích thước tập dữ liệu mạng có kích thước lớn để việc theo dõi quá trình suy luận dễ dàng hơn giữa các tập tin có kích thước gốc quá lớn hoặc quá nhỏ, đồng thời không

làm quá tải thiết bị. Đoạn gói tin được cắt ra đảm bảo liên tục với nhau để dữ liệu mạng sau xử lý không bị sai lệch ngữ cảnh tấn công.

STT	Tên file	Số gói tin	Thời lượng (s)	Kích thước (MB)
1	Benign.pcap	3,664,164	33,328.0	2,048.0
2	DDoS-ACK_Fragmentation.pcap	300,000	166.6	277.0
3	DDoS-HTTP_Flood.pcap	300,000	292.3	117.5
4	DDoS-ICMP_Flood.pcap	300,000	12.0	22.9
5	DDoS-ICMP_Fragmentation.pcap	300,000	580,244.6	265.8
6	DDoS-PSHACK_Flood.pcap	300,000	13.7	22.8
7	DDoS-RSTFINFlood.pcap	300,000	14.7	22.8
8	DDoS-SYN_Flood.pcap	300,000	21.2	22.9
9	DDoS-SlowLoris.pcap	300,000	1,409.8	89.4
10	DDoS-SynonymousIP_Flood.pcap	300,000	14.7	22.8
11	DDoS-TCP_Flood.pcap	300,000	15.5	23.0
12	DDoS-UDP_Flood.pcap	300,000	19.5	22.9
13	DDoS-UDP_Fragmentation.pcap	300,000	565.6	276.7
14	DoS-HTTP_Flood.pcap	300,000	219.7	258.7
15	DoS-SYN_Flood.pcap	300,000	27.7	23.7
16	DoS-TCP_Flood.pcap	300,000	22.8	23.1
17	DoS-UDP_Flood.pcap	300,000	27.6	23.2

Bảng 3.12. Bảng thông tin tập tin dữ liệu mạng PCAP đầu sử dụng để trích xuất và suy luận (tập tin thứ 1 đến 17)

STT	Tên file	Số gói tin	Thời lượng (s)	Kích thước (MB)
18	DictionaryBruteForce.pcap	133,138	254,640.0	39.1
19	DNS_Spoofing.pcap	300,000	13,333.2	154.5
20	MITM-ArpSpoofing.pcap	300,000	2,318.7	313.5
21	Recon-HostDiscovery.pcap	300,000	1,582.8	43.3
22	Recon-OSScan.pcap	300,000	5,087.9	109.5
23	Recon-Portscan.pcap	300,000	7,522.6	54.2
24	VulnerabilityScan.pcap	300,000	24,832.9	57.8
25	Backdoor_Malware.pcap	33,414	2,040.5	10.5
26	BrowserHijacking.pcap	59,821	1,560.0	35.0
27	CommandInjection.pcap	55,741	2,340.5	28.6
28	SqlInjection.pcap	53,462	3,119.7	9.8
29	Uploading_Attack.pcap	12,939	780.0	3.3
30	XSS.pcap	40,183	2,400.6	12.6
31	Mirai-greeth_flood.pcap	300,000	260.8	175.3
32	Mirai-greip_flood.pcap	300,000	145.2	174.3
33	Mirai-greip_flood.pcap	300,000	145.2	174.3
34	Mirai-udpplain.pcap	300,000	82.1	169.6

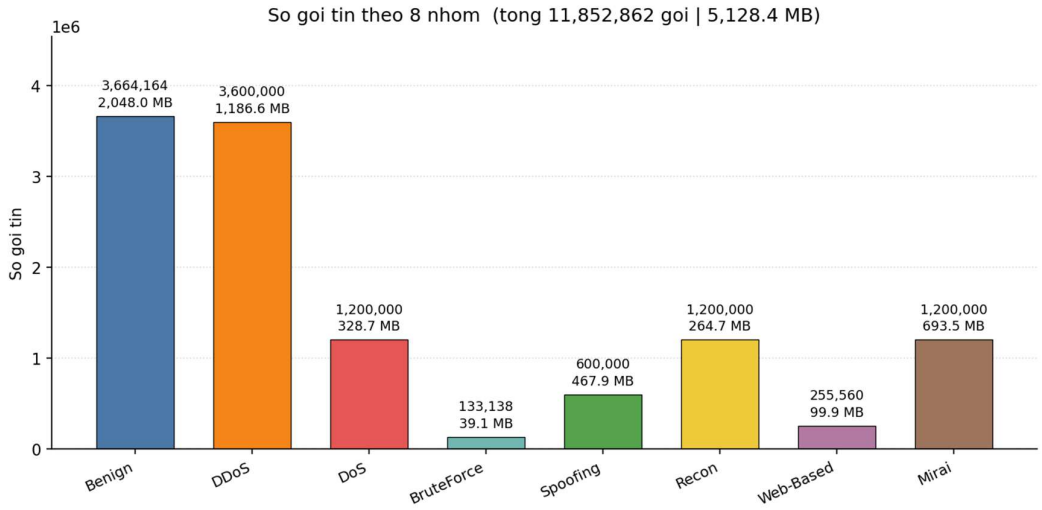
Bảng 3.13. Bảng thông tin tập tin dữ liệu mạng PCAP đầu sử dụng để trích xuất và suy luận (tập tin thứ 18 đến 34)

STT	Tên file	Nhãn	Tên nhãn	Số gói tin	Thời lượng (s)	Kích thước (MB)	Số lượng tập tin
1	Benign.pcap	0	Benign	3,664,164	33328	2,048.00	1
2	DDoS-ACK_Fragmentation.pcap	1	DDoS	3,600,000	582790.2	1,186.58	12
3	DDoS-HTTP_Flood.pcap						
4	DDoS-ICMP_Flood.pcap						
5	DDoS-ICMP_Fragmentation.pcap						
6	DDoS-PSHACK_Flood.pcap						
7	DDoS-RSTFINFlood.pcap						
8	DDoS-SYN_Flood.pcap						
9	DDoS-SlowLoris.pcap						
10	DDoS-SynonymousIP_Flood.pcap						
11	DDoS-TCP_Flood.pcap						
12	DDoS-UDP_Flood.pcap						
13	DDoS-UDP_Fragmentation.pcap						
14	DoS-HTTP_Flood.pcap	2	DoS	1,200,000	297.8	328.69	4
15	DoS-SYN_Flood.pcap						
16	DoS-TCP_Flood.pcap						
17	DoS-UDP_Flood.pcap						

Bảng 3.14. Bảng thông tin các tập tin dữ liệu mạng PCAP cho trích xuất và suy luận theo nhãn nhóm (từ tập tin thứ 1 đến 17)

STT	Tên file	Nhãn	Tên nhãn	Số gói tin	Thời lượng (s)	Kích thước (MB)	Số lượng tập tin
18	DictionaryBruteForce.pcap	3	BruteForce	133,138	254640	39.13	1
19	DNS_Spoofing.pcap	4	Spoofing	600,000	15651.9	467.92	2
20	MITM-ArpSpoofing.pcap						
21	Recon-HostDiscovery.pcap	5	Recon	1,200,000	39026.2	264.74	4
22	Recon-OSScan.pcap						
23	Recon-Portscan.pcap						
24	VulnerabilityScan.pcap						
25	Backdoor_Malware.pcap	6	Web-Based	255,560	12241.3	99.89	6
26	BrowserHijacking.pcap						
27	CommandInjection.pcap						
28	SqlInjection.pcap						
29	Uploading_Attack.pcap						
30	XSS.pcap						
31	Mirai-greeth_flood.pcap	7	Mirai	1,200,000	633.3	693.45	4
32	Mirai-greip_flood.pcap						
33	Mirai-greip_flood.pcap						
34	Mirai-udpplain.pcap						

Bảng 3.15. Bảng thông tin các tập tin dữ liệu mạng PCAP cho trích xuất và suy luận theo nhãn nhóm (từ tập tin thứ 18 đến 34)



Hình 3.4. Biểu đồ cột thể hiện số gói tin mỗi loại nhãn theo nhóm (8 nhóm)

Dữ liệu mạng PCAP tự thu thập thông qua chương trình thu thập dữ liệu mạng tcpdump từ nguồn là một máy tấn công Kali Linux. Các lưu lượng tấn công trực tuyến vào thiết bị IoT biên Raspberry Pi 4 B sẽ được thu thập dữ liệu, trích xuất đặc trưng và suy luận bằng mô hình. Lưu lượng sẽ được tạo thông qua các công cụ tấn công có sẵn trên Kali Linux [14].

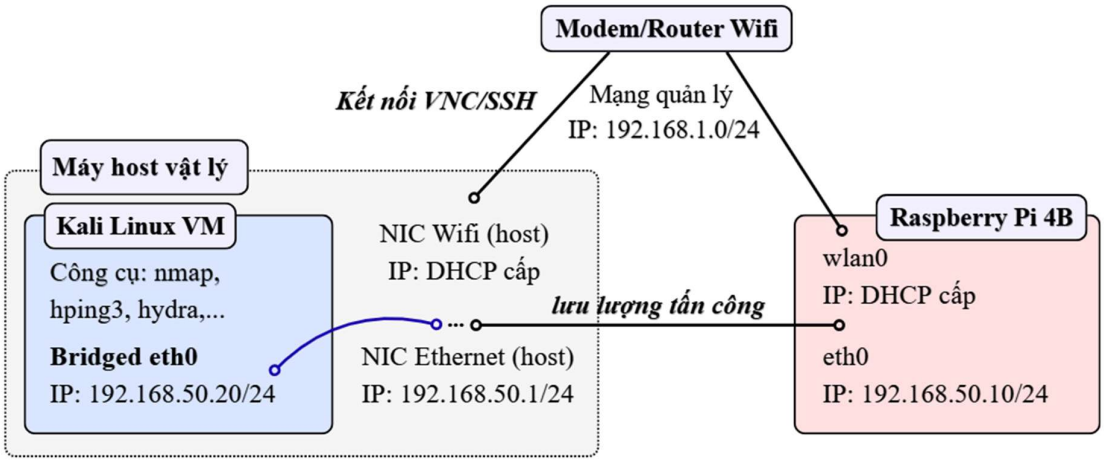
Nguồn dữ liệu này không phục vụ mục đích đánh giá độ chính xác, mà phục vụ kiểm thử tính khả thi triển khai đầu-cuối của toàn bộ hệ thống (thu thập - trích xuất – xử lý - suy luận) trong điều kiện gần với thực tế vận hành, nơi có sự tranh chấp tài nguyên giữa tiến trình bắt gói và tiến trình suy luận, cùng lưu lượng nằm ngoài phân bố của tập huấn luyện. Việc tách hai nguồn dữ liệu theo hai mục đích riêng biệt giúp quy trình đánh giá vừa đảm bảo tính định lượng, có thể đối chiếu (nhờ PCAP có nhãn), vừa đảm bảo tính thực tiễn, phản ánh điều kiện triển khai thật (nhờ PCAP tự thu thập). Các lưu lượng mạng sẽ được tạo bởi công cụ có sẵn trên Kali Linux, tham khảo bảng sau:

Công cụ	Lưu lượng
nmap	Recon
hydra	BruteForce
hping3	DDoS/DoS

Bảng 3.16. Công cụ của Kali Linux được sử dụng với lưu lượng tạo ra tương ứng

3.3.2. Mô tả môi trường triển khai quy trình đánh giá, so sánh

Các tập tin PCAP có nhãn tải từ trang chính chủ của UNB CIC sẽ được lưu trữ sẵn trên bộ nhớ thiết bị và được đọc ra, trích xuất đặc trưng trực tiếp mà không cần thu thập qua một công cụ nào khác. Còn khi triển khai suy luận trực tuyến sẽ xây dựng một môi trường mạng điểm-nối-điểm (point-to-point) giữa máy ảo Kali Linux (Virtual Box) trên máy tính Host vật lý và thiết bị biên IoT Raspberry 4 B thông qua một mạng cầu nối (bridged). Thiết bị Raspberry Pi 4 B sẽ được điều khiển bằng máy host vật lý thông qua một kết nối VNC (Virtual Network Computing). Sơ đồ mạng chi tiết như sau:



Hình 3.5. Minh họa sơ đồ mạng của môi trường triển khai quy trình

3.3.3. Các thành phần trong quy trình trích xuất đặc trưng, tiền xử lý và suy luận bằng các mô hình học sâu giáo viên và học sinh

Toàn bộ quy trình vận hành dựa vào 4 thành phần: thành phần thu thập dữ liệu mạng, thành phần trích xuất đặc trưng, tiền xử lý và suy luận bằng mô hình. Quy trình dùng chung một bộ trích xuất duy nhất cho cả nhánh đánh giá kết quả suy luận, nhánh đo hiệu năng trên Raspberry Pi 4 B và nhánh suy luận trực tuyến.

3.3.3.a. Thành phần thu thập dữ liệu mạng

Đối với trường hợp triển khai quy trình với dữ liệu mạng tự thu thập, không có nhãn ở mức cửa sổ, chỉ biết loại lưu lượng đã phát sinh từ phía tấn công. Nguồn này chỉ dùng để minh chứng hệ thống vận hành được trong điều kiện thực, không dùng để tính chỉ số chất lượng. Công cụ được sử dụng để thu thập là tcpdump, dữ liệu mạng sau thu thập được lưu trữ tại một thư mục dành riêng để chứa các tập tin PCAP tự thu thập ở dạng FIFO (First In First Out) hay là một đường ống (không lưu trữ lại tại đĩa), một đầu "in" nhận dữ liệu từ tiến trình thu thập của tcpdump và lập tức chuyển dữ

liệu đến tiến trình "out" đầu ra là đầu đọc dpkt. Đầu đọc sau đó chuyển dữ liệu đến bộ trích xuất.

Khi suy luận trên các tập tin PCAP có nhãn tải từ trang chính chủ của UNB CIC thì đã được lưu sẵn, chỉ cần dùng dpkt để đọc gói tin ra, không thu thập.

3.3.3.b. Bộ trích xuất đặc trưng từ dữ liệu mạng và tiền xử lý

Luận văn xây dựng một bộ trích xuất đặc trưng riêng hoàn toàn dựa trên bộ trích xuất từ mã nguồn trích xuất gốc mà UNB CIC đăng tải kèm với tập dữ liệu PCAP và CSV trên trang chính thức của mình. Thay đổi duy nhất đó là luận văn sẽ trích xuất ra 37 đặc trưng thay vì 39 đặc trưng như mã nguồn. Nguyên nhân chủ quan là do đặc trưng "Number" (số gói tin cho 1 cửa sổ) theo nhóm tác giả trích xuất và công bố [14] lựa chọn bằng 100 cho các gói tin có số lượng lớn mẫu thuộc các nhãn như DDoS, DoS và Mirai, các nhãn còn lại sử dụng kích thước bằng 10 để trích xuất, điều này không thực tế do khi dự đoán thật, mô hình hoàn toàn mù với nhãn, không biết loại lưu lượng nào đang đến để thay đổi kích thước cửa sổ. Việc loại bỏ đặc trưng "Number" dẫn đến cần thay đổi giá trị hoặc chuẩn hóa các đặc trưng khác bị ảnh hưởng bởi đặc trưng "Number". Cụ thể, đặc trưng "Tot sum" sẽ bị xóa đi cùng với number do "Tot_sum = Number * Tot_size" (có "number" ở trong công thức). Các đặc trưng số lượng đếm các cờ SYN/ACK/FIN/RST được đếm trên kích thước cửa sổ bằng "Number", nên phải chia các giá trị đếm SYN_count, ACK_count, FIN_count, RST_count cho "Number" đưa về dạng tỉ số (như một bước chuẩn hóa), như vậy mô hình chỉ học tỉ số các cờ trên kích thước cửa sổ thay vì số lượng cờ cứng. Các thay đổi này đã được áp dụng tiền xử lý cho tập dữ liệu CSV đầu vào trước khi đưa vào huấn luyện các mô hình. Bộ trích xuất ra 37 đặc trưng cũng áp dụng các thay đổi tương tự để kết quả được thống nhất.

> Luận văn cũng đã thực nghiệm huấn luyện, trích xuất và suy luận bằng 2 mô hình giáo viên trên tập dữ liệu CSV 37 đặc trưng và tập CSV 39 đặc trưng để so sách, nhằm chứng minh tính khả thi triển khai của tập dữ liệu 37 đặc trưng và bộ trích xuất 37 đặc trưng. Kết quả minh chứng xem ở phụ lục.

Quy trình trích xuất theo mô hình cửa sổ trượt không chồng lấp, bám theo mã nguồn của tác giả:

Bước 1: Xử lý từng gói tin:

- Lọc loại gói tin: hệ thống chỉ giữ lại gói tin thuộc hai loại IP và ARP; các gói tin khác bị loại ngay ở bước này. Đồng thời ghi nhận thời điểm

và kích thước: lưu lại thời điểm gói tin được bắt và độ dài toàn bộ khung Ethernet của gói tin (tính theo byte).

- Tính khoảng cách thời gian với gói trước (IAT): lấy thời điểm gói hiện tại trừ đi thời điểm của gói ngay trước đó, tính liên tục trong suốt cả file, không tính riêng theo từng cửa sổ 10 gói. Gói đầu tiên của file được quy ước IAT = 0 vì chưa có gói nào trước để so sánh.
- Đọc các trường cơ bản trong header IP: loại giao thức đang dùng (TCP/UDP/ICMP/IGMP) và thời gian sống của gói tin (TTL).
- Xác định giao thức vận chuyển và các cờ đi kèm: Nếu là TCP: đọc độ dài phần header, và giải mã 8 cờ điều khiển (FIN, SYN, RST, PSH, ACK, URG, ECE, CWR) trực tiếp từ byte cờ của gói tin. Nếu là UDP: gán độ dài header cố định 8 byte (theo đúng chuẩn giao thức UDP).
- Nhận diện giao thức tầng ứng dụng qua cổng (port): dựa vào cổng nguồn và cổng đích để đánh dấu gói tin có thuộc dịch vụ phổ biến hay không. Mỗi dịch vụ được đánh dấu 1 nếu khớp cổng, đánh dấu là 0 nếu không khớp.
- Kết thúc bước 1, mỗi gói tin trở thành một bản ghi đầy đủ các giá trị trên.

Bước 2: Gộp 10 gói tin thành 1 cửa sổ:

- Cứ đủ 10 gói tin liên tiếp trong cùng một file (không trộn gói giữa các file, không đảo thứ tự), hệ thống gộp lại thành 1 cửa sổ và tính ra 37 giá trị:

  header_length = trung bình cộng độ dài header của 10 gói.

  time_to_live = trung bình cộng TTL của 10 gói.

  rate (tốc độ gói tin) = lấy 10 (số gói trong cửa sổ) chia cho khoảng thời gian từ gói đầu đến gói cuối của cửa sổ đó.

- Nhóm thống kê độ dài gói tin: giá trị nhỏ nhất, lớn nhất, trung bình, độ lệch chuẩn và phương sai của 10 gói trong cửa sổ.

  Lưu được *min, max, avg, std, variance, tot_size*.

  iat = trung bình cộng khoảng cách thời gian (đã tính ở bước 1) của 10 gói trong cửa sổ.

- Loại giao thức đại diện cho cả cửa sổ: chọn giá trị giao thức xuất hiện nhiều lần nhất trong 10 gói.

  protocol_type = giá trị giao thức xuất hiện nhiều lần nhất trong 10 gói.

- Các cờ TCP và cờ giao thức ứng dụng: lấy tỉ lệ số gói mang cờ đó trên tổng 10 gói.

  7 cờ TCP: với mỗi cờ (FIN, SYN, RST, PSH, ACK, ECE, CWR), lấy tổng giá trị cờ đó chia 10 gói. Lưu được: *fin_flag_number, syn_flag_number, rst_flag_number, psh_flag_number, ack_flag_number, ece_flag_number, cwr_flag_number.*

  10 cờ giao thức/dịch vụ: với mỗi cờ (HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP), lấy tỉ lệ số gói mang cờ đó trên tổng 10 gói. Lưu được: *http, https, dns, telnet, smtp, ssh, irc, tcp, udp, dhcp.*

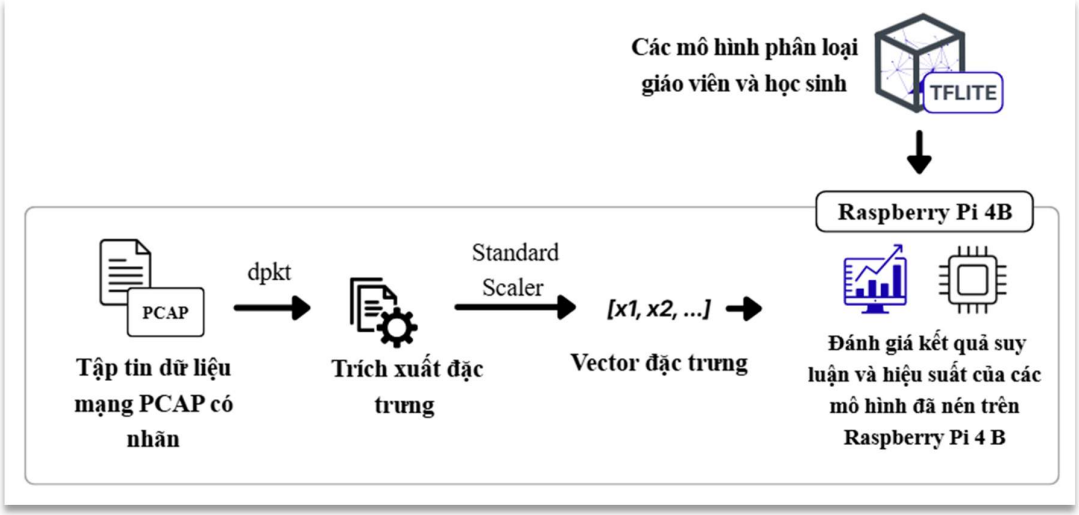
  5 cờ nhận diện còn lại: với mỗi cờ (ARP, ICMP, IGMP, IPv, LLC), lấy tỉ lệ số gói mang cờ đó trên tổng 10 gói. Lưu được: *arp, icmp, igmp, ipv, llc.*

  4 cờ ACK, SYN, FIN, RST ngoài lấy tổng giá trị, lấy số lượng chia 10 gói. Lưu được: *ack_count_ratio, syn_count_ratio, fin_count_ratio, rst_count_ratio.* Đây là các giá trị được đề tài đề xuất thay đổi so với các đặc trưng gốc là *ack_count, syn_count, fin_count, rst_count.*

- Toàn bộ 37 giá trị được sắp xếp đúng theo thứ tự cột đã dùng lúc huấn luyện mô hình, để mô hình luôn hiểu đúng ý nghĩa từng cột dù chạy trên máy chủ hay trên thiết bị biên.

Bước 3: Tiền xử lý đặc trưng: cửa sổ sau trích xuất ra đủ 37 đặc trưng được áp dụng phép chuẩn hóa đặc trưng StandardScaler có cùng (thang đo) scale tính ra từ tập dữ liệu huấn luyện trên Kaggle để đảm bảo vec-tơ đặc trưng sau trích xuất không bị lệch chuẩn.

3.3.4. Quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh sau chưng cất tri thức trên PCAP có nhãn trên thiết bị



Hình 3.6. Ảnh minh họa quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh trên dữ liệu PCAP có nhãn trên thiết bị

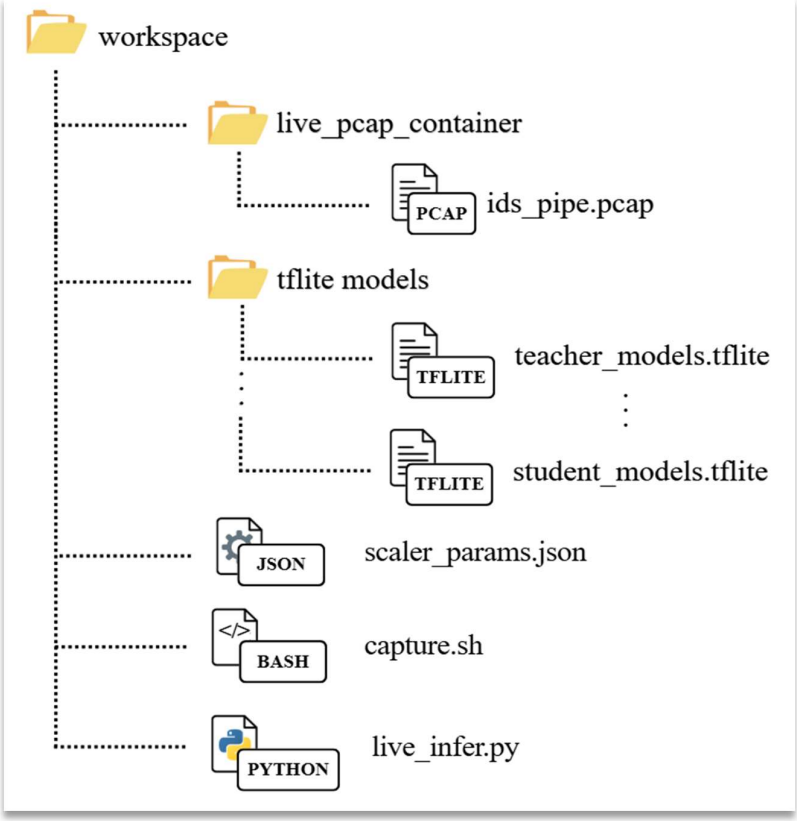
Các mô hình trước tiên hết cần được chuyển đổi về định dạng TFLITE, do tín gọn nhẹ của thư viện Tensorflow Lite phù hợp hơn trên môi trường thiết bị có tài nguyên hạn chế như Raspberry Pi 4 B [19]. Các file pcap có nhãn sau khi lưu trữ sẽ được đọc lần lượt qua bằng dpkt, trích xuất đặc trưng bằng bộ trích xuất, sau đó tiền xử lý rồi được suy luận lần lượt bằng tất cả các mô hình. 300 gói tin đầu sẽ không được dự đoán mà để làm nóng hệ thống trước khi đo lường thật. Đây là bước cần thiết để tránh sự việc khi các gói tin đầu tiên tới mô các thành phần, thư viện chỉ vừa được khởi động hoặc nạp nên hiệu quả chưa đạt chuẩn để bắt đầu suy luận và đo lường thật [20].

Kết quả suy luận thu thập trực tiếp tại Pi 4 B bao gồm có độ chính xác tổng thể (overall accuracy), macro-F1 (overall) và macro-AUC (overall), về hiệu suất sẽ thu thập các chỉ số tốc độ thực thi - exec rate (cửa sổ/giây), inference time (ms) từng model, độ trễ dự đoán của cá nhân một mô hình – model latency (ms), độ trễ quy trình từ lúc đọc gói tin cho đến lúc đưa ra dự đoán – pipeline latency (ms). Ngoài ra, đo chiếm dụng tài nguyên, đối với tài nguyên được tính toán bằng cách cứ mỗi 0.5s sẽ ghi nhận lại kết thông tin chiếm dụng CPU và RAM của chương trình python, dpkt được chạy để suy luận, các kết quả được ghi nhận này sẽ được sử dụng để tính ra chiếm dụng CPU trung bình và đỉnh (AVG/Peak CPU usages), chiếm dụng dung lượng RAM trung bình và chạm đỉnh (AVG/Peak RAM usages).

Các chỉ số sau khi được thu thập sẽ được tổng kết, so sánh giữa các mô hình học sinh với nhau và giữa học sinh với giáo viên để đưa ra nhận xét, đánh giá. Không thể bỏ qua so sánh giữa kết quả suy luận trên thiết bị với kết quả huấn luyện để đánh giá khả năng triển khai của các mô hình học sinh chưng cất tri thức trên môi trường thiết bị IoT biên có tài nguyên hạn chế.

3.3.4.a. Quy trình thử nghiệm suy luận mô hình trên dữ liệu mạng PCAP tự thu thập tại thiết bị

Cấu trúc thư mục của quy trình thử nghiệm được thiết lập như sau ngay trên thiết bị biên Raspberry Pi 4 B.



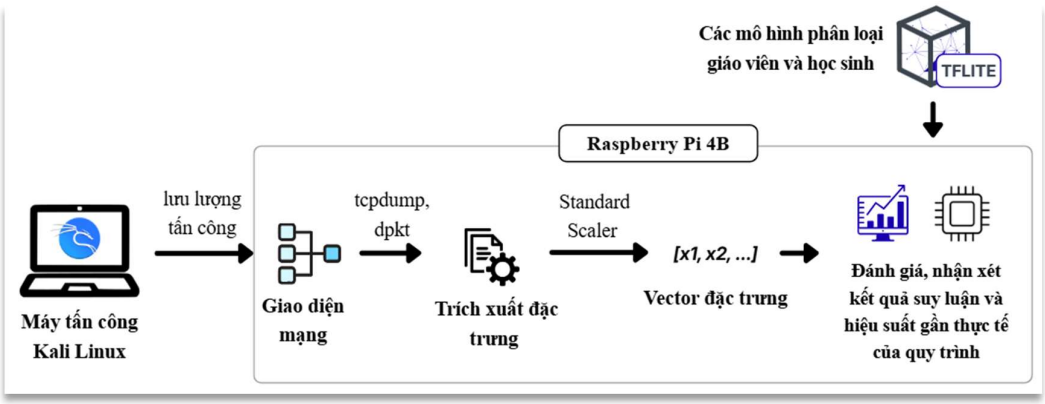
Hình 3.7. Cấu trúc cây thư mục của quy trình suy luận trên dữ liệu mạng tự thu thập

Quy trình triển khai cần khởi động 2 chương trình chính:

Chương trình capture.sh để bắt đầu mở công cụ tcpdump để thu dữ liệu mạng PCAP lưu vào thư mục live_pcap_container với tên file là ids_pipe.pcap (tên cố định và sau khi chạy xong sẽ bị xóa) dữ liệu bên trong ids_pipe.pcap không được lưu trữ mà lập tức gửi thẳng đến live_infer.py để đọc và bắt đầu trích xuất đặc trưng. Chương trình này nên mở sau khi mở giao diện chương trình live_infer.py.

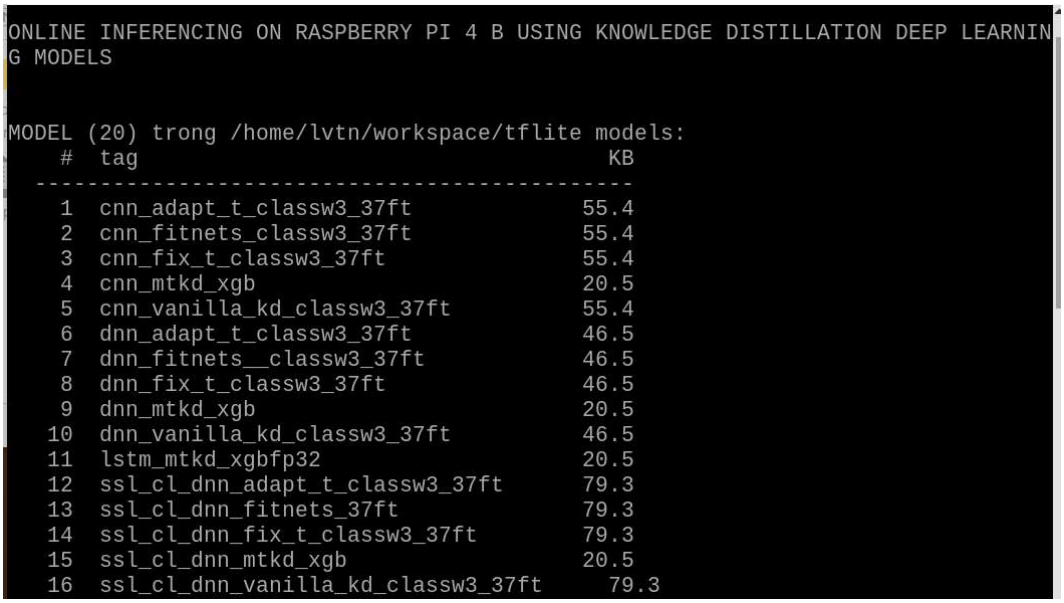
Chương trình live_infer.py thực hiện đọc dữ liệu mạng đến từ ids_pipe.pcap, mỗi gói tin được lần lượt đọc bằng dpkt, thực hiện trích xuất và tích lũy thông tin đặc trưng đến khi đủ 10 gói tin thì gom các thông tin lại thành một véc-tơ đặc trưng chưa xử lý (mỗi gói tin sau khi được đọc bằng dpkt sẽ bị xóa trực tiếp ra khỏi ids_pipe.pcap, tiền xử lý sử dụng scaler_params.json (tập tin lưu tham số chuẩn hóa để áp dụng cho véc-tơ đặc trưng sau trích xuất) và thực hiện dự đoán sử dụng các mô hình giáo viên và học sinh được lưu trữ sẵn trong thư mục tflite models.

Để đảm kết quả thực nghiệm được thu thập hiệu quả thì chỉ chạy với một mô hình cho một lần chạy để đảm bảo, các mô được lựa chọn thử ngiệm cân nhắc dựa vào kết quả suy luận của các mô hình trên dữ liệu PCAP có nhãn. Các mô hình có accuracy overall cao nhất, macro-F1 cao nhất hoặc mô hình có tốc độ thực thi tốt nhất là những mô hình lý tưởng hơn để chọn và chạy thực nghiệm.

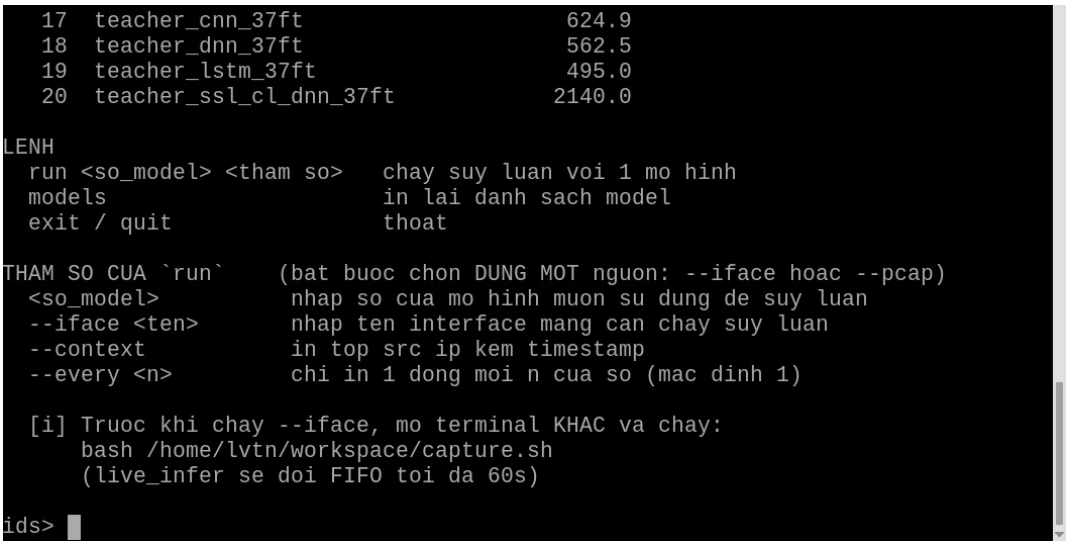


Hình 3.8. Ảnh minh họa quy trình thử nghiệm suy luận mô hình trên dữ liệu mạng PCAP tự thu thập tại thiết bị

Giao diện của live_infer.py khi khởi động, gồm có banner, danh sách tên các mô hình lựa chọn khi chạy triển khai thật, danh sách lệnh và tham số chọn để chạy kèm.



Hình 3.9. Giao diện chương trình live_infer.py khi khởi động (phần đầu)



Hình 3.10. Giao diện chương trình live_infer.py khi khởi động (phần cuối)

Để chạy cần chọn 1 trong 3 lệnh:

Lệnh	Chức năng
run <số mô hình> <tham số>	Chạy suy luận với 1 mô hình
models	In lại danh sách mô hình
exit/quit	Thoát

Bảng 3.17. Danh sách lệnh cơ bản của live_infer.py

Tham số cho "run"	Chức năng
<số mô hình>	Số thứ tự của mô hình muốn sử dụng để suy luận (bắt buộc phải có)
--iface	Giao diện mạng cần suy luận, dự đoán tấn công (bắt buộc phải có)
--context	In ngữ cảnh tấn công bao gồm mốc thời gian, và top IP nguồn
--every <n>	In kết quả 1 dòng mỗi n dự đoán

Bảng 3.18. Danh sách tham số truyền cho lệnh 'run' khi bắt đầu suy luận

Khi chạy suy luận bằng 'run' bắt buộc cần truyền 2 tham số: là lựa chọn mô hình và lựa chọn giao diện mạng cần dự đoán tấn công. Ngoài ra có thêm 2 tham số bổ sung là –context để in thêm ngữ cảnh tấn công bao gồm mốc thời gian và thứ tự giảm dần số lượng các IP nguồn đến thiết bị. Có thêm lựa chọn in kết quả 1 dòng mỗi n dự đoán để tiện theo dõi, đồng thời làm giảm nhẹ khối lượng công việc cho chương trình khi thiết bị có quá nhiều lưu lượng đến/bị làm ngập vì bản thân việc in kết quả liên tục cũng có ảnh hưởng đến hiệu suất của quy trình.

Sau khi mô hình suy luận, các kết quả hiển thị (output) trên giao diện cửa sổ terminal sẽ bao gồm có các thông tin lần lượt là:

[Số thứ tự cửa sổ/ dự đoán] | [xác xuất dự đoán nhãn 0] … [xác xuất dự đoán nhãn 7] | [Kết quả dự đoán] [tốc độ suy luận] [độ trễ từ lúc đọc tới lúc suy luận] [độ trễ từ lúc thu thập tới lúc suy luận]

Nếu quy trình kiểm tra trên PCAP có nhãn tập trung vào kết quả suy luận, hiệu suất và chiếm dụng tài nguyên để đánh giá khả năng triển khai của mô hình học sinh sau chưng cất tri thức, thì kết quả thực nghiệm của việc suy luận trên dữ liệu mạng PCAP đánh giá khả năng/ tính khả thi vận hành thực tế của cả quy trình thay vì chỉ mô hình phân loại.

Mục đích chính là kiểm tra tính toàn vẹn kiến trúc xem việc chạy quả quy trình đầy đủ từ bước thu thập đến bước suy luận và in kết quả có làm sập, treo hệ thống hay làm rớt gói,... (điều này không thể thấy được ở đọc file tĩnh). Đồng thời đo được chi phí hiệu năng phải trả khi triển khai (đôi khi con số này không lý tưởng như mình nghĩ). Quan trọng là bước thử nghiệm này phản ứng với lưu lượng có thể nằm ngoài tập dữ liệu và là phép thử thực tế hơn bước đánh giá kết quả suy luận bằng PCAP có sẵn nhãn.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

4.1. Cấu hình của môi trường huấn luyện và thiết bị IoT biên Rasberry Pi 4 B



Hình 4.1. Ảnh thiết bị Raspberry Pi 4 B

Môi trường	Cấu hình	Thông tin chi tiết cấu hình
Kaggle	OS	Linux 6.12.90+ (x86_64)
	CPU	Intel(R) Xeon(R) CPU @ 2.20GHz
	Số nhân	4
	RAM	33.66 GB
	Disk	214.75 GB
	GPU	2x NVIDIA Tesla T4
	VRAM	32 GB GDDR6 (16 GB cho mỗi GPU)
Raspberry Pi 4 Model B	OS	Linux 6.18.34+rpt-rpi-v8 (aarch64)
	CPU	Broadcom BCM2711 (Cortex-A72 @ 1.8GHz)
	Số nhân	4
	RAM	4GB
	Disk	59GB (MicroSSD)

Bảng 4.1. Cấu hình của môi trường huấn luyện Kaggle và thiết bị Raspberry Pi 4 B

Luận văn sử dụng môi trường huấn luyện của Kaggle để huấn luyện các mô hình học sâu do Kaggle cung cấp một nguồn tài nguyên vô cùng mạnh mẽ và hoàn toàn miễn phí sử dụng. Thiết bị Raspberry Pi 4 B với tài nguyên vô cùng hạn chế: dung lượng RAM chỉ bằng xấp xỉ 1/11 lần so với dung lượng RAM của Kaggle cung cấp, Raspberry Pi 4 B không có GPU, đồng thời dung lượng lưu trữ cũng ít hơn. Với

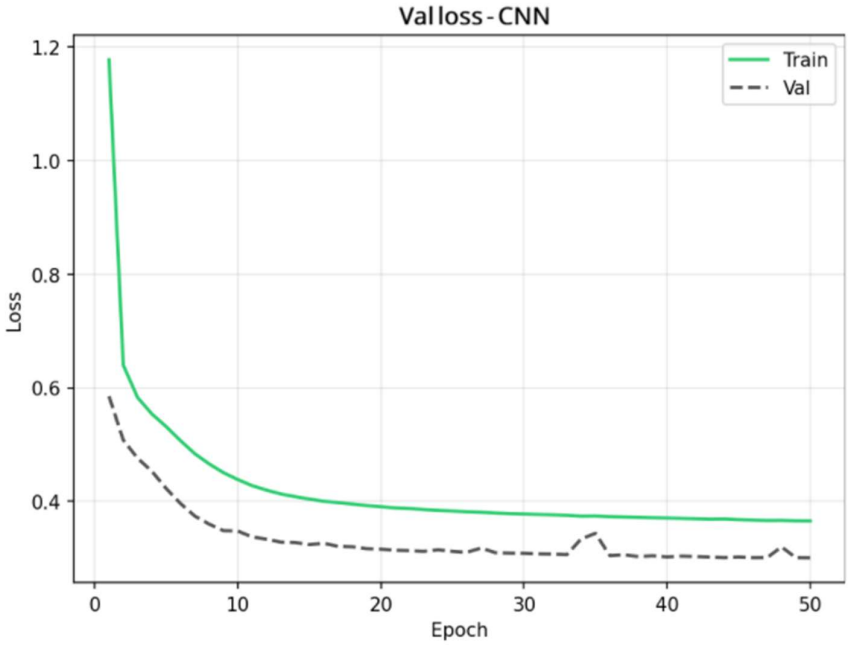
một môi trường thiết bị IoT biên có hiệu năng và tốc độ xử lý hạn chế, việc tối ưu hóa và vận hành thành công các mô hình học sâu gọn nhẹ chứng minh tính ứng dụng mạnh mẽ và tiềm năng triển khai thực tế của nghiên cứu.

4.2. Các kết quả huấn luyện các mô hình học sâu giáo viên và học sinh sau khi thực hiện các kỹ thuật chưng cất tri thức

Về sau các kết quả bảng đánh giá và biểu đồ sẽ sử dụng tên kĩ thuật: KD cho chưng cất tri thức truyền thống, Raid-KL (adapt t) cho chưng cất tri thức bằng phân kỳ JS-KL khi nhiệt độ thích ứng, Raid-KL (fix t) cho chưng cất tri thức bằng phân kỳ JS-KL khi nhiệt độ cố định và FitNets cho cho chưng cất tri thức bằng gợi ý tầng ẩn.

4.2.1. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức truyền thống

4.2.1.a. Kết quả chưng cất tri thức truyền thống trên mô hình CNN



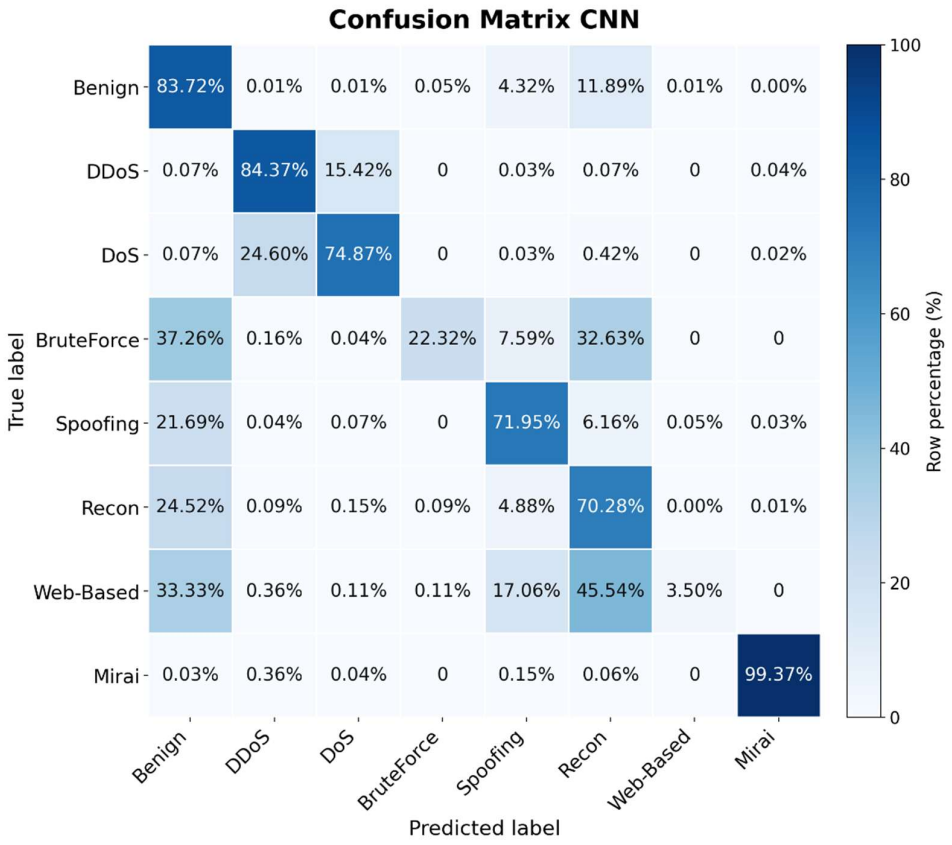
Hình 4.2. Biểu đồ đường mất mát trong quá trình huấn luyện CNN KD

Đường mất mát huấn luyện giảm rất mạnh trong khoảng mười vòng đầu, từ khoảng 1.18 xuống dưới 0.45, sau đó đi ngang và ổn định quanh 0.36 từ vòng thứ ba mươi trở đi. Đường mất mát trên tập kiểm định nằm thấp hơn đường huấn luyện trong toàn bộ quá trình huấn luyện chưng cất.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
CNN (teacher)	84.67	83.52	64.34	67.87	97.65	77.44	1,918
CNN KD	83.31	82.55	61.39	64.33	97.19	10.35	45.91

Bảng 4.2. Kết quả huấn luyện mô hình học sinh CNN KD so với giáo viên

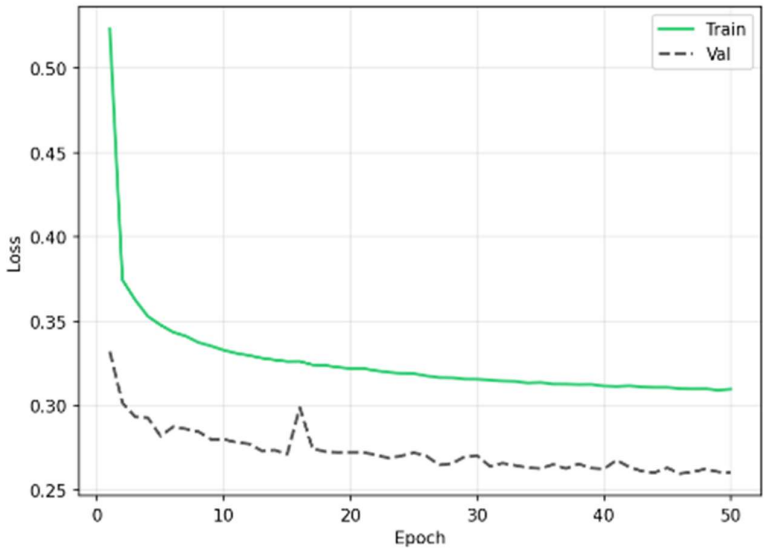
Mô hình học sinh CNN sau chưng cất truyền thống đạt 83.31% accuracy và 64.33% macro-F1, tương ứng giữ lại 98.4% accuracy và 94.8% macro-F1 của giáo viên trong khi số tham số chỉ còn 11,752 so với 158,600 và dung lượng lưu trữ giảm từ 1,918 KB xuống 45.91 KB. Accuracy gần như được bảo toàn (chỉ mất 1.36) còn macro-F1 mất 3.54 điểm phần trăm, cho thấy phần tri thức bị hao hụt trong quá trình nén tập trung ở các lớp thiểu số chứ không phân bố đều trên toàn bộ 8 nhãn. Chỉ số macro AUC chỉ giảm 0.46 điểm phần trăm, từ 97.65% xuống 97.19%. Đây là dấu hiệu cho thấy năng lực xếp hạng xác suất của học sinh vẫn bám sát giáo viên. Về thời gian suy luận, học sinh CNN đạt 10.35 µs mỗi mẫu so với 77.44 µs của giáo viên, nhanh hơn khoảng hơn 7 lần. Mức rút ngắn này bám khá sát tỉ lệ giảm số tham số, cho thấy phần chi phí tiết kiệm được đến trực tiếp từ việc thu hẹp chiều rộng các tầng chứ không phải từ một tối ưu phụ nào khác. Cần lưu ý các con số này được đo trên môi trường Kaggle nên chỉ phản ánh chi phí tương đối giữa hai kiến trúc, chưa phải chi phí thực tế tại thiết bị biên.



Hình 4.3. Ma trận nhầm lẫn của CNN KD

Ma trận nhầm lẫn cho thấy mô hình phân loại rất tốt trên các nhãn có số mẫu lớn: Mirai đạt 99.37%, DDoS 84.37%, Benign 83.72%, DoS 74.87%. Nhầm lẫn tập trung ở ba vùng. DDoS và DoS, hai loại lưu lượng có bản chất kỹ thuật gần như trùng nhau. Vùng thứ hai là nhãn BruteForce với recall chỉ 22.32%, phần lớn mẫu bị đẩy sang Benign và Recon. Vùng thứ ba là nhãn Web-Based với 3.50%, gần như bị hòa tan hoàn toàn vào Recon và Benign. Hai nhãn cuối cũng chính là hai nhãn hiếm nhất trong tập dữ liệu và là nguyên nhân ảnh hưởng macro-F1.

4.2.1.b. Kết quả chưng cất tri thức truyền thống trên mô hình DNN



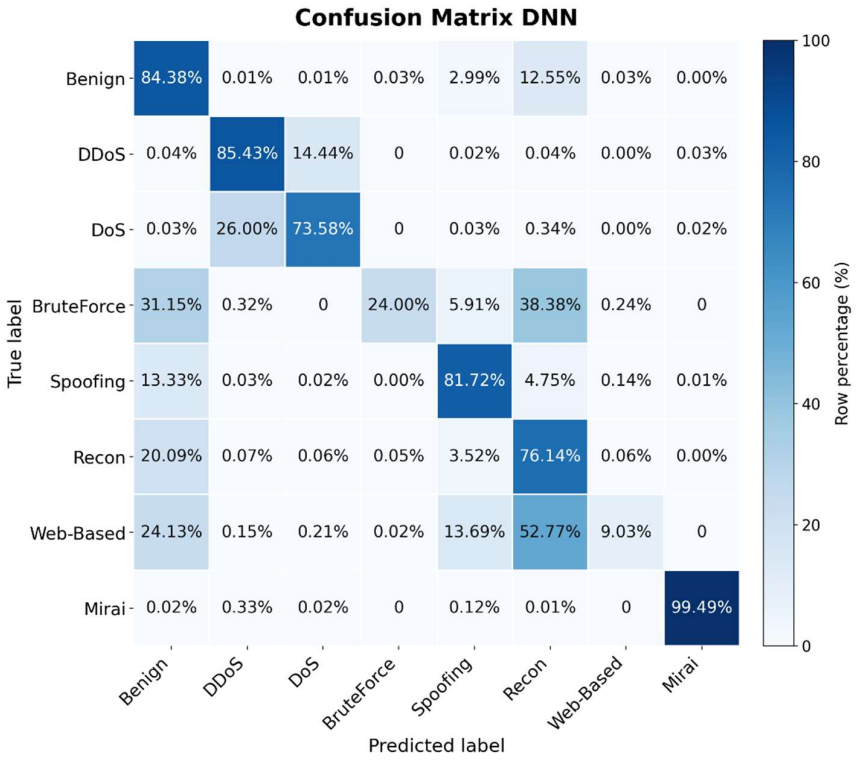
Hình 4.4. Biểu đồ đường mất mát quá trình huấn luyện DNN KD

Quá trình huấn luyện hội tụ nhanh hơn so với CNN. Mất mát huấn luyện giảm từ 0.52 xuống dưới 0.37 chỉ sau năm vòng đầu và ổn định quanh 0.31 từ vòng thứ hai mươi. Đường kiểm định giảm về khoảng 0.26 với một vài vòng bất thường tại trước vòng 20. Sau đó tiếp tục xu hướng giảm nhẹ và phẳng dần đến hết.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
DNN (teacher)	85.71	82.89	67.79	71.43	98.04	17.61	1,761
DNN KD	84.3	81.96	63.89	66.93	97.49	2.46	45.53

Bảng 4.3. Kết quả huấn luyện mô hình học sinh DNN KD so với giáo viên

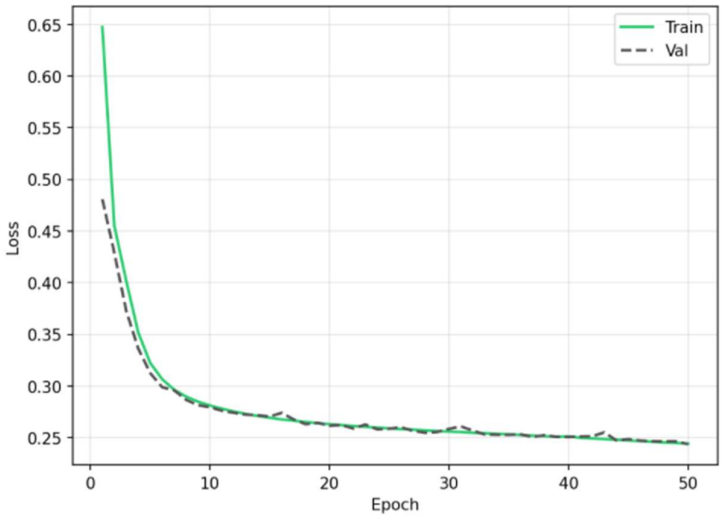
Học sinh DNN đạt 84.3% accuracy và 66.93% macro-F1, giữ lại 98.4% accuracy và 93.7% macro-F1 của giáo viên với 11,656 tham số so với 144,904, tương ứng mức nén 92.0% về số tham số và 97.4% về dung lượng. Cấu trúc hao hụt lặp lại đúng như ở CNN, ở macro precision gần như bảo toàn trong khi macro recall giảm 3.9 điểm phần trăm từ 67.79% xuống 63.89%, và macro AUC chỉ giảm 0.55 điểm phần trăm. Học sinh DNN có thời gian suy luận 2.46 µs mỗi mẫu, nhanh hơn giáo viên khoảng 7 lần và thấp hơn CNN, bản thân giáo viên DNN cũng chỉ tốn 17.61 µs, hơn CNN giáo viên, cho thấy kiến trúc toàn kết nối vốn đã rẻ về chi phí tính toán ngay cả trước khi được nén khi so với tích chập.



Hình 4.5. Ma trận nhầm lẫn DNN KD

Phân bố nhầm lẫn tương tự CNN nhưng ở mức nhẹ hơn trên các lớp thiểu số. Recall của BruteForce tăng lên 24.00% và Web-Based lên 9.03%, Spoofing đạt 81.72% và Recon đạt 76.14%, đều cao hơn đáng kể so với mô hình học sinh CNN tương ứng. Nhầm lẫn giữa DDoS và DoS vẫn là vùng lỗi lớn nhất và gần như không thay đổi về bản chất, cho thấy đây là giới hạn của biểu diễn đặc trưng đầu vào chứ không phải giới hạn của kiến trúc hay kỹ thuật chưng cất.

4.2.1.c. Kết quả chưng cất tri thức truyền thống trên mô hình SSL-CL-DNN



Hình 4.6. Biểu đồ đường mất mát quá trình huấn luyện SSL-CL-DNN KD

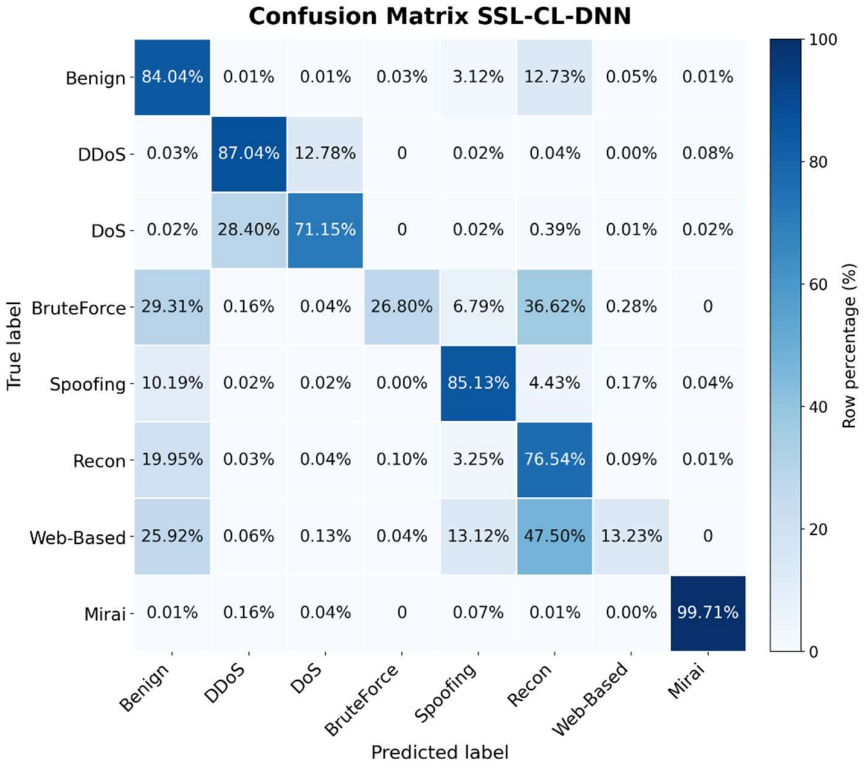
Đây là đồ thị có dạng hội tụ tốt nhất trong ba mô hình. Hai đường huấn luyện và kiểm định gần như đang xếp chồng lên nhau, cùng giảm từ khoảng 0.65 xuống 0.245 và phẳng dần từ vòng thứ ba mươi. Việc hai đường không tách rời cho thấy mô hình không quá khớp, cũng không chịu ảnh hưởng đáng kể từ chênh lệch chế độ huấn luyện và chế độ đánh giá như hai trường hợp trước, do học sinh SSL-CL-DNN chỉ được chưng cất ở phần tương đương giai đoạn tinh chỉnh có giám sát, khi Encoder đã mang sẵn biểu diễn học được từ giai đoạn tiền huấn luyện tương phản của giáo viên. Điểm khởi đầu của biểu diễn đầu vào tốt hơn giúp chưng cất dễ hơn và quỹ đạo hội tụ mượt hơn.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN (teacher)	85.58	82.80	67.12	70.93	97.97	84.38	2,137
SSL-CL-DNN KD	85.07	82.20	65.64	69.02	97.80	3.36	76.66

Bảng 4.4. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN KD so với giáo viên

Học sinh SSL-CL-DNN đạt 85.07% accuracy và 69.02% macro-F1, giữ lại 99.4% accuracy và 97.3% macro-F1 của giáo viên. Đây là tỉ lệ bảo toàn tri thức cao nhất trong mô hình KD, đạt được đồng thời với mức nén mạnh nhất: 19,624 tham số so với 547,080, tương ứng giảm 96.4%. Macro AUC gần như không suy giảm. Kết

quả này củng cố nhận định rút ra từ đồ thị mất mát. Khi học sinh được khởi tạo trên một kiến trúc đã có sẵn cơ chế học biểu diễn mạnh, quá trình chưng cất chỉ cần đảm nhiệm việc truyền tri thức phân loại chứ không phải đồng thời gánh cả việc học biểu diễn từ đầu, nên tổn thất do nén giảm rõ rệt. Đổi lại, mô hình này có dung lượng lớn nhất trong ba học sinh (76.66 KB, 19,624 tham số) và thời gian suy luận 3.36 µs mỗi mẫu, cao hơn học sinh DNN nhưng vẫn rất thấp. Đáng chú ý là mức rút ngắn so với giáo viên ở đây lớn nhất trong ba kiến trúc, từ 84.38 µs xuống 3.36 µs, tương ứng nhanh hơn khoảng 25 lần, hệ quả của việc giáo viên SSL-CL-DNN là mô hình đồ sộ nhất trong nhóm học sâu còn học sinh lại bị cắt bớt cả một tầng Encoder ngoài việc thu hẹp chiều rộng.



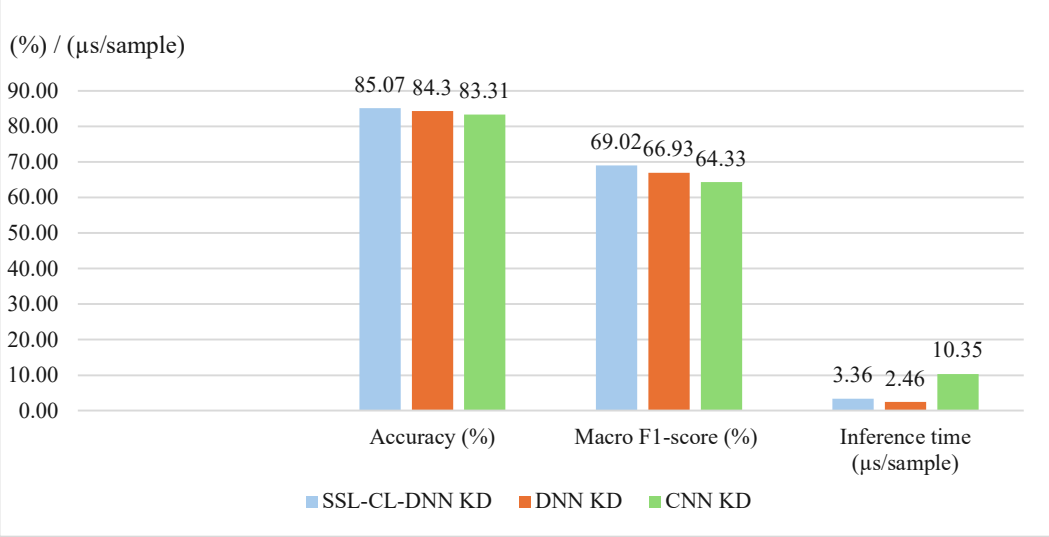
Hình 4.7. Ma trận nhầm lẫn của SSL-CL-DNN KD

Ma trận nhầm lẫn cho thấy cải thiện nhất quán trên nhóm lớp khó so với hai mô hình trước: BruteForce đạt 26.80%, Web-Based đạt 13.23%, Spoofing đạt 85.13%, Recon đạt 76.54%, Mirai đạt 99.71%. Các vùng nhầm lẫn chủ đạo vẫn giữ nguyên cấu trúc là DoS lệch sang DDoS, Web-Based lệch sang Recon, BruteForce lệch sang Recon và Benign. Ưu thế của kiến trúc SSL-CL-DNN đến từ việc phân biệt tốt hơn trong nhóm lớp hiếm.

4.2.1.d. Nhận xét chung kết quả các mô hình bằng KD truyền thống

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN KD	**85.07**	82.20	**65.64**	**69.02**	**97.80**	3.36	76.66
DNN KD	84.3	81.96	63.89	66.93	97.49	**2.46**	45.91
CNN KD	83.31	**82.55**	61.39	64.33	97.19	10.35	45.53

Bảng 4.5. Bảng kết quả huấn luyện các mô hình học sinh bằng chưng cất tri thức truyền thống

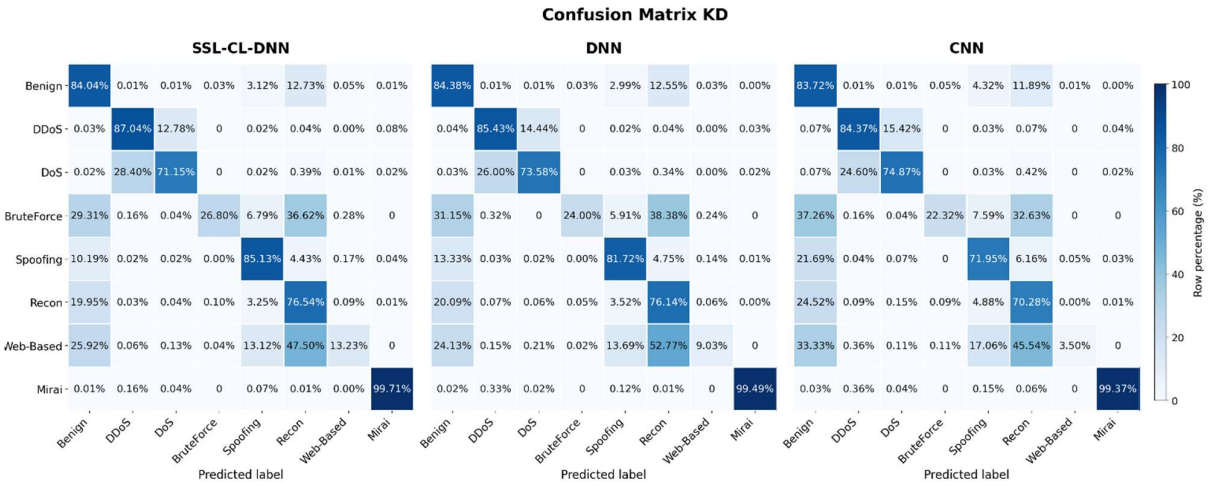


Hình 4.8. Biểu đồ cột Accuracy, Macro-F1 và inference time của các mô hình học sinh KD

Khi cùng áp dụng một kỹ thuật chưng cất duy nhất lên ba kiến trúc học sinh, thứ tự hiệu quả phân loại được xác lập rõ ràng và nhất quán trên cả bốn chỉ số: SSL-CL-DNN (85.07% accuracy, 69.02% macro-F1, 97.80% macro AUC) đứng đầu, DNN (84.3%, 66.93%, 97.49%) đứng thứ hai, CNN đứng cuối. Khoảng cách giữa mô hình tốt nhất và kém nhất là 1.76 điểm accuracy và 4.69 điểm macro-F1. Chênh lệch ở macro-F1 lớn gấp gần ba lần chênh lệch ở accuracy khẳng định rằng khác biệt nhiều giữa các kiến trúc nằm ở năng lực xử lý các lớp ít mẫu.

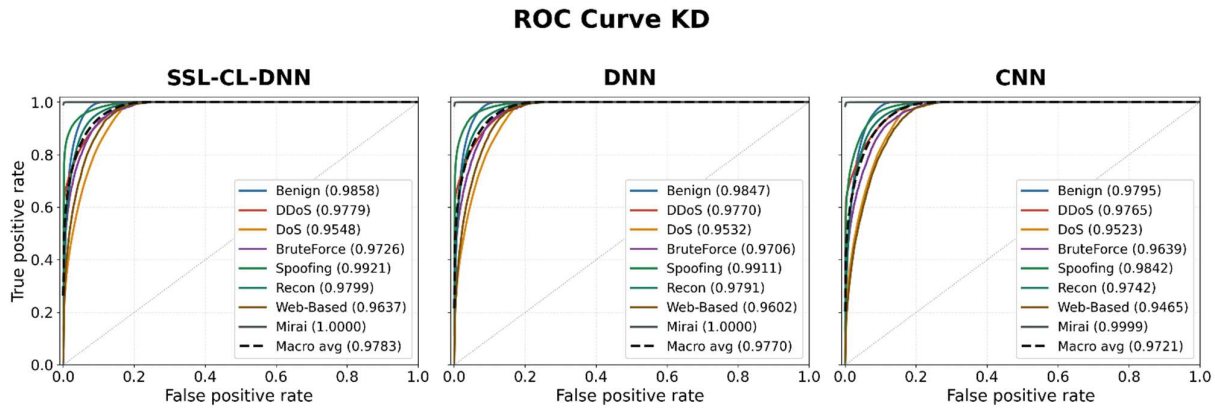
Đổi lại ưu thế về hiệu quả, mô hình SSL-CL-DNN có dung lượng lớn nhất (76.66 KB so với 45.53 KB của DNN) và thời gian suy luận 3.36 µs so với 2.46 µs của DNN. Điểm bất lợi rõ nhất lại thuộc về CNN: với số tham số gần như bằng DNN, học sinh CNN vẫn tốn 10.35 µs mỗi mẫu, chậm hơn khoảng bốn lần. Nguyên

nhân nằm ở bản chất của phép tích chập, khi cùng một bộ lọc được áp dụng lặp lại trên nhiều vị trí của chuỗi đặc trưng, nên khối lượng tính toán không tỉ lệ thuận với số tham số như ở các tầng kết nối đầy đủ. Nói cách khác, trong ba kiến trúc học sinh, CNN vừa cho hiệu quả phân loại thấp nhất vừa có chi phí suy luận cao nhất.



Hình 4.9. Ma trận nhầm lẫn của các mô hình học sinh sau chưng cất tri thức truyền thống

Đặt ba ma trận cạnh nhau, cấu trúc lỗi của cả ba mô hình gần như đồng dạng: đường chéo chính đậm ở Benign, DDoS, DoS, Spoofing, Recon và Mirai; hai hàng BruteForce và Web-Based nhạt gần như toàn bộ. Khác biệt giữa ba mô hình chỉ nằm ở mức độ đậm nhạt chứ không nằm ở vị trí các ô lỗi. Điều này cho thấy giới hạn hiệu quả trên hai nhãn hiếm bắt nguồn từ bản thân biểu diễn đặc trưng và mức độ mất cân bằng của tập dữ liệu, không phải từ lựa chọn kiến trúc học sinh.

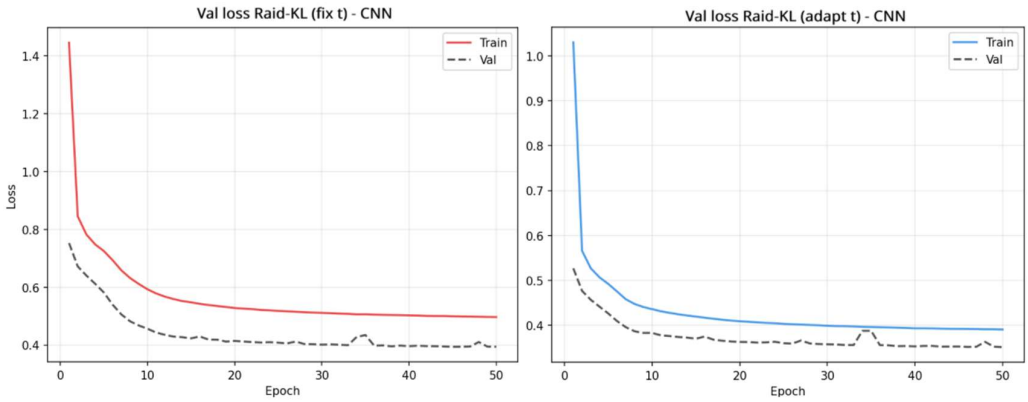


Hình 4.10. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức truyền thống

Cả ba mô hình đều cho đường ROC bám sát góc trên bên trái với macro-AUC từ 0.9721 đến 0.9783. Mirai đạt AUC bằng hoặc xấp xỉ 1.0000 ở cả ba mô hình, các nhãn còn lại nằm trong khoảng 0.94 đến 0.99, trong đó DoS là nhãn có AUC thấp nhất (0.9523 đến 0.9548), phù hợp với nhầm lẫn DoS–DDoS đã quan sát trên ma trận nhầm lẫn. Điểm đáng chú ý là BruteForce và Web-Based vẫn đạt AUC trên 0.94 dù recall của chúng rất thấp. Sự tương phản này xác nhận rằng mô hình vẫn xếp hạng xác suất đúng cho hai nhãn hiếm nhưng ngưỡng quyết định argmax (tìm chỉ số lớn nhất) mặc định không đủ để chúng thắng được các lớp đa số.

4.2.2. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức bằng phân kì JS-KL (Raid-KL)

4.2.2.a. Kết quả chưng cất tri thức Raid-KL trên mô hình CNN



Hình 4.11. Hai biểu đồ đường mất mát huấn luyện của mô hình CNN Raid-KL

Hai đồ thị cần được đọc song song vì chúng cùng một kiến trúc và chỉ khác cơ chế nhiệt độ. Trường hợp nhiệt độ cố định khởi đầu ở mức mất mát huấn luyện khoảng 1.45 và hội tụ về khoảng 0.49; trường hợp nhiệt độ thích ứng khởi đầu ở khoảng 1.05 và hội tụ về khoảng 0.38. Về dạng quỹ đạo, cả hai đều hội tụ ổn định và cùng xuất hiện một điểm nhô lên tại vòng thứ xấp xỉ ba mươi lăm trên đường kiểm định, điều này khá tương đồng với đồ thị của CNN KD truyền thống, cho thấy đây là đặc điểm của dữ liệu và lịch huấn luyện chứ không phải của kỹ thuật chưng cất. Khác biệt đáng kể là trường hợp nhiệt độ thích ứng có khoảng cách giữa đường huấn luyện và đường kiểm định hẹp hơn rõ rệt, tức là tín hiệu chưng cất mà học sinh nhận được ở chế độ nhiệt độ thích ứng gần với tín hiệu mà nó tái tạo được trên dữ liệu chưa thấy hơn.

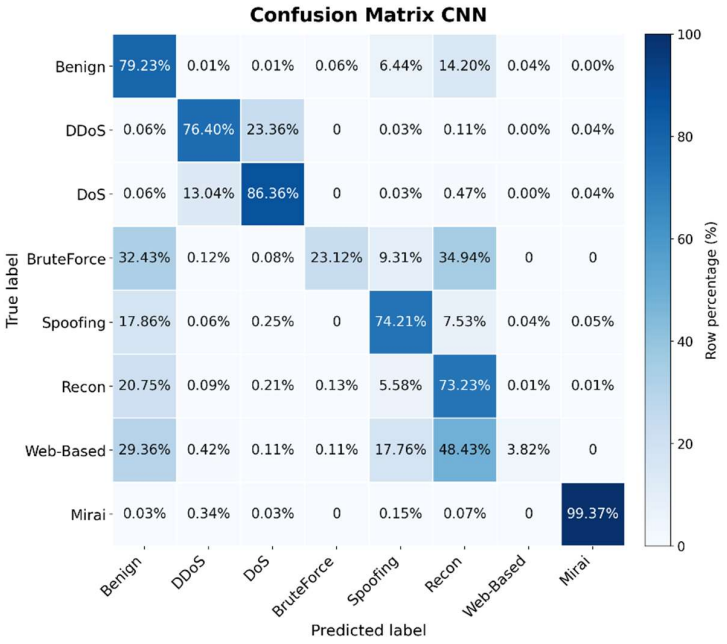
Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
CNN (teacher)	84.67	83.52	64.34	67.87	97.65	77.44	1,918
CNN Raid-KL (fixed temp)	83.05	71.88	60.62	63.34	97.04	**10.39**	45.91
CNN Raid-KL (adaptive temp)	**83.4**	**83.08**	**61.44**	**64.55**	**97.21**	10.51	45.91

Bảng 4.6. Kết quả huấn luyện mô hình học sinh CNN Raid-KL so với giáo viên

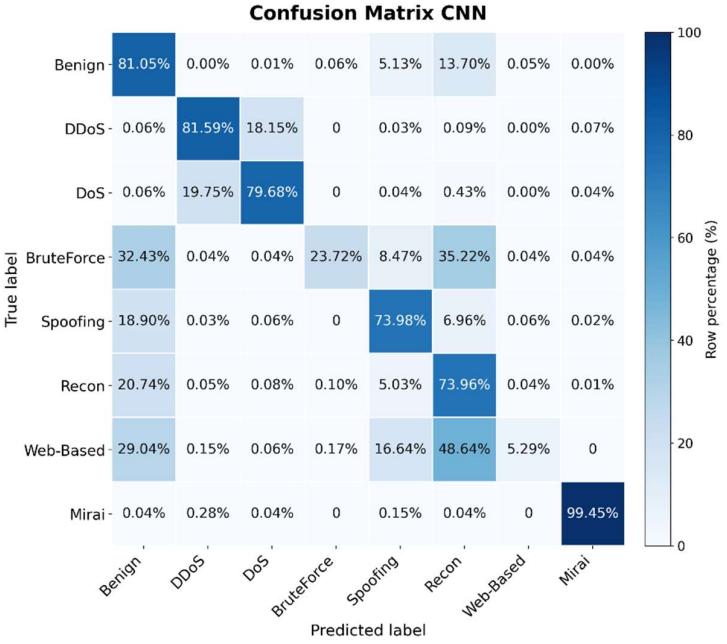
Trong cùng kiến trúc CNN, biến thể nhiệt độ thích ứng vượt biến thể nhiệt độ cố định trên toàn bộ các chỉ số hiệu quả: accuracy 83.4% so với 83.05%, macro-F1 64.55% so với 63.34%, macro AUC 97.21% so với 97.04%. Khoảng cách rõ nhất nằm ở macro precision, 83.08% so với 71.88%, tức chênh lệch 11.2 điểm phần trăm trong khi macro recall chỉ chênh 0.82 điểm. Mức sụt precision này không đến từ việc mô hình gán sai đồng đều trên mọi lớp mà đến từ một lớp thiểu số có precision suy giảm nghiêm trọng, biến thể nhiệt độ cố định gán nhãn Web-Based cho rất ít mẫu và phần lớn trong số đó là gán sai, kéo giá trị trung bình macro xuống mạnh.

Nguyên nhân có thể giải thích từ cơ chế. Với $\tau = 7$ áp dụng đồng loạt, phân phối đầu ra của giáo viên bị làm mềm rất mạnh trên mọi mẫu, kể cả những mẫu mà giáo viên vốn đã rất chắc chắn. Tri thức về quan hệ giữa các lớp được bộc lộ nhưng biên độ phân biệt giữa lớp đúng và các lớp gần kề cũng bị san phẳng, khiến học sinh khó tách các lớp hiếm nằm sát nhau trong không gian biểu diễn. Cơ chế nhiệt độ thích ứng khắc phục đúng điểm này bằng cách chỉ làm mềm mạnh ở những mẫu mà biểu diễn của học sinh còn lệch xa giáo viên, và giữ nhiệt độ thấp ở những mẫu hai bên đã đồng thuận.

Về chi phí, hai biến thể có cùng dung lượng 45.91 KB và thời gian suy luận gần như bằng nhau (10.39 µs so với 10.51 µs) với nhiệt độ cố định nhanh hơn chỉ 1 tí (0.12 µs) nhưng về sự chính xác thì thua ở mọi chỉ số.



Hình 4.12. Ma trận nhầm lẫn CNN Raid-KL (fix t)

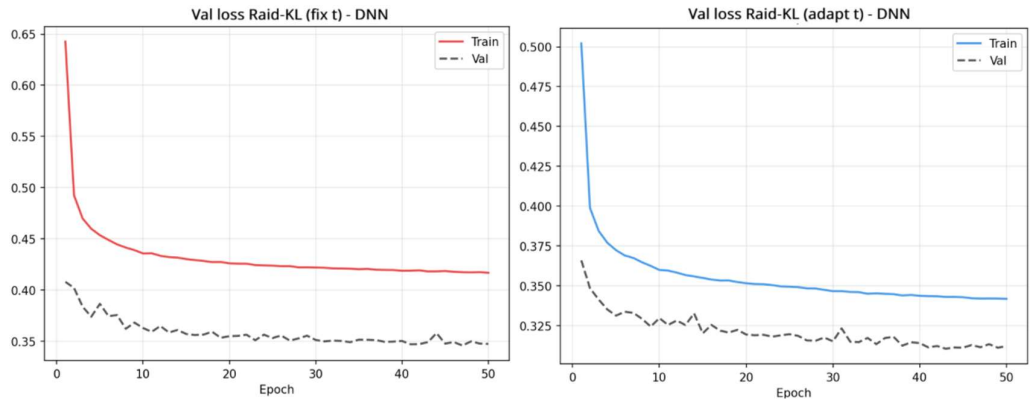


Hình 4.13. Ma trận nhầm lẫn CNN Raid-KL (adapt t)

Hai ma trận cho thấy cùng một cấu trúc lỗi đã quan sát ở KD truyền thống. Điểm khác biệt đáng ghi nhận nằm ở cặp DDoS-DoS: biến thể nhiệt độ cố định đạt recall DoS rất cao (86.36%) nhưng đánh đổi bằng recall DDoS thấp (76.40%), trong khi

biến thể thích ứng cân bằng hơn (79.68% và 81.59%). Nhãn Web-Based giữ mức recall thấp ở cả hai (3.82% và 5.29%) và nhãn Mirai vẫn gần như hoàn hảo.

4.2.2.b. Kết quả chưng cất tri thức Raid-KL trên mô hình DNN



Hình 4.14. Hai biểu đồ đường mất mát huấn luyện mô hình DNN Raid-KL

Cả hai biến thể hội tụ nhanh và phẳng sớm, phù hợp với đặc điểm đã thấy ở DNN KD truyền thống. Khoảng cách giữa hai đường ở biến thể thích ứng hẹp hơn, tiếp tục lặp lại quan sát ở CNN Raid-KL. Đường kiểm định của cả hai đều đi ngang từ vòng thứ xấp xỉ 30 mà không đi lên, xác nhận mô hình hội tụ tốt trong 50 vòng huấn luyện.

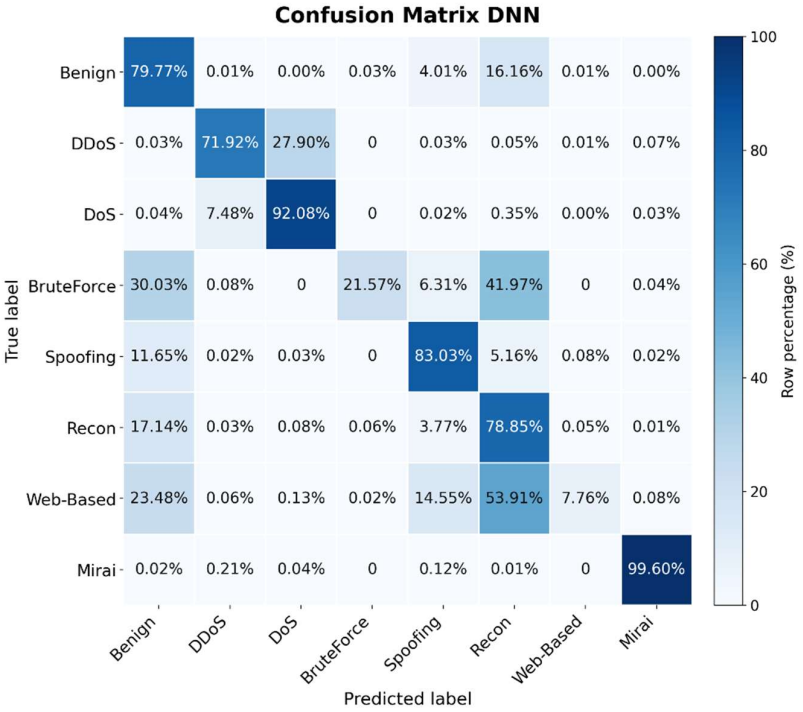
Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
DNN (teacher)	85.71	82.89	67.79	71.43	98.04	17.61	1,761
DNN Raid-KL (fixed temp)	84.24	**85.17**	62.99	65.85	97.36	**2.46**	45.53
DNN Raid-KL (adaptive temp)	**84.45**	81.87	**63.88**	**66.83**	**97.53**	**2.46**	45.53

Bảng 4.7. Kết quả huấn luyện mô hình học sinh DNN Raid-KL so với giáo viên

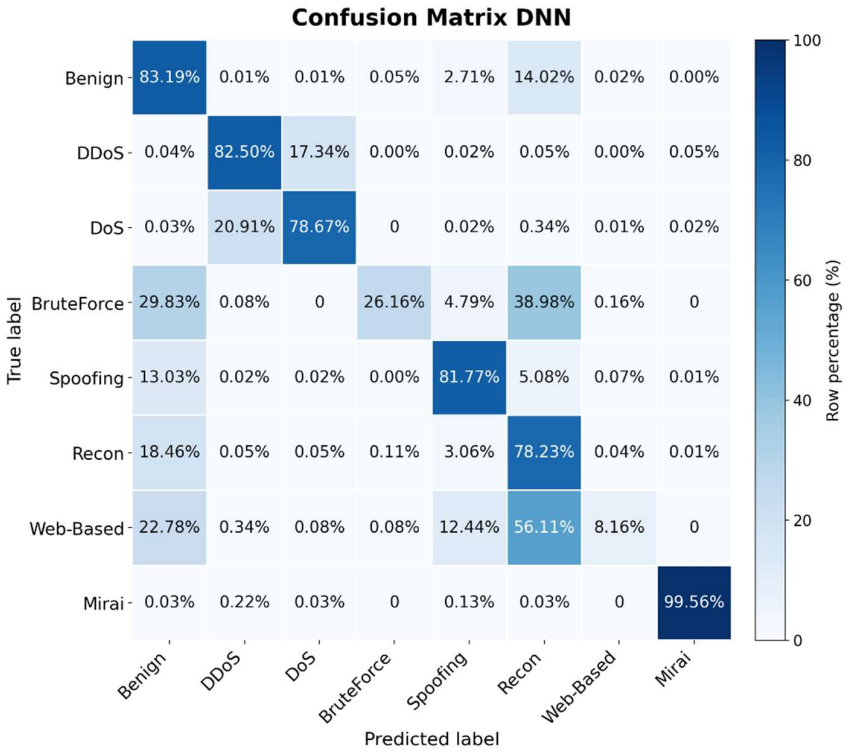
Kết luận về nhiệt độ được lặp lại nhất quán: biến thể thích ứng đạt 84.45% accuracy, 66.83% macro-F1 và 97.53% macro AUC, đều cao hơn biến thể cố định (84.24%, 65.85%, 97.36%). Trường hợp DNN cho thấy một dạng đánh đổi khác so với CNN. Ở đây biến thể nhiệt độ cố định lại đạt macro precision cao hơn, thậm chí hơn cả giáo viên (85.17%) nhưng macro recall thấp nhất (62.99%), tức là mô hình trở nên thận trọng quá mức: nó chỉ gán mẫu vào một lớp khi rất chắc chắn, nên gần như

không tạo cảnh báo sai nhưng bỏ sót nhiều mẫu tấn công thuộc lớp hiếm. Với bài toán phát hiện xâm nhập, kiểu đánh đổi này bất lợi vì bỏ sót tấn công gây thiệt hại lớn hơn cảnh báo sai. Cả hai biến thể có cùng thời gian suy luận 2.46 µs và cùng dung lượng 45.53 KB, đúng như kỳ vọng vì hai mô hình chỉ khác nhau ở cách tính hàm mất mát trong huấn luyện chứ không khác nhau về kiến trúc triển khai.

Ma trận nhầm lẫn tương ứng làm rõ điều này: biến thể cố định đẩy recall DoS lên 92.08% nhưng làm recall DDoS tụt xuống 71.92% và recall Web-Based chỉ còn 7.76%, trong khi biến thể thích ứng phân bổ đều hơn với 78.67%, 82.50% và 8.16%. Ngoài ra, cả hai giữ nguyên ba vùng nhầm lẫn đặc trưng đã mô tả. Biến thể thích ứng cho đường chéo chính đồng đều hơn trên sáu nhãn đầu, đặc biệt ở Benign (83.19% so với 79.77%) và BruteForce (26.16% so với 21.57%), trong khi biến thể cố định chỉ vượt trội riêng ở DoS. Xem chi tiết 2 ma trận nhầm lẫn bên dưới.

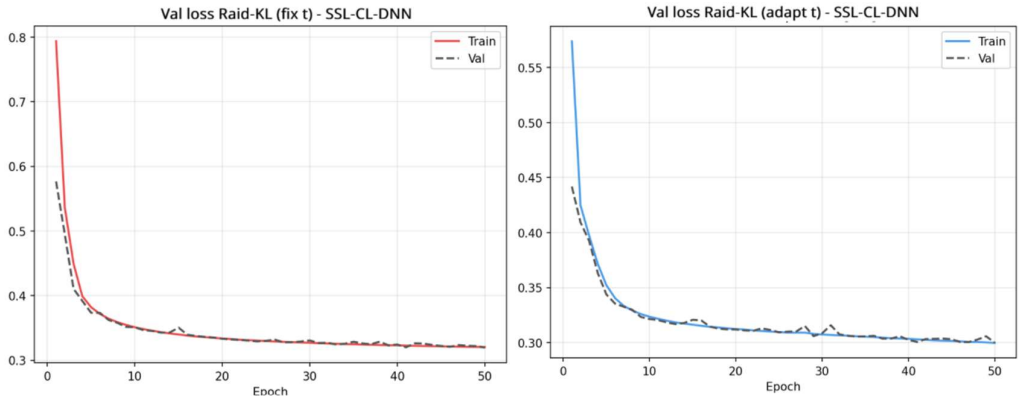


Hình 4.15. Ma trận nhầm lẫn DNN Raid-KL (fix t)



Hình 4.16. Ma trận nhầm lẫn DNN Raid-KL (adapt t)

4.2.2.c. Kết quả chưng cất tri thức Raid-KL trên mô hình SSL-CL-DNN



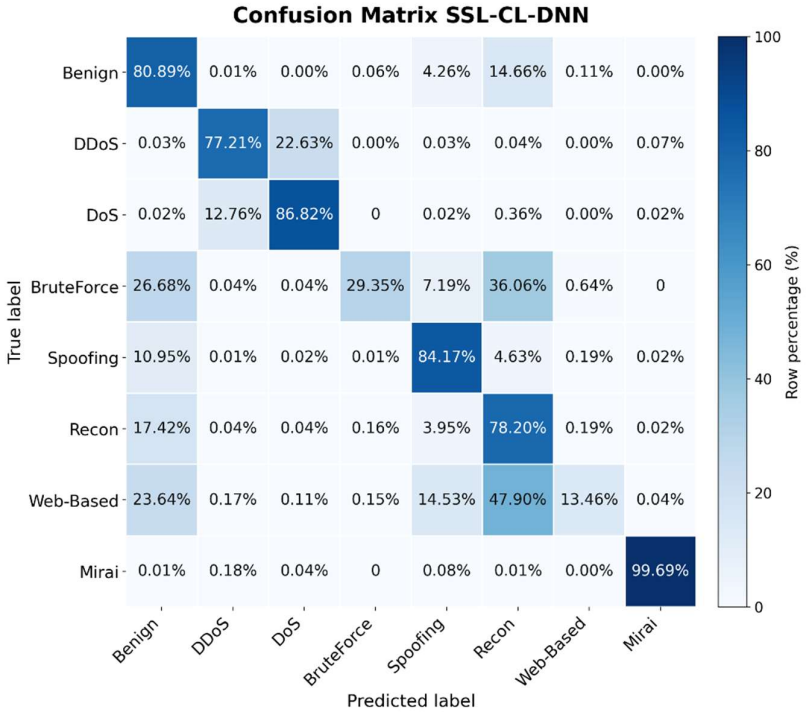
Hình 4.17. Hai biểu đồ đường mất mát huấn luyện mô hình SSL-CL-DNN Raid-KL

Hai đồ thị đều có đường huấn luyện và đường kiểm định chồng khít nhau gần như toàn bộ quá trình, giống hệt đặc điểm đã thấy ở SSL-CL-DNN KD truyền thống. Biến thể cố định hội tụ về khoảng 0.31, biến thể thích ứng về khoảng thấp hơn 0.3. Trong ba phân hệ kiến trúc, đây là phân hệ có quỹ đạo hội tụ ổn định nhất ở cả hai chế độ nhiệt độ, cho thấy Encoder DNN được kế thừa từ giai đoạn học tương phản làm giảm độ nhạy của quá trình tối ưu đối với lựa chọn siêu tham số nhiệt độ.

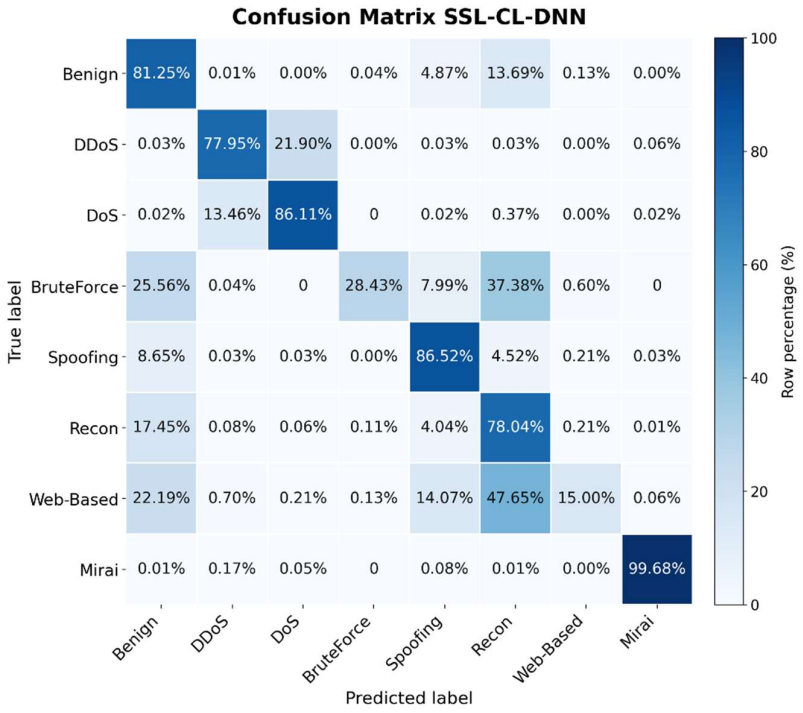
Bảng 4.8. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN Raid-KL so với giáo viên

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN (teacher)	85.58	82.80	67.12	70.93	97.97	84.38	2,137
SSL-CL-DNN Raid-KL (fixed temp)	84.92	83.60	64.97	68.21	97.71	**3.35**	76.66
SSL-CL-DNN Raid-KL (adaptive temp)	**85.05**	**83.44**	**65.24**	**68.71**	**97.80**	**3.35**	76.66

Biến thể thích ứng tiếp tục dẫn trước biến thể cố định (85.05% so với 84.92% accuracy, 68.71% so với 68.21% macro-F1, 97.80% so với 97.71% macro AUC), nhưng khoảng cách ở đây là nhỏ nhất trong ba kiến trúc: chỉ 0.13 điểm accuracy và 0.50 điểm macro-F1. Xu hướng thu hẹp này cho thấy lợi ích của cơ chế nhiệt độ thích ứng tỉ lệ nghịch với chất lượng khởi điểm của biểu diễn học sinh. Khi biểu diễn của học sinh vốn đã gần với giáo viên, độ tương đồng cosine giữa hai bên đã cao ngay từ đầu, nhiệt độ thích ứng luôn nằm gần cận dưới τ_min và hành vi của cơ chế hội tụ về gần với một nhiệt độ cố định thấp, do đó cơ chế thích ứng không tạo khác biệt. Cả hai mô hình tương tự như DNN đều có cùng kích thước và tốc độ thực thi.



Hình 4.18. Ma trận nhầm lẫn SSL-CL-DNN Raid-KL (fix t)



Hình 4.19. Ma trận nhầm lẫn SSL-CL-DNN Raid-KL (adapt t)

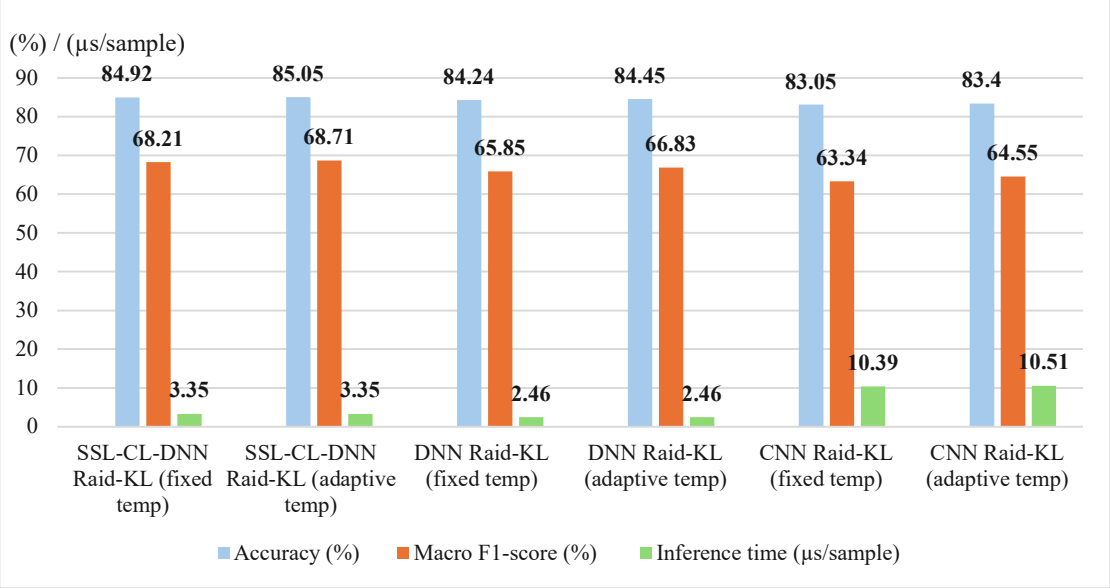
Hai ma trận rất gần nhau về hình dạng. Biến thể thích ứng nhỉnh hơn ở Spoofing (86.52% so với 84.17%), Web-Based (15.00% so với 13.46%) và Benign (81.25% so với 80.89%). Đáng chú ý là biến thể thích ứng đạt recall Web-Based 15.00%, mức cao nhất ghi nhận được cao nhất trong toàn bộ nhóm học sinh.

4.2.2.d. Nhận xét chung kết quả chưng cất các mô hình bằng Raid-KL

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN Raid-KL (fixed temp)	84.92	83.60	64.97	68.21	97.71	3.35	76.66
SSL-CL-DNN Raid-KL (adaptive temp)	**85.05**	83.44	**65.24**	**68.71**	**97.80**	3.35	76.66
DNN Raid-KL (fixed temp)	84.24	**85.17**	62.99	65.85	97.36	**2.46**	45.53
DNN Raid-KL (adaptive temp)	84.45	81.87	63.88	66.83	97.53	**2.46**	45.53

CNN Raid-KL (fixed temp)	83.05	71.88	60.62	63.34	97.04	10.39	45.91
CNN Raid-KL (adaptive temp)	83.4	83.08	61.44	64.55	97.21	10.51	45.91

Bảng 4.9. Kết quả huấn luyện các mô hình học sinh chưng cất tri thức Raid-KL



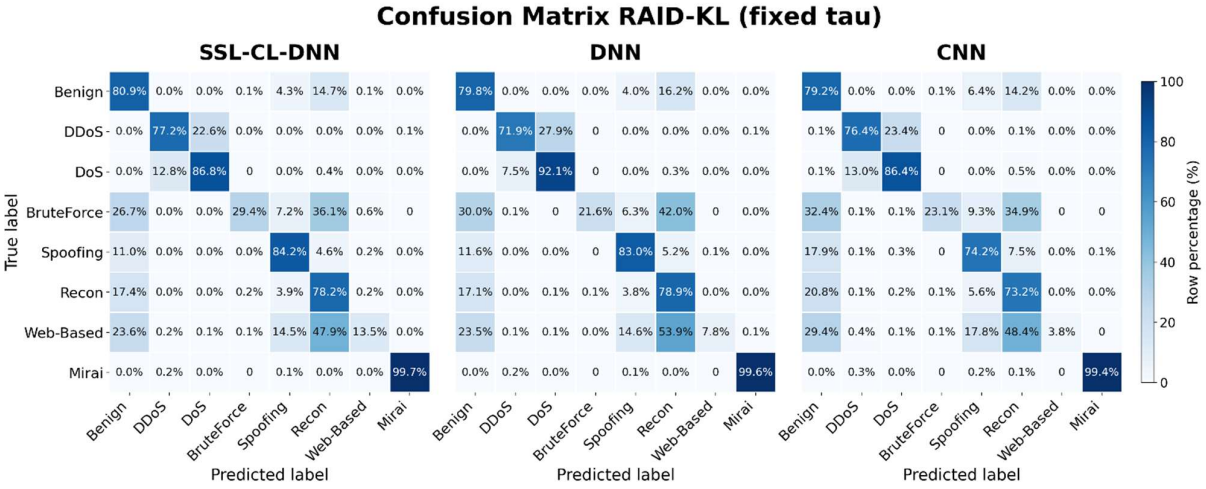
Hình 4.20. Biểu đồ cột Accuracy, Macro-F1 và Inference time các mô hình học sinh chưng cất tri thức Raid-KL

Bảng tổng hợp và biểu đồ của sáu mô hình cho phép rút ra hai kết luận độc lập nhau.

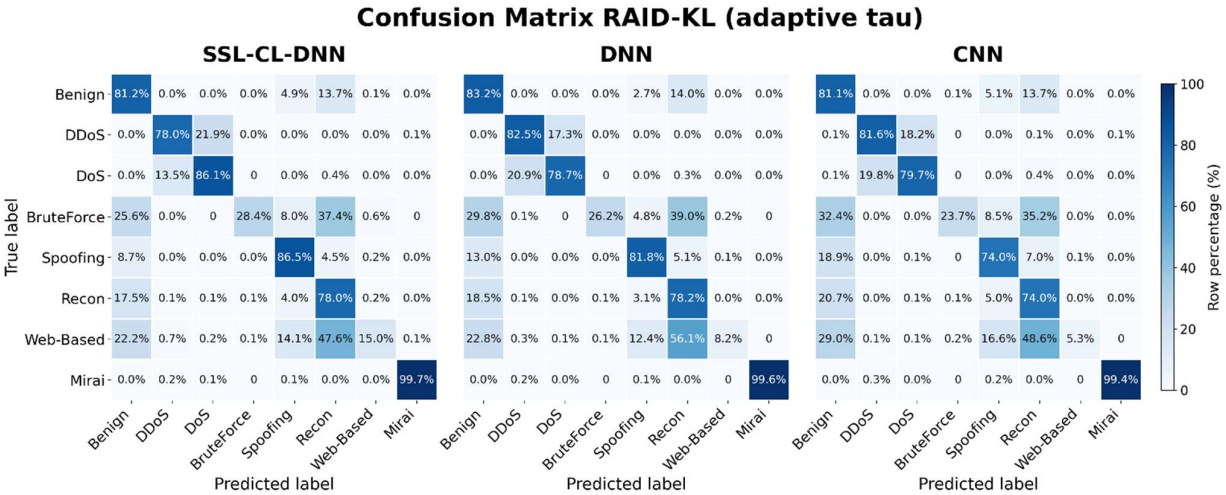
Kết luận về nhiệt độ mang tính hệ thống và không có ngoại lệ: trên cả ba kiến trúc, biến thể nhiệt độ thích ứng vượt biến thể nhiệt độ cố định ở toàn bộ ba chỉ số accuracy, macro-F1 và macro AUC. Mức cải thiện macro-F1 lần lượt là 1.21 điểm ở CNN, 0.98 điểm ở DNN và 0.50 điểm ở SSL-CL-DNN. Tính nhất quán trên ba kiến trúc khác biệt về bản chất (CNN tích chập một chiều, DNN kết nối dày đặc và SSL-CL-DNN tự giám sát tương phản) cho phép khẳng định rằng lợi ích của cơ chế nhiệt độ thích ứng không phải là hiệu ứng ngẫu nhiên của một cấu hình cụ thể mà là thuộc tính của chính cơ chế.

Kết luận về kiến trúc lặp lại đúng thứ tự đã thấy ở KD: SSL-CL-DNN dẫn đầu, DNN đứng giữa, CNN đứng cuối, ở cả hai chế độ nhiệt độ. Khoảng cách macro-F1 giữa mô hình tốt nhất (SSL-CL-DNN thích ứng, 68.71%) và kém nhất (CNN cố định, 63.34%) là 5.37 điểm. Cho thế lựa chọn kiến trúc và giáo viên tác động đến hiệu quả phân loại mạnh hơn lựa chọn biến thể của kỹ thuật chưng cất.

Về chi phí, sáu mô hình phân tách theo đúng ba kiến trúc chứ không theo chế độ nhiệt độ: nhóm DNN đạt 2.46 µs với 45.53 KB, nhóm SSL-CL-DNN đạt 3.35 µs với 76.66 KB, còn nhóm CNN tốn khoảng 10.4 µs với 45.91 KB. Việc hai biến thể trong cùng một kiến trúc luôn cho thời gian suy luận trùng nhau khẳng định lại rằng lựa chọn kỹ thuật chưng cất chỉ tác động đến chất lượng tri thức mà học sinh nhận được, không tác động đến chi phí vận hành, về chi phí do ảnh hưởng từ kiến trúc nhiều hơn. Riêng nhóm CNN tiếp tục là trường hợp bất lợi: chậm nhất trong ba kiến trúc dù không phải là nhóm có nhiều tham số nhất, đồng thời cũng cho hiệu quả phân loại thấp nhất.

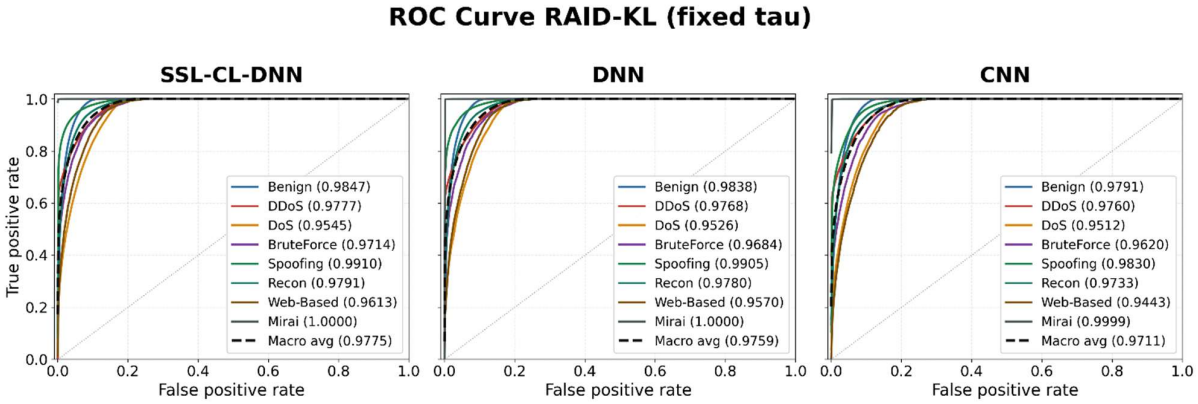


Hình 4.21. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức Raid-KL (fix t)

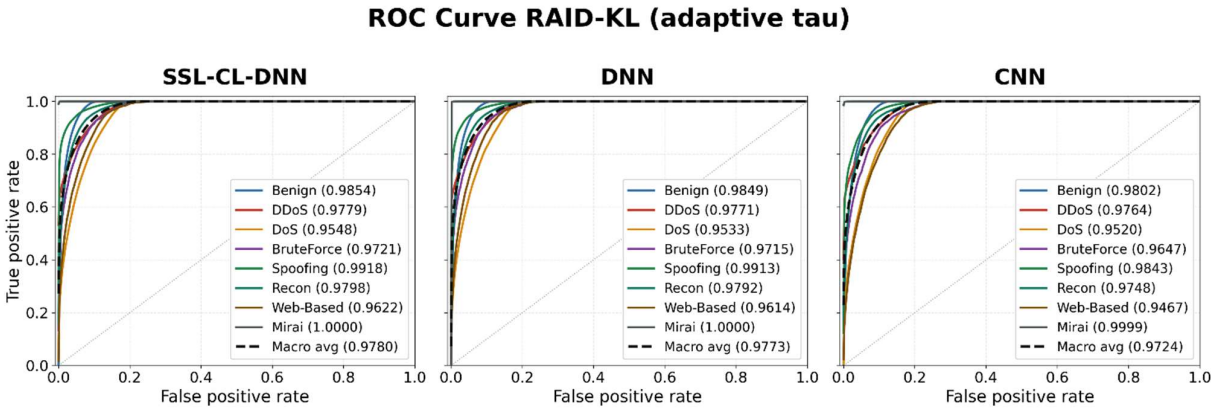


Hình 4.22. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức Raid-KL (adapt t)

Sáu ma trận đặt cạnh nhau cho thấy cấu trúc lỗi ổn định xuyên suốt: nhóm nhãn nhiều mẫu được phân loại tốt, cặp DDoS-DoS nhầm lẫn qua lại, hai nhãn BruteForce và Web-Based bị hòa vào Benign và Recon.



Hình 4.23. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức Raid-KL (fix t)

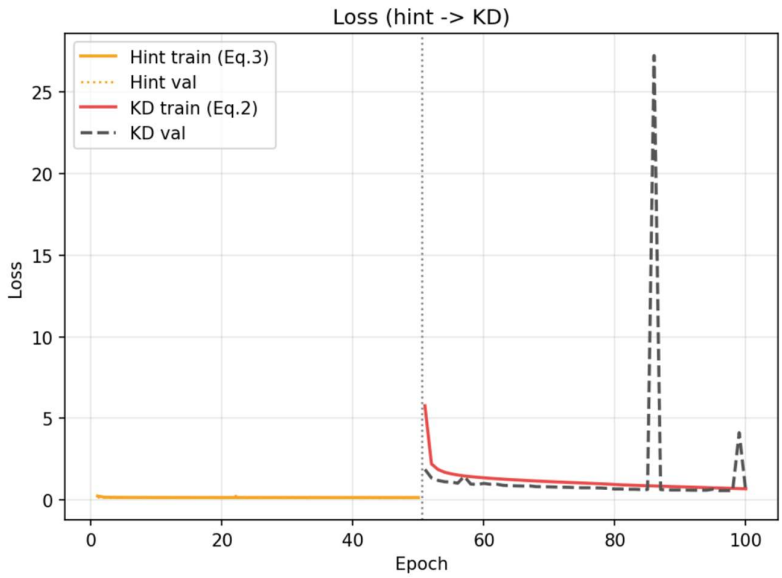


Hình 4.24. Các biểu đồ đường ROC của các mô hình học sinh sau chưng cất tri thức Raid-KL (adapt t)

Nhìn chung kết quả macro-AUC của cả 6 mô hình đều vẫn rất cao như ghi nhận với các mô hình học sinh chưng cất tri thức KD. Mirai đạt xấp xỉ 1.0000, BruteForce và Web-Based tuy có recall rất thấp nhưng vẫn giữ AUC cao. Macro-AUC trung bình trường hợp nhiệt độ thích ứng cao hơn so với nhóm nhiệt độ cố định.

4.2.3. Kết quả huấn luyện các mô hình học sinh sử dụng kỹ thuật chưng cất tri thức bằng gợi ý tầng ẩn (FitNets)

4.2.3.a. Kết quả chưng cất tri thức FitNets trên mô hình CNN



Hình 4.25. Biểu đồ đường mất mát ở 2 giai đoạn huấn luyện CNN FitNets

Đồ thị gồm hai giai đoạn ngăn cách bởi đường thẳng đứng tại vòng thứ năm mươi. Giá trị mất mát của hai giai đoạn không cùng đơn vị và không so sánh được với nhau: giai đoạn khớp gợi ý tối thiểu hóa sai số bình phương giữa hai đặc trưng tầng ẩn, còn giai đoạn chưng cất tối thiểu hóa tổ hợp cross-entropy và phân kỳ KL trên phân phối đầu ra.

Trong giai đoạn khớp gợi ý, đường mất mát nằm sát trục hoành và gần như phẳng ngay từ vòng đầu tiên. Điều này cho thấy bộ hồi quy tích chập kernel 1×1 khớp được đặc trưng của giáo viên gần như tức thời, tức là tín hiệu gợi ý ở vị trí đã chọn không đặt ra bài toán tối ưu thực sự cho học sinh. Hệ quả là giai đoạn một đóng góp rất ít vào việc dẫn dắt trọng số thân mạng, và mô hình bước vào giai đoạn hai với điểm khởi tạo không khác nhiều so với khởi tạo ngẫu nhiên.

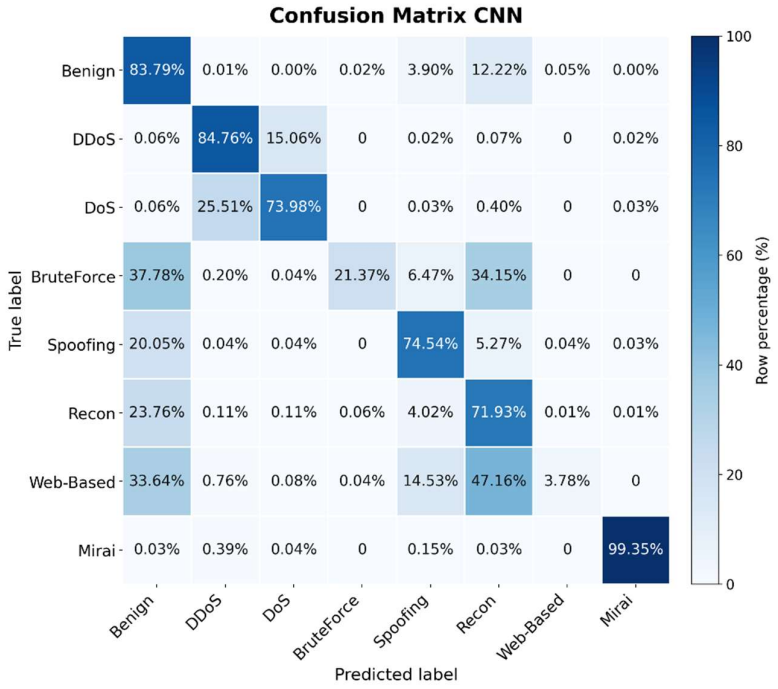
Trong giai đoạn chưng cất, mất mát huấn luyện giảm từ khoảng 5.9 xuống dưới 1.0 và ổn định, nhưng đường kiểm định xuất hiện hai đột biến rất lớn tại khoảng vòng thứ khoảng sau 80 và trước 95, trong đó đột biến mất mát đạt giá trị khoảng hơn 25. Các đột biến này trùng với giai đoạn cuối của lịch giảm tuyến tính hệ số λ từ 4 về 1, khi trọng số dành cho thành phần nhãn cứng tăng lên tương đối và mô hình phải điều chỉnh lại quyết định trên các lớp có trọng số lớp cao. Vì cơ chế lưu trạng thái tốt nhất

chỉ giữ lại tham số tại vòng có mất mát kiểm định thấp nhất, các đột biến này không ảnh hưởng đến mô hình cuối cùng, nhưng chúng là dấu hiệu cho thấy quá trình huấn luyện FitNets trên kiến trúc CNN kém ổn định.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
CNN (teacher)	84.67	83.52	64.34	67.87	97.65	77.44	1,918
CNN FitNets	83.58	73.29	61.43	64.13	97.27	10.44	45.91

Bảng 4.10. Kết quả mô hình học sinh CNN FitNets so với giáo viên

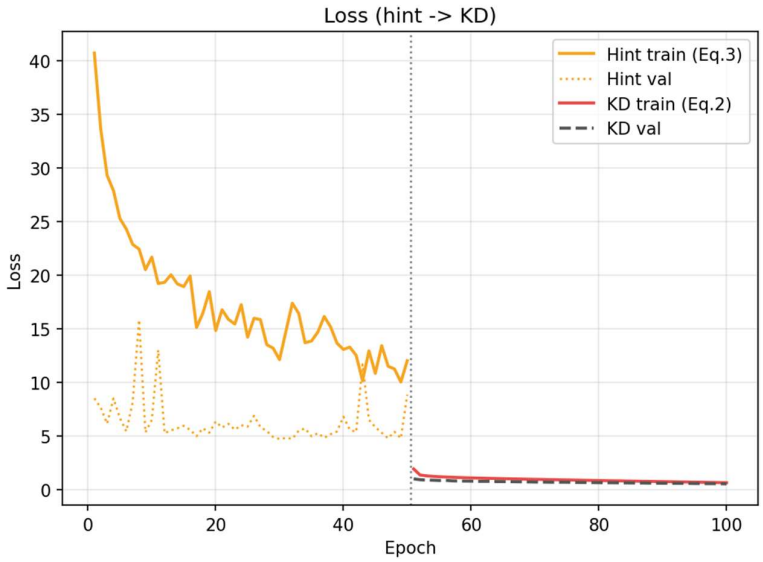
Học sinh CNN FitNets đạt 83.58% accuracy, khá cao nhưng macro-F1 chỉ 64.13%, thấp hơn cả KD truyền thống (64.33%) và Raid-KL thích ứng (64.55%). Sự phân ly giữa accuracy và macro-F1 này cho thấy giai đoạn khớp gợi ý giúp mô hình bám sát phân phối tổng thể của dữ liệu, vốn bị chi phối bởi các lớp đa số, nhưng không cải thiện được năng lực phân biệt các lớp hiếm. Kết quả phù hợp với quan sát trên đồ thị mất mát: khi tín hiệu gợi ý bão hòa ngay từ đầu, giai đoạn một không truyền được thông tin có ý nghĩa về cấu trúc quan hệ giữa các lớp mà chỉ đưa thân mạng về một vùng tham số hợp lý theo nghĩa tái tạo đặc trưng. Về chi phí, kích thước không đổi so với 2 phương pháp còn lại và tốc độ suy luận cũng giảm gấp hơn 7 lần so với giáo viên.



Hình 4.26. Ma trận nhầm lẫn CNN FitNets

Ma trận giữ nguyên cấu trúc lỗi đặc trưng của nhóm học sinh CNN. Recall Web-Based ở mức 3.78% và BruteForce ở mức 21.37%, đều thuộc nhóm thấp nhất trong các mô hình học sinh CNN, xác nhận nhận định rút ra từ bảng kết quả.

4.2.3.b. Kết quả chưng cất tri thức FitNets trên mô hình DNN



Hình 4.27. Biểu đồ đường mất mát 2 giai đoạn huấn luyện DNN FitNets

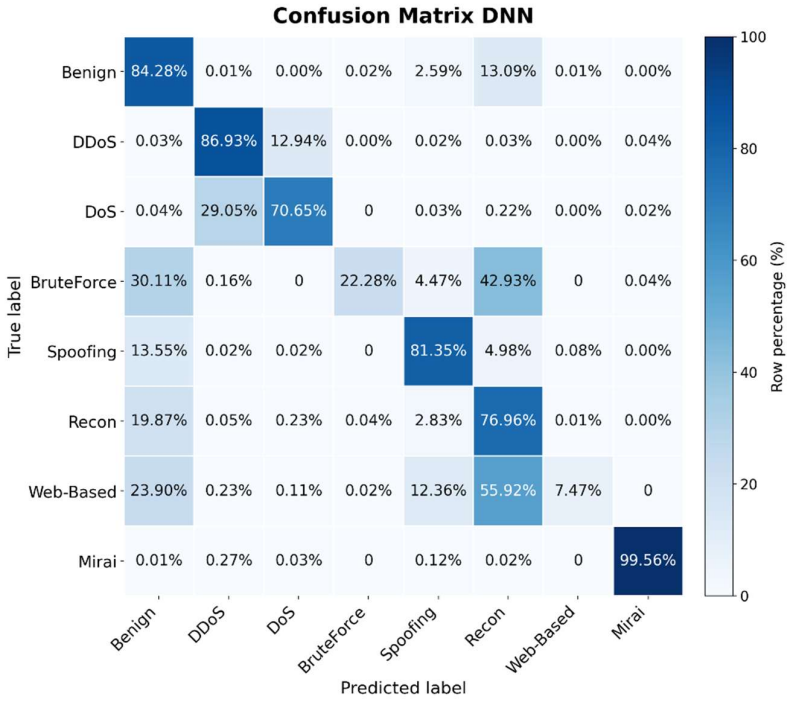
Trái ngược với trường hợp CNN, giai đoạn khớp gợi ý ở đây có mất mát khởi đầu rất cao (khoảng 40.5) và giảm dần xuống khoảng 11 sau năm mươi vòng, với dao động mạnh trong suốt quá trình. Đường mất mát khớp gợi ý trên tập kiểm định dao động trong dải 5 đến 15 và cũng không ổn định. Diễn biến này cho thấy bài toán khớp đặc trưng ở đây là bài toán thực sự: tầng hồi quy phải ánh xạ 64 chiều của học sinh lên 256 chiều của giáo viên, và sai số bình phương trên không gian chưa chuẩn hóa có độ lớn đáng kể.

Giai đoạn chưng cất sau đó diễn ra rất phẳng, mất mát nằm quanh mức 1.0 và hai đường huấn luyện, kiểm định gần như trùng nhau ngay từ vòng đầu của giai đoạn hai. Điều này chứng tỏ giai đoạn một đã đưa thân mạng học sinh về một vùng tham số mà giai đoạn hai chỉ cần tinh chỉnh nhẹ, đúng như kỳ vọng của phương pháp gốc.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
DNN (teacher)	85.71	82.89	67.79	71.43	98.04	17.61	1,761
DNN FitNets	84.35	80.96	63.79	66.84	97.49	2.46	45.53

Bảng 4.11. Kết quả huấn luyện mô hình học sinh DNN FitNets so với giáo viên

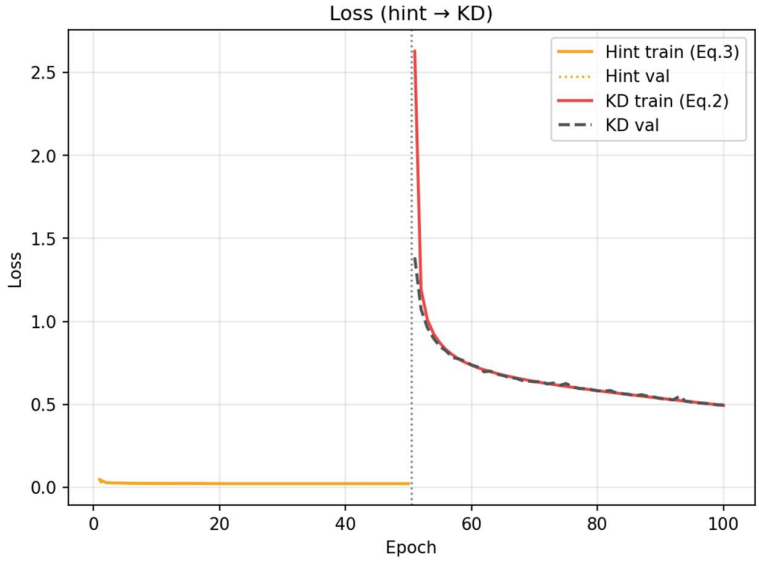
Học sinh DNN FitNets đạt 84.35% accuracy và 66.84% macro-F1, chỉ mất lần lượt 1.36 và 4.59 điểm phần trăm so với giáo viên, với thời gian suy luận 2.46 µs mỗi mẫu, bằng đúng ba mô hình DNN còn lại, do bộ hồi quy đã bị loại bỏ hoàn toàn sau giai đoạn một và mô hình triển khai chỉ còn thân mạng học sinh nguyên bản. Điều đáng lưu ý là dù giai đoạn khớp gợi ý ở kiến trúc DNN hoạt động đúng như thiết kế (khác hẳn trường hợp CNN) hiệu quả cuối cùng vẫn không vượt được KD truyền thống. Kết quả này gợi ý rằng trên dữ liệu bảng đặc trưng, lợi ích của việc dẫn dắt tầng ẩn trung gian bị hạn chế vì mạng học sinh chỉ có ba tầng ẩn, độ sâu chưa đủ lớn để vấn đề tối ưu hóa mà FitNets nhắm tới trở thành nút thắt thực sự. Phương pháp gốc được đề xuất cho các mạng sâu hơn hẳn và cho dữ liệu ảnh, nơi tầng ẩn trung gian mang biểu diễn phân cấp rõ rệt.



Hình 4.28. Ma trận nhầm lẫn DNN FitNets

Ma trận cho recall cao trên nhóm nhãn nhiều mẫu (Mirai 99.56%, DDoS 86.93%, Benign 84.28%, Spoofing 81.35%, Recon 76.96%) và giữ nguyên hai vùng yếu ở BruteForce (22.28%) và Web-Based (7.47%). Không có khác biệt về hình thái so với các mô hình DNN đã trình bày.

4.2.3.c. Kết quả chưng cất tri thức FitNets trên mô hình SSL-CL-DNN



Hình 4.29. Biểu đồ đường mất mát 2 giai đoạn huấn luyện SSL-CL-DNN FitNets

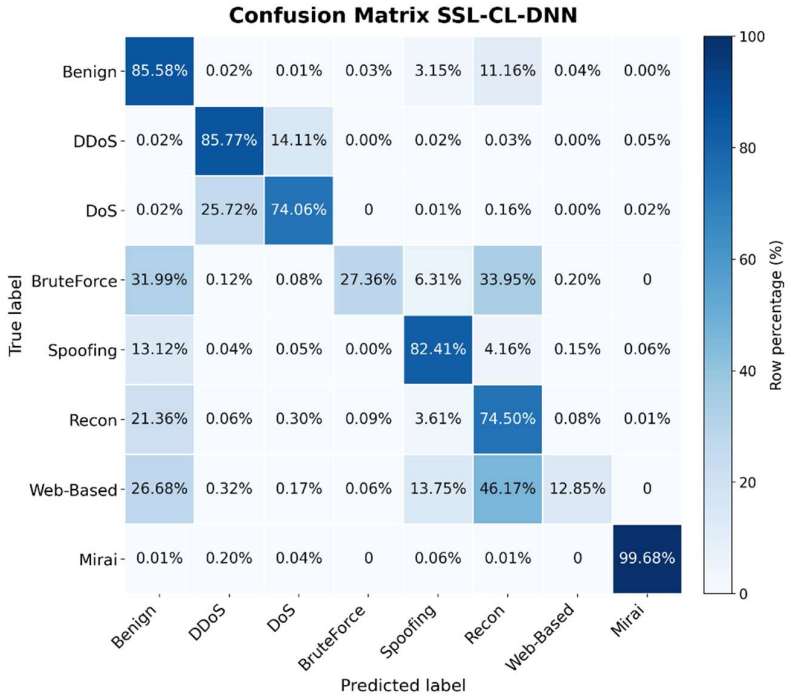
Giai đoạn khớp gợi ý cho mất mát rất thấp và phẳng, tương tự trường hợp CNN, phản ánh việc Encoder của học sinh đã kế thừa cấu trúc biểu diễn tương thích với Encoder của giáo viên nên bài toán khớp đặc trưng gần như được giải ngay lập tức. Giai đoạn chưng cất khởi đầu ở khoảng hớn 2.5 và giảm liên tục xuống khoảng 0.50 tại vòng thứ một trăm, với hai đường huấn luyện và kiểm định chồng khít nhau. Tuy nhiên đường mất mát vẫn còn xu hướng đi xuống ở vòng cuối, cho thấy mô hình chưa hội tụ hoàn toàn trong giới hạn năm mươi vòng của giai đoạn hai và có thể còn cải thiện nếu kéo dài lịch huấn luyện. Đây là điểm hạn chế cần ghi nhận khi so sánh công bằng với hai kỹ thuật còn lại, vốn đã hội tụ phẳng trong cùng số vòng.

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN (teacher)	85.58	82.80	67.12	70.93	97.97	84.38	2,137
SSL-CL-DNN FitNets	85.12	82.97	65.69	68.88	97.80	3.35	76.66

Bảng 4.12. Kết quả huấn luyện mô hình học sinh SSL-CL-DNN FitNets so với giáo viên

Học sinh SSL-CL-DNN FitNets đạt 85.12% accuracy và 68.88% macro-F1, chỉ mất 0.46 và 2.05 điểm phần trăm so với giáo viên. Đây cũng là mô hình học sinh có

accuracy cao nhất trong toàn bộ khảo sát. Về chi phí, mô hình giữ nguyên 3.35 µs mỗi mẫu và 76.66 KB như ba mô hình SSL-CL-DNN còn lại, xác nhận rằng hai giai đoạn huấn luyện của FitNets chỉ làm tăng chi phí huấn luyện chứ không để lại dấu vết nào trong mô hình triển khai.



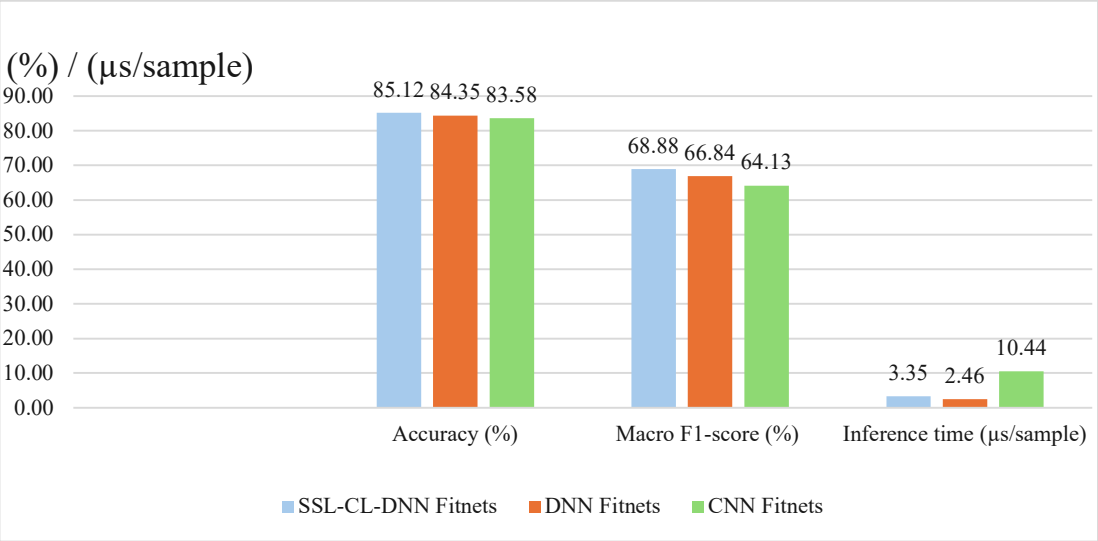
Hình 4.30. Ma trận nhầm lẫn SSL-CL-DNN FitNets

Ma trận cho thấy recall cao nhất trên nhãn Benign trong toàn bộ nhóm học sinh (85.58%), cùng với Mirai 99.68%, DDoS 85.77%, Spoofing 82.41% và Recon 74.50%. Recall Web-Based đạt 12.85% và BruteForce đạt 27.36%, đều nằm ở nhóm cao so với mặt bằng chung của các mô hình học sinh.

4.2.3.d. Nhận xét chung kết quả chưng cất các mô hình bằng FitNets

Model name	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)
SSL-CL-DNN FitNets	**85.12**	**82.97**	**65.69**	**68.88**	**97.80**	3.35	76.66
DNN FitNets	84.35	80.96	63.79	66.84	97.49	**2.46**	45.53
CNN FitNets	83.58	73.29	61.43	64.13	97.27	10.44	45.91

Bảng 4.13. Kết quả huấn luyện các mô hình học sinh chưng cất tri thức FitNets

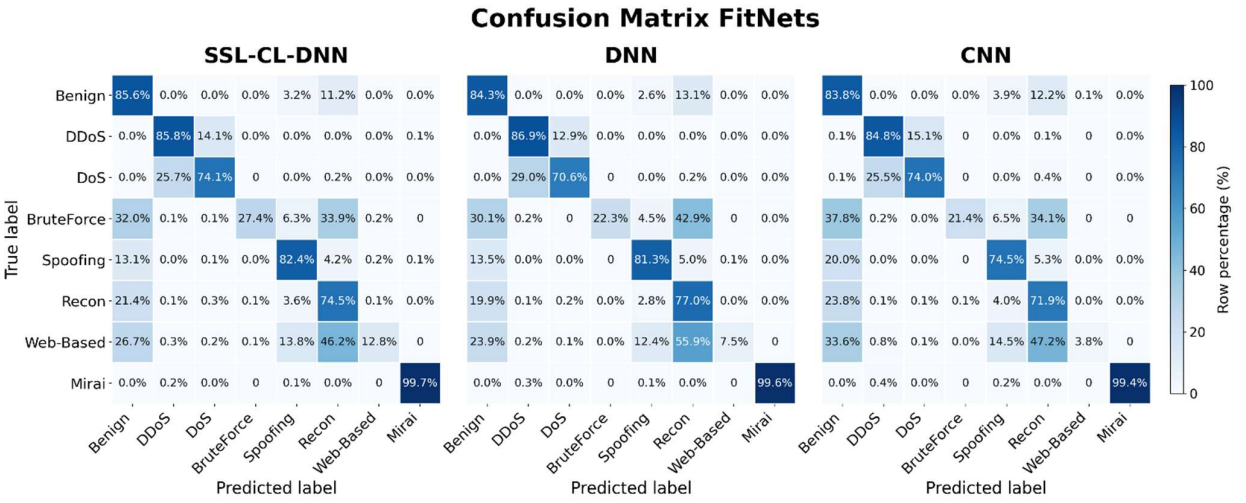


Hình 4.31. Biểu đồ cột Accuracy, Macro-F1 và Inference time các mô hình học sinh chưng cất tri thức FitNets

Thứ tự ba kiến trúc một lần nữa được giữ nguyên: SSL-CL-DNN (85.12% accuracy, 68.88% macro-F1), DNN (84.35%, 66.84%), CNN (83.58%, 64.13%). Tính lặp lại của thứ tự này qua cả ba kỹ thuật chưng cất khảo sát là bằng chứng mạnh cho thấy nó phản ánh đặc tính của kiến trúc và của cặp giáo viên tương ứng chứ không phải kết quả của một cấu hình huấn luyện may mắn.

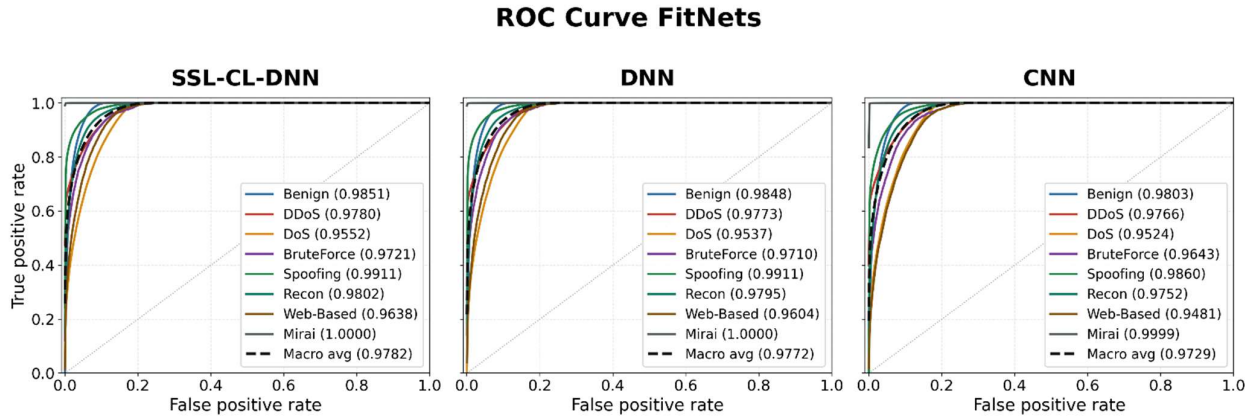
Về chi phí, ba mô hình FitNets lặp lại đúng phân tầng đã thấy ở hai kỹ thuật trước: SSL-CL-DNN 3.35 µs với 76.66 KB, DNN 2.46 µs với 45.53 KB, CNN 10.44 µs với 45.91 KB. Không có kỹ thuật chưng cất nào trong ba kỹ thuật khảo sát làm thay đổi chi phí suy luận, nên toàn bộ khác biệt về hiệu suất giữa các mô hình học sinh đều quy về lựa chọn kiến trúc.

Riêng với FitNets, cần ghi nhận một quan sát xuyên suốt ba kiến trúc: phương pháp cho accuracy tốt (cao nhất nhóm ở SSL-CL-DNN) nhưng macro-F1 luôn thấp hơn KD truyền thống. Chênh lệch lần lượt là 0.20 điểm ở CNN, 0.09 điểm ở DNN và 0.14 điểm ở SSL-CL-DNN. Nguyên nhân đã được chỉ ra qua phân tích đồ thị mất mát: hiệu quả của giai đoạn khớp gợi ý phụ thuộc mạnh vào việc lựa chọn cặp tầng gợi ý, tầng được dẫn dắt có thực sự đặt ra bài toán tối ưu hay không. Ở CNN và SSL-CL-DNN, mất mát khớp gợi ý bão hòa ngay từ vòng đầu nên giai đoạn một không đóng góp nhiều; ở DNN thì giai đoạn một hoạt động đúng thiết kế nhưng độ sâu ba tầng ẩn của học sinh không đủ để lợi ích thể hiện thành cải thiện đo được.



Hình 4.32. Ma trận nhầm lẫn các mô hình học sinh chưng cất tri thức bằng FitNets

Ba ma trận nhầm lẫn của học sinh chưng cất bằng FitNets đồng dạng với các nhóm kết quả trước. Không xuất hiện hình dạng gì mới nào riêng của phương pháp FitNets.



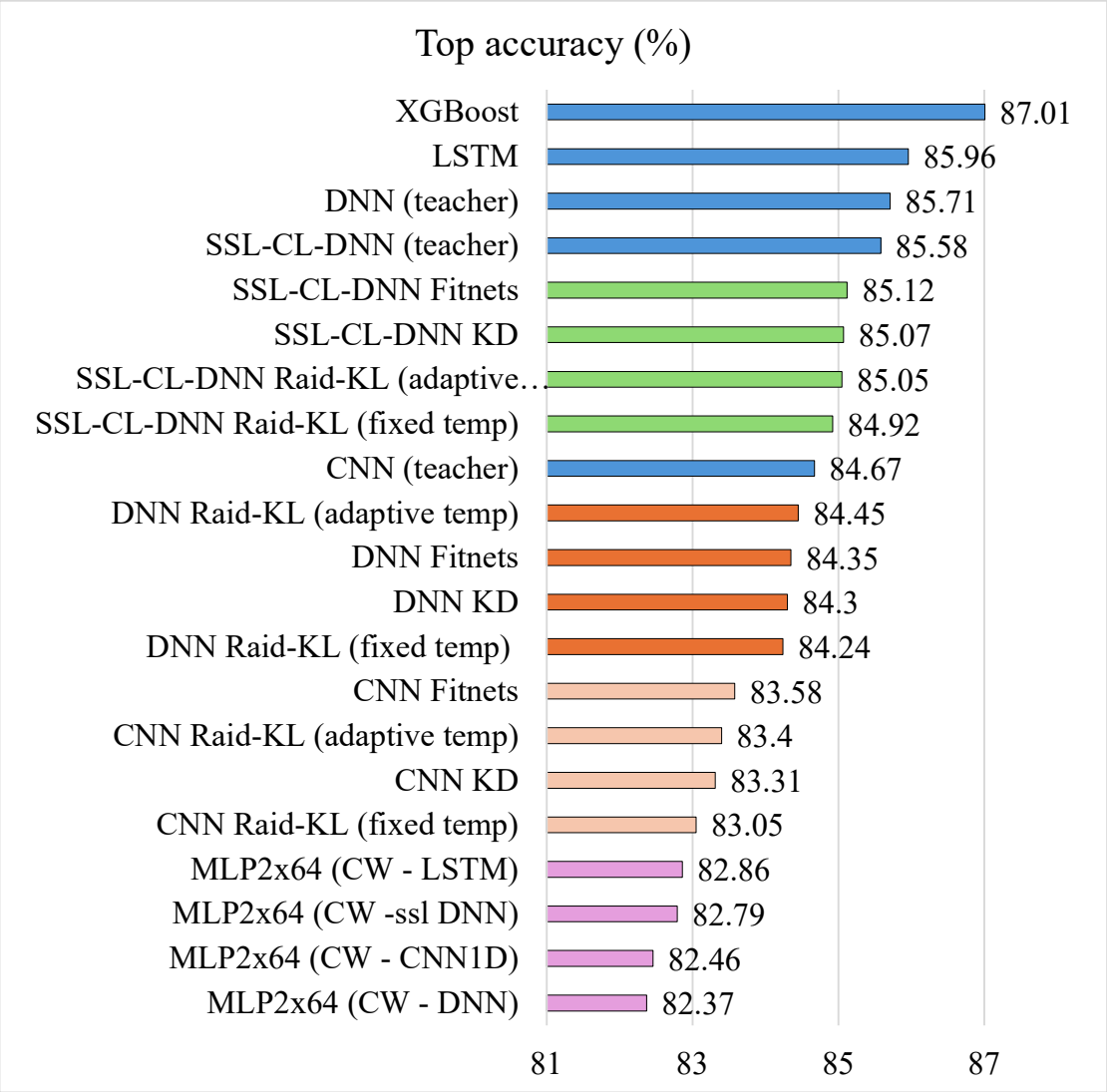
Hình 4.33. Các biểu đồ đường ROC các mô hình học sinh chưng cất tri thức bằng FitNets

Macro AUC vẫn rất cao (hơn 0.97), nằm cùng dải với hai kỹ thuật đã thực hiện. Phân bố AUC theo nhãn không thay đổi: Mirai xấp xỉ 1.0000, DoS thấp nhất trong khoảng 0.952 đến 0.955.

4.2.4. Đánh giá, so sánh kết quả tổng kết sau quá trình huấn luyện các mô hình

Sau quá trình huấn luyện các mô hình học sinh CNN, DNN và SSL-CL-DNN bằng các kĩ thuật chưng cất KD, Raid-KL, FitNets, quan sát lại các biểu đồ tỉ lệ độ

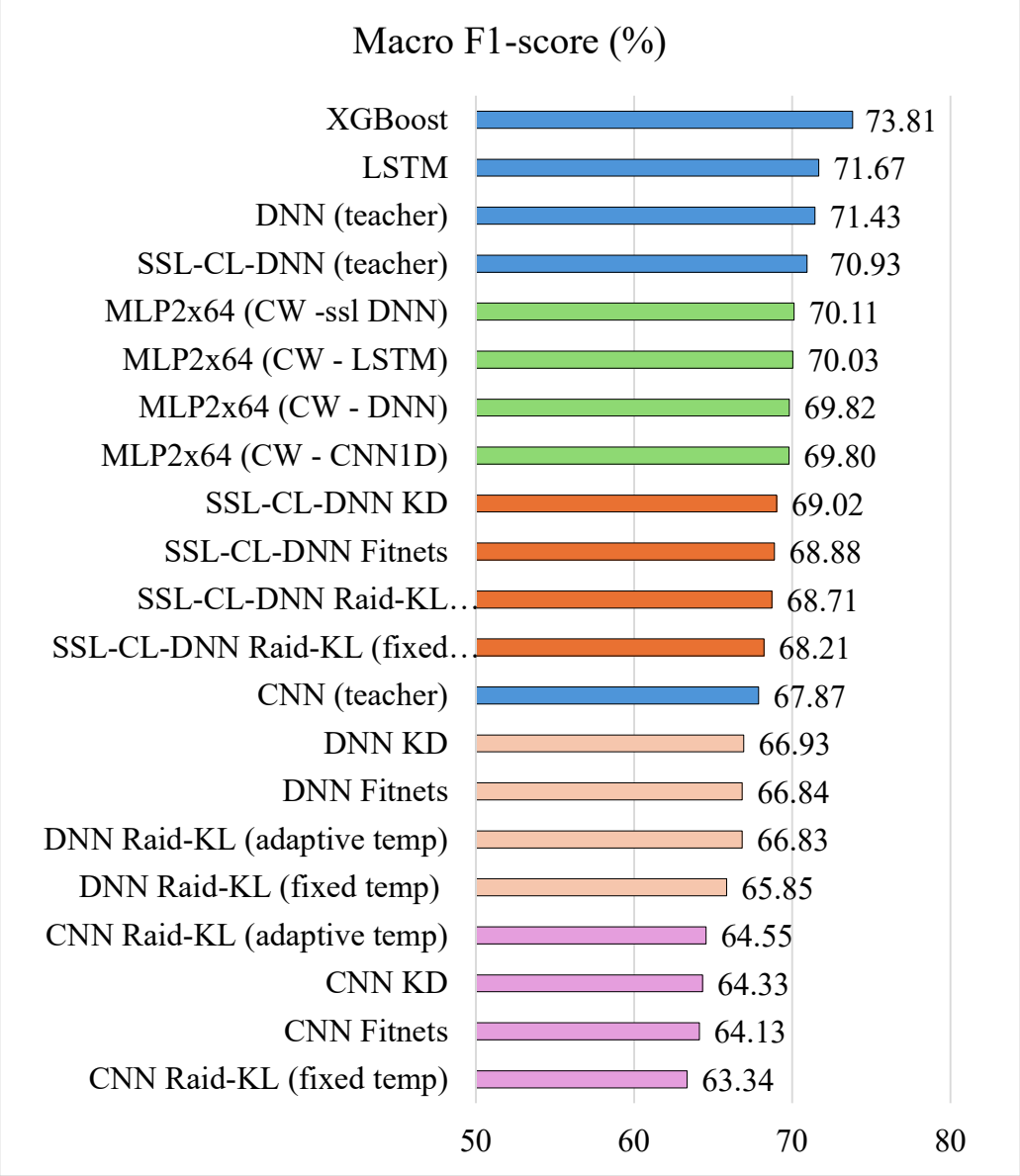
chính xác, tỉ lệ macro-F1 và thời gian suy luận giảm dần kèm bảng tổng hợp các kết quả để rút ra các đánh giá tổng kết:



Hình 4.34. Biểu đồ cột thể hiện Accuracy giảm dần của tất cả các mô hình học sinh và giáo viên

Biểu đồ cho thấy tất cả 21 mô hình giáo viên và học sinh phân thành 5 cụm rõ rệt theo phân hệ kiến trúc. Ở nhóm học sinh, bốn mô hình SSL-CL-DNN chiếm trọn nhóm dẫn đầu từ 84.92% đến 85.12%, thậm chí với kiến trúc nhỏ đã vượt xa mô hình giáo viên CNN, bốn mô hình DNN nằm liền kề từ 84.24% đến 84.45%, bốn mô hình CNN từ 83.05% đến 83.58%, và bốn mô hình MLP2x64 đa giáo viên xếp cuối từ 82.37% đến 82.86%. Ranh giới giữa các cụm rõ hơn ranh giới giữa các kỹ thuật trong cùng cụm, minh họa trực quan kết luận rằng kiến trúc là yếu tố phân định chính ở chỉ

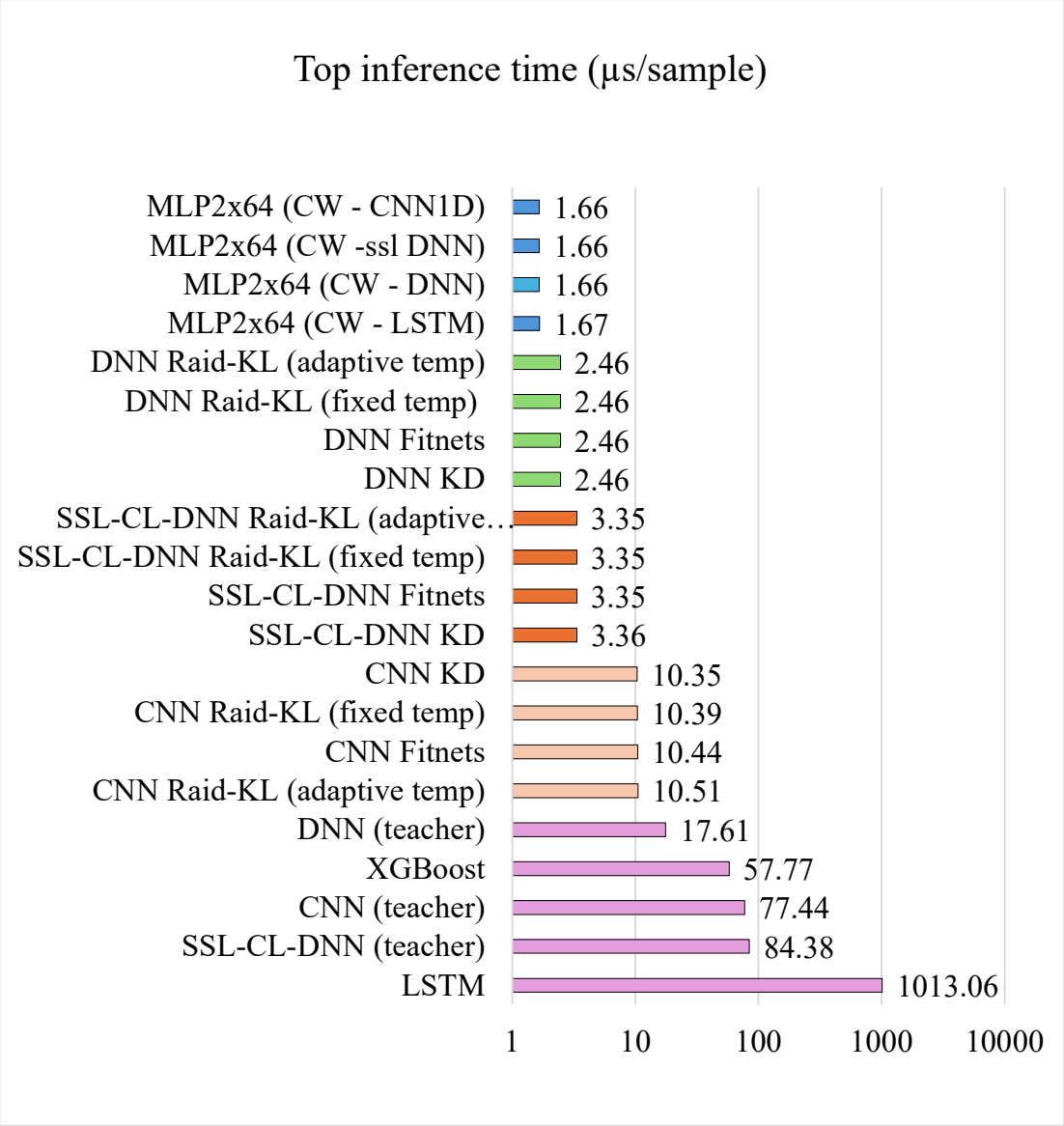
số này. Nhưng vẫn phải so sánh hiệu quả các giữa các phương pháp: FitNets đạt cao nhất ở SSL-CL-DNN và CNN, còn Raid-KL nhiệt độ thích ứng đạt cao nhất ở DNN ở hạng mục accuracy, các mô hình chưng cất đa giáo viên đều nằm ở nhóm cuối.



Hình 4.35. Biểu đồ cột thể hiện Macro-F1 giảm dần của tất cả các mô hình học sinh và giáo viên

So với biểu đồ accuracy giảm dần, thứ tự ở đây thay đổi đáng kể ở phần đầu bảng không xét mô hình giáo viên. Bốn mô hình MLP2x64 chưng cất đa giáo viên chuyển từ vị trí cuối lên vị trí dẫn đầu trong nhóm học sinh khi nói về macro-F1 và vượt cả bốn mô hình SSL-CL-DNN. Ở đây đặc biệt thấy rằng bốn mô hình MLP2x64

và bốn mô hình kiến trúc SSL-CL-DNN vượt xa giáo viên CNN. Hai cụm còn lại giữ nguyên thứ tự tương đối. Trong cùng một kiến trúc học sinh, khác biệt giữa bốn kỹ thuật chưng cất là nhỏ, dao động từ 0.81 điểm macro-F1 ở nhóm SSL-CL-DNN đến 1.21 điểm ở nhóm CNN. Chưng cất truyền thống cho macro-F1 cao nhất ở hai trong ba kiến trúc là SSL-CL-DNN và DNN, với kiến trúc CNN thì Raid-KL nhiệt độ thích ứng cho ra macro-F1 tốt hơn.



Hình 4.36. Biểu đồ cột thể hiện Inference time (µs) giảm dần của các mô hình học sinh

Khác với hai biểu đồ trước vốn phân tách theo kiến trúc, biểu đồ thời gian suy luận cho thấy một trật tự bám khá sát quy mô mô hình. Nhóm MLP2x64 với 4,776 tham số nhanh nhất (1.66 µs), kế đến là nhóm DNN với 11,656 tham số (2.46 µs), rồi nhóm SSL-CL-DNN với 19,624 tham số (3.35 µs). Ba nhóm này đều là mạng kết nối đầy đủ, nơi mỗi tham số tham gia đúng một lần vào mỗi lần dự đoán, nên chi phí tăng gần như tuyến tính theo kích thước.

Hai trường hợp lệch khỏi quy luật đó đều có nguyên nhân từ cách kiến trúc tổ chức phép tính chứ không phải từ số tham số. Nhóm CNN chỉ có 11,752 tham số, xấp xỉ nhóm DNN, nhưng tốn khoảng 10.4 µs vì bộ lọc tích chập được áp dụng lặp lại trên nhiều vị trí của chuỗi đặc trưng, khiến khối lượng tính toán lớn hơn nhiều so với những gì số tham số gợi ý. Mô hình LSTM giáo viên là trường hợp cực đoan nhất với 1013.06 µs dù chỉ có 117,256 tham số, ít hơn cả giáo viên DNN do phải xử lý tuần tự qua từng bước thời gian nên không tận dụng được khả năng tính toán song song. Ở chiều ngược lại, XGBoost có hơn 1.37 triệu tham số nhưng chỉ tốn 57.77 µs vì mỗi dự đoán chỉ đi qua một số nhánh nhất định của các cây thay vì kích hoạt toàn bộ mô hình.

Models	Models name	Accuracy (%)	Macro F1-score (%)	Macro AUC (%)	Inference time (µs/sample)	Model size (KB)	Params
1	**SSL-CL-DNN**	85.58	70.93	97.97	84.38	2,137	547,080
2	**DNN**	85.71	71.43	98.04	17.61	1,761	144,904
3	**XGBoost**	87.01	73.81	98.32	57.77	82,326	1,377,292
4	**CNN1D**	84.67	67.87	97.65	77.44	1,918	158,600
5	**LSTM**	85.96	71.67	98	1013.06	1,422	117,256
6	SSL-CL-DNN KD	85.07	69.02	97.80	3.36	76.66	19,624
7	SSL-CL-DNN Fitnets	85.12	68.88	97.80	3.35	76.66	19,624
8	SSL-CL-DNN Raid-KL (fixed temp)	84.92	68.21	97.71	3.35	76.66	19,624
9	SSL-CL-DNN Raid-KL (adaptive temp)	85.05	68.71	97.80	3.35	76.66	19,624
10	DNN KD	84.3	66.93	97.49	2.46	45.53	11,656
11	DNN Fitnets	84.35	66.84	97.49	2.46	45.53	11,656
12	DNN Raid-KL (fixed temp)	84.24	65.85	97.36	2.46	45.53	11,656

13	DNN Raid-KL (adaptive temp)	84.45	66.83	97.53	2.46	45.53	11,656
14	CNN KD	83.31	64.33	97.19	10.35	45.53	11,752
15	CNN Fitnets	83.58	64.13	97.27	10.44	45.91	11,752
16	CNN Raid-KL (fixed temp)	83.05	63.34	97.04	10.39	45.91	11,752
17	CNN Raid-KL (adaptive temp)	83.4	64.55	97.21	10.51	45.91	11,752
18	MLP2x64 (CW - DNN)	82.37	69.82	97.69	1.66	36.65	4,776
19	MLP2x64 (CW -ssl DNN)	82.79	70.11	97.77	1.66	36.65	4,776
20	MLP2x64 (CW - CNN1D)	82.46	69.80	97.70	1.66	36.65	4,776
21	MLP2x64 (CW - LSTM)	82.86	70.03	97.69	1.67	36.65	4,776

Bảng 4.14. Bảng tổng hợp kết quả huấn luyện tất cả các mô hình

Đối chiếu giữa giáo viên và học sinh, mức rút ngắn dao động từ khoảng bảy lần ở nhóm DNN và CNN đến khoảng hai mươi lăm lần ở nhóm SSL-CL-DNN, và lên tới hơn sáu trăm lần nếu so học sinh MLP2x64 với giáo viên LSTM. Điều này cho thấy lợi ích của chưng cất tri thức về mặt hiệu suất không cố định mà phụ thuộc vào việc mô hình giáo viên ban đầu "đắt" đến mức nào.

Tổng kết giai đoạn huấn luyện:

Nhìn ba biểu đồ cột cùng bảng tổng hợp, có thể rút ra các nhận định sau về giai đoạn huấn luyện:

Chưng cất tri thức đạt được mục tiêu cơ bản mà đề tài đặt ra. Các mô hình học sinh giảm từ khoảng 92% đến gần 99.7% số tham số so với giáo viên tương ứng, trong khi chỉ mất từ 0.46 đến 1.62 điểm accuracy và từ 1.91 đến 5.58 điểm macro-F1 ở nhóm chưng cất đơn giáo viên. Tổn thất tập trung rõ rệt ở macro-F1 chứ không ở accuracy, phản ánh việc phần tri thức khó truyền tải nhất chính là tri thức về các lớp tấn công hiếm như BruteForce và Web-Based. Trong khi đó macro-AUC gần như được bảo toàn ở mọi mô hình, cho thấy học sinh vẫn kế thừa được năng lực xếp hạng xác suất của giáo viên và phần thiệt hại nằm ở quyết định cuối cùng chứ không ở khả năng phân tách lớp.

Bốn mô hình học sinh SSL-CL-DNN đều vượt giáo viên CNN1D về macro-F1 (68.21 đến 69.02 so với 67.87), SSL-CL-DNN vượt qua

CNN1D về accuracy. Cho thấy một mô hình học sinh nhỏ hơn tám lần vẫn có thể tốt hơn một mô hình giáo viên thuộc kiến trúc khác.

Trong cùng một kiến trúc học sinh, khác biệt giữa bốn kỹ thuật chưng cất là nhỏ, dao động từ 0.81 điểm macro-F1 ở nhóm SSL-CL-DNN đến 1.21 điểm ở nhóm CNN. Chưng cất truyền thống cho macro-F1 cao nhất ở hai trong ba kiến trúc là SSL-CL-DNN và DNN, với FitNets bám ngay phía sau ở khoảng cách chưa tới 0.15 điểm; riêng ở kiến trúc CNN thì Raid-KL nhiệt độ thích ứng vươn lên dẫn đầu về macro-F1. Xét trên accuracy thì trật tự lại khác: FitNets đạt cao nhất ở SSL-CL-DNN và CNN, còn Raid-KL nhiệt độ thích ứng đạt cao nhất ở DNN. Việc thứ hạng thay đổi tùy theo kiến trúc và tùy theo chỉ số, trong khi biên độ chênh lệch chỉ khoảng một điểm, cho thấy không có kỹ thuật nào trong ba kỹ thuật này chiếm ưu thế rõ rệt trên dữ liệu bảng đặc trưng của bài toán.

Kiến trúc là thành phần tác động đến hiệu quả mạnh hơn hẳn lựa chọn kỹ thuật chưng cất, với khoảng cách macro-F1 giữa nhóm tốt nhất và nhóm kém nhất lên tới hơn 3 điểm, trong khi khoảng cách giữa bốn kỹ thuật trong cùng một kiến trúc chỉ khoảng 1 điểm. Các kiến trúc giáo viên trong khảo sát được kế thừa từ những nghiên cứu đi trước chứ không do luận văn tự thiết kế, và được chọn sao cho phủ được nhiều họ mô hình khác nhau, từ mạng toàn kết nối, mạng tích chập, mạng hồi quy, mô hình cây quyết định đến mô hình có giai đoạn học biểu diễn tự giám sát; mục tiêu là để phép so sánh giữa các kỹ thuật chưng cất không bị ràng buộc vào một kiểu kiến trúc duy nhất. Trong bốn nhóm đó, nhóm SSL-CL-DNN giữ lại được nhiều hiệu quả của giáo viên nhất sau khi nén. Nguyên nhân nằm ở chỗ Encoder của học sinh kế thừa cấu trúc biểu diễn đã hình thành trong giai đoạn học tương phản tự giám sát, nên khi chưng cất chỉ cần việc truyền tri thức phân loại, thay vì đồng thời gánh cả việc học biểu diễn từ đầu như hai kiến trúc còn lại. Nhóm CNN1D ngược lại tỏ ra kém phù hợp với dữ liệu bảng đặc trưng, vừa cho hiệu quả thấp nhất vừa tốn chi phí suy luận cao nhất trong ba kiến trúc học sinh.

Đáng chú ý là thứ hạng của học sinh không sao chép nguyên thứ hạng của giáo viên. Giáo viên DNN luôn có accuracy và macro-F1 cao hơn giáo viên SSL-CL-DNN, nhưng ở nhóm học sinh thì kết quả đảo chiều. Như vậy hiệu quả của mô hình giáo viên không dự báo trực tiếp được hiệu quả của mô hình học sinh, và mức hao hụt khi nén là một đại lượng riêng, phải đo bằng thực nghiệm cho từng cặp giáo viên và học sinh cụ thể.

Chưng cất đa giáo viên CW-MTKD tạo ra nhóm mô hình có đặc tính riêng biệt: nhỏ nhất, nhanh nhất, macro-F1 cao nhất trong toàn bộ học sinh, nhưng accuracy lại thấp nhất.

> Dù nhóm MLP2x64 đứng cuối bảng accuracy nhưng đứng đầu nhóm học sinh ở macro-F1, và chỉ còn kém giáo viên SSL-CL-DNN 0.82 điểm, sự đảo chiều hoàn toàn giữa hai biểu đồ này là minh họa trực quan nhất cho đặc tính của chưng cất đa giáo viên. Việc XGBoost tham gia vào cả bốn cặp giáo viên là yếu tố quyết định, vì đây là mô hình xử lý các lớp thiểu số tốt nhất trong khảo sát nhưng lại quá lớn để triển khai trực tiếp lên thiết bị biên.

Nhóm MLP2x64 vì vậy đóng vai trò là con đường duy nhất trong khảo sát này để đưa tri thức của mô hình cây quyết định lên thiết bị hạn chế tài nguyên, và dạng đánh đổi mà nó thể hiện là hy sinh một phần hiệu quả trên lớp đa số để phát hiện đồng đều hơn trên các lớp tấn công hiếm, đây là dạng đánh đổi phù hợp với bài toán an ninh mạng.

Cuối cùng, cần nhấn mạnh rằng toàn bộ nhận định trên mới chỉ dựa vào tập kiểm tra tách ra từ chính tập dữ liệu huấn luyện và các chỉ số chi phí đo trên môi trường Kaggle. Câu hỏi thực sự của đề tài là các mô hình này hoạt động ra sao trên lưu lượng mạng thực và trên phần cứng thực, và điều đó chỉ có thể trả lời ở phần tiếp theo.

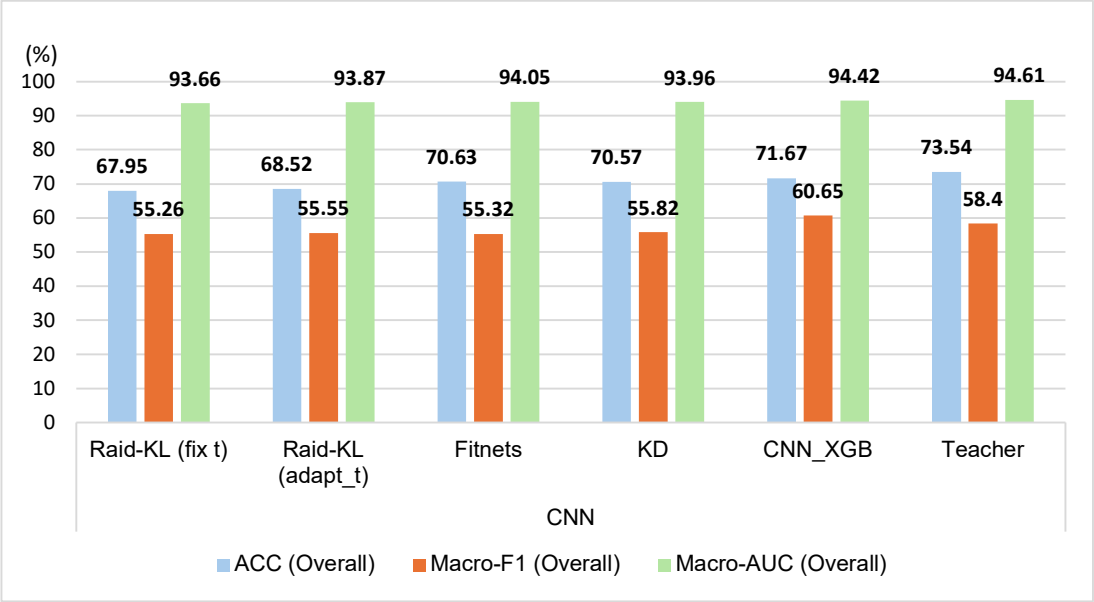
4.3. Các kết quả triển khai suy luận bằng các mô hình trên dữ liệu mạng PCAP tại thiết bị biên

4.3.1. Kết quả đánh giá của các mô hình học sinh và giáo viên trên dữ liệu mạng PCAP có nhãn tại thiết bị biên

Kết quả của các mô hình học sinh so với giáo viên khi suy luận trên dữ liệu mạng PCAP có sẵn nhãn có thể quan sát được qua các bảng và biểu đồ sau:

Model	**CNN**					
KD methods	Raid-KL (fix t)	Raid-KL (adapt_t)	FitNets	KD	CNN_XGB	Teacher
ACC (Overall)	67.95	68.52	70.63	70.57	**71.67**	73.54
Macro-F1 (Overall)	55.26	55.55	55.32	55.82	**60.65**	58.4
Macro-AUC (Overall)	93.66	93.87	94.05	93.96	**94.42**	94.61

Bảng 4.15. Kết quả đánh giá các mô hình học sinh CNN trên dữ liệu mạng PCAP



Hình 4.37. Kết quả đánh giá các mô hình học sinh CNN trên dữ liệu mạng PCAP

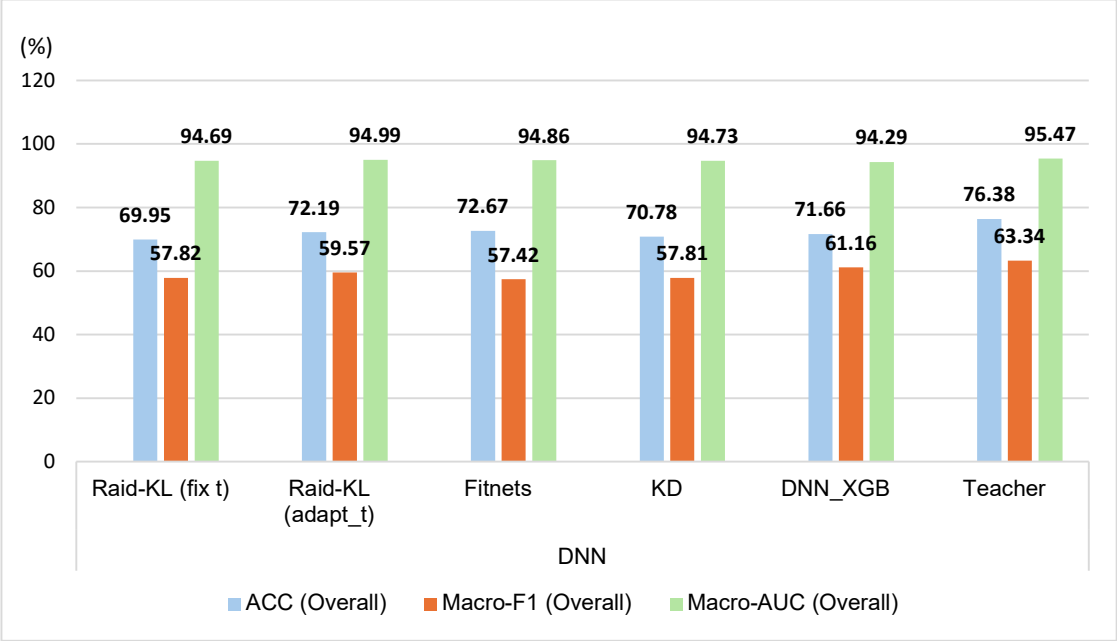
Điểm đầu tiên cần ghi nhận là toàn bộ các chỉ số của nhóm CNN đều thấp hơn đáng kể so với giai đoạn huấn luyện: accuracy nằm trong khoảng 67.95% đến 71.67% thay vì 83% đến 84%, macro-F1 chỉ còn 55.26% đến 60.65% thay vì 63% đến 65%. Mức sụt này xuất hiện ở cả mô hình giáo viên (73.54% so với 84.67% khi huấn luyện), nên không phải hệ quả của việc nén mô hình mà đến từ sự khác biệt giữa hai nguồn dữ liệu, sẽ được phân tích ở phần so sánh phía sau.

Trong nội bộ nhóm, hai kỹ thuật FitNets và KD truyền thống cho accuracy cao nhất (70.63% và 70.57%), tốt hơn nhiều Raid-KL. Riêng Raid-KL nhiệt độ cố định tiếp tục xếp cuối ở cả ba chỉ số, đúng như đã ghi nhận ở giai đoạn huấn luyện. Tuy nhiên khoảng cách macro-F1 giữa bốn kỹ thuật đơn giáo viên chỉ vỏn vẹn 0.56 điểm.

Kết quả đáng chú ý nhất của nhóm này thuộc về mô hình MLP2x64 chưng cất từ cặp XGBoost và CNN1D: dẫn đầu cả accuracy (71.67%) lẫn macro-F1 (60.65%), và vượt chính mô hình giáo viên CNN về macro-F1 (58.40%) tới 2.25 điểm. Đây là lần đầu tiên trong khảo sát một mô hình học sinh vượt được một mô hình giáo viên về hiệu quả phân loại, và điều đó xảy ra trên dữ liệu mạng thực chứ không phải trên tập kiểm tra. Nguyên nhân nằm ở chỗ mô hình này thừa hưởng tri thức từ XGBoost, vốn là mô hình xử lý các lớp thiểu số tốt nhất trong khảo sát, trong khi kiến trúc CNN1D lại là kiến trúc yếu nhất trên dữ liệu bảng đặc trưng.

Model	DNN					
KD methods	Raid-KL (fix t)	Raid-KL (adapt_t)	FitNets	KD	DNN_XGB	Teacher
ACC (Overall)	69.95	72.19	**72.67**	70.78	71.66	76.38
Macro-F1 (Overall)	57.82	59.57	57.42	57.81	**61.16**	63.34
Macro-AUC (Overall)	94.69	**94.99**	94.86	94.73	94.29	95.47

Bảng 4.16. Kết quả đánh giá của các mô hình học sinh DNN trên dữ liệu mạng PCAP



Hình 4.38. Kết quả đánh giá các mô hình học sinh DNN trên dữ liệu mạng PCAP

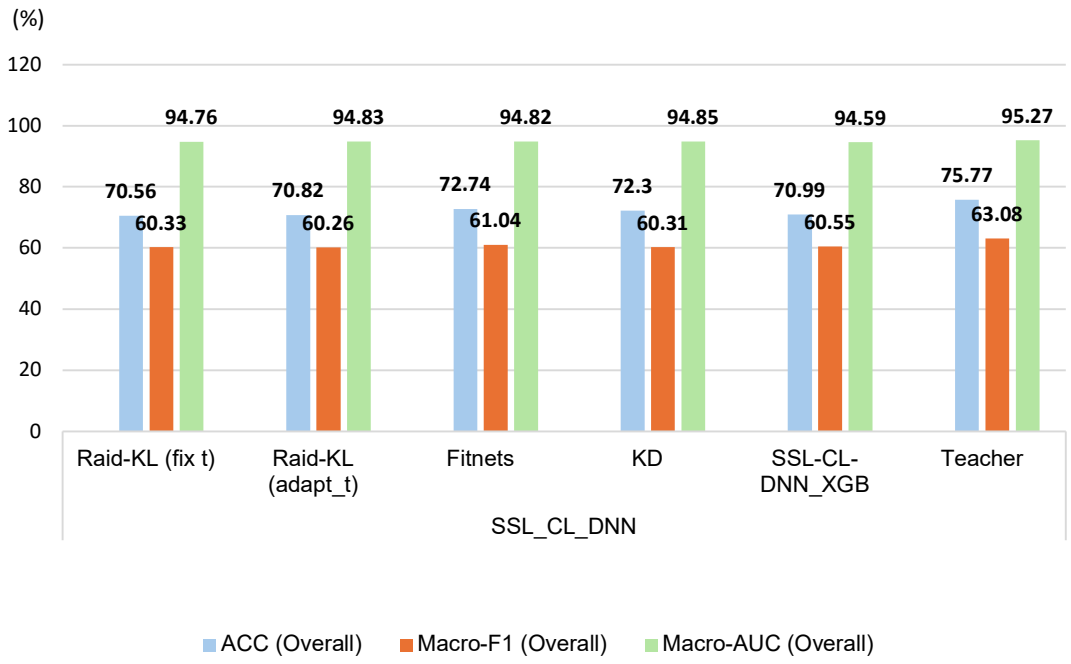
Nhóm DNN cho mức hiệu quả cao hơn nhóm CNN trên toàn bộ ba chỉ số, với giáo viên đạt 76.38% accuracy và 63.34% macro-F1 đây là cao nhất trong bốn mô hình giáo viên. Các mô hình học sinh đạt accuracy từ 69.95% đến 72.67% và macro-F1 từ 57.42% đến 61.16%.

Thứ hạng nội bộ nhóm thay đổi rõ rệt so với giai đoạn huấn luyện. Raid-KL nhiệt độ thích ứng vươn lên dẫn đầu macro-F1 trong nhóm đơn giáo viên (59.57%), vượt KD truyền thống 1.76 điểm. Ngược lại, FitNets đạt accuracy cao nhất (72.67%) nhưng lại có macro-F1 thấp nhất (57.42%), đúng đặc tính thiên về lớp đa số đã ghi nhận ở giai đoạn huấn luyện, và ở đây biểu hiện rõ hơn hẳn. Mô hình MLP2x64 từ cặp XGBoost và DNN tiếp tục dẫn đầu macro-F1 toàn nhóm (61.16%), dù accuracy chỉ ở mức trung bình.

Một chi tiết đáng lưu ý là Raid-KL nhiệt độ cố định, vốn luôn xếp cuối ở giai đoạn huấn luyện, lại đứng thứ hai về macro-F1 trong nhóm này (57.82%). Điều này gợi ý rằng thứ hạng giữa các kỹ thuật chưng cất không ổn định khi chuyển sang dữ liệu khác.

Model	**SSL_CL_DNN**					
KD methods	Raid-KL (fix t)	Raid-KL (adapt_t)	FitNets	KD	SSL-CL-DNN_XGB	Teacher
ACC (Overall)	70.56	70.82	**72.74**	72.3	70.99	75.77
Macro-F1 (Overall)	60.33	60.26	**61.04**	60.31	60.55	63.08
Macro-AUC (Overall)	94.76	94.83	94.82	**94.85**	94.59	95.27

Bảng 4.17. Kết quả đánh giá của các mô hình học sinh SSL-CL-DNN trên dữ liệu mạng PCAP



Hình 4.39. Kết quả đánh giá các mô hình học sinh SSL-CL-DNN trên dữ liệu mạng PCAP

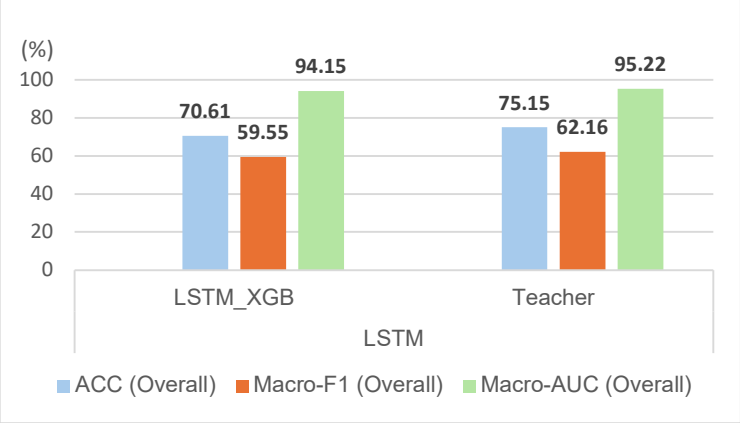
Nhóm SSL-CL-DNN là nhóm giữ được chất lượng tốt nhất khi chuyển sang dữ liệu thực. Bốn mô hình học sinh đơn giáo viên đạt macro-F1 rất sát nhau, từ 60.26% đến 61.04%, chỉ kém giáo viên khoảng hai điểm và khoảng cách hẹp nhất trong ba kiến trúc chính. Kết luận rút ra ở giai đoạn huấn luyện về việc kiến trúc này hao hụt ít nhất khi nén vì vậy được xác nhận lại trên dữ liệu thực.

FitNets đạt đồng thời accuracy cao nhất (72.74%) và macro-F1 cao nhất (61.04%) trong nhóm, còn KD truyền thống bám sát phía sau. Khác biệt giữa bốn kỹ thuật đơn giáo viên về macro-F1 chỉ 0.78 điểm, tức gần như không phân biệt được.

Điểm khác biệt so với hai nhóm trước là mô hình MLP2x64 không còn dẫn đầu macro-F1 (60.55%, xếp sau FitNets). Điều này hợp lý vì các học sinh SSL-CL-DNN vốn đã xử lý các lớp hiếm tương đối tốt, nên lợi thế mà chưng cất đa giáo viên mang lại bị thu hẹp, chưng cất đa giáo viên phát huy tác dụng mạnh nhất ở đúng những kiến trúc mà chưng cất đơn giáo viên còn yếu.

Model	Models	
KD methods	LSTM_XGB	Teacher
ACC (Overall)	70.61	75.15
Macro-F1 (Overall)	59.55	62.16
Macro-AUC (Overall)	94.15	95.22

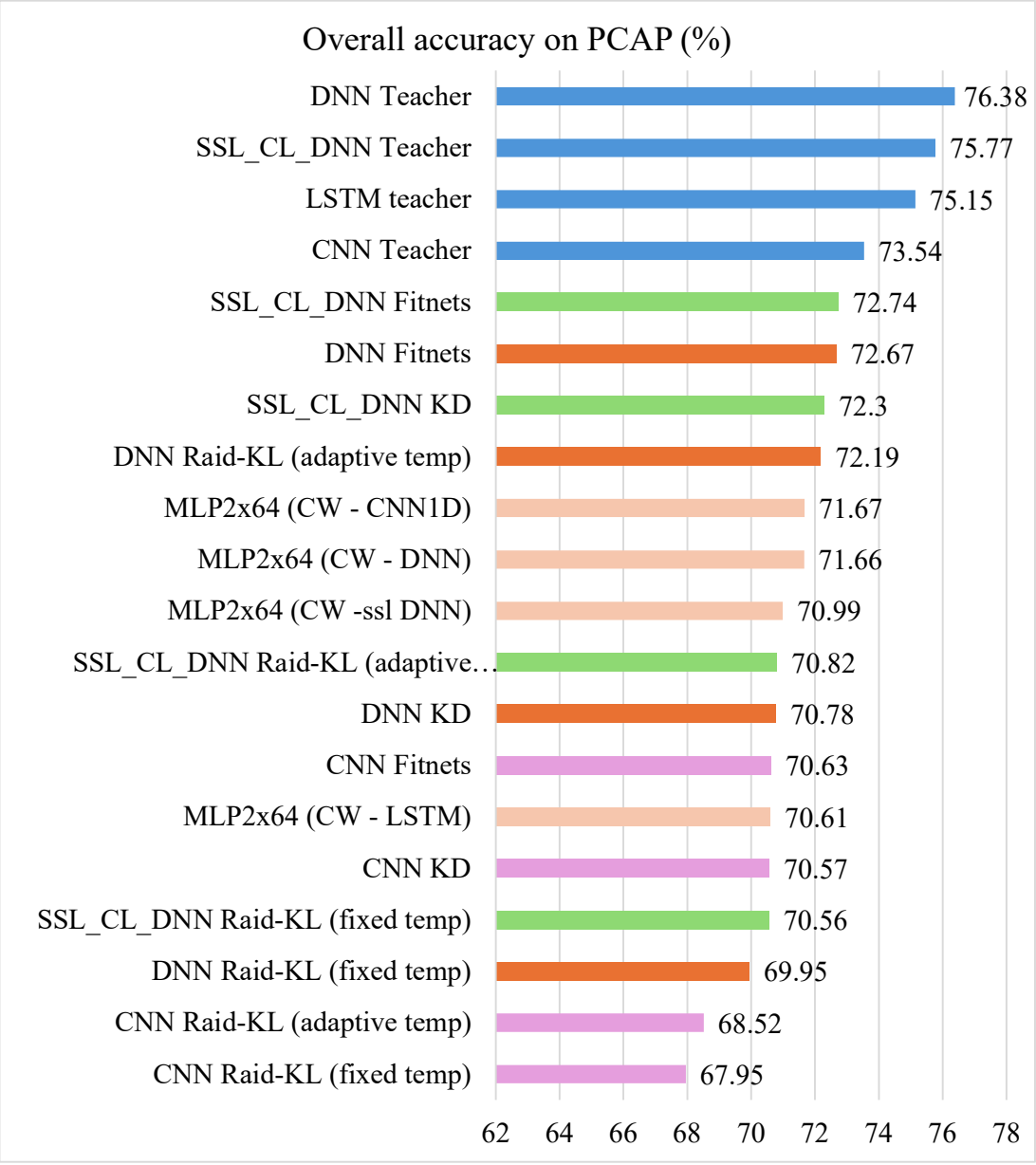
Bảng 4.18. Kết quả đánh giá của các mô hình học sinh LSTM trên dữ liệu mạng PCAP



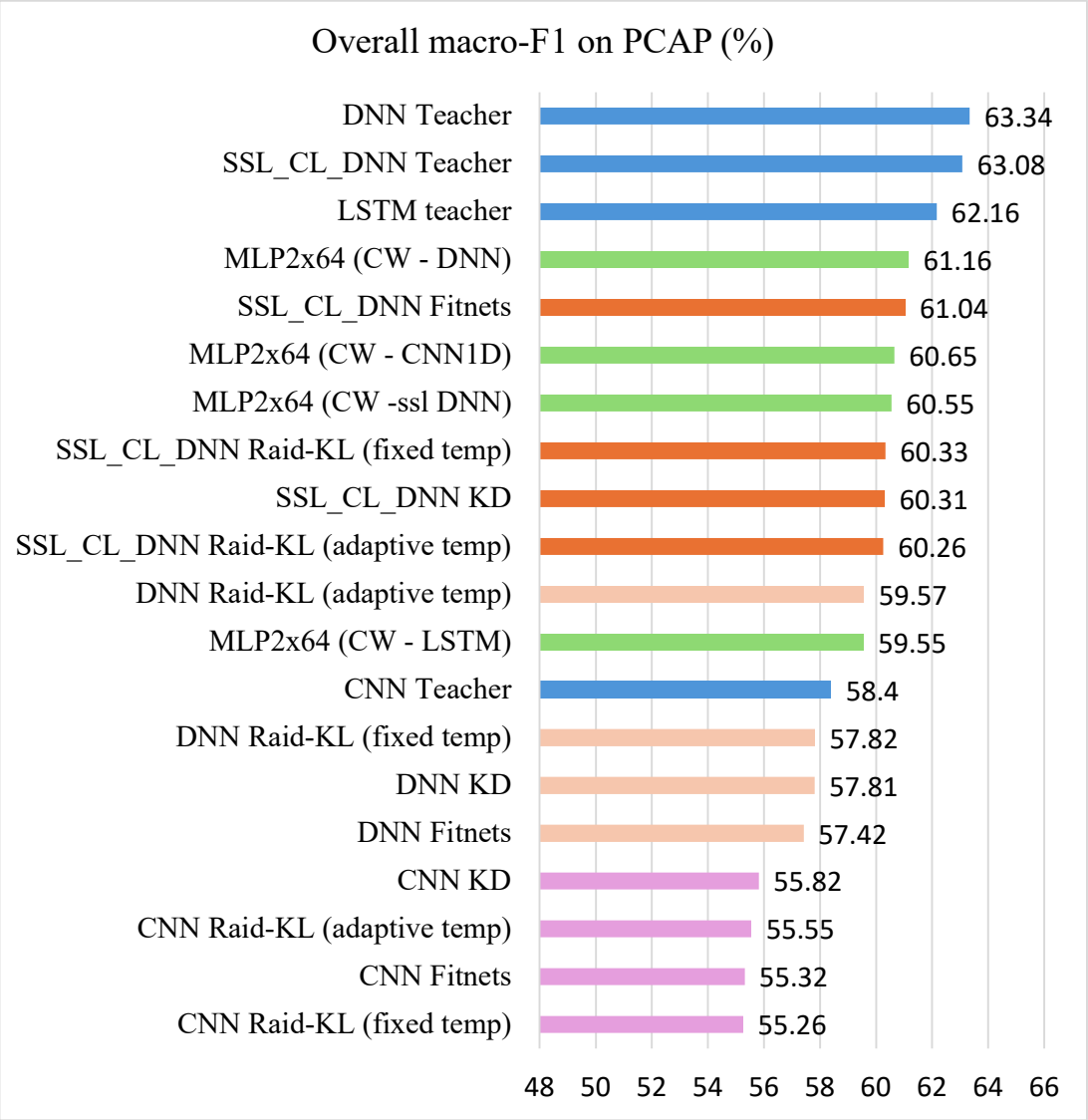
Hình 4.40. Kết quả đánh giá mô hình LSTM học sinh so với giáo viên trên dữ liệu mạng PCAP

Giáo viên LSTM đạt 75.15% accuracy và 62.16% macro-F1, thuộc nhóm giáo viên mạnh trên dữ liệu thực. Mô hình học sinh MLP2x64 chưng cất từ cặp XGBoost và LSTM đạt 70.61% và 59.55%, tức kém giáo viên 4.54 và 2.61 điểm. Đây là cặp duy nhất trong khảo sát chuyển tri thức từ một mạng hồi quy sang một mạng truyền thẳng đơn giản, và với mức hao hụt chỉ khoảng 2.6 điểm macro-F1 trên dữ liệu thực trong khi mô hình học sinh nhỏ hơn giáo viên khoảng hai mươi bốn lần về số tham số, có thể nói việc chuyển đổi này là khả thi. Về hiệu suất: giáo viên LSTM là mô

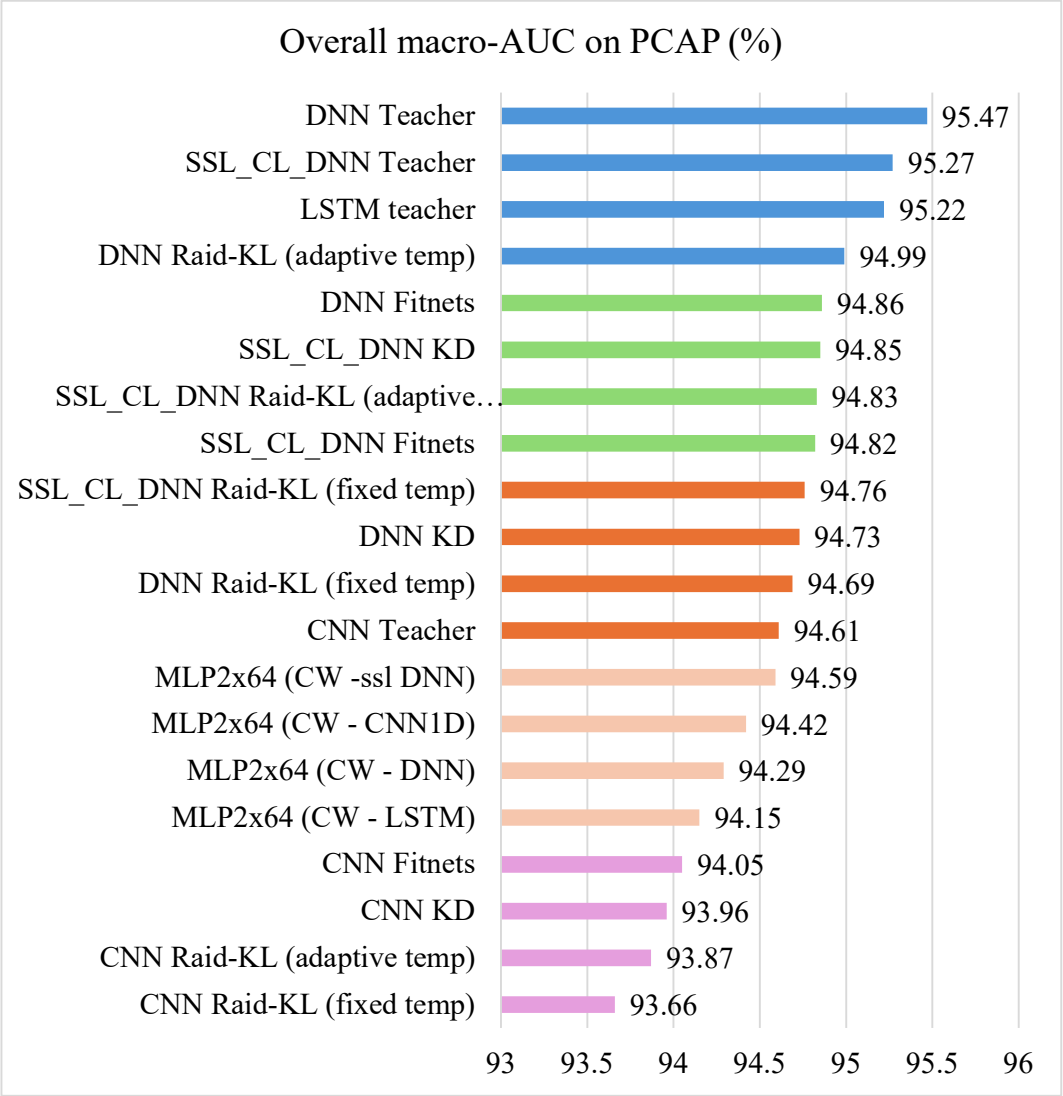
hình chậm nhất trong toàn bộ khảo sát, nên đây cũng là cặp mà chưng cất mang lại lợi ích lớn nhất về chi phí triển khai.



Hình 4.41. Kết quả accuracy của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn



Hình 4.42. Kết quả macro-F1 của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn



Hình 4.43. Kết quả macro-AUC của tất cả mô hình sau khi suy luận trên dữ liệu mạng PCAP có nhãn

Quan sát ba biểu đồ xếp hạng theo accuracy, macro-F1 và macro-AUC, có thể rút ra các nhận định sau.

Bốn mô hình giáo viên chiếm trọn nhóm dẫn đầu về accuracy, từ 73.54% đến 76.38%, tách hẳn khỏi các mô hình học sinh nằm trong khoảng 67.95% đến 72.74%. Nhưng ở biểu đồ macro-F1, giáo viên CNN tụt xuống vị trí thứ mười ba, đứng sau chín mô hình học sinh. Sự đảo chiều này lặp lại đúng hiện tượng đã thấy ở giai đoạn huấn luyện, khẳng định accuracy không phải chỉ số phù hợp cho bài toán phát hiện xâm nhập trên dữ liệu mất cân bằng.

Thứ hạng giữa các nhóm kiến trúc được bảo toàn. Ở macro-F1, nhóm MLP2x64 và nhóm SSL-CL-DNN đan xen ở phía trên (59.55% đến 61.16%), nhóm DNN nằm giữa (57.42% đến 59.57%), nhóm CNN xếp cuối (55.26% đến 55.82%) đúng trật tự đã xác lập khi huấn luyện. Việc trật tự này giữ nguyên trên dữ liệu mạng thực cho thấy các nhận định về kiến trúc phản ánh đặc tính thật của mô hình, không phải hiệu ứng riêng của tập kiểm tra.

Ngược lại, thứ hạng giữa các kỹ thuật chưng cất trong cùng một kiến trúc thì không giữ nguyên. Khi huấn luyện, KD truyền thống dẫn đầu macro-F1 ở hai trong ba kiến trúc; trên dữ liệu PCAP, vị trí dẫn đầu lần lượt thuộc về KD ở nhóm CNN, Raid-KL thích ứng ở nhóm DNN và FitNets ở nhóm SSL-CL-DNN. Raid-KL nhiệt độ cố định vốn luôn xếp cuối khi huấn luyện thì nay lên vị trí thứ hai ở hai nhóm. Đáng chú ý là chênh lệch macro-F1 giữa bốn kỹ thuật trong cùng một nhóm chỉ khoảng 0.5 đến 2.1 điểm, nhỏ hơn nhiều so với chênh lệch giữa các kiến trúc. Khoảng cách nhỏ như vậy dễ bị thay đổi khi đổi dữ liệu đánh giá, nên không đủ cơ sở để khẳng định kỹ thuật nào tốt hơn kỹ thuật nào; kết luận hợp lý là bốn kỹ thuật cho hiệu quả tương đương trên bài toán này.

Chỉ một quy luật về kỹ thuật là bền vững qua cả hai giai đoạn: xét trên accuracy, FitNets cao nhất ở cả ba kiến trúc còn Raid-KL nhiệt độ cố định thấp nhất ở cả ba. Điều này khớp với đặc tính đã nêu khi huấn luyện, rằng giai đoạn khớp gợi ý tầng ẩn giúp mô hình bám sát phân phối tổng thể của dữ liệu vốn bị chi phối bởi các lớp đa số nhưng không cải thiện khả năng phân biệt các lớp hiếm.

Về macro-AUC, cả hai mươi mốt mô hình nằm trong dải hẹp từ 93.66% đến 95.47%, thứ tự gần trùng với macro-F1. Mô hình DNN Raid-KL thích ứng đạt 94.99%, vượt cả giáo viên CNN (94.61%).

So sánh với kết quả huấn luyện, mức sụt giảm là đồng đều: accuracy giảm 9 đến 13 điểm, macro-F1 giảm 8 đến 9.5 điểm, macro-AUC chỉ giảm 2.5 đến 3.5 điểm. Quan trọng là mức giảm này xảy ra với cả giáo viên lẫn học sinh ở biên độ tương đương, nên nguyên nhân không nằm ở chưng cất tri thức.

Nguyên nhân đến từ khác biệt giữa hai nguồn dữ liệu. Tập CSV do UNB công bố được trích xuất với kích thước cửa sổ thay đổi theo loại lưu lượng, còn quy trình của luận văn phải dùng cửa sổ cố định mười gói tin vì khi vận hành thật mô hình không biết trước loại lưu lượng đang đến. Bên cạnh đó, bộ trích xuất đặc trưng của luận văn là bản tự triển khai bám theo mã nguồn gốc của UNB CIC, trong đó tập đặc trưng đã được điều chỉnh còn 37 đặc trưng nhằm loại bỏ sự phụ thuộc vào kích thước cửa sổ thay đổi theo nhãn (một điều chỉnh để tăng chính xác cho kết quả). Tuy vậy,

bản triển khai này vẫn chưa đạt độ chính xác hoàn toàn tương đương với bộ trích xuất gốc, nên véc-tơ đặc trưng sinh ra từ PCAP còn sai lệch nhất định so với véc-tơ mà mô hình đã học, và đây là một phần nguyên nhân của mức sụt giảm. Ngoài ra nhãn được gán theo tập tin nguồn, nên một cửa sổ mười gói tin có thể lẫn lưu lượng nền, làm nhòe ranh giới giữa các lớp.

Điểm đáng chú ý nhất là khoảng cách macro-F1 giữa học sinh và giáo viên không rộng hơn khi chuyển sang dữ liệu thực. Ở nhóm CNN, khoảng cách thu hẹp từ 3.32 xuống 2.58 điểm và mô hình MLP2x64 còn vượt giáo viên. Ở nhóm DNN, khoảng cách giảm từ 4.50 xuống 3.77 điểm. Ở nhóm SSL-CL-DNN, khoảng cách giữ nguyên quanh 2 điểm. Như vậy phần tri thức học sinh kế thừa được vẫn phát huy tác dụng khi dữ liệu đầu vào thay đổi, tức chưng cất không tạo ra mô hình chỉ chạy tốt trong điều kiện phòng thí nghiệm.

Xét trên accuracy thì ngược lại, khoảng cách từ khoảng 1 điểm lên khoảng 3 điểm ở cả ba kiến trúc. Gộp hai quan sát: khi phân bố dữ liệu thay đổi, mô hình học sinh mất nhiều hơn giáo viên ở các lớp đa số nhưng giữ được tương đương ở các lớp thiểu số.

Cuối cùng, việc macro-AUC chỉ giảm khoảng ba điểm trong khi accuracy giảm hơn mười điểm cho thấy các mô hình vẫn phân biệt được giữa các lớp trên dữ liệu mới; phần lớn tổn thất đến từ ngưỡng quyết định học được từ tập huấn luyện không còn phù hợp với phân bố dữ liệu mới. Điều này có giá trị thực tiễn, vì hiệu chỉnh lại ngưỡng theo dữ liệu triển khai có thể thu hồi một phần đáng kể khoảng cách mà không cần huấn luyện lại mô hình.

4.3.2. Kết quả hiệu suất của các mô hình học sinh và mô hình giáo viên trên dữ liệu mạng PCAP tại thiết bị biên.

Qua quy trình suy luận bằng các mô hình học sâu giáo viên và học sinh sau chưng cất tri thức, đề tài thu thập được các bảng kết quả của các chỉ số đánh giá đã lựa chọn:

Models	Packet/s (On Pi)	Packet/s (On Kaggle)
MLP(SSL_CL_DNN_XGB)	4171.6	11610.5
DNN_adapt_t	4166.8	11597.0
DNN_KD	4165.8	11600.8
DNN_Fitnets	4165.3	11611.4

DNN_fix_t	4165.2	11601.6
SSL_CL_DNN_fix_t	4159.8	11603.0
SSL_CL_DNN_KD	4160.2	11562.5
MLP(LSTM_XGB)	4159.2	11566.0
SSL_CL_DNN_adapt_t	4159	11621.5
MLP(CNN_XGB)	4158.4	11538.9
SSL_CL_DNN_Fitnets	4156.8	11616.7
MLP(DNN_XGB)	4156	11563.8
CNN_adapt_t	4089.9	11416.4
CNN_Fitnets	4086.8	11381.8
CNN_KD	4087.1	11379.7
CNN_fix_t	4086	11434.4
teacher_DNN	3873.4	11375.3
teacher_CNN	3293.3	10422.4
teacher_SSL_CL_DNN	3198.5	10443.3
teacher_LSTM	1544.5	5089.2

Bảng 4.19. Kết quả số gói tin trên giây của các mô hình trên thiết bị so với trên môi trường huấn luyện

Kết quả thông lượng cho thấy một điều mà giai đoạn huấn luyện không thể bộc lộ: mười sáu mô hình học sinh cho tốc độ xử lý gần như bằng nhau, dao động từ 4,086 đến 4,172 gói tin mỗi giây, tức chênh lệch chưa tới 2.1%. Trong khi đó ở giai đoạn huấn luyện, thời gian suy luận giữa mô hình học sinh nhanh nhất và chậm nhất chênh nhau hơn sáu lần. Nguyên nhân là khi đưa vào quy trình hoàn chỉnh, chi phí đọc gói tin và trích xuất đặc trưng chiếm phần áp đảo, còn chi phí chạy mô hình chỉ là một phần nhỏ. Nói cách khác, nút thắt của hệ thống nằm ở khâu tiền xử lý chứ không nằm ở mô hình phân loại.

Chỉ khi mô hình đủ nặng thì ảnh hưởng mới hiện ra rõ. Bốn mô hình giáo viên đều kéo thông lượng xuống thấy được, từ 3,873 gói mỗi giây với DNN, xuống 3,293 với CNN, 3,198 với SSL-CL-DNN, và chỉ còn 1,544 với LSTM. Riêng LSTM làm mất hơn 60% thông lượng so với các mô hình học sinh, phù hợp với đặc tính xử lý tuần tự đã ghi nhận ở giai đoạn huấn luyện. Đây chính là bằng chứng thực tế cho thấy vì sao các mô hình giáo viên không phù hợp để triển khai trực tiếp lên thiết bị biên.

So với môi trường Kaggle, thiết bị Raspberry Pi 4 B chậm hơn khoảng 2.8 lần với các mô hình học sinh (khoảng 4,160 so với khoảng 11,600 gói mỗi giây). Tỉ lệ này ổn định trên toàn bộ các mô hình, cho thấy thiết bị không gặp hiện tượng nghẽn bất thường mà chỉ đơn thuần có năng lực tính toán thấp hơn.

Models	Inferencing time on Pi (µs/sample)	Inferencing time on Kaggle (µs/sample)
MLP(SSL_CL_DNN_XGB)	122.3	1.66
DNN_adapt_t	125.1	2.46
DNN_KD	125.6	2.46
DNN_Fitnets	126	2.46
DNN_fix_t	126	2.46
SSL_CL_DNN_fix_t	129.1	3.35
SSL_CL_DNN_KD	128.9	3.36
MLP(LSTM_XGB)	129.5	1.67
SSL_CL_DNN_adapt_t	129.6	3.35
MLP(CNN_XGB)	129.9	1.66
SSL_CL_DNN_Fitnets	130.9	3.35
MLP(DNN_XGB)	131.3	1.66
CNN_adapt_t	170.2	10.51
CNN_Fitnets	172.1	10.44
CNN_KD	171.9	10.35
CNN_fix_t	172.6	10.39
teacher_DNN	306.9	17.61
teacher_CNN	761.6	77.44
teacher_SSL_CL_DNN	851.7	84.38
teacher_LSTM	4199.9	1013.06

Bảng 4.20. Kết quả chỉ số thời gian suy luận bằng các mô hình trên thiết bị so với trên môi trường huấn luyện

Bảng này tách riêng chi phí bản thân mô hình khỏi cấu trúc chi phí của quy trình. Trên thiết bị, toàn bộ mô hình học sinh nằm trong khoảng 122.3 đến 172.6 µs mỗi cửa sổ, trong khi độ trễ cả quy trình vào khoảng 2,400 µs. Như vậy mô hình chỉ chiếm khoảng 5% đến 7% tổng chi phí, phần còn lại thuộc về khâu đọc gói và trích xuất đặc trưng và con số này giải thích trực tiếp vì sao thông lượng của mười sáu mô hình học sinh gần như không phân biệt được. Trật tự giữa các nhóm học sinh vẫn được giữ nhưng biên độ co lại mạnh. Trên Kaggle, nhóm CNN chậm hơn nhóm DNN khoảng 4 lần; trên thiết bị, khoảng cách chỉ còn khoảng 1.4 lần (khoảng 171 µs so với khoảng 126 µs). Sự co lại này là do phần chi phí cố định của môi trường thực thi TensorFlow Lite trên thiết bị chiếm tỉ trọng lớn hơn, làm nhòe bớt khác biệt về khối lượng tính toán thuần túy giữa các kiến trúc.

Ở phía giáo viên, khác biệt vẫn rất rõ: DNN tốn 306.9 µs, CNN 761.6 µs, SSL-CL-DNN 851.7 µs và LSTM 4,199.9 µs. Riêng LSTM chiếm tới khoảng hai phần ba tổng độ trễ quy trình, tức mô hình đã trở thành nút thắt chính thay vì khâu trích xuất.

Models	Pipeline latency (ms) on Pi	Pipeline latency (ms) on Kaggle
MLP(SSL_CL_DNN_XGB)	2.3972	0.8613
DNN_adapt_t	2.3999	0.8623
DNN_KD	2.4005	0.862
DNN_Fitnets	2.4008	0.8612
DNN_fix_t	2.4008	0.8619
SSL_CL_DNN_fix_t	2.4039	0.8618
SSL_CL_DNN_KD	2.4037	0.8649
MLP(LSTM_XGB)	2.4043	0.8646
SSL_CL_DNN_adapt_t	2.4044	0.8605
MLP(CNN_XGB)	2.4047	0.8666
SSL_CL_DNN_Fitnets	2.4057	0.8608
MLP(DNN_XGB)	2.4061	0.8648
CNN_adapt_t	2.4451	0.8759
CNN_Fitnets	2.4469	0.8786
CNN_KD	2.4467	0.8788
CNN_fix_t	2.4474	0.8746
teacher_DNN	2.5817	0.8791
teacher_CNN	3.0365	0.9595
teacher_SSL_CL_DNN	3.1265	0.9575
teacher_LSTM	6.4747	1.965

Bảng 4.21. Kết quả chỉ số độ trễ quy trình khi suy luận bằng các mô hình trên thiết bị so với trên môi trường huấn luyện

Độ trễ quy trình củng cố toàn bộ nhận định trên. Mười sáu mô hình học sinh nằm trong dải rất hẹp từ 2.397 đến 2.447 ms cho mỗi cửa sổ, trong đó bốn mô hình CNN nhỉnh hơn phần còn lại khoảng 0.05 ms đúng với việc kiến trúc tích chập tốn nhiều phép tính hơn nhưng không quá đáng để kể. Các mô hình giáo viên làm độ trễ tăng lên 2.58 ms với DNN, khoảng 3.0 đến 3.1 ms với CNN và SSL-CL-DNN, và 6.47 ms với LSTM. Với độ trễ khoảng 2.4 ms cho mỗi cửa sổ mười gói tin, hệ thống đưa ra phán đoán gần như tức thời theo thang thời gian của con người.

Models	Pipeline CPU usages (%)	Pipeline AVG RAM usages (MB)	Pipeline Peak RAM (MB)
MLP(SSL_CL_DNN_XGB)	25	105.5	105.5
DNN_adapt_t	25	105.5	105.5
DNN_KD	25	105.5	105.5
DNN_Fitnets	25	105.5	105.5
DNN_fix_t	25	105.5	105.5
SSL_CL_DNN_fix_t	25	105.6	105.6
SSL_CL_DNN_KD	25	105.6	105.6
MLP(LSTM_XGB)	25	105.5	105.5
SSL_CL_DNN_adapt_t	25	105.6	105.6
MLP(CNN_XGB)	25	105.5	105.5
SSL_CL_DNN_Fitnets	25	105.6	105.6
MLP(DNN_XGB)	25	105.5	105.5
CNN_adapt_t	25	105.3	105.3
CNN_Fitnets	25	105.5	105.5
CNN_KD	25	105.5	105.5
CNN_fix_t	25	105.5	105.5
teacher_DNN	25	106.1	106.1
teacher_CNN	25	106.1	106.1
teacher_SSL_CL_DNN	25.1	107.7	107.7
teacher_LSTM	25	106.1	106.1

Bảng 4.22. Kết quả chỉ số chiếm dụng CPU và RAM của quy trình suy luận bằng mô hình trên thiết bị biên

Mức chiếm dụng tài nguyên gần như không phân biệt được giữa các mô hình: CPU luôn ở mức 25% và RAM đỉnh dao động rất hẹp quanh 105.5 MB với các mô hình học sinh, 106.1 đến 107.7 MB với các mô hình giáo viên. Con số CPU 25% trên thiết bị bốn nhân cho thấy quy trình đang chạy đơn luồng và bão hòa đúng một nhân.

Về bộ nhớ, mức tiêu thụ khoảng 105 MB trên tổng 4 GB của thiết bị là rất nhẹ, và điều đáng chú ý là ngay cả mô hình giáo viên nặng nhất cũng chỉ làm tăng thêm khoảng 2 MB. Phần lớn bộ nhớ được dùng cho bản thân chương trình và thư viện chứ không phải cho trọng số mô hình. Hệ quả là ưu thế về dung lượng của các mô hình học sinh rất ấn tượng trên giấy tờ, từ hàng nghìn KB xuống vài chục KB tuy nhiên lại không chuyển hóa thành lợi ích rõ rệt về RAM khi vận hành. Lợi ích thực sự của việc nén mô hình trên thiết bị biên nằm ở tốc độ và độ trễ, không nằm ở bộ nhớ.

Thông số so sánh	Hệ thống Realguard [21]	Hệ thống Kitsune [22]	Quy trình suy luận của luận văn
Tập dữ liệu	CICIDS2017	Tự thu thập dữ liệu mạng	CICIoT2023
Phương pháp học	Có giám sát (DNN)	Không giám sát (Autoencoders)	Có giám sát (nhiều mô hình)
Độ chính xác (%)	**99.93%**	94.47%	72.74% (trên PCAP có nhãn, SSL-CL-DNN_FitNets)
Thiết bị IoT biên	Raspberry Pi 4 B	Raspberry Pi 3 B	Raspberry Pi 4 B
Tốc độ thực thi	**10.600 gói/giây**	5.400 gói/giây	4171.6 gói/giây (học sinh MLP(SSL-CL-DNN_XGB))
Sử dụng CPU	36.00%	33.80%	**25.1%**
Sử dụng RAM	114.5 MB	156.3 MB	**105.71 MB**

Bảng 4.23. Kết quả suy luận và hiệu suất của các hệ thống IDS liên quan đến đề tài

Về độ chính xác, ba con số trong bảng không so sánh trực tiếp được với nhau và cần đọc kèm điều kiện đo. Kitsune giải bài toán phát hiện bất thường hai lớp theo hướng không giám sát trên dữ liệu tự thu thập, Realguard đánh giá trên CICIDS2017 bằng mô hình học trực tiếp trên dữ liệu PCAP và cách huấn luyện rất khác huấn luyện offline truyền thống, còn luận văn phân loại 8 nhãn nhóm trên CICIoT2023 với dữ liệu đi lại từ tập tin PCAP thô qua bộ trích xuất tự triển khai dù bám theo mã nguồn tác giả, có cải thiện nhưng chưa hiệu quả.

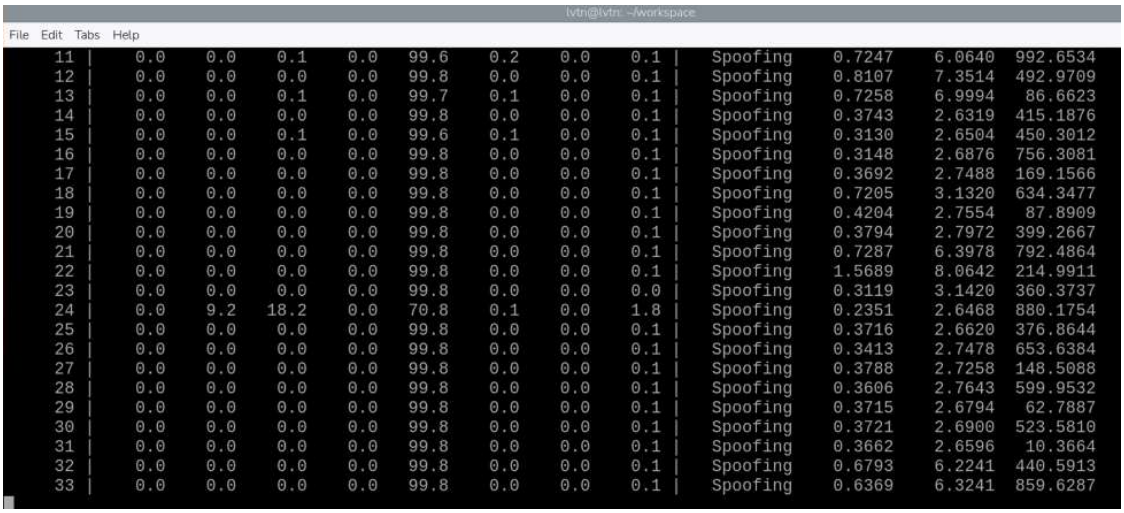
So với hai hệ thống đối chiếu, quy trình của luận văn tiêu tốn ít tài nguyên hơn ở cả CPU và bộ nhớ, dù phải giải bài toán phân loại tám nhãn nhóm trên tập dữ liệu IoT có mức độ đa dạng lưu lượng cao. Điều này xác nhận rằng các mô hình học sinh sau chưng cất đủ gọn để không trở thành gánh nặng cho thiết bị biên. Đổi lại, thông lượng thấp hơn cả hai hệ thống còn lại. Nguyên nhân chính đã được chỉ ra qua các bảng trước: quy trình hiện được triển khai đơn luồng, chỉ khai thác một trong bốn nhân của thiết bị, và phần lớn thời gian dành cho khâu trích xuất đặc trưng chứ không phải cho mô hình phân loại.

4.3.3. Kết quả thực nghiệm khi suy luận trực tuyến trên dữ liệu mạng tự thu thập trên thiết bị biên

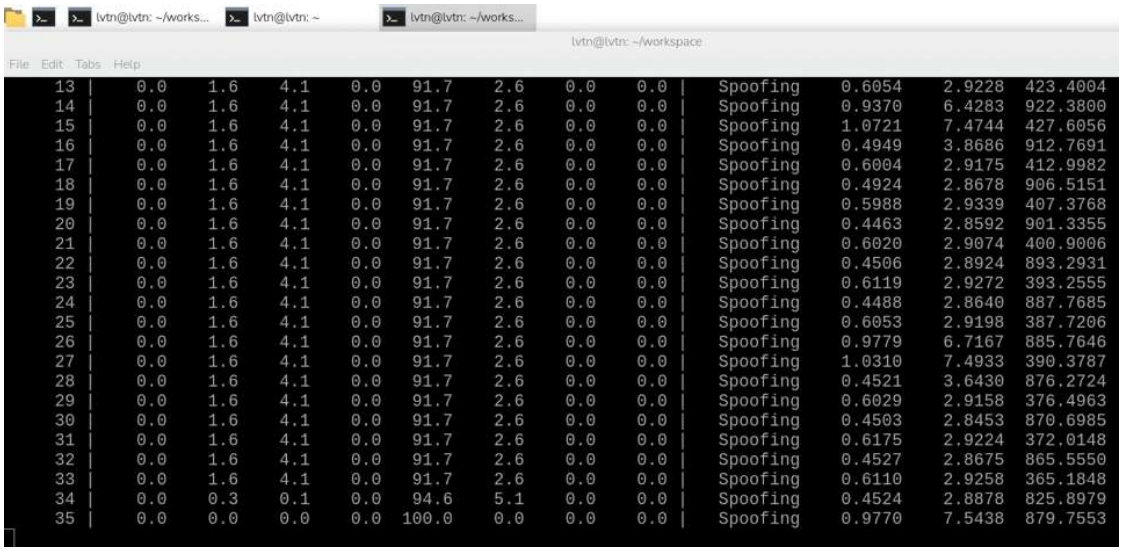
Vì trong bài toán phát hiện tấn công mạng đa lớp, với dữ liệu mất cân bằng thì chỉ số macro-F1 thường được chú trọng hơn. Dựa vào kết quả phân loại, suy luận của các mô hình học sinh và giáo viên trên dữ liệu mạng PCAP có nhãn, bước thử nghiệm suy luận trực tuyến sử dụng mô hình học sinh có macro-F1 tốt nhất và mô hình giáo viên có macro-F1 cao nhất đó là mô hình học sinh chưng cất bằng đa giáo viên MLP2x64 (CW-DNN) với macro-F1 là 61.16%, còn mô hình giáo viên là DNN (63.34% macro-F1). Quy trình được thực hiện làm nóng (warm-up) thiết bị và quy trình trước, sau đó mới chính thức tính kết quả phân loại và đo hiệu suất.

Lần lượt sau khi được warm-up, luận văn sẽ kiểm tra kết nối ping tới thiết bị Pi 4, quét host sống trên nhánh mạng, quét cổng bằng kĩ thuật quét nửa mở hoặc quét ẩn (half-open scan/stealth scan), quét hệ điều hành, tấn công vét cạn mật khẩu (brute-force) trên cổng dịch vụ 22 (SSH), làm ngập TCP SYN liên tục thông thường và với IP ngẫu nhiên. Các kết quả output sinh ra bởi giáo viên và học sinh sẽ được ghi nhận để nhận xét, so sánh, đánh giá và đưa ra kết luận.

Bước warm-up:



Hình 4.44. Dự đoán của mô hình học sinh cho các gói tin warm-up ban đầu



13	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6054	2.9228	423.4004
14	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.9370	6.4283	922.3800
15	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	1.0721	7.4744	427.6056
16	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4949	3.8686	912.7691
17	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6004	2.9175	412.9982
18	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4924	2.8678	906.5151
19	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.5988	2.9339	407.3768
20	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4463	2.8592	901.3355
21	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6020	2.9074	400.9006
22	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4506	2.8924	893.2931
23	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6119	2.9272	393.2555
24	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4488	2.8640	887.7685
25	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6053	2.9198	387.7206
26	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.9779	6.7167	885.7646
27	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	1.0310	7.4933	390.3787
28	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4521	3.6430	876.2724
29	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6029	2.9158	376.4963
30	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4503	2.8453	870.6985
31	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6175	2.9224	372.0148
32	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.4527	2.8675	865.5550
33	0.0	1.6	4.1	0.0	91.7	2.6	0.0	0.0	Spoofing	0.6110	2.9258	365.1848
34	0.0	0.3	0.1	0.0	94.6	5.1	0.0	0.0	Spoofing	0.4524	2.8878	825.8979
35	0.0	0.0	0.0	0.0	100.0	0.0	0.0	0.0	Spoofing	0.9770	7.5438	879.7553

Hình 4.45. Dự đoán của mô hình giáo viên sau bước warm-up

Lệnh sử dụng để warm-up: *sudo hping3 -S 192.168.50.10 --fast*

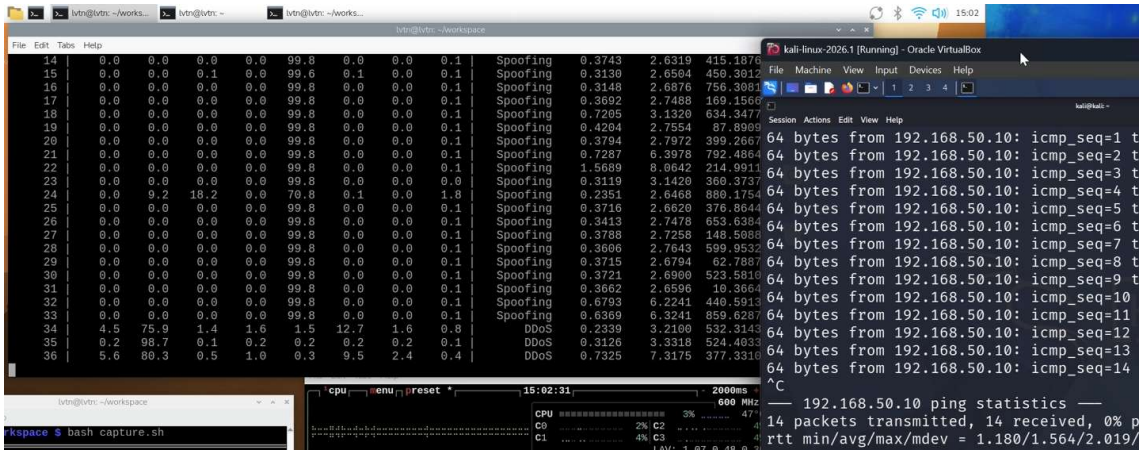
Trong khoảng hơn 30 gói tin dự đoán đầu tiên phục vụ cho việc warm-up quy trình lên bằng mô hình học sinh và giáo viên đều dự đoán là Spoofing. Dù chỉ là để warm-up thì kết quả hiệu suất cũng có khát biệt:

Mô hình học sinh có tốc độ suy luận trải từ 0.25 ms đến 0.72 ms (có 1 dự đoán mất hơn 1 ms), mô hình giáo viên lại trải từ 0.44 đến 0.97 ms (có 2 gói tin mất hơn 1 ms). Và độ trễ cả quy trình ở mô hình học sinh nhìn chung ít hơn mô hình giáo viên ở rất nhiều dự đoán, tuy nhiên đây chỉ là bước khởi động.

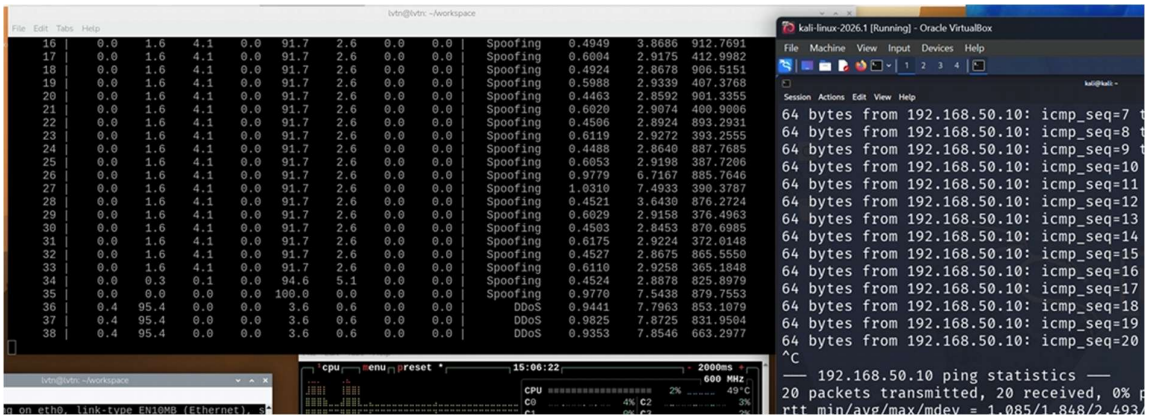
Lệnh kiểm tra kết nối bằng ICMP ping:

Lệnh sử dụng: *ping 192.168.50.10*

Kết quả thực hiện quan sát các ảnh thu thập:



Hình 4.46. Dự đoán của mô hình học sinh sau khi ping



Hình 4.47. Dự đoán của mô hình giáo viên sau khi ping

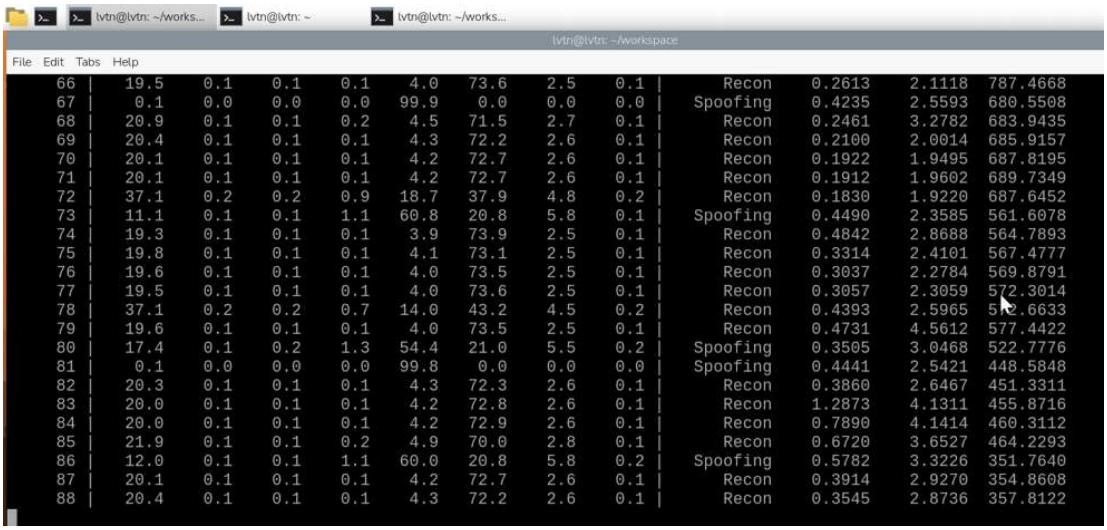
Khi ping hơn 14 gói tin, cả hai mô hình học sinh MLP2x64 (CW-DNN) và giáo viên DNN đều dự đoán ra DDoS giống nhau (trong đó giáo viên tự tin hơn học sinh một chút khi các xác xuất đều hơn 95%). Về hiệu suất, trong 3 dự đoán DDoS cả quy trình khi chạy với mô hình học sinh đều nhanh hơn giáo viên đến khoảng 300 ms.

Việc cả hai mô hình gán nhãn DDoS cho lưu lượng ICMP ping thông thường là một dự đoán sai đáng chú ý, vì nhãn kỳ vọng ở đây phải là Benign. Nguyên nhân nằm ở cách bài toán được đặt: trong tập dữ liệu gốc là lưu lượng ứng dụng của thiết bị IoT với lưu lượng Benign chủ yếu là thiết bị khi treo rảnh (idle), dữ liệu cảm biến, dữ liệu nhận dạng giọng nói,… chứ hiếm khi là ICMP thuần túy được ping liên tục.

Quét host sống trên nhánh mạng bằng công cụ nmap:

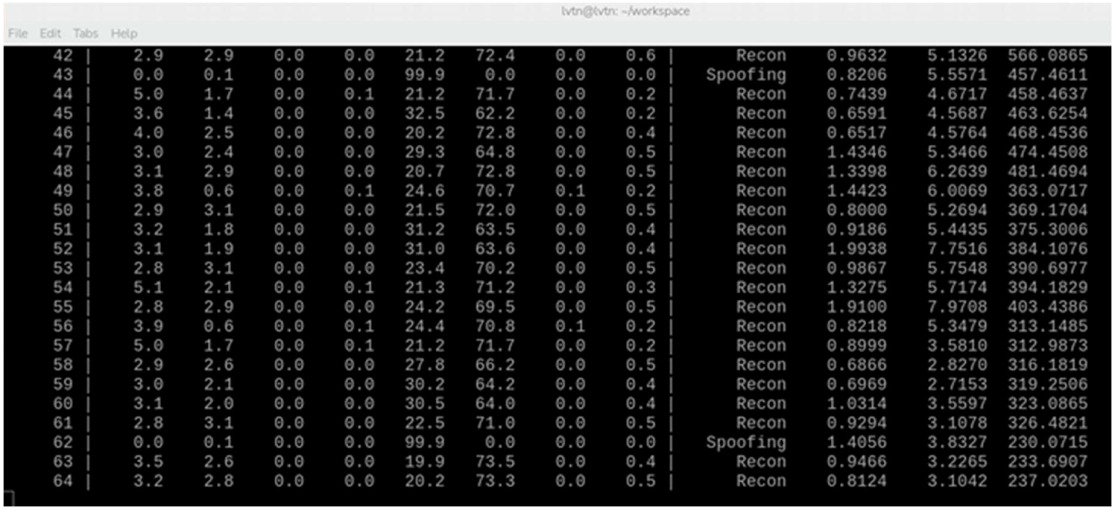
Lệnh sử dụng: *nmap -sn 192.168.50.0/24*

Các kết quả suy luận như sau:



66	19.5	0.1	0.1	0.1	4.0	73.6	2.5	0.1	Recon	0.2613	2.1118	787.4668
67	0.1	0.0	0.0	0.0	99.9	0.0	0.0	0.0	Spoofing	0.4235	2.5593	680.5508
68	20.9	0.1	0.1	0.2	4.5	71.5	2.7	0.1	Recon	0.2461	3.2782	683.9435
69	20.4	0.1	0.1	0.1	4.3	72.2	2.6	0.1	Recon	0.2100	2.0014	685.9157
70	20.1	0.1	0.1	0.1	4.2	72.7	2.6	0.1	Recon	0.1922	1.9495	687.8195
71	20.1	0.1	0.1	0.1	4.2	72.7	2.6	0.1	Recon	0.1912	1.9602	689.7349
72	37.1	0.2	0.2	0.9	18.7	37.9	4.8	0.2	Recon	0.1830	1.9220	687.6452
73	11.1	0.1	0.1	1.1	60.8	20.8	5.8	0.1	Spoofing	0.4490	2.3585	561.6078
74	19.3	0.1	0.1	0.1	3.9	73.9	2.5	0.1	Recon	0.4842	2.8688	564.7893
75	19.8	0.1	0.1	0.1	4.1	73.1	2.5	0.1	Recon	0.3314	2.4101	567.4777
76	19.6	0.1	0.1	0.1	4.0	73.5	2.5	0.1	Recon	0.3037	2.2784	569.8791
77	19.5	0.1	0.1	0.1	4.0	73.6	2.5	0.1	Recon	0.3057	2.3059	572.3014
78	37.1	0.2	0.2	0.7	14.0	43.2	4.5	0.2	Recon	0.4393	2.5965	574.6633
79	19.6	0.1	0.1	0.1	4.0	73.5	2.5	0.1	Recon	0.4731	4.5612	577.4422
80	17.4	0.1	0.2	1.3	54.4	21.0	5.5	0.2	Spoofing	0.3505	3.0468	522.7776
81	0.1	0.0	0.0	0.0	99.8	0.0	0.0	0.0	Spoofing	0.4441	2.5421	448.5848
82	20.3	0.1	0.1	0.1	4.3	72.3	2.6	0.1	Recon	0.3860	2.6467	451.3311
83	20.0	0.1	0.1	0.1	4.2	72.8	2.6	0.1	Recon	1.2873	4.1311	455.8716
84	20.0	0.1	0.1	0.1	4.2	72.9	2.6	0.1	Recon	0.7890	4.1414	460.3112
85	21.9	0.1	0.1	0.2	4.9	70.0	2.8	0.1	Recon	0.6720	3.6527	464.2293
86	12.0	0.1	0.1	1.1	60.0	20.8	5.8	0.2	Spoofing	0.5782	3.3226	351.7640
87	20.1	0.1	0.1	0.1	4.2	72.7	2.6	0.1	Recon	0.3914	2.9270	354.8608
88	20.4	0.1	0.1	0.1	4.3	72.2	2.6	0.1	Recon	0.3545	2.8736	357.8122

Hình 4.48. Dự đoán của mô hình học sinh khi bị quét host trên nhánh mạng bằng nmap



42	2.9	2.9	0.0	0.0	21.2	72.4	0.0	0.6	Recon	0.9632	5.1326	566.0865
43	0.0	0.1	0.0	0.0	99.9	0.0	0.0	0.0	Spoofing	0.8206	5.5571	457.4611
44	5.0	1.7	0.0	0.1	21.2	71.7	0.0	0.2	Recon	0.7439	4.6717	458.4637
45	3.6	1.4	0.0	0.0	32.5	62.2	0.0	0.2	Recon	0.6591	4.5687	463.6254
46	4.0	2.5	0.0	0.0	20.2	72.8	0.0	0.4	Recon	0.6517	4.5764	468.4536
47	3.0	2.4	0.0	0.0	29.3	64.8	0.0	0.5	Recon	1.4346	5.3466	474.4508
48	3.1	2.9	0.0	0.0	20.7	72.8	0.0	0.5	Recon	1.3398	6.2639	481.4694
49	3.8	0.6	0.0	0.1	24.6	70.7	0.1	0.2	Recon	1.4423	6.0069	363.0717
50	2.9	3.1	0.0	0.0	21.5	72.0	0.0	0.5	Recon	0.8000	5.2694	369.1704
51	3.2	1.8	0.0	0.0	31.2	63.5	0.0	0.4	Recon	0.9186	5.4435	375.3006
52	3.1	1.9	0.0	0.0	31.0	63.6	0.0	0.4	Recon	1.9938	7.7516	384.1076
53	2.8	3.1	0.0	0.0	23.4	70.2	0.0	0.5	Recon	0.9867	5.7548	390.6977
54	5.1	2.1	0.0	0.1	21.3	71.2	0.0	0.3	Recon	1.3275	5.7174	394.1829
55	2.8	2.9	0.0	0.0	24.2	69.5	0.0	0.5	Recon	1.9100	7.9708	403.4386
56	3.9	0.6	0.0	0.1	24.4	70.8	0.1	0.2	Recon	0.8218	5.3479	313.1485
57	5.0	1.7	0.0	0.1	21.2	71.7	0.0	0.2	Recon	0.8999	3.5810	312.9873
58	2.9	2.6	0.0	0.0	27.8	66.2	0.0	0.5	Recon	0.6866	2.8270	316.1819
59	3.0	2.1	0.0	0.0	30.2	64.2	0.0	0.4	Recon	0.6969	2.7153	319.2506
60	3.1	2.0	0.0	0.0	30.5	64.0	0.0	0.4	Recon	1.0314	3.5597	323.0865
61	2.8	3.1	0.0	0.0	22.5	71.0	0.0	0.5	Recon	0.9294	3.1078	326.4821
62	0.0	0.1	0.0	0.0	99.9	0.0	0.0	0.0	Spoofing	1.4056	3.8327	230.0715
63	3.5	2.6	0.0	0.0	19.9	73.5	0.0	0.4	Recon	0.9466	3.2265	233.6907
64	3.2	2.8	0.0	0.0	20.2	73.3	0.0	0.5	Recon	0.8124	3.1042	237.0203

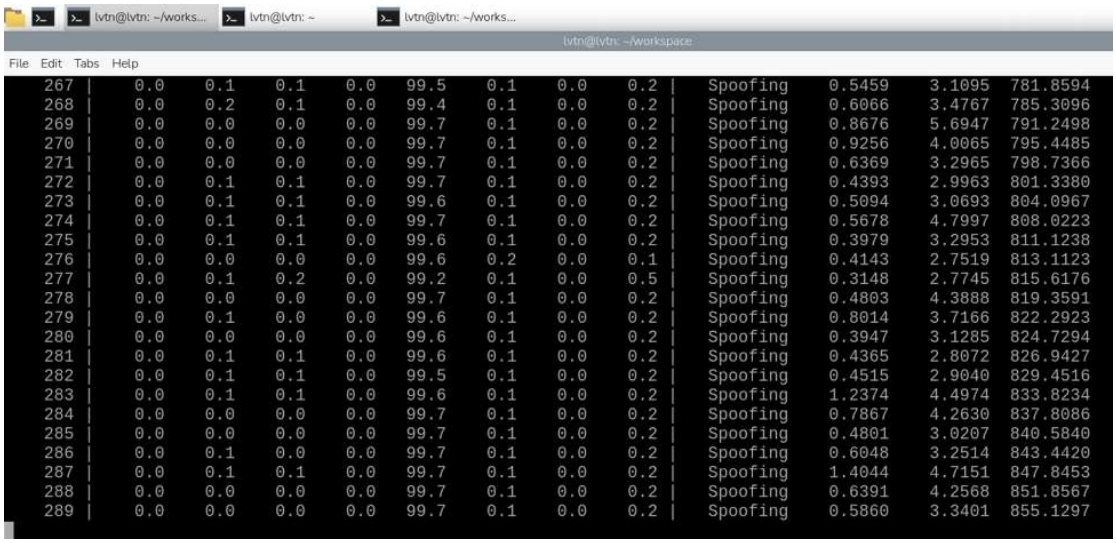
Hình 4.49. Dự đoán của mô hình giáo viên khi bị quét host trên nhánh mạng bằng nmap

Giữa 2 mô hình học sinh và giáo viên, khi bị quét host trên nhánh mạng, mô hình học sinh còn nhầm lẫn giữa Spoofing và Recon trong khi giáo viên chính xác nhiều hơn một chút. Về hiệu suất, mô hình học sinh nhỉnh hơn 1 tí ở tốc độ suy luận.

Quét cổng bằng kĩ thuật quét bán mở/quét ẩn:

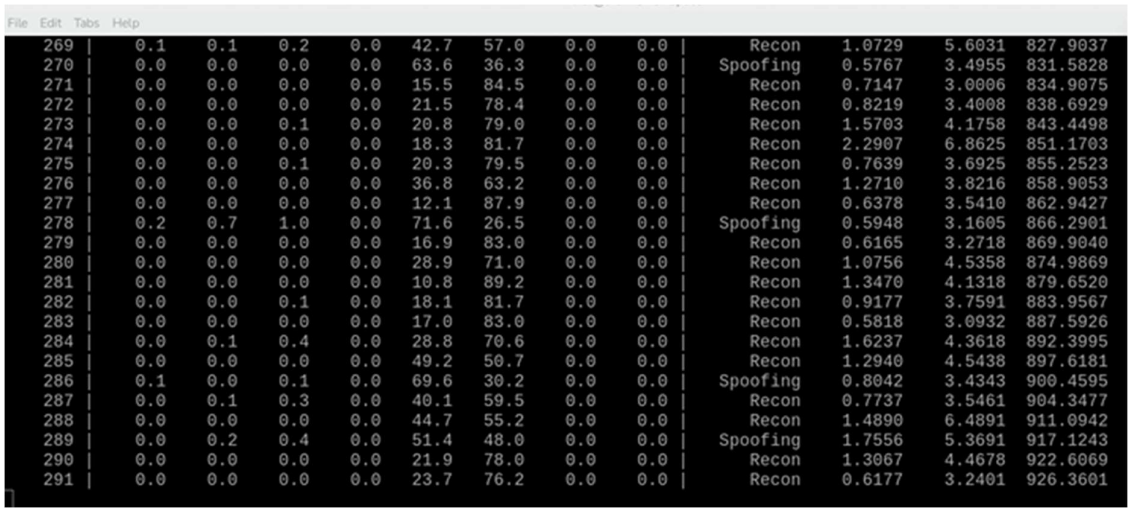
Lệnh sử dụng: nmap -sS 192.168.50.10

Các kết quả suy luận quan sát ảnh sau:



267	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.5459	3.1095	781.8594
268	0.0	0.2	0.1	0.0	99.4	0.1	0.0	0.2	Spoofing	0.6066	3.4767	785.3096
269	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.8676	5.6947	791.2498
270	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.9256	4.0065	795.4485
271	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.6369	3.2965	798.7366
272	0.0	0.1	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	0.4393	2.9963	801.3380
273	0.0	0.1	0.1	0.0	99.6	0.1	0.0	0.2	Spoofing	0.5094	3.0693	804.0967
274	0.0	0.1	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	0.5678	4.7997	808.0223
275	0.0	0.1	0.1	0.0	99.6	0.1	0.0	0.2	Spoofing	0.3979	3.2953	811.1238
276	0.0	0.0	0.0	0.0	99.6	0.2	0.0	0.1	Spoofing	0.4143	2.7519	813.1123
277	0.0	0.1	0.2	0.0	99.2	0.1	0.0	0.5	Spoofing	0.3148	2.7745	815.6176
278	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.4803	4.3888	819.3591
279	0.0	0.1	0.0	0.0	99.6	0.1	0.0	0.2	Spoofing	0.8014	3.7166	822.2923
280	0.0	0.0	0.0	0.0	99.6	0.1	0.0	0.2	Spoofing	0.3947	3.1285	824.7294
281	0.0	0.1	0.1	0.0	99.6	0.1	0.0	0.2	Spoofing	0.4365	2.8072	826.9427
282	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.4515	2.9040	829.4516
283	0.0	0.1	0.1	0.0	99.6	0.1	0.0	0.2	Spoofing	1.2374	4.4974	833.8234
284	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.7867	4.2630	837.8086
285	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.4801	3.0207	840.5840
286	0.0	0.1	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.6048	3.2514	843.4420
287	0.0	0.1	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	1.4044	4.7151	847.8453
288	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.6391	4.2568	851.8567
289	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.5860	3.3401	855.1297

Hình 4.50.Dự đoán của mô hình khi bị quét Stealth scan bằng nmap



269	0.1	0.1	0.2	0.0	42.7	57.0	0.0	0.0	Recon	1.0729	5.6031	827.9037
270	0.0	0.0	0.0	0.0	63.6	36.3	0.0	0.0	Spoofing	0.5767	3.4955	831.5828
271	0.0	0.0	0.0	0.0	15.5	84.5	0.0	0.0	Recon	0.7147	3.0006	834.9075
272	0.0	0.0	0.0	0.0	21.5	78.4	0.0	0.0	Recon	0.8219	3.4008	838.6929
273	0.0	0.0	0.1	0.0	20.8	79.0	0.0	0.0	Recon	1.5703	4.1758	843.4498
274	0.0	0.0	0.0	0.0	18.3	81.7	0.0	0.0	Recon	2.2907	6.8625	851.1703
275	0.0	0.0	0.1	0.0	20.3	79.5	0.0	0.0	Recon	0.7639	3.6925	855.2523
276	0.0	0.0	0.0	0.0	36.8	63.2	0.0	0.0	Recon	1.2710	3.8216	858.9053
277	0.0	0.0	0.0	0.0	12.1	87.9	0.0	0.0	Recon	0.6378	3.5410	862.9427
278	0.2	0.7	1.0	0.0	71.6	26.5	0.0	0.0	Spoofing	0.5948	3.1605	866.2901
279	0.0	0.0	0.0	0.0	16.9	83.0	0.0	0.0	Recon	0.6165	3.2718	869.9040
280	0.0	0.0	0.0	0.0	28.9	71.0	0.0	0.0	Recon	1.0756	4.5358	874.9869
281	0.0	0.0	0.0	0.0	10.8	89.2	0.0	0.0	Recon	1.3470	4.1318	879.6520
282	0.0	0.0	0.1	0.0	18.1	81.7	0.0	0.0	Recon	0.9177	3.7591	883.9567
283	0.0	0.0	0.0	0.0	17.0	83.0	0.0	0.0	Recon	0.5818	3.0932	887.5926
284	0.0	0.1	0.4	0.0	28.8	70.6	0.0	0.0	Recon	1.6237	4.3618	892.3995
285	0.0	0.0	0.0	0.0	49.2	50.7	0.0	0.0	Recon	1.2940	4.5438	897.6181
286	0.1	0.0	0.1	0.0	69.6	30.2	0.0	0.0	Spoofing	0.8042	3.4343	900.4595
287	0.0	0.1	0.3	0.0	40.1	59.5	0.0	0.0	Recon	0.7737	3.5461	904.3477
288	0.0	0.0	0.0	0.0	44.7	55.2	0.0	0.0	Recon	1.4890	6.4891	911.0942
289	0.0	0.2	0.4	0.0	51.4	48.0	0.0	0.0	Spoofing	1.7556	5.3691	917.1243
290	0.0	0.0	0.0	0.0	21.9	78.0	0.0	0.0	Recon	1.3067	4.4678	922.6069
291	0.0	0.0	0.0	0.0	23.7	76.2	0.0	0.0	Recon	0.6177	3.2401	926.3601

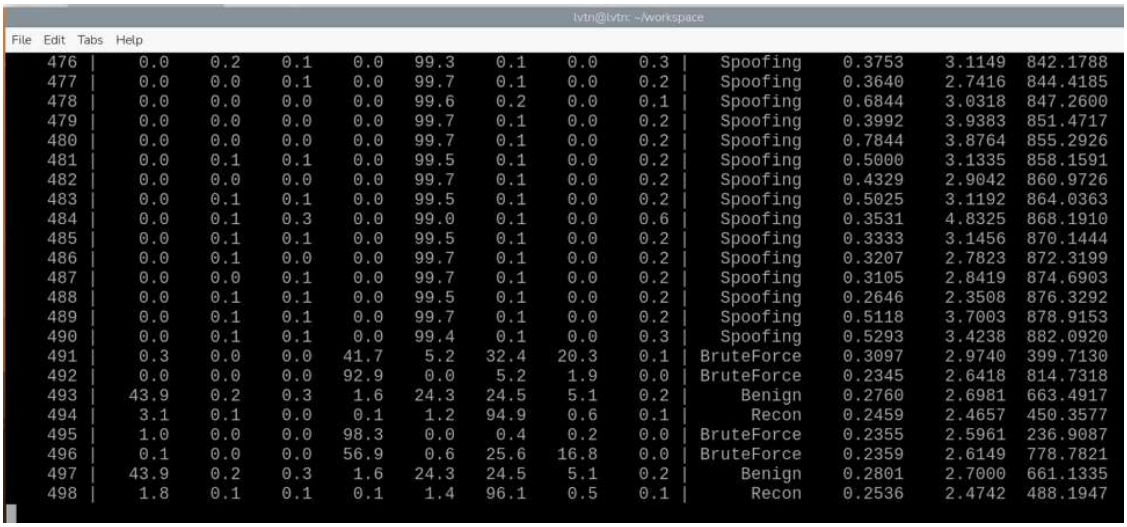
Hình 4.51. Dự đoán của mô hình giáo viên khi bị quét Stealth Scan bằng nmap

Khi bị quét bằng kĩ thuật bán mở, mô hình học sinh đều dự đoán ra Spoofing so với nhãn mong đợi là Recon, còn mô hình giáo viên chính xác hơn trong suy luận khi dự đoán ra Recon tuy vẫn có lẫn vài dự đoán Spoofing. Về hiệu suất, nhìn chung tốc độ suy luận của mô hình học sinh nhanh hơn giáo viên (có nhiều gói tin suy luận trên 1 ms), mô hình học sinh chạy với độ trễ trung bình quy trình thấp hơn, nhưng dự đoán thì chưa chính xác.

Quét dò thông tin hệ điều hành bằng công cụ nmap:

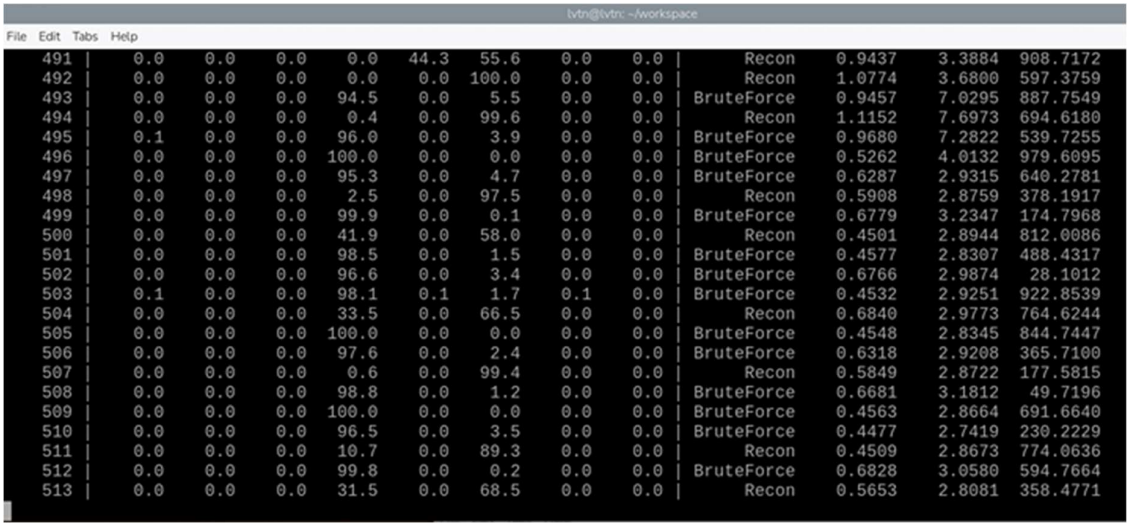
Lệnh sử dụng*: nmap -O 192.168.50.10* (tham số -O lựa chọn quét hệ điều hành)

Các kết quả suy luận như sau:



476	0.0	0.2	0.1	0.0	99.3	0.1	0.0	0.3	Spoofing	0.3753	3.1149	842.1788
477	0.0	0.0	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	0.3640	2.7416	844.4185
478	0.0	0.0	0.0	0.0	99.6	0.2	0.0	0.1	Spoofing	0.6844	3.0318	847.2600
479	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.3992	3.9383	851.4717
480	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.7844	3.8764	855.2926
481	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.5000	3.1335	858.1591
482	0.0	0.0	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.4329	2.9042	860.9726
483	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.5025	3.1192	864.0363
484	0.0	0.1	0.3	0.0	99.0	0.1	0.0	0.6	Spoofing	0.3531	4.8325	868.1910
485	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.3333	3.1456	870.1444
486	0.0	0.1	0.0	0.0	99.7	0.1	0.0	0.2	Spoofing	0.3207	2.7823	872.3199
487	0.0	0.0	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	0.3105	2.8419	874.6903
488	0.0	0.1	0.1	0.0	99.5	0.1	0.0	0.2	Spoofing	0.2646	2.3508	876.3292
489	0.0	0.1	0.1	0.0	99.7	0.1	0.0	0.2	Spoofing	0.5118	3.7003	878.9153
490	0.0	0.1	0.1	0.0	99.4	0.1	0.0	0.3	Spoofing	0.5293	3.4238	882.0920
491	0.3	0.0	0.0	41.7	5.2	32.4	20.3	0.1	BruteForce	0.3097	2.9740	399.7130
492	0.0	0.0	0.0	92.9	0.0	5.2	1.9	0.0	BruteForce	0.2345	2.6418	814.7318
493	43.9	0.2	0.3	1.6	24.3	24.5	5.1	0.2	Benign	0.2760	2.6981	663.4917
494	3.1	0.1	0.0	0.1	1.2	94.9	0.6	0.1	Recon	0.2459	2.4657	450.3577
495	1.0	0.0	0.0	98.3	0.0	0.4	0.2	0.0	BruteForce	0.2355	2.5961	236.9087
496	0.1	0.0	0.0	56.9	0.6	25.6	16.8	0.0	BruteForce	0.2359	2.6149	778.7821
497	43.9	0.2	0.3	1.6	24.3	24.5	5.1	0.2	Benign	0.2801	2.7000	661.1335
498	1.8	0.1	0.1	0.1	1.4	96.1	0.5	0.1	Recon	0.2536	2.4742	488.1947

Hình 4.52. Dự đoán của mô hình học sinh khi bị quét hệ điều hành bằng nmap



491	0.0	0.0	0.0	0.0	44.3	55.6	0.0	0.0	Recon	0.9437	3.3884	908.7172
492	0.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	Recon	1.0774	3.6800	597.3759
493	0.0	0.0	0.0	94.5	0.0	5.5	0.0	0.0	BruteForce	0.9457	7.0295	887.7549
494	0.0	0.0	0.0	0.4	0.0	99.6	0.0	0.0	Recon	1.1152	7.6973	694.6180
495	0.1	0.0	0.0	96.0	0.0	3.9	0.0	0.0	BruteForce	0.9680	7.2822	539.7255
496	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.5262	4.0132	979.6095
497	0.0	0.0	0.0	95.3	0.0	4.7	0.0	0.0	BruteForce	0.6287	2.9315	640.2781
498	0.0	0.0	0.0	2.5	0.0	97.5	0.0	0.0	Recon	0.5908	2.8759	378.1917
499	0.0	0.0	0.0	99.9	0.0	0.1	0.0	0.0	BruteForce	0.6779	3.2347	174.7968
500	0.0	0.0	0.0	41.9	0.0	58.0	0.0	0.0	Recon	0.4501	2.8944	812.0086
501	0.0	0.0	0.0	98.5	0.0	1.5	0.0	0.0	BruteForce	0.4577	2.8307	488.4317
502	0.0	0.0	0.0	96.6	0.0	3.4	0.0	0.0	BruteForce	0.6766	2.9874	28.1012
503	0.1	0.0	0.0	98.1	0.1	1.7	0.1	0.0	BruteForce	0.4532	2.9251	922.8539
504	0.0	0.0	0.0	33.5	0.0	66.5	0.0	0.0	Recon	0.6840	2.9773	764.6244
505	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.4548	2.8345	844.7447
506	0.0	0.0	0.0	97.6	0.0	2.4	0.0	0.0	BruteForce	0.6318	2.9208	365.7100
507	0.0	0.0	0.0	0.6	0.0	99.4	0.0	0.0	Recon	0.5849	2.8722	177.5815
508	0.0	0.0	0.0	98.8	0.0	1.2	0.0	0.0	BruteForce	0.6681	3.1812	49.7196
509	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.4563	2.8664	691.6640
510	0.0	0.0	0.0	96.5	0.0	3.5	0.0	0.0	BruteForce	0.4477	2.7419	230.2229
511	0.0	0.0	0.0	10.7	0.0	89.3	0.0	0.0	Recon	0.4509	2.8673	774.0636
512	0.0	0.0	0.0	99.8	0.0	0.2	0.0	0.0	BruteForce	0.6828	3.0580	594.7664
513	0.0	0.0	0.0	31.5	0.0	68.5	0.0	0.0	Recon	0.5653	2.8081	358.4771

Hình 4.53. Dự đoán của mô hình giáo viên khi bị quét hệ điều hành bằng nmap

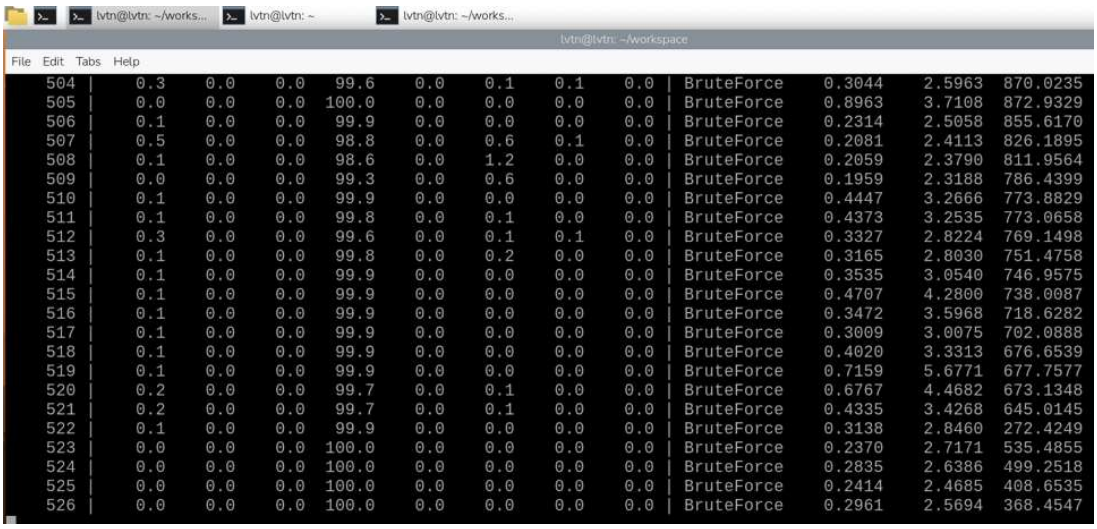
Kết quả dự đoán của cả 2 mô hình nhìn chung khá dao động, ở mô hình học sinh thì vẫn như kĩ thuật trước, mô hình học sinh đều dự đoán ra Spoofing, ở cuối quy trình thì xuất hiện lẫn lộn các dự đoán BruteForce, Benign và Recon. Ở giáo viên sự dao động ấy ít hơn, chỉ nhầm lẫn qua lại giữa BruteForce và Recon. Về hiệu suất, khi bị quét OS, độ trễ khá tương đồng, mô hình học sinh nhỉnh hơn 1 chút ở tốc độ suy luận.

Ba kịch bản quét cho thấy khác biệt rõ nhất giữa học sinh và giáo viên: giáo viên đưa ra nhãn Recon tương đối ổn định, còn học sinh gần như luôn gán về Spoofing. Recon và Spoofing là hai nhóm gần nhau vì đều gồm lưu lượng thưa, gói nhỏ, không mang tải trọng ứng dụng; giáo viên có dung lượng lớn hơn nên giữ được các đặc trưng phân biệt tinh tế mà học sinh mất đi khi nén. Sự thiếu hụt này không lộ ra trên dữ liệu quen thuộc, chỉ bộc lộ khi gặp lưu lượng lạ.

Tấn công vét cạn mật khẩu bằng công cụ Hydra:

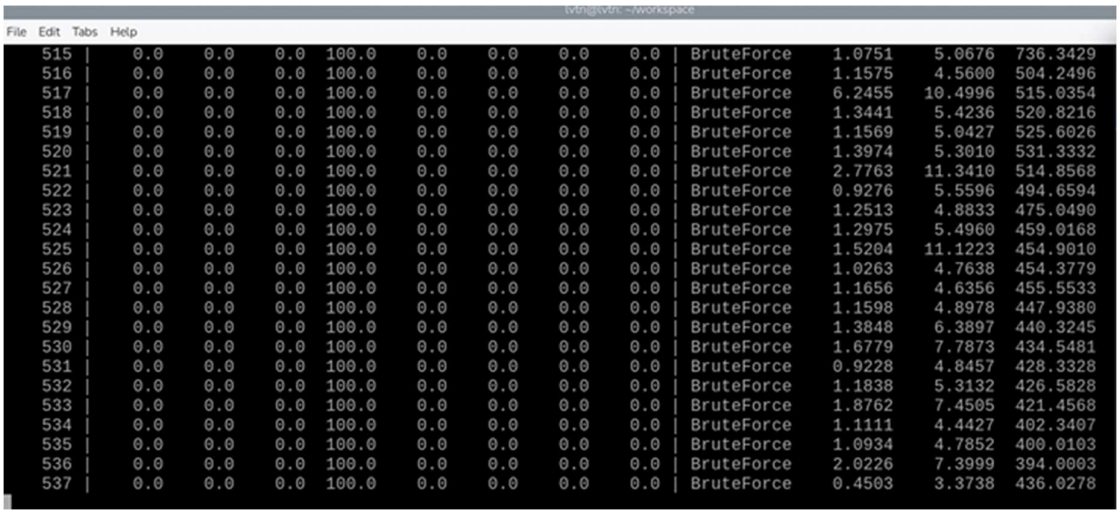
Lệnh sử dụng: *hydra -l lvtn -P pass.txt 192.168.50.10 ssh*

Kết quả suy luận 2 mô hình quan sát các ghi nhận sau:



504	0.3	0.0	0.0	99.6	0.0	0.1	0.1	0.0	BruteForce	0.3044	2.5963	870.0235
505	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.8963	3.7108	872.9329
506	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.2314	2.5058	855.6170
507	0.5	0.0	0.0	98.8	0.0	0.6	0.1	0.0	BruteForce	0.2081	2.4113	826.1895
508	0.1	0.0	0.0	98.6	0.0	1.2	0.0	0.0	BruteForce	0.2059	2.3790	811.9564
509	0.0	0.0	0.0	99.3	0.0	0.6	0.0	0.0	BruteForce	0.1959	2.3188	786.4399
510	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.4447	3.2666	773.8829
511	0.1	0.0	0.0	99.8	0.0	0.1	0.0	0.0	BruteForce	0.4373	3.2535	773.0658
512	0.3	0.0	0.0	99.6	0.0	0.1	0.1	0.0	BruteForce	0.3327	2.8224	769.1498
513	0.1	0.0	0.0	99.8	0.0	0.2	0.0	0.0	BruteForce	0.3165	2.8030	751.4758
514	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.3535	3.0540	746.9575
515	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.4707	4.2800	738.0087
516	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.3472	3.5968	718.6282
517	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.3009	3.0075	702.0888
518	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.4020	3.3313	676.6539
519	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.7159	5.6771	677.7577
520	0.2	0.0	0.0	99.7	0.0	0.1	0.0	0.0	BruteForce	0.6767	4.4682	673.1348
521	0.2	0.0	0.0	99.7	0.0	0.1	0.0	0.0	BruteForce	0.4335	3.4268	645.0145
522	0.1	0.0	0.0	99.9	0.0	0.0	0.0	0.0	BruteForce	0.3138	2.8460	272.4249
523	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.2370	2.7171	535.4855
524	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.2835	2.6386	499.2518
525	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.2414	2.4685	408.6535
526	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.2961	2.5694	368.4547

Hình 4.54. Dự đoán của mô hình học sinh khi bị tấn công BruteForce trên dịch vụ SSH bằng hydra



515	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.0751	5.0676	736.3429
516	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1575	4.5600	504.2496
517	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	6.2455	10.4996	515.0354
518	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.3441	5.4236	520.8216
519	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1569	5.0427	525.6026
520	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.3974	5.3010	531.3332
521	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	2.7763	11.3410	514.8568
522	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.9276	5.5596	494.6594
523	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.2513	4.8833	475.0490
524	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.2975	5.4960	459.0168
525	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.5204	11.1223	454.9010
526	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.0263	4.7638	454.3779
527	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1656	4.6356	455.5533
528	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1598	4.8978	447.9380
529	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.3848	6.3897	440.3245
530	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.6779	7.7873	434.5481
531	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.9228	4.8457	428.3328
532	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1838	5.3132	426.5828
533	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.8762	7.4505	421.4568
534	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.1111	4.4427	402.3407
535	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	1.0934	4.7852	400.0103
536	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	2.0226	7.3999	394.0003
537	0.0	0.0	0.0	100.0	0.0	0.0	0.0	0.0	BruteForce	0.4503	3.3738	436.0278

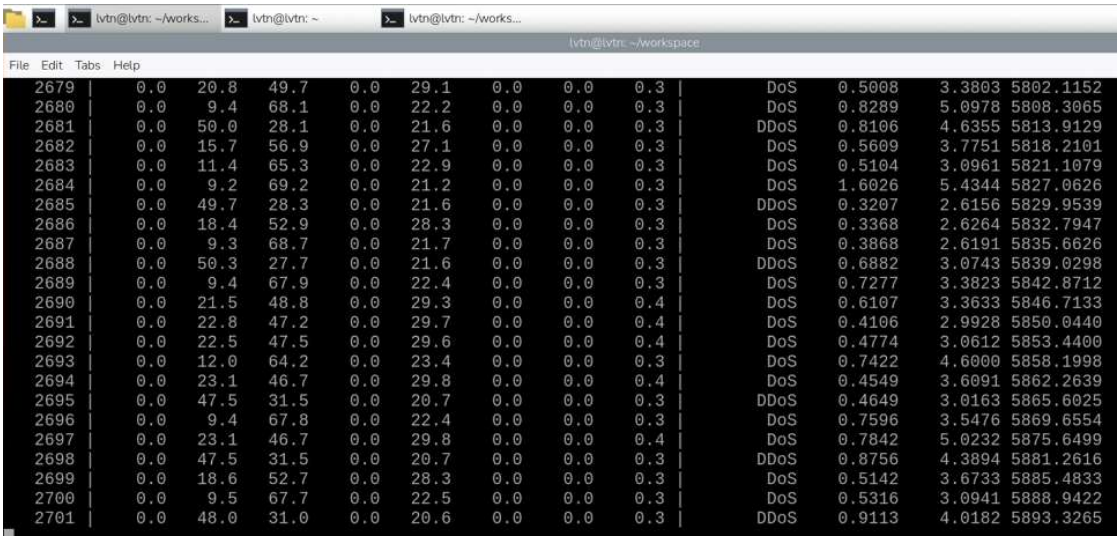
Hình 4.55. Dự đoán của mô hình giáo viên khi bị tấn công BruteForce trên dịch vụ SSH bằng hydra

Khi bị tấn công bruteforce, cả 2 mô hình học sinh và giáo viên đều dự đoán tốt. Về mặt hiệu suất mô hình học sinh suy luận với tốc độ nhanh hơn (ở giáo viên rất nhiều gói tin mất từ 1-2 ms để dự đoán, ở học sinh đều mất dưới 1 ms để dự đoán). Tấn công vét cạn mật khẩu qua SSH tạo chuỗi kết nối TCP liên tục trên cùng một cổng với mẫu cờ lặp lại, đủ dày để chiếm trọn cửa sổ và không lẫn với lưu lượng nền. So với các kịch bản quét, có thể thấy quy luật: mô hình dự đoán đúng khi lưu lượng tấn công đủ dày và có dấu hiệu giao thức riêng biệt, dự đoán sai khi lưu lượng thưa hoặc trộn với lưu lượng nền.

Tấn công làm ngập bằng công cụ hping3:

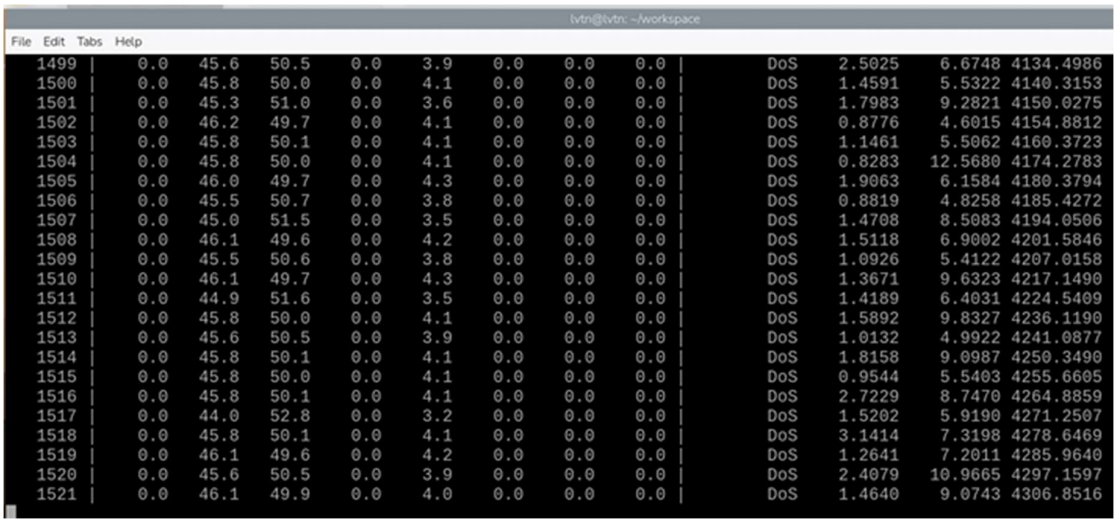
Lệnh sử dụng: sudo hping3 -S 192.168.50.10 –flood

Các kết quả dự đoán của 2 mô hình như sau:



2679	0.0	20.8	49.7	0.0	29.1	0.0	0.0	0.3	DoS	0.5008	3.3803 5802.1152
2680	0.0	9.4	68.1	0.0	22.2	0.0	0.0	0.3	DoS	0.8289	5.0978 5808.3065
2681	0.0	50.0	28.1	0.0	21.6	0.0	0.0	0.3	DDoS	0.8106	4.6355 5813.9129
2682	0.0	15.7	56.9	0.0	27.1	0.0	0.0	0.3	DoS	0.5609	3.7751 5818.2101
2683	0.0	11.4	65.3	0.0	22.9	0.0	0.0	0.3	DoS	0.5104	3.0961 5821.1079
2684	0.0	9.2	69.2	0.0	21.2	0.0	0.0	0.3	DoS	1.6026	5.4344 5827.0626
2685	0.0	49.7	28.3	0.0	21.6	0.0	0.0	0.3	DDoS	0.3207	2.6156 5829.9539
2686	0.0	18.4	52.9	0.0	28.3	0.0	0.0	0.3	DoS	0.3368	2.6264 5832.7947
2687	0.0	9.3	68.7	0.0	21.7	0.0	0.0	0.3	DoS	0.3868	2.6191 5835.6626
2688	0.0	50.3	27.7	0.0	21.6	0.0	0.0	0.3	DDoS	0.6882	3.0743 5839.0298
2689	0.0	9.4	67.9	0.0	22.4	0.0	0.0	0.3	DoS	0.7277	3.3823 5842.8712
2690	0.0	21.5	48.8	0.0	29.3	0.0	0.0	0.4	DoS	0.6107	3.3633 5846.7133
2691	0.0	22.8	47.2	0.0	29.7	0.0	0.0	0.4	DoS	0.4106	2.9928 5850.0440
2692	0.0	22.5	47.5	0.0	29.6	0.0	0.0	0.4	DoS	0.4774	3.0612 5853.4400
2693	0.0	12.0	64.2	0.0	23.4	0.0	0.0	0.3	DoS	0.7422	4.6000 5858.1998
2694	0.0	23.1	46.7	0.0	29.8	0.0	0.0	0.4	DoS	0.4549	3.6091 5862.2639
2695	0.0	47.5	31.5	0.0	20.7	0.0	0.0	0.3	DDoS	0.4649	3.0163 5865.6025
2696	0.0	9.4	67.8	0.0	22.4	0.0	0.0	0.3	DoS	0.7596	3.5476 5869.6554
2697	0.0	23.1	46.7	0.0	29.8	0.0	0.0	0.4	DoS	0.7842	5.0232 5875.6499
2698	0.0	47.5	31.5	0.0	20.7	0.0	0.0	0.3	DDoS	0.8756	4.3894 5881.2616
2699	0.0	18.6	52.7	0.0	28.3	0.0	0.0	0.3	DoS	0.5142	3.6733 5885.4833
2700	0.0	9.5	67.7	0.0	22.5	0.0	0.0	0.3	DoS	0.5316	3.0941 5888.9422
2701	0.0	48.0	31.0	0.0	20.6	0.0	0.0	0.3	DDoS	0.9113	4.0182 5893.3265

Hình 4.56. Dự đoán của mô hình học sinh khi bị tấn công làm ngập bằng hping3



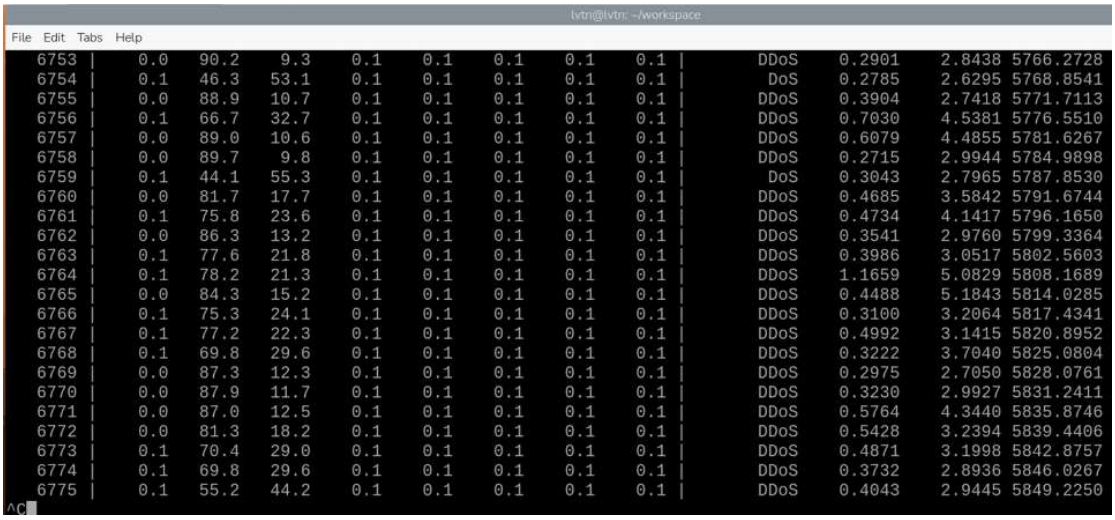
Hình 4.57. Dự đoán của mô hình giáo viên khi bị tấn công làm ngập bằng hping3

Khi bị tấn công làm ngập, mô hình học sinh cho ra dự đoán cả DDoS và DoS, độ tự tin chưa cao và có 20% xác suất dự đoán là Spoofing. Còn ở mô hình giáo viên, xác suất chỉ rơi vào DDoS và DoS với DoS là dự đoán chính, hầu như không dao động. Về hiệu suất mô hình học sinh vẫn có tốc độ suy luận nhìn chung cao hơn (dựa vào số lượng gói tin mất nhiều hơn 1 ms để xử lý bên mô hình giáo viên), độ trễ của quy trình đọc đến suy luận của học sinh thấp hơn, nhưng độ trễ tổng thể quy trình thì giáo viên nhẹ hơn (ảnh hưởng 1 phần do thời điểm ghi nhận kết quả khi học sinh đã dự đoán đến gói tin thứ 2701, còn của giáo viên chỉ ở 1521).

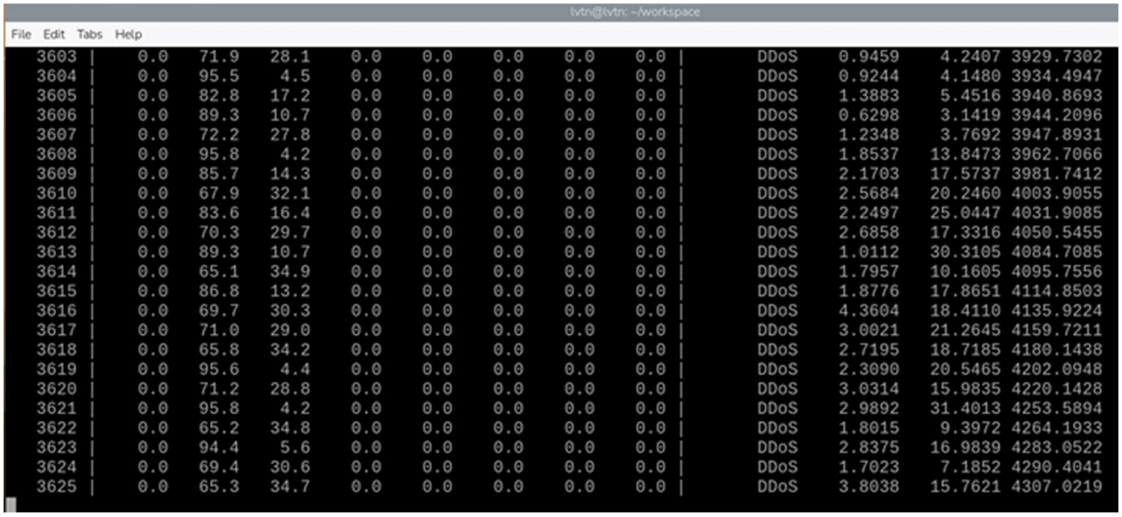
Tấn công làm ngập với IP ngẫu nhiên bằng công cụ hping3:

Lệnh sử dụng: sudo hping3 -S 192.168.50.10 --flood --rand-source

Kết quả suy luận chi tiết tham khảo ảnh ghi nhận:



Hình 4.58. Dự đoán của mô hình khi bị tấn công làm ngập với IP ngẫu nhiên bằng hping3



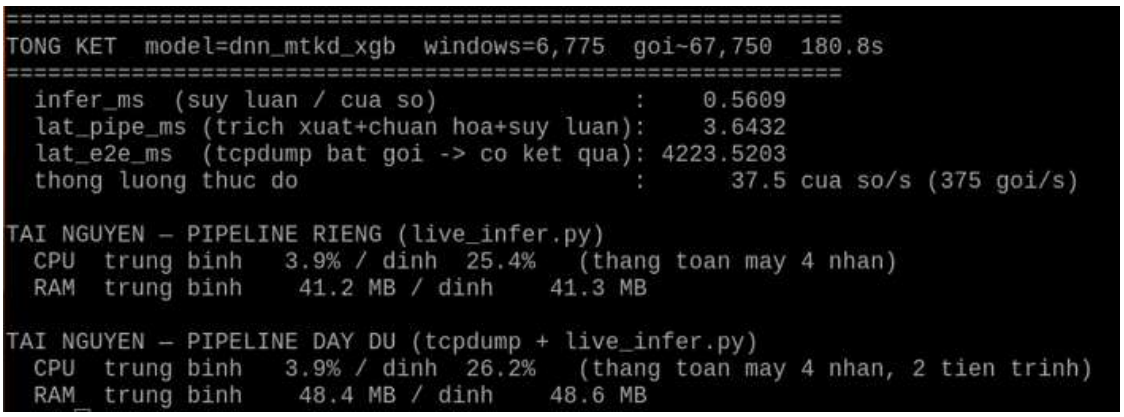
Hình 4.59. Dự đoán của mô hình giáo viên khi bị tấn công làm ngập bằng IP ngẫu nhiên với công cụ hping3

Mô hình học sinh khi giao diện mạng bị làm ngập với nhiều IP ngẫu nhiên đều dự đoán nhãn là DDoS (có nhầm DoS nhưng rất ít), còn mô hình giáo viên dự đoán DDoS một cách ổn định. Tốc độ suy luận của mô hình học sinh như các kỹ thuật trước đều nhỉnh hơn mô hình giáo viên. Đặc biệt độ trễ của quy trình đọc-trích xuất và suy luận của mô hình giáo viên đôi lúc đến vài chục ms.

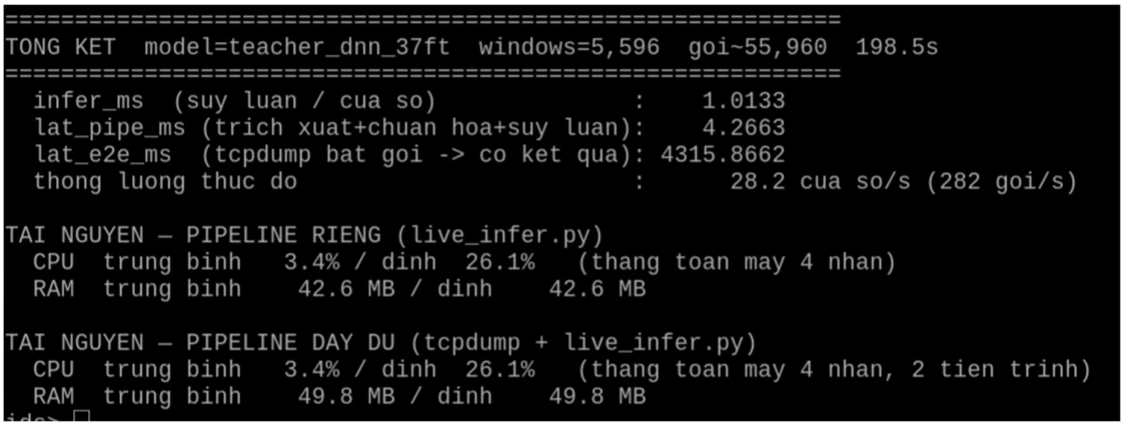
Hai kịch bản này xác nhận quy luật trên: khi lưu lượng SYN chiếm trọn mọi cửa sổ, cả hai mô hình đều dự đoán đúng nhóm. Đáng chú ý là làm ngập từ một nguồn cho kết quả nghiêng về DoS, còn làm ngập với IP ngẫu nhiên nghiêng hẳn về DDoS,

đúng với tiêu chí phân biệt hai nhóm này, cho thấy mô hình học được đặc trưng về số lượng nguồn tấn công chứ không chỉ nhận diện cường độ. Khác biệt còn lại là độ ổn định: học sinh vẫn để lẫn khoảng 20% xác suất cho nhãn Spoofing, giáo viên gần như không dao động.

Kết quả đo hiệu suất tổng thể của quy trình suy luận với mô hình học sinh và giáo viên:



Hình 4.60. Kết quả đo hiệu suất sau khi chạy chương trình suy luận trực tuyến với mô hình học sinh DNN_XGB



Hình 4.61. Kết quả đo hiệu suất sau khi chạy chương trình suy luận trực tuyến với mô hình giáo viên DNN

Qua kết quả thấy mô hình học sinh suy luận nhanh hơn giáo viên và có độ trễ quy trình thấp hơn. Về chiếm dụng tài nguyên, ở quy trình đầy đủ từ bước thu thập đến in kết quả suy luận, mô hình học sinh và giáo viên chiếm CPU gần như tương đương nhau (26.1-26.2%) CPU tăng nhẹ lên so nhánh PCAP tĩnh (xấp xỉ 25%), tức chi phí do tiến trình thu thập chạy song song là không đáng kể. Bộ nhớ toàn bộ quy trình chỉ còn khoảng 48.6 MB với học sinh và 49.8 MB với giáo viên, chưa bằng một

nửa so với nhánh PCAP tĩnh, do gói tin đi qua đường ống và được xử lý ngay thay vì phải đọc các tập tin PCAP lớn từ đĩa. Chênh lệch RAM giữa hai mô hình chỉ 1.2 MB, trong khi chênh lệch tốc độ suy luận quan sát được rõ ở mọi kịch bản, khẳng định lại rằng lợi ích của chưng cất nằm ở tốc độ chứ không ở bộ nhớ.

Qua bước đánh giá trên kết quả huấn luyện, kết quả suy luận trên dữ liệu PCAP có nhãn cho thấy xu hướng nhất quán: điều kiện đánh giá càng gần thực tế thì hiệu quả phân loại càng giảm và khoảng cách giữa học sinh với giáo viên càng rộng. Khi huấn luyện, học sinh MLP2x64 kém giáo viên DNN 1.6 điểm macro-F1; trên PCAP có nhãn là 2.2 điểm và hiệu quả tuyệt đối của cả hai cùng giảm tám đến chín điểm; khi suy luận trực tuyến, khác biệt thể hiện định tính khi giáo viên nhận đúng nhóm Recon còn học sinh gần như luôn gán về Spoofing. Có thể hiểu rằng học sinh kế thừa được phần tri thức cốt lõi nhưng vẫn chưa được đầy đủ nên khó khăn hơn tron việc quyết định khi gặp dữ liệu lạ. Ở chiều hiệu suất thì ngược lại: trên PCAP tĩnh ưu thế tốc độ của học sinh bị che khuất vì khâu trích xuất chiếm phần lớn chi phí, còn khi chạy trực tuyến ưu thế này hiện ra rõ và ổn định ở mọi kịch bản.

Mục tiêu của bước suy luận trên dữ liệu PCAP tự thu thập như đã đề ra không phải đo độ chính xác mà là kiểm chứng tính khả thi vận hành đầu-cuối trong điều kiện gần thực tế. Về tính toàn vẹn kiến trúc, hệ thống chạy liên tục qua cả các kịch bản mà không treo, không sập, không mất kết nối, kể cả khi bị làm ngập TCP SYN ở cường độ cao nhất, điều mà đọc tập tin PCAP tĩnh không kiểm chứng được. Về chi phí vận hành, quy trình chỉ chiếm khoảng 26% CPU và dưới 50 MB RAM. Về chất lượng phân loại, mô hình dự đoán đúng ở các kịch bản lưu lượng đủ dày và có dấu hiệu riêng biệt như brute-force và hai kiểu làm ngập, nhưng lệch nhãn ở các kịch bản lưu lượng thưa như quét mạng và ping; nguyên nhân chính là mạng thực nghiệm chỉ có hai máy, khác xa môi trường 105 thiết bị nơi tập dữ liệu được thu thập, nên thành phần lưu lượng nền trong mỗi cửa sổ lệch hẳn so với dữ liệu huấn luyện.

Như vậy có thể thấy chưng cất tri thức tạo ra được mô hình đủ nhẹ và đủ nhanh để chạy trực tuyến được trên thiết bị IoT biên.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Luận văn đã khảo sát một cách có hệ thống khả năng của các kỹ thuật chưng cất tri thức trong việc xây dựng mô hình phát hiện xâm nhập nhẹ cho thiết bị IoT biên, với minh chứng thực nghiệm trên tập dữ liệu CICIoT2023 và trên thiết bị vật lý Raspberry Pi 4 B. Đối chiếu với các mục tiêu đề ra ở Chương 1, nghiên cứu đã đạt được những kết quả chính sau đây.

Trước hết, luận văn đã xây dựng và huấn luyện thành công các mô hình giáo viên đa dạng về kiến trúc gồm CNN-1D, DNN, SSL-CL-DNN, XGBoost và LSTM, đều có tối ưu siêu tham số bằng Optuna, làm cơ sở tri thức vững chắc cho quá trình chưng cất về sau.

Tiếp theo, luận văn đã triển khai và so sánh bốn hướng chưng cất tri thức gồm KD truyền thống, RAID-KL với hai chế độ nhiệt độ, FitNets, và chưng cất đa giáo viên theo AW-MTKD cùng CW-MTKD, qua đó tạo ra mười sáu mô hình học sinh. Kết quả cho thấy chưng cất tri thức nén được mô hình từ 92.0% đến 99.65% về số tham số mà chỉ mất từ 1.91 đến 5.58 điểm phần trăm macro-F1, đủ để khẳng định tính khả thi của hướng tiếp cận. Trong đó, kiến trúc và cặp giáo viên là yếu tố quyết định hiệu quả phân loại, tác động mạnh hơn nhiều so với lựa chọn kỹ thuật chưng cất. Nhóm SSL-CL-DNN chịu nén tốt nhất và đạt độ chính xác cao nhất, còn nhóm MLP đa giáo viên CW-MTKD đạt macro-F1 cao nhất với kích thước nhỏ nhất. Đáng chú ý là cơ chế nhiệt độ thích ứng của RAID-KL vượt nhiệt độ cố định một cách nhất quán trên cả ba kiến trúc.

Sau cùng, luận văn đã đánh giá được hiệu quả phân loại và hiệu suất suy luận của các mô hình trên lưu lượng mạng thực, với hiệu suất được đo trực tiếp trên thiết bị Raspberry Pi 4 B tài nguyên hạn chế. Trên lưu lượng thực, độ chính xác giảm còn khoảng 68% đến 73% do khác biệt của bộ trích xuất đặc trưng và dịch chuyển phân phối, song macro-AUC vẫn duy trì khoảng 94% đến 95% và các mô hình học sinh vẫn giữ lại khoảng 90% đến 95% độ chính xác của giáo viên. Về hiệu suất triển khai, các mô hình học sinh đạt thông lượng khoảng 4.100 đến 4.170 gói tin mỗi giây với độ trễ khoảng 2,4 ms cho mỗi cửa sổ mười gói tin, chiếm khoảng 25% CPU và 105 MB RAM, trong khi các mô hình giáo viên chỉ đạt 1.500 đến 3.900 gói tin mỗi giây và làm độ trễ tăng lên đến 6,5 ms. So với các hệ thống phát hiện xâm nhập nhẹ đã công bố như Realguard và Kitsune, quy trình của luận văn tiêu tốn ít CPU và bộ nhớ hơn dù giải bài toán phân loại tám nhãn nhóm. Ở chế độ suy luận trực tuyến trên lưu

lượng tự thu thập, hệ thống vận hành liên tục và ổn định qua toàn bộ các kịch bản tấn công thử nghiệm, kể cả khi bị làm ngập ở cường độ cao nhất, với mức chiếm dụng khoảng 26% CPU và dưới 50 MB RAM. Các kết quả này cho thấy quy trình hoàn toàn đáp ứng được yêu cầu vận hành thời gian thực tại biên.

Tổng hợp lại, chưng cất tri thức là một hướng tiếp cận khả thi và hiệu quả để đưa các mô hình phát hiện xâm nhập học sâu về kích thước phù hợp với thiết bị IoT biên. Không có mô hình nào tối ưu trên mọi tiêu chí, nên việc lựa chọn mô hình triển khai cần căn cứ vào mục tiêu ưu tiên cụ thể. Nếu ưu tiên độ trễ thấp, nhóm DNN là lựa chọn phù hợp; nếu ưu tiên độ chính xác cao, SSL-CL-DNN tỏ ra vượt trội; còn nếu cần phát hiện đồng đều các lớp tấn công hiếm, CW-MTKD là phương án đáng cân nhắc. Tuy nhiên, kết quả đo trên thiết bị thực cũng cho thấy khi vận hành trong một quy trình hoàn chỉnh, phần lớn chi phí xử lý nằm ở khâu trích xuất đặc trưng chứ thể hiện mạnh mẽ ở mô hình phân loại, nên khác biệt về tốc độ giữa các mô hình học sinh gần như không đáng kể và tiêu chí lựa chọn chủ yếu vẫn nên là chất lượng phân loại.

5.2. Hạn chế

Bên cạnh những kết quả đạt được, luận văn vẫn còn một số hạn chế. Nghiên cứu mới dừng ở việc triển khai và đo benchmark riêng module phân loại trên thiết bị thực, chưa xây dựng được một hệ thống IDS hoàn chỉnh vận hành liên tục từ khâu thu thập, trích xuất, phân loại cho đến cảnh báo và phản ứng.

Về mặt phân loại, một số lớp tấn công vẫn bị phát hiện kém. Cặp DDoS và DoS thường xuyên bị nhầm lẫn do bản chất kỹ thuật gần như trùng nhau, còn hai lớp hiếm là Web-Based và BruteForce có recall rất thấp do dữ liệu mất cân bằng nghiêm trọng và do ngưỡng quyết định argmax mặc định. Ngoài ra, hiệu quả trên lưu lượng mạng thực giảm khá nhiều so với trên tập đặc trưng dựng sẵn, cho thấy bộ trích xuất đặc trưng tự triển khai và phạm vi phân bố của tập huấn luyện chưa bao phủ tốt điều kiện thực tế. Cuối cùng, phạm vi thực nghiệm giới hạn ở một tập dữ liệu là CICIoT2023 và một loại thiết bị biên là Raspberry Pi 4 B, nên tính tổng quát của các kết luận sang những tập dữ liệu và nền tảng phần cứng khác vẫn cần được kiểm chứng thêm.

5.3. Hướng phát triển

Từ những hạn chế trên, luận văn đề xuất một số hướng phát triển tiếp theo. Về mô hình, có thể kết hợp chưng cất tri thức với các kỹ thuật làm nhẹ khác như lượng tử hóa và cắt tỉa mạng để giảm thêm kích thước và tăng tốc độ suy luận trên biên,

đồng thời thử nghiệm hiệu chỉnh ngưỡng quyết định theo từng lớp thay cho argmax hoặc dùng hàm mất mát nhạy chi phí nhằm cải thiện recall của các lớp tấn công hiếm.

Về dữ liệu, có thể mở rộng sang nhiều tập dữ liệu và nhiều loại tấn công hơn, bổ sung các kỹ thuật cân bằng và tăng cường dữ liệu cho lớp thiểu số, cũng như tinh chỉnh bộ trích xuất đặc trưng để thu hẹp khoảng cách giữa dữ liệu huấn luyện và lưu lượng mạng thực. Một hướng đáng cân nhắc là thu thập dữ liệu huấn luyện ngay tại môi trường mạng sẽ triển khai, hoặc tinh chỉnh mô hình học sinh theo dữ liệu cục bộ sau khi đã chưng cất, nhằm khắc phục hiện tượng lệch nhãn đã quan sát được ở bước suy luận trực tuyến.

Về hệ thống, hướng phát triển tự nhiên là xây dựng một hệ thống IDS hoàn chỉnh, hoạt động thời gian thực trên thiết bị biên, tích hợp cơ chế cảnh báo và phản ứng, đồng thời kiểm thử trên nhiều nền tảng phần cứng biên khác nhau để khẳng định tính tổng quát và khả năng ứng dụng thực tế của phương pháp.

TÀI LIỆU THAM KHẢO

[1] "Number of connected IoT devices growing 14% to 21.1 billion." Accessed: Aug. 04, 2026. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[2] "Reimagined Partner Program, Cybersecurity Solutions | SonicWall." Accessed: Aug. 04, 2026. [Online]. Available: https://www.sonicwall.com/news/2023-cyber-threat-report-iot-malware-attacks-climb-87

[3] "Press Releases & News | Kaspersky | Kaspersky." Accessed: Aug. 04, 2026. [Online]. Available: https://www.kaspersky.com/about/press-releases

[4] H. Alzahrani, T. Sheltami, A. Barnawi, M. Imam, and A. Yaser, "A Lightweight Intrusion Detection System Using Convolutional Neural Network and Long Short-Term Memory in Fog Computing," *Comput. Mater. Contin.*, vol. 80, no. 3, pp. 4703–4728, 2024, doi: 10.32604/cmc.2024.054203.

[5] O. D. Okey, S. Dadkhah, D. Z. Rodríguez, and J. H. Kleinschmidt, "Explainable resource-Aware IoT security model via knowledge distillation and adaptive loss function optimization," *Expert Syst. Appl.*, vol. 302, p. 130460, Mar. 2026, doi: 10.1016/j.eswa.2025.130460.

[6] C.-W. Kuo and C.-X. Wu, "A Lightweight Two-Stage Intrusion Detection Framework Optimized for Edge-Based IoT Environments," *Comput. Mater. Contin.*, vol. 87, no. 3, pp. 1–10, 2026, doi: 10.32604/cmc.2026.076767.

[7] S. F. Misrak and H. M. Melaku, "Lightweight intrusion detection system for IoT with improved feature engineering and advanced dynamic quantization," *Discov. Internet Things*, vol. 5, no. 1, p. 97, Sep. 2025, doi: 10.1007/s43926-025-00203-8.

[8] M. Ogab, S. Zaidi, A. Bourouis, and C. T. Calafate, "Intrusion Detection Systems for the Internet of Drones Security Using Multi-Teacher Knowledge Distillation," *IEEE Access*, vol. 13, pp. 210134–210152, 2025, doi: 10.1109/ACCESS.2025.3642632.

[9] T. Wisanwanichthan and M. Thammawichai, "A Lightweight Intrusion Detection System for IoT and UAV Using Deep Neural Networks with Knowledge Distillation," *Computers*, vol. 14, no. 7, p. 291, Jul. 2025, doi: 10.3390/computers14070291.

[10] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," Mar. 09, 2015, *arXiv*: arXiv:1503.02531. doi: 10.48550/arXiv.1503.02531.

[11] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for Thin Deep Nets," Mar. 27, 2015, *arXiv*: arXiv:1412.6550. doi: 10.48550/arXiv.1412.6550.

[12] "Big IoT Data Analytics: Architecture, Opportunities, and Open Research Challenges," *IEEE Access*, vol. 5, pp. 5247–5261, 2017, doi: 10.1109/ACCESS.2017.2689040.

[13] E. Gyamfi and A. Jurcut, "Intrusion Detection in Internet of Things Systems: A Review on Design Approaches Leveraging Multi-Access Edge Computing, Machine Learning, and Datasets," *Sensors*, vol. 22, no. 10, p. 3744, May 2022, doi: 10.3390/s22103744.

[14] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023, doi: 10.3390/s23135941.

[15] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, "Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset," *Future Internet*, vol. 16, no. 8, p. 284, Aug. 2024, doi: 10.3390/fi16080284.

[16] I. Koukoulis, I. Syrigos, and T. Korakis, "Self-Supervised Transformer-based Contrastive Learning for Intrusion Detection Systems," May 12, 2025, *arXiv*: arXiv:2505.08816. doi: 10.48550/arXiv.2505.08816.

[17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," Jul. 25, 2019, *arXiv*: arXiv:1907.10902. doi: 10.48550/arXiv.1907.10902.

[18] M. Mushtaq, M. G. Madden, and I. Ullah, "Explainable lightweight neural networks: A survey on methods, metrics, and challenges," *Neurocomputing*, vol. 700, p. 134504, Nov. 2026, doi: 10.1016/j.neucom.2026.134504.

[19] H. Wasswa, H. Abbass, and T. Lynar, "A Quantized VAE-MLP Botnet Detection Model: A Systematic Evaluation of Quantization-Aware Training and Post-Training Quantization Strategies".

[20] TensorFlow, "SavedModel Warmup." Accessed: Aug. 13, 2026. [Online]. Available: https://www.tensorflow.org/tfx/serving/saved_model_warmup

[21] X.-H. Nguyen, X.-D. Nguyen, H.-H. Huynh, and K.-H. Le, "Realguard: A Lightweight Network Intrusion Detection System for IoT Gateways," *Sensors*, vol. 22, no. 2, p. 432, Jan. 2022, doi: 10.3390/s22020432.

[22] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in *Proceedings 2018 Network and Distributed System Security Symposium*, San Diego, CA: Internet Society, 2018. doi: 10.14722/ndss.2018.23204.

[23] Canadian Institute for Cybersecurity, University of New Brunswick, "CICIoT2023 Dataset." Accessed: Aug. 13, 2026. [Online]. Available: https://cicresearch.ca/IOTDataset/CIC_IOT_Dataset2023/

PHỤ LỤC

Đối chiếu đặc trưng CICIoT2023 giữa mô tả gốc và mã nguồn trích xuất

a) Mục đích

Phần phụ lục này trình bày sự khác biệt giữa tập dữ liệu CICIoT2023 46 đặc trưng (nguồn từ Kaggle, giống mô tả [14]) và tập dữ liệu CICIoT2023 39 đặc trưng (tải từ web chính chủ của UNB CIC), từ đó đưa ra quyết định của đề tài về tập dữ liệu nào sẽ được sử dụng.

Bảng 4 trong [14] mô tả 46 đặc trưng, phần lớn được định nghĩa theo luồng (flow), tức tổng hợp qua toàn bộ các gói tin thuộc cùng một kết nối hai chiều. Đối chiếu với mã nguồn chính thức do nhóm tác giả công bố để sinh ra tệp CSV 39 đặc trưng, cho thấy phần lớn khác biệt không nằm ở từng đặc trưng riêng lẻ, mà ở đơn vị tổng hợp: mã nguồn thay khái niệm "flow" bằng cửa sổ cố định 10 bản ghi gói tin liên tiếp [23]. Sự khác biệt về số lượng đặc trưng trình bày cũng như phương pháp thu thập khiến cho đề tài nghi vấn đề độ trung thực giữa kết quả nghiên cứu và mã nguồn công bố của [14].

Vì vậy tiếp theo đề tài sẽ trình bày các kết quả đã thực hiện đã dẫn đến quyết định sử dụng tập dữ liệu CICIoT2023 bản 37 đặc trưng.

b) Kết quả huấn luyện và suy luận trên tập dữ liệu CICIoT2023 46 đặc trưng

Dưới đây là bảng danh sách 46 đặc trưng của tập dữ liệu CICIoT2023 của [14], chi tiết như sau:

STT	Tên đặc trưng	Ý nghĩa
1	Flow duration	Thời gian thực thi luồng dữ liệu
2	Header length	Độ dài header
3	Protocol type	IP, UDP, TCP, ICMP, IGMP, Unknown (Integers)
4	Duration	Time-to-live (TTL)
5	Rate	Tỷ lệ truyền gói tin trong một luồng
6	Srate	Tốc độ truyền gói tin đi (outbound) từ thiết bị nguồn trong một luồng
7	Drate	Tốc độ truyền gói tin đến (inbound) thiết bị đích trong một luồng

8	fin flag number	Giá trị cờ FIN
9	syn flag number	Giá trị cờ SYN
10	rst flag number	Giá trị cờ RST
11	psh flag number	Giá trị cờ PSH
12	ack flag number	Giá trị cờ ACK
13	ece flag number	Giá trị cờ ECE
14	cwr flag number	Giá trị cờ CWR
15	ack count	Số lượng gói tin có cờ ACK trong cùng luồng
16	syn count	Số lượng gói tin có cờ SYN trong cùng luồng
17	fin count	Số lượng gói tin có cờ FIN trong cùng luồng
18	urg count	Số lượng gói tin có cờ URG trong cùng luồng
19	rst count	Số lượng gói tin có cờ RST trong cùng luồng
20	HTTP	Giao thức tầng ứng dụng sử dụng
21	HTTPS	Giao thức tầng ứng dụng sử dụng
22	DNS	Giao thức tầng ứng dụng sử dụng
23	Telnet	Giao thức tầng ứng dụng sử dụng
24	SMTP	Giao thức tầng ứng dụng sử dụng
25	SSH	Giao thức tầng ứng dụng sử dụng
26	IRC	Giao thức tầng ứng dụng sử dụng
27	TCP	Giao thức tầng vận chuyển sử dụng
28	UDP	Giao thức tầng vận chuyển sử dụng
29	DHCP	Giao thức tầng ứng dụng sử dụng
30	ARP	Giao thức tầng liên kết dữ liệu
31	ICMP	Giao thức tầng mạng sử dụng
32	IPv	Giao thức IP tầng mạng sử dụng
33	LLC	Giao thức tầng liên kết dữ liệu

34	Tot sum	Tổng chiều dài gói tin trong luồng
35	Min	Chiều dài gói tin nhỏ nhất trong luồng
36	Max	Chiều dài gói tin lớn nhất trong luồng
37	AVG	Trung bình chiều dài gói tin trong luồng
38	Std	Độ lệch chuẩn chiều dài gói tin
39	Tot size	Tổng kích thước gói tin
40	IAT	Khoảng thời gian giữa 2 gói tin liên tiếp
41	Number	Số lượng gói tin trong flow
42	Magnitue	= (AVG(chiều dài incoming packets) + AVG(chiều dài outgoing packets)) / 2
43	Radius	Độ phân tán = (Variance(incoming) + Variance(outgoing)) / 2
44	Covariance	Phương sai giữa độ dài của incoming và outgoing
45	Variance	Độ biến thiên giữa độ dài incoming và outgoing
46	Weight	Trọng số đặc trưng = Số gói tin vào x Số gói tin ra

Bảng phụ lục 1. Danh sách đặc trưng của tập dữ liệu CICIoT2023 có 46 đặc trưng

Ở những giai đoạn đầu thực hiện, đề tài đã nghiên cứu, thực hiện huấn luyện các mô hình giáo viên SSL-CL-DNN và XGBoost để áp dụng các kĩ thuật chưng cất tri thức KD truyền thống, Raid-KL (nhiệt độ cố định và nhiệt độ thích ứng) và chưng cất đa giáo viên cho các mô hình mạng MLP nhỏ sử dụng 2 giáo viên trên, quá trình thực hiện đầu tiên trên tập dữ liệu CICIoT2023 có 46 đặc trưng trên.

Với tập dữ liệu CICIoT2023 46 đặc trưng, quá trình thực hiện tiền xử lý tương tự như đối với tập dữ liệu 37 đặc trưng (ngoại trừ việc phải xóa hoặc chuẩn hóa các đặc trưng bị ảnh hưởng bởi "Number") bao gồm: làm sạch, giảm mẫu bằng kĩ thuật RandomUnderSampler sao cho mỗi nhãn trên 33 loại nhãn tấn công không quá 300,000 mẫu để có được tập dữ liệu gọn nhẹ đủ sử dụng (lưu lượng Benign không giảm mẫu), sau đó chia tập dữ liệu theo tỉ lệ 64:16:20 cho các tập dữ liệu train:validation:test, cuối cùng là chuẩn hóa dữ liệu bằng StandardScaler. Kết quả huấn luyện các mô hình trên tập dữ liệu CICIoT2023 có 46 đặc trưng chi tiết như sau:

Models	ACC (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Macro-AUC (%)	Model size	Params
SSL-CL-DNN	***96.59***	***92.72***	***82.2***	***87.15***	***99.67***	***2.19 MB***	***551,688***
SSL-CL-DNN KD	96.08	93.87	78.68	82.33	99.59	81.16 KB	20,776
SSL-CL-DNN Raid-KL (fixed temp)	96.03	93.83	78.54	82.14	99.56	81.16 KB	20,776
SSL-CL-DNN Raid-KL (adaptive temp)	96.24	94.2	79.19	82.93	99.61	145.16 KB	37,160
XGBOOST	***95.94***	***93***	***82***	***86***	***99.63***	***53MB***	***449,555***
MLP2x16 (XGB_SSL-CL-DNN)	94.75	87.14	74.08	75.83	98.14	2.71 KB	960
MLP2x32 (XGB_SSL-CL-DNN)	95.41	88.24	78.33	81.13	99.36	3.3 KB	2168
MLP2x64 (XGB_SSL-CL-DNN)	95.25	82.38	79.84	80.73	99.47	38.91 KB	5352

Bảng phụ lục 2. Kết quả huấn luyện các mô hình giáo viên và học sinh trên tập dữ liệu CICIoT2023 có 46 đặc trưng

Qua quá trình huấn luyện, kết quả trên tập dữ liệu CICIoT2023 có 46 đặc trưng cho ra kết quả huấn luyện vô cùng ấn tượng với độ chính xác các mô hình đều trên 94%, các mô hình học sinh SSL-CL-DNN chỉ mất từ 0.34 tới 0.51 điểm phần trăm, nhiệt độ thích ứng vẫn tốt hơn hẳn nhiệt độ cố định, các mô hình vẫn xảy ra hiện tượng mất mát ở macro-F1 nhiều hơn Accuracy. Đặc biệt hơn, các mô hình MLP với kích thước siêu nhỏ 3.3 KB lại sở hữu năng lực đánh giá vẫn rất cao, hao hụt tri thức từ giáo viên hầu như không đổi ở accuracy và mất 5 điểm ở macro-F1. Nhìn chung kết quả của các mô hình ban đầu trên tập dữ liệu này cao hơn rất nhiều, xấp xỉ 10% so với kết quả trên tập dữ liệu 37 đặc trưng ở mỗi chỉ số đánh giá ngoại trừ kích thước.

Mô hình giáo viên SSL-CL-DNN sau đó được chạy thử nghiệm với bộ trích xuất 46 đặc trưng bám theo định nghĩa các đặc trưng từ [14] và theo mô tả của tác giả. Quy trình trích xuất từ dữ liệu mạng PCAP có sẵn nhãn ban đầu chỉ được thực hiện trên 16 tập dữ liệu PCAP, kết quả suy luận như sau:

Pcap files	Vote (%)	Predicted label	True label
Backdoor_Malware	60.8	Recon	Web-based
Benign	60.8	Spoofing	Benign
CommandInjection	54.4	Recon	Web-based
DDoS-ACK_Fragmentation	88.2	Recon	DDoS
DDoS-ICMP_Flood	98.2	DDoS	DDoS
DDoS-SlowLoris	52.9	Spoofing	DDoS
DDoS-UDP_Flood	52.8	Mirai	DDoS
DNS_Spoofing	50.7	Spoofing	Spoofing
DictionaryBruteForce	61.1	Recon	BruteForce
DoS-TCP_Flood	61.9	DoS	DoS
MITM_ArpSpoofing	89.4	Spoofing	Spoofing
Recon-HostDiscovery	74.9	Recon	Recon
Recon-OSScan	50.3	Recon	Recon
Recon-Portscan	54.5	Recon	Recon
Uploading_Attack	67.1	Recon	Web-based
VulnerabilityScan	61.4	Recon	Recon

Bảng phụ lục 3. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 46 đặc trưng

Tuy nhiên khác với kết quả huấn luyện, khi cố gắng trích xuất 46 đặc trưng theo phương pháp và định nghĩa các đặc trưng vừa trình bày thì kết quả dự đoán lại không được tốt như mong đợi khi suy luận trên dữ liệu PCAP có nhãn, mô hình giáo viên SSL-CL-DNN hầu như chỉ phát hiện được Recon, Spoofing và DoS cũng như 1 phần DDoS. Việc phải trích xuất 46 đặc trưng theo định nghĩa thay vì theo mã nguồn vì UNB CIC chỉ công bố công thức tính cho 39 đặc trưng (cách tính thực tế trong mã nguồn tác giả rất khác so với định nghĩa trong bảng danh sách đặc trưng công bố của [14] như đã trình bày ở phụ lục A) khiến cho độ hiệu quả của phương pháp trích xuất 46 đặc trưng không ổn định.

Sau bước này, đề tài chuyển hướng sang sử dụng tập dữ liệu CSV 39 đặc trưng được công bố trên trang chính thức của UNB CIC công bố và tham khảo chính xác mã nguồn trích xuất ra 39 đặc trưng.

c) Kết quả huấn luyện và suy luận của các mô hình trên tập dữ liệu CICIoT2023 có 39 đặc trưng

Bảng sau trình bày sự khác biệt của tập dữ liệu 39 đặc trưng so với 46 đặc trưng ở trên:

Số thứ tự theo 46 đặc trưng ở trên	Đặc trưng	Trạng thái trong tập dữ liệu 39 đặc trưng
2	flow duration	Không có
5	Rate	Có, nhưng tính theo cửa sổ tổng hợp, không theo luồng
6, 7	Srate, Drate	Không có, chương trình có tính nhưng không đưa vào kết quả xuất ra
9–15	7 cờ _flag_number	Có, nhưng giá trị là tỉ lệ trung bình các gói mang cờ đó trong cửa sổ (số thực từ 0 đến 1), không phải giá trị cờ của một gói đại diện
16–18, 20	ack/syn/fin/rst_count	Có, nhưng đếm theo cửa sổ — chương trình khởi tạo lại biến đếm ở mỗi gói nên không tích lũy theo luồng như mô tả gốc
19	urg_count	Không có, chương trình có tính nhưng không ghi vào kết quả
35–39	Tot sum, Min, Max, AVG, Std	Có, nhưng tính trên toàn bộ gói trong cửa sổ, không theo luồng
42	Number	Có, nhưng giá trị luôn xấp xỉ bằng kích thước cửa sổ
46	Variance	Tính bằng phương sai của độ dài gói trong đề tài mà không có phân biệt luồng đến/đi
43–45, 47	Magnitude, Radius, Covariance, Weight	Không có, chương trình có hàm tính sẵn nhưng đoạn gọi hàm này bị vô hiệu hóa, không được thực thi

Bảng phụ lục 4. Khác biệt trong định nghĩa đặc trưng giữa tập dữ liệu 39 đặc trưng so với tập dữ liệu 46 đặc trưng

Models	Accuracy (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Macro-AUC (%)	Model size (MB)	Params
SSL-CL-DNN	85.87	81.59	70.04	73.03	98.62	2.07	542,472

Bảng phụ lục 5. Kết quả huấn luyện mô hình SSL-CL-DNN trên tập dữ liệu CICIoT2023 39 đặc trưng

Kết quả huấn luyện mô hình SSL-CL-DNN trên tập dữ liệu 39 đặc trưng khá tương đồng với kết quả trên tập dữ liệu 37 đặc trưng (do cùng nguồn dữ liệu). Kết quả suy luận trên 16 file PCAP có nhãn như sau:

Pcap files	**Vote (%)**	**Predicted label**	**True label**
Backdoor_Malware	39.1	Benign	Web-Based
Benign	79.0	Benign	Benign
CommandInjection	44.1	Spoofing	Web-Based
DDoS-ACK_Fragmentation	96.6	DDoS	DDoS
DDoS-ICMP_Flood	99.7	DDoS	DDoS
DDoS-SlowLoris	58.1	Recon	DDoS
DDoS-UDP_Flood	99.5	Spoofing	DDoS
DNS_Spoofing	43.3	Spoofing	Spoofing
DictionaryBruteForce	37.0	Benign	BruteForce
DoS-TCP_Flood	69.2	Spoofing	DoS
MITM-ArpSpoofing	72.6	Spoofing	Spoofing
Recon-HostDiscovery	89.6	Recon	Recon
Recon-OSScan	51.3	Benign	Recon
Recon-Portscan	78.2	Recon	Recon
Uploading_Attack	42.0	Benign	Web-Based
VulnerabilityScan	81.3	Recon	Recon

Bảng phụ lục 6. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng

Kết quả trên cho thấy mô hình SSL-CL-DNN trên tập dữ liệu 39 đặc trưng vẫn chưa suy luận tốt hơn mô hình học từ tập 46 đặc trưng, nhưng ở đây đề tài phát hiện ra vấn đề của đặc trưng "Number" như đã trình bày ở chương 3. Bước sau đề tài tiền xử lý lại, huấn luyện lại mô hình SSL-CL-DNN tập dữ liệu 37 đặc trưng và suy luận lại trên tập dữ liệu PCAP có nhãn để so sánh với kết quả trên tập dữ liệu 39 đặc trưng.

d) Kết quả huấn luyện và suy luận của các mô hình trên tập dữ liệu CICIoT2023 có 37 đặc trưng

Kết quả huấn luyện mô hình SSL-CL-DNN trên 37 đặc trưng đã được trình bày ở Chương 4, dưới đây là kết quả suy luận trên 16 tập tin mạng PCAP có nhãn:

Pcap files	Vote (%)	Predicted label	True label
Backdoor_Malware	42.6	Recon	Web-Based
Benign	82.8	Benign	Benign
CommandInjection	29.0	Web-Based	Web-Based
DDoS-ACK_Fragmentation	98.4	DDoS	DDoS
DDoS-ICMP_Flood	99.9	DDoS	DDoS
DDoS-SlowLoris	39.8	DDoS	DDoS
DDoS-UDP_Flood	96.6	DDoS	DDoS
DNS_Spoofing	52.3	Spoofing	Spoofing
DictionaryBruteForce	38.5	Benign	BruteForce
DoS-TCP_Flood	71.3	DoS	DoS
MITM-ArpSpoofing	75.0	Spoofing	Spoofing
Recon-HostDiscovery	91.6	Recon	Recon
Recon-OSScan	50.2	Benign	Recon
Recon-Portscan	82.3	Recon	Recon
Uploading_Attack	40.3	Recon	Web-Based
VulnerabilityScan	85.9	Recon	Recon

Bảng phụ lục 7. Kết quả suy luận của mô hình SSL-CL-DNN trên dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng

Kết suy luận cho thấy mô hình SSL-CL-DNN sau khi huấn luyện trên tập dữ liệu 37 đặc trưng và suy luận với đặc trưng từ bộ trích xuất 37 đặc trưng thì mô hình cho ra kết quả suy luận tốt hơn rất nhiều, đúng 12/16 lưu lượng (so với 8/12 lưu lượng ở cả 2 kết quả suy luận trước.

Sau đó để kiểm tra kết quả này có lặp lại một lần nữa không thì đề tài huấn luyện thêm mô hình LSTM trên tập 39 đặc trưng và 37 đặc trưng và đánh giá trên đầy đủ 34 tập dữ liệu thuộc 34 loại lưu lượng để kiểm tra lại một lần nữa thì cho ra kết quả suy luận tốt hơn ở mô hình huấn luyện trên 37 so với 39, quan sát các ảnh kết quả sau:

Pcap files	Vote (%)	Accuracy (%)	Predicted label	True label
Backdoor_Malware.pcap	39.1	0.2	Benign	Web-Based
Benign.pcap	79.0	79.0	Benign	Benign
BrowserHijacking.pcap	38.7	11.9	Benign	Web-Based
CommandInjection.pcap	44.1	0.4	Spoofing	Web-Based
DDoS-ACK.pcap	96.6	96.6	DDoS	DDoS
DDoS-HTTP_Flood.pcap	49.6	0.1	Recon	DDoS
DDoS-ICMP_Flood.pcap	99.7	99.7	DDoS	DDoS
DDoS-ICMP.pcap	93.2	93.2	DDoS	DDoS
DDoS-PSHACK_Flood.pcap	93.4	93.4	DDoS	DDoS
DDoS-RSTFINFlood.pcap	78.8	78.8	DDoS	DDoS
DDoS-SYN_Flood.pcap	69.8	30.2	Recon	DDoS
DDoS-SlowLoris.pcap	58.1	0.9	Recon	DDoS
DDoS-SynonymousIP_Flood.pcap	53.7	46.3	Recon	DDoS
DDoS-TCP_Flood.pcap	98.6	98.6	DDoS	DDoS
DDoS-UDP_Flood.pcap	99.5	0.0	Spoofing	DDoS
DDoS-UDP_Fragmentation.pcap	93.7	93.7	DDoS	DDoS
DNS_Spoofing.pcap	43.3	43.3	Spoofing	Spoofing
DictionaryBruteForce.pcap	37.0	21.3	Benign	BruteForce
DoS-HTTP_Flood.pcap	31.3	31.3	DoS	DoS
DoS-SYN_Flood.pcap	73.6	0.0	Recon	DoS
DoS-TCP_Flood.pcap	69.2	0.0	Spoofing	DoS
DoS-UDP_Flood.pcap	93.4	4.4	Spoofing	DoS
MITM-ArpSpoofing.pcap	72.6	72.6	Spoofing	Spoofing
Mirai-greeth_flood.pcap	94.6	94.6	Mirai	Mirai
Mirai-greip_flood.pcap	98.3	98.3	Mirai	Mirai
Mirai-greip_flood.pcap	98.3	98.3	Mirai	Mirai
Mirai-udpplain.pcap	76.7	76.7	Mirai	Mirai
Recon-HostDiscovery.pcap	89.6	89.6	Recon	Recon
Recon-OSScan.pcap	51.3	43.6	Benign	Recon
Recon-Portscan.pcap	78.2	78.2	Recon	Recon
SqlInjection.pcap	60.1	1.3	Recon	Web-Based
Uploading_Attack.pcap	42.0	0.2	Benign	Web-Based
VulnerabilityScan.pcap	81.3	81.3	Recon	Recon
XSS.pcap	41.4	0.1	Benign	Web-Based
Tổng số dự đoán đúng của SSL-CL-DNN:			**18**	

Bảng phụ lục 8. Kết quả suy luận của mô hình SSL-CL-DNN trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng

Pcap files	Vote (%)	Accuracy (%)	Predicted label	True label
Backdoor_Malware.pcap	42.6	9.9	Recon	Web-Based
Benign.pcap	82.8	82.8	Benign	Benign
BrowserHijacking.pcap	34.7	19.4	Benign	Web-Based
CommandInjection.pcap	29.0	29.0	Web-Based	Web-Based
DDoS-ACK.pcap	98.4	98.4	DDoS	DDoS
DDoS-HTTP_Flood.pcap	53.0	22.9	DoS	DDoS
DDoS-ICMP_Flood.pcap	99.9	99.9	DDoS	DDoS
DDoS-ICMP.pcap	95.0	95.0	DDoS	DDoS
DDoS-PSHACK_Flood.pcap	99.6	99.6	DDoS	DDoS
DDoS-RSTFINFlood.pcap	100.0	100.0	DDoS	DDoS
DDoS-SYN_Flood.pcap	70.0	70.0	DDoS	DDoS
DDoS-SlowLoris.pcap	39.8	39.8	DDoS	DDoS
DDoS-SynonymousIP_Flood.pcap	98.7	98.7	DDoS	DDoS
DDoS-TCP_Flood.pcap	65.4	34.5	DoS	DDoS
DDoS-UDP_Flood.pcap	96.6	96.6	DDoS	DDoS
DDoS-UDP_Fragmentation.pcap	98.0	98.0	DDoS	DDoS
DNS_Spoofing.pcap	52.3	52.3	Spoofing	Spoofing
DictionaryBruteForce.pcap	38.5	28.7	Benign	BruteForce
DoS-HTTP_Flood.pcap	55.1	55.1	DoS	DoS
DoS-SYN_Flood.pcap	59.0	40.0	DDoS	DoS
DoS-TCP_Flood.pcap	71.3	71.3	DoS	DoS
DoS-UDP_Flood.pcap	90.9	8.2	DDoS	DoS
MITM-ArpSpoofing.pcap	75.0	75.0	Spoofing	Spoofing
Mirai-greeth_flood.pcap	96.2	96.2	Mirai	Mirai
Mirai-greip_flood.pcap	99.1	99.1	Mirai	Mirai
Mirai-greip_flood.pcap	99.1	99.1	Mirai	Mirai
Mirai-udpplain.pcap	98.8	98.8	Mirai	Mirai
Recon-HostDiscovery.pcap	91.6	91.6	Recon	Recon
Recon-OSScan.pcap	50.2	46.8	Benign	Recon
Recon-Portscan.pcap	82.3	82.3	Recon	Recon
SqlInjection.pcap	65.7	5.8	Recon	Web-Based
Uploading_Attack.pcap	40.3	12.4	Recon	Web-Based
VulnerabilityScan.pcap	85.9	85.9	Recon	Recon
XSS.pcap	40.1	11.8	Recon	Web-Based
Tổng số dự đoán đúng của SSL-CL-DNN:			**23**	

Bảng phụ lục 9. Kết quả suy luận của mô hình SSL-CL-DNN trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng

Pcap files	Vote (%)	Accuracy (%)	Predicted label	True label
Backdoor_Malware.pcap	45.3	8.4	Recon	Web-Based
Benign.pcap	79.2	79.2	Benign	Benign
BrowserHijacking.pcap	41.0	16.9	Recon	Web-Based
CommandInjection.pcap	34.9	26.3	Benign	Web-Based
DDoS-ACK.pcap	97.0	97.0	DDoS	DDoS
DDoS-HTTP_Flood.pcap	74.4	1.6	Recon	DDoS
DDoS-ICMP_Flood.pcap	74.0	74.0	DDoS	DDoS
DDoS-ICMP.pcap	93.7	93.7	DDoS	DDoS
DDoS-PSHACK_Flood.pcap	98.0	98.0	DDoS	DDoS
DDoS-RSTFINFlood.pcap	98.7	98.7	DDoS	DDoS
DDoS-SYN_Flood.pcap	96.8	96.8	DDoS	DDoS
DDoS-SlowLoris.pcap	61.0	0.1	Recon	DDoS
DDoS-SynonymousIP_Flood.pcap	97.9	97.9	DDoS	DDoS
DDoS-TCP_Flood.pcap	97.4	97.4	DDoS	DDoS
DDoS-UDP_Flood.pcap	96.9	96.9	DDoS	DDoS
DDoS-UDP_Fragmentation.pcap	96.6	96.6	DDoS	DDoS
DNS_Spoofing.pcap	48.1	48.1	Spoofing	Spoofing
DictionaryBruteForce.pcap	35.4	26.1	Recon	BruteForce
DoS-HTTP_Flood.pcap	54.9	8.6	Recon	DoS
DoS-SYN_Flood.pcap	87.1	0.8	DDoS	DoS
DoS-TCP_Flood.pcap	68.5	0.0	DDoS	DoS
DoS-UDP_Flood.pcap	92.5	3.2	DDoS	DoS
MITM-ArpSpoofing.pcap	73.1	73.1	Spoofing	Spoofing
Mirai-greeth_flood.pcap	92.3	92.3	Mirai	Mirai
Mirai-greip_flood.pcap	96.7	96.7	Mirai	Mirai
Mirai-greip_flood.pcap	96.7	96.7	Mirai	Mirai
Mirai-udpplain.pcap	94.1	94.1	Mirai	Mirai
Recon-HostDiscovery.pcap	92.1	92.1	Recon	Recon
Recon-OSScan.pcap	49.9	49.9	Recon	Recon
Recon-Portscan.pcap	86.8	86.8	Recon	Recon
SqlInjection.pcap	73.5	5.2	Recon	Web-Based
Uploading_Attack.pcap	46.2	6.4	Recon	Web-Based
VulnerabilityScan.pcap	88.4	88.4	Recon	Recon
XSS.pcap	42.5	8.3	Recon	Web-Based
Tổng số dự đoán đúng của LSTM:			**21**	

Bảng phụ lục 10. Kết quả suy luận của mô hình LSTM trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 39 đặc trưng

Pcap files	Vote (%)	Accuracy (%)	Predicted label	True label
Backdoor_Malware.pcap	47.9	7.6	Recon	Web-Based
Benign.pcap	80.4	80.4	Benign	Benign
BrowserHijacking.pcap	38.6	17.0	Recon	Web-Based
CommandInjection.pcap	33.6	29.3	Recon	Web-Based
DDoS-ACK.pcap	98.4	98.4	DDoS	DDoS
DDoS-HTTP_Flood.pcap	45.9	32.4	DoS	DDoS
DDoS-ICMP_Flood.pcap	99.9	99.9	DDoS	DDoS
DDoS-ICMP.pcap	94.8	94.8	DDoS	DDoS
DDoS-PSHACK_Flood.pcap	99.5	99.5	DDoS	DDoS
DDoS-RSTFINFlood.pcap	100.0	100.0	DDoS	DDoS
DDoS-SYN_Flood.pcap	72.8	72.8	DDoS	DDoS
DDoS-SlowLoris.pcap	40.4	40.4	DDoS	DDoS
DDoS-SynonymousIP_Flood.pcap	99.0	99.0	DDoS	DDoS
DDoS-TCP_Flood.pcap	61.4	61.4	DDoS	DDoS
DDoS-UDP_Flood.pcap	98.6	98.6	DDoS	DDoS
DDoS-UDP_Fragmentation.pcap	98.2	98.2	DDoS	DDoS
DNS_Spoofing.pcap	49.4	49.4	Spoofing	Spoofing
DictionaryBruteForce.pcap	34.5	29.4	Recon	BruteForce
DoS-HTTP_Flood.pcap	56.8	56.8	DoS	DoS
DoS-SYN_Flood.pcap	71.4	25.3	DDoS	DoS
DoS-TCP_Flood.pcap	51.7	47.9	DDoS	DoS
DoS-UDP_Flood.pcap	91.7	7.0	DDoS	DoS
MITM-ArpSpoofing.pcap	73.5	73.5	Spoofing	Spoofing
Mirai-greeth_flood.pcap	95.6	95.6	Mirai	Mirai
Mirai-greip_flood.pcap	98.7	98.7	Mirai	Mirai
Mirai-greip_flood.pcap	98.7	98.7	Mirai	Mirai
Mirai-udpplain.pcap	98.8	98.8	Mirai	Mirai
Recon-HostDiscovery.pcap	92.1	92.1	Recon	Recon
Recon-OSScan.pcap	50.3	47.4	Benign	Recon
Recon-Portscan.pcap	85.4	85.4	Recon	Recon
SqlInjection.pcap	71.1	6.3	Recon	Web-Based
Uploading_Attack.pcap	47.8	8.7	Recon	Web-Based
VulnerabilityScan.pcap	87.6	87.6	Recon	Recon
XSS.pcap	44.3	9.9	Recon	Web-Based
Tổng số dự đoán đúng của LSTM:			**22**	

Bảng phụ lục 11. Kết quả suy luận của mô hình LSTM trên 34 tập dữ liệu PCAP có nhãn bằng bộ trích xuất 37 đặc trưng

Qua các kết quả cho thấy mô hình SSL-CL-DNN đúng 23/34 nhãn với tập dữ liệu 37 đặc trưng so với 18/34 nhãn của tập 39 đặc trưng, ở LSTM thì đúng 22/34 so với 21/34 một lần nữa khẳng định tập dữ liệu CICIoT2023 37 đặc trưng cho ra kết quả suy luận mô hình tốt hơn, đề tài từ các kết quả này đã quyết định lựa chọn tập dữ liệu CICIoT2023 có 37 đặc trưng (bám theo mã nguồn và có tiền xử lí bổ sung từ tập dữ liệu CICIoT2023 có 39 đặc trưng tải từ trang chính của UNB CIC).

Dù kết quả đã trình bày ở chưng 4 khi suy luận với các mô hình học sinh cho giá trị các chỉ số đều bị sụp xuống, cho thấy bộ trích xuất 37 đặc trưng vẫn chưa phù hợp và còn nhiều hạn chế nhưng đây là lựa chọn có kiểm nghiệm từ đề tài. Như hướng phát triển của đề tài đã trình bày, việc thay thế hướng tiếp cận bằng một phương pháp trích xuất khác hay tập dữ liệu khác hoàn toàn có thể thay đổi kết quả suy luận của các mô hình.

Tuy nhiên ảnh hưởng từ phương pháp trích xuất của đề tài vẫn không thể phủ định được hiệu quả của các phương pháp chưng cất tri thức khi mà với sức nén rất cao và kích thước mô hình rất nhỏ, các mô hình học sinh vẫn bảo toàn được rất nhiều tri thức từ giáo viên, cho thấy được tiềm năng của chưng cất tri thức trong phát hiện xâm nhập tại mạng IoT biên.